

Operating Systems for Software Development Engineers

Complete Interview Preparation Handbook

Aditya Udai

2026

Contents

I	Fundamentals	1
1	Introduction to Operating Systems	3
1.1	Chapter Overview	3
1.2	Learning Objectives	3
1.3	What is an Operating System?	4
1.3.1	Definition	4
1.3.2	Simple Analogy	4
1.3.3	Examples of Operating Systems	4
1.4	Goals of an Operating System	5
1.4.1	Convenience	5
1.4.2	Efficiency	5
1.4.3	Resource Sharing	5
1.4.4	Reliability	5
1.4.5	Security	5
1.4.6	Scalability	5
1.5	Evolution of Operating Systems	6
1.6	Batch Systems	6
1.6.1	Definition	6
1.6.2	Working	6
1.6.3	Advantages	6
1.6.4	Disadvantages	6
1.6.5	Example	7
1.7	Multiprogramming Systems	7
1.7.1	Definition	7
1.7.2	Goal	7

1.7.3	Example	7
1.8	Time Sharing Systems	7
1.8.1	Definition	7
1.8.2	How It Works	7
1.8.3	Advantages	7
1.8.4	Examples	8
1.9	Distributed Operating Systems	8
1.9.1	Definition	8
1.9.2	Goals	8
1.9.3	Examples	8
1.10	Real-Time Operating Systems	8
1.10.1	Definition	8
1.10.2	Types	8
1.10.3	Examples	9
1.11	Embedded Operating Systems	9
1.11.1	Definition	9
1.11.2	Characteristics	9
1.11.3	Examples	9
1.12	Components of an Operating System	9
1.13	Kernel Overview	10
1.13.1	Definition	10
1.13.2	Kernel Responsibilities	10
1.13.3	Types of Kernels	10
1.13.4	Examples	10
1.14	User Mode vs Kernel Mode	11
1.15	System Calls	11
1.15.1	Examples	11
1.15.2	Flow	11
1.16	Boot Process Overview	12
1.17	Operating System Services	12

1.18	Why Operating Systems Matter for SDEs	12
1.18.1	Interview Perspective	12
1.18.2	Real-World Examples	13
1.18.3	Interview Tip	13
1.19	Key Takeaways	13
1.20	15 Interview Questions with Answers	13
1.21	15 MCQs with Answers	14
1.22	Revision Notes	15
1.23	Cheat Sheet	15
2	Computer System Architecture	17
2.1	Introduction	17
2.2	Computer System Overview	17
2.2.1	Major Components	17
2.2.2	High-Level Architecture	18
2.2.3	Von Neumann Architecture	18
2.3	CPU Architecture	18
2.3.1	CPU Components	18
2.4	Registers	19
2.4.1	Characteristics	19
2.4.2	Common Registers	19
2.5	Program Counter	19
2.5.1	Operation	20
2.5.2	Example	20
2.6	Stack Pointer	20
2.6.1	Uses	20
2.6.2	Stack Layout	21
2.7	Memory Hierarchy	21
2.7.1	Hierarchy Levels	21
2.7.2	Hierarchy Comparison	21
2.8	Instruction Cycle	22

2.9	Fetch Decode Execute Cycle	22
2.9.1	Fetch	22
2.9.2	Decode	22
2.9.3	Execute	22
2.10	Interrupts	22
2.10.1	Types	23
2.10.2	Interrupt Handling Flow	23
2.11	Trap Handling	23
2.11.1	Examples	23
2.11.2	Flow	23
2.12	Exceptions	23
2.12.1	Examples	24
2.12.2	Classification	24
2.13	DMA	24
2.13.1	Advantages	24
2.13.2	DMA Operation	24
2.14	Cache Memory	24
2.14.1	Why Cache Works	25
2.14.2	Cache Levels	25
2.14.3	Cache Metrics	25
2.14.4	Interview Insight	25
2.15	Multicore Systems	25
2.15.1	Benefits	25
2.15.2	Architecture	26
2.16	NUMA Architecture	26
2.16.1	Concept	26
2.16.2	Architecture	26
2.16.3	Characteristics	26
2.17	Hardware Protection Mechanisms	26
2.17.1	Goals	26

2.17.2	Mechanisms	27
2.18	User Mode and Privileged Instructions	27
2.18.1	User Mode	27
2.18.2	Kernel Mode	27
2.18.3	Privileged Instructions	27
2.18.4	Mode Transition	28
2.19	Interview Explanations	28
2.19.1	Why Are Registers Faster Than RAM?	28
2.19.2	Why Are Interrupts Important?	28
2.19.3	Why Is Cache Needed?	28
2.19.4	Difference Between Interrupt and Trap	28
2.19.5	Why Does NUMA Exist?	28
2.20	Key Takeaways	28
II	Processes and Threads	29
3	Processes	31
3.1	Chapter Overview	31
3.2	Learning Objectives	31
3.3	Process Concept	31
3.3.1	Definition	31
3.3.2	Real-World Analogy	32
3.3.3	Examples	32
3.4	Program vs Process	32
3.4.1	Interview Tip	33
3.5	Process Memory Layout	33
3.5.1	Text Segment	33
3.5.2	Data Segment	33
3.5.3	BSS Segment	33
3.5.4	Heap	34
3.5.5	Stack	34

3.5.6	Interview Question	34
3.6	Process States	34
3.6.1	Five-State Model	34
3.6.2	Ready State	35
3.6.3	Running State	35
3.6.4	Waiting State	35
3.6.5	Terminated State	35
3.7	Process Control Block (PCB)	35
3.7.1	Definition	35
3.7.2	Contents of PCB	35
3.8	Context Switching	36
3.8.1	Definition	36
3.8.2	Steps	36
3.8.3	Interview Insight	36
3.9	Process Creation	37
3.9.1	Linux Model	37
3.9.2	Windows Model	37
3.10	fork()	37
3.10.1	Definition	37
3.10.2	Return Values	37
3.10.3	Example	37
3.10.4	Interview Insight	38
3.11	exec()	38
3.11.1	Definition	38
3.11.2	Common Variants	38
3.11.3	Example	38
3.11.4	Important Point	38
3.12	wait()	39
3.12.1	Definition	39
3.12.2	Example	39

3.12.3 Purpose	39
3.13 Process Scheduling Overview	39
3.14 Process Termination	40
3.15 Zombie Processes	40
3.15.1 Definition	40
3.15.2 Reason	40
3.15.3 Characteristics	40
3.15.4 Fix	40
3.15.5 Linux Example	41
3.16 Orphan Processes	41
3.16.1 Definition	41
3.16.2 What Happens?	41
3.16.3 Interview Question	41
3.17 Daemon Processes	41
3.17.1 Definition	41
3.17.2 Characteristics	41
3.18 Process Hierarchies	42
3.19 Linux Process Management	42
3.19.1 Useful Commands	42
3.19.2 Examples	42
3.20 Interview Insights	43
3.21 Key Takeaways	43
3.22 20 Interview Questions with Answers	44
3.23 20 MCQs with Answers	45
3.24 Revision Notes	45
3.25 Cheat Sheet	46
4 Threads	47
4.1 Motivation for Threads	47
4.2 What is a Thread	47
4.3 Process vs Thread	48

4.4	Thread States	48
4.5	User Threads	48
4.6	Kernel Threads	49
4.7	Multithreading Models	49
4.8	Many-to-One	49
4.9	One-to-One	50
4.10	Many-to-Many	50
4.11	Thread Libraries	50
4.12	POSIX Threads	50
4.12.1	Pthreads Example	50
4.13	Java Threads	51
4.13.1	Java Example	51
4.14	Python Threads	51
4.14.1	Python Example	51
4.15	Thread Pools	52
4.16	Thread Safety	52
4.17	Thread Local Storage	53
4.18	Multicore Execution	53
4.19	Interview Notes	53
4.20	Key Takeaways	53
5	CPU Scheduling	55
5.1	Chapter Overview	55
5.2	Learning Objectives	55
5.3	CPU Scheduling Basics	55
5.3.1	Definition	55
5.3.2	Why Scheduling is Needed	56
5.3.3	Scheduling Cycle	56
5.4	Scheduling Goals	56
5.5	Throughput	56
5.5.1	Definition	56

5.5.2	Example	56
5.6	Turnaround Time	57
5.6.1	Definition	57
5.6.2	Example	57
5.7	Waiting Time	57
5.7.1	Definition	57
5.7.2	Example	57
5.8	Response Time	58
5.8.1	Definition	58
5.8.2	Importance	58
5.9	CPU Utilization	58
5.9.1	Definition	58
5.9.2	Goal	58
5.10	Preemptive Scheduling	58
5.10.1	Definition	58
5.10.2	Examples	58
5.10.3	Advantages	59
5.10.4	Disadvantages	59
5.11	Non-Preemptive Scheduling	59
5.11.1	Definition	59
5.11.2	Examples	59
5.11.3	Advantages	59
5.11.4	Disadvantages	59
5.12	First Come First Serve (FCFS)	59
5.12.1	Definition	59
5.12.2	Working	60
5.12.3	Advantages	60
5.12.4	Disadvantages	60
5.12.5	Complexity	60
5.12.6	Example	60

5.13	Shortest Job First (SJF)	61
5.13.1	Definition	61
5.13.2	Working	61
5.13.3	Advantages	61
5.13.4	Disadvantages	61
5.13.5	Complexity	61
5.13.6	Example	61
5.14	Shortest Remaining Time First (SRTF)	62
5.14.1	Definition	62
5.14.2	Working	62
5.14.3	Advantages	62
5.14.4	Disadvantages	62
5.14.5	Complexity	62
5.14.6	Example	62
5.15	Priority Scheduling	63
5.15.1	Definition	63
5.15.2	Working	63
5.15.3	Advantages	63
5.15.4	Disadvantages	63
5.15.5	Complexity	63
5.15.6	Example	63
5.16	Round Robin Scheduling	64
5.16.1	Definition	64
5.16.2	Working	64
5.16.3	Advantages	64
5.16.4	Disadvantages	64
5.16.5	Complexity	64
5.16.6	Example	64
5.17	Multilevel Queue Scheduling	65
5.17.1	Definition	65

5.17.2	Working	65
5.17.3	Advantages	65
5.17.4	Disadvantages	65
5.17.5	Complexity	65
5.18	Multilevel Feedback Queue (MLFQ)	65
5.18.1	Definition	65
5.18.2	Working	66
5.18.3	Advantages	66
5.18.4	Disadvantages	66
5.18.5	Complexity	66
5.18.6	Example	66
5.19	Dispatcher	66
5.19.1	Definition	66
5.19.2	Dispatch Latency	67
5.20	Context Switch Impact	67
5.21	Starvation	67
5.21.1	Definition	67
5.21.2	Common In	67
5.22	Aging	67
5.22.1	Definition	67
5.22.2	Purpose	67
5.22.3	Example	68
5.23	Comparative Summary	68
5.24	Common Interview Traps	68
5.25	How to Solve Scheduling Numericals Quickly	68
5.25.1	Step 1	68
5.25.2	Step 2	68
5.25.3	Step 3	68
5.25.4	Step 4	69
5.25.5	Step 5	69

5.25.6	Golden Interview Formula	69
5.26	Key Takeaways	69
5.27	25 Interview Questions with Answers	70
5.28	25 MCQs with Answers	71
5.29	Revision Notes	72
5.30	Cheat Sheet	72
III	Synchronization	73
6	The Critical Section Problem	75
6.1	Introduction to Concurrency	75
6.1.1	Concurrent Execution	75
6.2	Shared Resources	75
6.2.1	Shared Counter Example	76
6.3	Race Conditions	76
6.3.1	Example	76
6.3.2	Execution Timeline	76
6.4	Critical Sections	77
6.4.1	Structure	77
6.5	The Critical Section Problem	77
6.6	Requirements for a Correct Solution	78
6.6.1	Mutual Exclusion	78
6.6.2	Progress	78
6.6.3	Bounded Waiting	78
6.7	Software-Based Solutions	78
6.8	Peterson's Algorithm	78
6.8.1	Shared Variables	79
6.8.2	Process i	79
6.8.3	Working Principle	79
6.8.4	State Diagram	79
6.8.5	Interview Note	79

6.9 Bakery Algorithm	79
6.9.1 Idea	80
6.9.2 Example	80
6.9.3 Advantages	80
6.10 Hardware Support for Synchronization	80
6.11 Atomic Operations	80
6.11.1 Atomicity Illustration	81
6.12 Test-and-Set Instruction	81
6.12.1 Pseudo Code	81
6.12.2 Spinlock Example	81
6.13 Compare-and-Swap (CAS)	81
6.13.1 Pseudo Code	82
6.13.2 Example	82
6.14 Exchange Instruction	82
6.14.1 Pseudo Code	82
6.14.2 Use Cases	82
6.15 Spinlocks	82
6.15.1 Operation	83
6.15.2 Visualization	83
6.15.3 Advantages	83
6.15.4 Disadvantages	83
6.16 Modern CPU Challenges	83
6.17 Memory Reordering Basics	84
6.17.1 Example	84
6.17.2 Memory Illustration	84
6.18 Real-World Race Condition Examples	84
6.18.1 Bank Account Update	84
6.18.2 Inventory Management	84
6.18.3 Ticket Booking	84
6.18.4 Web Counter	85

6.19	Interview Explanations	85
6.19.1	Why is the Critical Section Problem Important?	85
6.19.2	Why Isn't Peterson's Algorithm Used in Practice?	85
6.19.3	Why Is CAS Popular?	85
6.19.4	Spinlock vs Mutex	85
6.20	Common Mistakes	85
6.21	Key Takeaways	86
7	The Critical Section Problem	87
7.1	Introduction to Concurrency	87
7.1.1	Concurrent Execution	87
7.2	Shared Resources	87
7.2.1	Shared Counter Example	88
7.3	Race Conditions	88
7.3.1	Example	88
7.3.2	Execution Timeline	88
7.4	Critical Sections	89
7.4.1	Structure	89
7.5	The Critical Section Problem	89
7.6	Requirements for a Correct Solution	90
7.6.1	Mutual Exclusion	90
7.6.2	Progress	90
7.6.3	Bounded Waiting	90
7.7	Software-Based Solutions	90
7.8	Peterson's Algorithm	90
7.8.1	Shared Variables	91
7.8.2	Process i	91
7.8.3	Working Principle	91
7.8.4	State Diagram	91
7.8.5	Interview Note	91
7.9	Bakery Algorithm	91

7.9.1	Idea	92
7.9.2	Example	92
7.9.3	Advantages	92
7.10	Hardware Support for Synchronization	92
7.11	Atomic Operations	92
7.11.1	Atomicity Illustration	93
7.12	Test-and-Set Instruction	93
7.12.1	Pseudo Code	93
7.12.2	Spinlock Example	93
7.13	Compare-and-Swap (CAS)	93
7.13.1	Pseudo Code	94
7.13.2	Example	94
7.14	Exchange Instruction	94
7.14.1	Pseudo Code	94
7.14.2	Use Cases	94
7.15	Spinlocks	94
7.15.1	Operation	95
7.15.2	Visualization	95
7.15.3	Advantages	95
7.15.4	Disadvantages	95
7.16	Modern CPU Challenges	95
7.17	Memory Reordering Basics	96
7.17.1	Example	96
7.17.2	Memory Illustration	96
7.18	Real-World Race Condition Examples	96
7.18.1	Bank Account Update	96
7.18.2	Inventory Management	96
7.18.3	Ticket Booking	96
7.18.4	Web Counter	97
7.19	Interview Explanations	97

7.19.1	Why is the Critical Section Problem Important?	97
7.19.2	Why Isn't Peterson's Algorithm Used in Practice?	97
7.19.3	Why Is CAS Popular?	97
7.19.4	Spinlock vs Mutex	97
7.20	Common Mistakes	97
7.21	Key Takeaways	98
8	Classical Synchronization Problems	99
8.1	Introduction	99
8.2	Producer Consumer Problem	99
8.2.1	Problem Statement	99
8.2.2	Visualization	100
8.2.3	Synchronization Challenges	100
8.2.4	Semaphore Solution	100
8.2.5	Monitor Solution	100
8.2.6	Modern Implementation	100
8.2.7	Complexity Discussion	101
8.2.8	Execution Example	101
8.2.9	C Example	101
8.2.10	Java Example	101
8.2.11	Python Example	101
8.2.12	Interview Follow-Up	101
8.3	Bounded Buffer Problem	102
8.3.1	Problem Statement	102
8.3.2	Visualization	102
8.3.3	Synchronization Challenges	102
8.3.4	Semaphore Solution	102
8.3.5	Monitor Solution	102
8.3.6	Modern Implementation	102
8.3.7	Complexity Discussion	102
8.4	Readers Writers Problem	103

8.4.1	Problem Statement	103
8.4.2	Visualization	103
8.4.3	Synchronization Challenges	103
8.4.4	Semaphore Solution	103
8.4.5	Monitor Solution	103
8.4.6	Modern Implementation	104
8.4.7	Complexity Discussion	104
8.4.8	Interview Follow-Up	104
8.5	Dining Philosophers Problem	104
8.5.1	Problem Statement	104
8.5.2	Visualization	104
8.5.3	Synchronization Challenges	104
8.5.4	Semaphore Solution	105
8.5.5	Monitor Solution	105
8.5.6	Modern Implementation	105
8.5.7	Complexity Discussion	105
8.5.8	C Example	105
8.5.9	Interview Follow-Up	105
8.6	Sleeping Barber Problem	105
8.6.1	Problem Statement	105
8.6.2	Visualization	106
8.6.3	Synchronization Challenges	106
8.6.4	Semaphore Solution	106
8.6.5	Monitor Solution	106
8.6.6	Modern Implementation	106
8.6.7	Complexity Discussion	106
8.7	Cigarette Smokers Problem	107
8.7.1	Problem Statement	107
8.7.2	Visualization	107
8.7.3	Synchronization Challenges	107

8.7.4	Semaphore Solution	107
8.7.5	Monitor Solution	107
8.7.6	Modern Implementation	107
8.7.7	Complexity Discussion	107
8.8	River Crossing Problem	107
8.8.1	Problem Statement	107
8.8.2	Visualization	108
8.8.3	Synchronization Challenges	108
8.8.4	Semaphore Solution	108
8.8.5	Monitor Solution	108
8.8.6	Modern Implementation	108
8.8.7	Complexity Discussion	108
8.9	Barrier Synchronization Problem	108
8.9.1	Problem Statement	108
8.9.2	Visualization	109
8.9.3	Synchronization Challenges	109
8.9.4	Semaphore Solution	109
8.9.5	Monitor Solution	109
8.9.6	Modern Implementation	109
8.9.7	Complexity Discussion	109
8.9.8	Java Example	109
8.9.9	Python Example	110
8.10	Key Takeaways	110
IV	Deadlocks	111
9	Deadlocks	113
9.1	Chapter Overview	113
9.2	Learning Objectives	113
9.3	Introduction to Deadlocks	114
9.3.1	Motivation	114

9.3.2	Real-World Analogy	114
9.4	Resource Allocation Concepts	115
9.4.1	Resource	115
9.4.2	Resource Usage Cycle	115
9.4.3	Example	115
9.5	Reusable Resources	115
9.5.1	Definition	115
9.5.2	Characteristics	115
9.5.3	Examples	116
9.5.4	Illustration	116
9.6	Consumable Resources	116
9.6.1	Definition	116
9.6.2	Characteristics	116
9.6.3	Examples	116
9.6.4	Example	116
9.7	Deadlock Definition	117
9.7.1	Formal Definition	117
9.7.2	Example	117
9.7.3	Visualization	117
9.8	Coffman Conditions	117
9.8.1	Overview	117
9.8.2	Mutual Exclusion	118
9.8.3	Hold and Wait	118
9.8.4	No Preemption	119
9.8.5	Circular Wait	119
9.9	Resource Allocation Graphs	120
9.9.1	Definition	120
9.9.2	Components	120
9.9.3	Edges	120
9.9.4	Example	121

9.9.5	Important Observation	121
9.10	Wait-For Graphs	121
9.10.1	Definition	121
9.10.2	Construction	121
9.10.3	Example	122
9.10.4	Deadlock Rule	122
9.11	Deadlock Examples	122
9.11.1	Example 1: Printer and Scanner	122
9.11.2	Example 2: Two Mutex Locks	122
9.11.3	Example 3: Dining Philosophers	123
9.12	Deadlocks in Databases	123
9.12.1	Scenario	123
9.12.2	Example	123
9.12.3	Database Handling	123
9.12.4	Case Study	124
9.13	Deadlocks in Distributed Systems	124
9.13.1	Challenge	124
9.13.2	Examples	124
9.13.3	Problem	124
9.13.4	Case Study	124
9.14	Deadlocks in Modern Applications	125
9.14.1	Microservices	125
9.14.2	Cloud Systems	125
9.14.3	Multithreaded Applications	125
9.15	Deadlock Detection Concepts	125
9.15.1	Purpose	125
9.15.2	Basic Idea	125
9.15.3	For Single Instance Resources	126
9.15.4	For Multiple Instances	126
9.15.5	Detection Frequency	126

9.15.6	Deadlock Recovery	126
9.16	Interview Case Studies	126
9.16.1	Case Study 1	126
9.16.2	Case Study 2	126
9.16.3	Case Study 3	127
9.17	Key Takeaways	127
9.18	20 Interview Questions with Answers	127
9.19	20 MCQs with Answers	128
9.20	Revision Notes	129
9.21	Cheat Sheet	130
10	Deadlock Handling	131
10.1	Introduction	131
10.2	Deadlock Prevention	131
10.2.1	Necessary Conditions	131
10.2.2	Prevention Techniques	131
10.3	Resource Ordering	132
10.3.1	Advantages	132
10.4	Resource Preemption	132
10.4.1	Example	132
10.5	Deadlock Avoidance	133
10.6	Safe State	133
10.6.1	Safe Sequence	133
10.6.2	Visualization	133
10.7	Unsafe State	133
10.8	Banker's Algorithm	134
10.8.1	Data Structures	134
10.9	Numerical Example	134
10.9.1	Available	134
10.9.2	Allocation Matrix	134
10.9.3	Max Matrix	135

10.9.4 Need Matrix	135
10.10 Safety Algorithm	135
10.10.1 Algorithm Steps	135
10.10.2 Walkthrough	135
10.11 Resource Request Algorithm	137
10.11.1 Checks	137
10.11.2 Example	137
10.12 Deadlock Detection	137
10.12.1 Resource Allocation Graph	138
10.12.2 Detection Algorithm	138
10.13 Deadlock Recovery	138
10.14 Recovery by Process Termination	138
10.14.1 Option 1	138
10.14.2 Option 2	139
10.15 Recovery by Resource Preemption	139
10.15.1 Steps	139
10.15.2 Victim Selection Criteria	139
10.16 Flowchart for Deadlock Handling	139
10.17 Practical Deadlock Handling	140
10.18 Database Deadlock Handling	140
10.18.1 Solutions	140
10.18.2 Wait-Die	140
10.18.3 Wound-Wait	141
10.19 Distributed Deadlock Handling	141
10.19.1 Techniques	141
10.19.2 Probe-Based Detection	141
10.20 Interview Strategies	141
10.21 Common Pitfalls	141
10.22 Key Takeaways	142

V	Memory Management	143
11	Memory Management Fundamentals	145
11.1	Chapter Overview	145
11.2	Learning Objectives	145
11.3	Memory Management Overview	146
11.3.1	Definition	146
11.3.2	Responsibilities	146
11.3.3	Why It Matters	146
11.3.4	Real-World Analogy	146
11.4	Address Binding	147
11.4.1	Definition	147
11.4.2	Why Binding is Needed	147
11.4.3	Stages	147
11.5	Compile-Time Binding	147
11.5.1	Definition	147
11.5.2	Characteristics	147
11.5.3	Example	147
11.5.4	Advantages	148
11.5.5	Disadvantages	148
11.6	Load-Time Binding	148
11.6.1	Definition	148
11.6.2	Characteristics	148
11.6.3	Example	148
11.6.4	Advantages	149
11.6.5	Disadvantages	149
11.7	Execution-Time Binding	149
11.7.1	Definition	149
11.7.2	Characteristics	149
11.7.3	Advantages	149
11.7.4	Disadvantages	149

11.8 Logical Addresses	150
11.8.1 Definition	150
11.8.2 Characteristics	150
11.8.3 Example	150
11.9 Physical Addresses	150
11.9.1 Definition	150
11.9.2 Characteristics	150
11.9.3 Example	150
11.10MMU (Memory Management Unit)	151
11.10.1 Definition	151
11.10.2 Working	151
11.10.3 Example	151
11.10.4 Interview Note	152
11.11Dynamic Loading	152
11.11.1 Definition	152
11.11.2 Example	152
11.11.3 Advantages	152
11.11.4 Disadvantages	152
11.12Dynamic Linking	152
11.12.1 Definition	152
11.12.2 Working	152
11.12.3 Advantages	153
11.12.4 Disadvantages	153
11.13Shared Libraries	153
11.13.1 Definition	153
11.13.2 Linux Examples	153
11.13.3 Windows Examples	153
11.13.4 Advantages	153
11.14Static Linking vs Dynamic Linking	154
11.15Swapping	154

11.15.1 Definition	154
11.15.2 Purpose	154
11.15.3 Diagram	154
11.15.4 Advantages	154
11.15.5 Disadvantages	154
11.16 Memory Protection	155
11.16.1 Definition	155
11.16.2 Need	155
11.16.3 Protection Mechanisms	155
11.16.4 Example	155
11.17 Relocation	155
11.17.1 Definition	155
11.17.2 Why Needed	155
11.17.3 Interview Insight	156
11.18 Contiguous Memory Allocation	156
11.18.1 Definition	156
11.18.2 Example	156
11.18.3 Advantages	156
11.18.4 Disadvantages	156
11.19 Fragmentation	157
11.19.1 Definition	157
11.20 Internal Fragmentation	157
11.20.1 Definition	157
11.20.2 Example	157
11.20.3 Visualization	157
11.21 External Fragmentation	158
11.21.1 Definition	158
11.21.2 Example	158
11.21.3 Visualization	158
11.22 Memory Compaction	158

11.22.1 Definition	158
11.22.2 Before Compaction	158
11.22.3 After Compaction	159
11.22.4 Advantages	159
11.22.5 Disadvantages	159
11.23 Linux Examples	159
11.24 Windows Examples	159
11.25 Interview Notes	160
11.25.1 Golden Rule	160
11.26 Key Takeaways	160
11.27 20 Interview Questions with Answers	160
11.28 20 MCQs with Answers	161
11.29 Revision Notes	162
11.30 Cheat Sheet	163
12 Paging	165
12.1 Motivation for Paging	165
12.2 Paging Fundamentals	165
12.3 Pages and Frames	166
12.3.1 Pages	166
12.3.2 Frames	166
12.4 Logical Address Structure	167
12.5 Physical Address Structure	167
12.6 Address Translation	168
12.6.1 Example	168
12.7 Page Tables	168
12.7.1 Example	169
12.7.2 Purpose	169
12.8 Page Table Entries	169
12.9 Translation Lookaside Buffer (TLB)	169
12.9.1 Motivation	169

12.10 Effective Memory Access Time	170
12.10.1 Formula	170
12.10.2 Numerical Example	170
12.11 Hierarchical Paging	171
12.12 Multilevel Paging	171
12.13 Inverted Page Tables	172
12.13.1 Advantages	172
12.13.2 Disadvantages	172
12.14 Hashed Page Tables	172
12.15 Shared Pages	173
12.16 Huge Pages	173
12.16.1 Advantages	173
12.17 Paging in Linux	174
12.18 Paging in Windows	174
12.19 TLB Miss Handling	174
12.19.1 TLB Hit	174
12.19.2 TLB Miss	175
12.20 TLB Shootdown	175
12.21 Interview-Focused Explanations	175
12.21.1 Why Paging?	175
12.21.2 Why TLB?	175
12.21.3 Why Multilevel Paging?	176
12.21.4 Why Huge Pages?	176
12.21.5 Most Common Interview Formula	176
12.22 Key Takeaways	176
13 Segmentation	177
13.1 Introduction	177
13.2 Motivation for Segmentation	177
13.2.1 Problem with Linear Address Spaces	177
13.2.2 Logical Organization	178

13.3 Segmentation Fundamentals	178
13.3.1 Definition	178
13.3.2 Examples	178
13.3.3 Logical Address Format	178
13.4 User View vs Physical Memory View	179
13.4.1 User View	179
13.4.2 Physical Memory View	179
13.4.3 Illustration	179
13.5 Segment Structure	179
13.5.1 Example	180
13.6 Segment Tables	180
13.6.1 Definition	180
13.6.2 Fields	180
13.6.3 Example Segment Table	180
13.7 Address Translation in Segmentation	180
13.7.1 Translation Process	180
13.7.2 Formula	181
13.7.3 Numerical Example	181
13.8 Address Translation Diagram	181
13.9 Protection Mechanisms	181
13.9.1 Protection Bits	182
13.9.2 Example	182
13.10 Sharing Mechanisms	182
13.10.1 Example	182
13.10.2 Benefits	182
13.11 Segmentation Hardware	182
13.11.1 STBR	183
13.11.2 STLR	183
13.12 External Fragmentation	183
13.12.1 Problem	183

13.12.2 Example	183
13.12.3 Result	183
13.13 Compaction	183
13.13.1 Definition	183
13.13.2 Before	183
13.13.3 After	184
13.13.4 Drawback	184
13.14 Pure Segmentation	184
13.14.1 Definition	184
13.14.2 Characteristics	184
13.15 Segmentation with Paging	184
13.15.1 Motivation	184
13.15.2 Approach	184
13.15.3 Benefits	184
13.15.4 Architecture	185
13.16 Intel x86 Segmentation Overview	185
13.16.1 Historical Design	185
13.16.2 Segment Registers	185
13.16.3 Modern x86	185
13.17 Modern Operating Systems and Segmentation	185
13.17.1 Linux	185
13.17.2 Windows	186
13.17.3 Reason	186
13.18 Advantages of Segmentation	186
13.19 Disadvantages of Segmentation	186
13.20 Real-World Use Cases	187
13.20.1 Program Organization	187
13.20.2 Shared Libraries	187
13.20.3 Security	187
13.20.4 Legacy x86 Systems	187

13.21	Segmentation vs Paging	187
13.22	Key Takeaways	187
13.23	20 Interview Questions with Answers	188
13.24	20 MCQs with Answers	188
13.25	Revision Notes	190
13.26	Cheat Sheet	190
14	Virtual Memory	191
14.1	Introduction	191
14.2	Motivation for Virtual Memory	191
14.3	Virtual Address Space	192
14.3.1	Advantages	192
14.4	Demand Paging	193
14.5	Demand Segmentation	193
14.6	Lazy Loading	193
14.7	Page Faults	194
14.7.1	Causes	194
14.8	Page Fault Handling Procedure	194
14.8.1	Detailed Walkthrough	194
14.9	Valid and Invalid References	195
14.9.1	Valid Reference	195
14.9.2	Invalid Reference	195
14.10	Copy-on-Write (COW)	196
14.10.1	Motivation	196
14.10.2	Operation	196
14.11	Memory Mapped Files	196
14.11.1	Example	197
14.12	Shared Memory	197
14.13	Working Set Model	197
14.14	Thrashing	198
14.14.1	Solutions	198

14.15	Local Allocation Policies	198
14.16	Global Allocation Policies	199
14.17	Overcommitment	199
14.18	Swap Space	200
14.18.1	Purpose	200
14.18.2	Drawbacks	200
14.19	Virtual Memory in Linux	200
14.19.1	Linux Commands	200
14.20	Virtual Memory in Windows	201
14.20.1	Important Components	201
14.21	Case Study: Launching a Large Application	201
14.22	Case Study: fork() in Linux	201
14.23	Interview Focus: What Happens During a Page Fault?	202
14.24	Interview Focus: Why Virtual Memory Exists	202
14.25	Interview Focus: How Copy-on-Write Works	202
14.26	Key Takeaways	203
15	Page Replacement Algorithms	205
15.1	Introduction	205
15.2	Learning Objectives	205
15.3	Why Page Replacement is Needed	205
15.3.1	Example	206
15.4	Frame Allocation Basics	206
15.4.1	Frame	206
15.4.2	Page	206
15.4.3	Relationship	206
15.4.4	Allocation Strategies	206
15.5	Page Faults	207
15.5.1	Definition	207
15.5.2	Steps	207
15.5.3	Impact	207

15.6	Reference Strings	207
15.6.1	Definition	207
15.6.2	Example	207
15.7	Performance Considerations	207
15.7.1	Goal	208
15.8	FIFO (First-In First-Out)	208
15.8.1	Theory	208
15.8.2	Working Principle	208
15.8.3	Example	208
15.8.4	Advantages	208
15.8.5	Disadvantages	208
15.8.6	Complexity	209
15.9	Optimal Page Replacement	209
15.9.1	Theory	209
15.9.2	Example	209
15.9.3	Advantages	209
15.9.4	Disadvantages	209
15.9.5	Complexity	209
15.10	LRU (Least Recently Used)	210
15.10.1	Theory	210
15.10.2	Example	210
15.10.3	Advantages	210
15.10.4	Disadvantages	210
15.10.5	Complexity	210
15.11	Second Chance Algorithm	210
15.11.1	Theory	210
15.11.2	Working	210
15.11.3	Advantages	211
15.11.4	Disadvantages	211
15.11.5	Complexity	211

15.12	Clock Algorithm	211
15.12.1	Theory	211
15.12.2	Structure	211
15.12.3	Advantages	211
15.12.4	Disadvantages	211
15.13	Enhanced Clock Algorithm	212
15.13.1	Theory	212
15.13.2	Priority Order	212
15.13.3	Advantages	212
15.13.4	Disadvantages	212
15.14	LFU (Least Frequently Used)	213
15.14.1	Theory	213
15.14.2	Example	213
15.14.3	Advantages	213
15.14.4	Disadvantages	213
15.14.5	Complexity	213
15.15	MFU (Most Frequently Used)	213
15.15.1	Theory	213
15.15.2	Idea	213
15.15.3	Advantages	214
15.15.4	Disadvantages	214
15.15.5	Complexity	214
15.16	NRU (Not Recently Used)	214
15.16.1	Theory	214
15.16.2	Classes	214
15.16.3	Advantages	214
15.16.4	Disadvantages	214
15.17	Random Replacement	215
15.17.1	Theory	215
15.17.2	Advantages	215

15.17.3 Disadvantages	215
15.17.4 Complexity	215
15.18 Solved Numerical Example 1	215
15.19 Solved Numerical Example 2	216
15.20 Solved Numerical Example 3	216
15.21 Belady's Anomaly	216
15.21.1 Definition	216
15.21.2 Example	216
15.21.3 Observation	217
15.22 Stack Algorithms	217
15.22.1 Definition	217
15.22.2 Examples	217
15.22.3 Property	217
15.23 Working Set Based Replacement	217
15.23.1 Theory	217
15.23.2 Working Set	217
15.23.3 Advantages	218
15.23.4 Disadvantages	218
15.24 Page Buffering	218
15.24.1 Definition	218
15.24.2 Benefits	218
15.24.3 Working	218
15.25 Algorithm Comparison	218
15.26 Interview Shortcuts	219
15.26.1 Shortcut 1	219
15.26.2 Shortcut 2	219
15.26.3 Shortcut 3	219
15.26.4 Shortcut 4	219
15.26.5 Shortcut 5	219
15.27 Key Takeaways	219

15.2825 Interview Questions with Answers	219
15.2925 MCQs with Answers	220
15.30Revision Notes	221
15.31Cheat Sheet	222
16 Memory Allocation Strategies	223
16.1 Introduction	223
16.2 Memory Allocation Overview	223
16.3 Fixed Partition Allocation	223
16.3.1 Example	224
16.3.2 Advantages	224
16.3.3 Disadvantages	224
16.4 Dynamic Partition Allocation	225
16.4.1 Example	225
16.4.2 Advantages	225
16.4.3 Disadvantages	225
16.5 First Fit	226
16.5.1 Example	226
16.5.2 Advantages	226
16.5.3 Disadvantages	226
16.6 Best Fit	226
16.6.1 Example	227
16.6.2 Advantages	227
16.6.3 Disadvantages	227
16.7 Worst Fit	227
16.7.1 Example	227
16.7.2 Advantages	228
16.7.3 Disadvantages	228
16.8 Next Fit	228
16.8.1 Advantages	228
16.8.2 Disadvantages	228

16.9 Buddy System	229
16.9.1 Advantages	229
16.9.2 Disadvantages	230
16.10 Slab Allocation	230
16.10.1 Advantages	230
16.11 Kernel Memory Allocation	230
16.12 Heap Management	231
16.13 Free Lists	231
16.13.1 Example	231
16.13.2 Advantages	231
16.13.3 Disadvantages	231
16.14 Fragmentation Analysis	232
16.14.1 Internal Fragmentation	232
16.14.2 External Fragmentation	232
16.15 Memory Pools	232
16.16 Allocators in Modern Systems	233
16.17 Linux Memory Allocation	233
16.18 malloc Internals Overview	233
16.18.1 Common Data Structures	234
16.19 jemalloc and tcmalloc Overview	234
16.19.1 jemalloc	234
16.19.2 tcmalloc	234
16.20 Detailed Comparison of Allocation Methods	235
16.21 Performance Analysis	235
16.22 Interview Discussions	235
16.22.1 Why is Buddy Allocation Popular in Kernels?	235
16.22.2 Why is Slab Allocation Used?	236
16.22.3 Why Do Modern Servers Use jemalloc or tcmalloc?	236
16.23 Key Takeaways	236

VI	File Systems	237
17	File System Fundamentals	239
17.1	Introduction	239
17.2	Learning Objectives	239
17.3	What is a File System	239
17.3.1	Definition	239
17.3.2	Responsibilities	240
17.3.3	Examples	240
17.4	File Concept	240
17.4.1	Definition	240
17.4.2	Examples	240
17.4.3	Real-World Analogy	240
17.5	File Attributes	241
17.5.1	Common Attributes	241
17.5.2	Example	241
17.6	File Operations	241
17.6.1	Linux Example	242
17.6.2	Windows Example	242
17.7	Access Methods	242
17.8	Sequential Access	242
17.8.1	Definition	242
17.8.2	Example	243
17.8.3	Advantages	243
17.8.4	Disadvantages	243
17.9	Direct Access	243
17.9.1	Definition	243
17.9.2	Example	243
17.9.3	Advantages	243
17.9.4	Disadvantages	243
17.10	Indexed Access	244

17.10.1 Definition	244
17.10.2 Example	244
17.10.3 Advantages	244
17.10.4 Disadvantages	244
17.11 Directory Structures	244
17.12 Single-Level Directory	245
17.12.1 Definition	245
17.12.2 Advantages	245
17.12.3 Disadvantages	245
17.13 Two-Level Directory	245
17.13.1 Definition	245
17.13.2 Advantages	245
17.13.3 Disadvantages	245
17.14 Tree-Structured Directory	246
17.14.1 Definition	246
17.14.2 Advantages	246
17.14.3 Disadvantages	246
17.15 Acyclic Graph Directories	246
17.15.1 Definition	246
17.15.2 Advantages	246
17.15.3 Disadvantages	246
17.16 General Graph Directories	247
17.16.1 Definition	247
17.16.2 Advantages	247
17.16.3 Disadvantages	247
17.17 File Metadata	247
17.17.1 Definition	247
17.17.2 Examples	247
17.18 File Permissions	247
17.18.1 Linux Permissions	247

17.18.2 Meaning	248
17.18.3 Permission Groups	248
17.19 Links	248
17.20 Hard Links	248
17.20.1 Definition	248
17.20.2 Linux Example	248
17.20.3 Properties	248
17.20.4 Illustration	249
17.21 Symbolic Links	249
17.21.1 Definition	249
17.21.2 Linux Example	249
17.21.3 Properties	249
17.21.4 Illustration	249
17.22 Hard Link vs Soft Link	249
17.23 Linux Examples	250
17.24 Windows Examples	250
17.25 Directory Structure Comparison	251
17.26 Interview Focus	251
17.26.1 Frequently Asked Topics	251
17.26.2 Golden Rule	251
17.27 Key Takeaways	251
17.28 20 Interview Questions with Answers	252
17.29 20 MCQs with Answers	253
17.30 Revision Notes	253
17.31 Cheat Sheet	254
18 File System Implementation	255
18.1 Introduction	255
18.2 File System Architecture	255
18.3 On-Disk Structures	256
18.4 In-Memory Structures	256

18.5 Boot Block	256
18.6 Superblock	257
18.7 Inodes	257
18.8 File Control Blocks	258
18.9 Allocation Methods	258
18.10 Contiguous Allocation	258
18.11 Linked Allocation	259
18.12 Indexed Allocation	259
18.13 Free Space Management	260
18.14 Bitmaps	260
18.15 Free Lists	261
18.16 Journaling	261
18.17 Log Structured File Systems	262
18.18 ext4 Overview	262
18.19 NTFS Overview	263
18.20 File System Consistency	263
18.21 fsck Overview	263
18.22 Dedicated Interview Section: How Linux Stores Files Internally	264
18.23 Allocation Method Comparison	265
18.24 Key Takeaways	265
19 Disk Management and Disk Scheduling	267
19.1 Introduction	267
19.2 Learning Objectives	267
19.3 Magnetic Disks Overview	267
19.3.1 Definition	267
19.3.2 Components	268
19.3.3 Characteristics	268
19.4 SSD Overview	268
19.4.1 Definition	268
19.4.2 Characteristics	268

19.4.3 Advantages over HDDs	268
19.5 Disk Structure	268
19.6 Tracks	269
19.6.1 Definition	269
19.6.2 Characteristics	269
19.7 Sectors	269
19.7.1 Definition	269
19.7.2 Typical Sizes	269
19.8 Cylinders	270
19.8.1 Definition	270
19.9 Seek Time	270
19.9.1 Definition	270
19.9.2 Formula	270
19.9.3 Example	270
19.10 Rotational Latency	271
19.10.1 Definition	271
19.10.2 Average Rotational Latency	271
19.10.3 Example	271
19.11 Transfer Time	271
19.11.1 Definition	271
19.11.2 Factors	272
19.12 Disk Access Time	272
19.13 Disk Scheduling Concepts	272
19.13.1 Goals	272
19.13.2 Example Queue	272
19.14 FCFS (First Come First Serve)	272
19.14.1 Working	272
19.14.2 Sequence	273
19.14.3 Head Movement	273
19.14.4 Advantages	273

19.14.5 Disadvantages	273
19.15 SSTF (Shortest Seek Time First)	273
19.15.1 Working	273
19.15.2 Sequence	273
19.15.3 Head Movement	274
19.15.4 Advantages	274
19.15.5 Disadvantages	274
19.16 SCAN Algorithm	274
19.16.1 Working	274
19.16.2 Nickname	274
19.16.3 Example	274
19.16.4 Advantages	275
19.16.5 Disadvantages	275
19.17 CSCAN Algorithm	275
19.17.1 Working	275
19.17.2 Advantages	275
19.17.3 Disadvantages	276
19.18 LOOK Algorithm	276
19.18.1 Working	276
19.18.2 Advantages	276
19.18.3 Disadvantages	276
19.19 CLOOK Algorithm	276
19.19.1 Working	276
19.19.2 Advantages	276
19.19.3 Disadvantages	276
19.20 Scheduling Diagram	276
19.21 RAID Levels	277
19.21.1 Definition	277
19.22 RAID 0	277
19.22.1 Concept	277

19.22.2 Advantages	277
19.22.3 Disadvantages	277
19.23 RAID 1	277
19.23.1 Concept	277
19.23.2 Advantages	278
19.23.3 Disadvantages	278
19.24 RAID 5	278
19.24.1 Concept	278
19.24.2 Characteristics	278
19.24.3 Advantages	278
19.24.4 Disadvantages	278
19.25 RAID 6	278
19.25.1 Concept	278
19.25.2 Characteristics	278
19.25.3 Advantages	278
19.25.4 Disadvantages	279
19.26 RAID 10	279
19.26.1 Concept	279
19.26.2 Advantages	279
19.26.3 Disadvantages	279
19.27 RAID Comparison	279
19.28 SSD Internals	279
19.28.1 Organization	280
19.28.2 Characteristics	280
19.29 Wear Leveling	280
19.29.1 Problem	280
19.29.2 Solution	280
19.29.3 Goal	280
19.29.4 Types	280
19.30 TRIM	280

19.30.1 Definition	280
19.30.2 Benefits	280
19.30.3 Linux Example	281
19.31 Solved Numerical Example	281
19.32 Interview Tips	281
19.33 Key Takeaways	282
19.34 25 Interview Questions with Answers	282
19.35 25 MCQs with Answers	283
19.36 Revision Notes	284
19.37 Cheat Sheet	285
VII Input/Output Systems	287
20 Disk Management and Disk Scheduling	289
20.1 Introduction	289
20.2 Learning Objectives	289
20.3 Magnetic Disks Overview	289
20.3.1 Definition	289
20.3.2 Components	290
20.3.3 Characteristics	290
20.4 SSD Overview	290
20.4.1 Definition	290
20.4.2 Characteristics	290
20.4.3 Advantages over HDDs	290
20.5 Disk Structure	290
20.6 Tracks	291
20.6.1 Definition	291
20.6.2 Characteristics	291
20.7 Sectors	291
20.7.1 Definition	291
20.7.2 Typical Sizes	291

20.8 Cylinders	292
20.8.1 Definition	292
20.9 Seek Time	292
20.9.1 Definition	292
20.9.2 Formula	292
20.9.3 Example	292
20.10 Rotational Latency	293
20.10.1 Definition	293
20.10.2 Average Rotational Latency	293
20.10.3 Example	293
20.11 Transfer Time	293
20.11.1 Definition	293
20.11.2 Factors	294
20.12 Disk Access Time	294
20.13 Disk Scheduling Concepts	294
20.13.1 Goals	294
20.13.2 Example Queue	294
20.14 FCFS (First Come First Serve)	294
20.14.1 Working	294
20.14.2 Sequence	295
20.14.3 Head Movement	295
20.14.4 Advantages	295
20.14.5 Disadvantages	295
20.15 SSTF (Shortest Seek Time First)	295
20.15.1 Working	295
20.15.2 Sequence	295
20.15.3 Head Movement	296
20.15.4 Advantages	296
20.15.5 Disadvantages	296
20.16 SCAN Algorithm	296

20.16.1 Working	296
20.16.2 Nickname	296
20.16.3 Example	296
20.16.4 Advantages	297
20.16.5 Disadvantages	297
20.17CSCAN Algorithm	297
20.17.1 Working	297
20.17.2 Advantages	297
20.17.3 Disadvantages	298
20.18LOOK Algorithm	298
20.18.1 Working	298
20.18.2 Advantages	298
20.18.3 Disadvantages	298
20.19CLOOK Algorithm	298
20.19.1 Working	298
20.19.2 Advantages	298
20.19.3 Disadvantages	298
20.20Scheduling Diagram	298
20.21RAID Levels	299
20.21.1 Definition	299
20.22RAID 0	299
20.22.1 Concept	299
20.22.2 Advantages	299
20.22.3 Disadvantages	299
20.23RAID 1	299
20.23.1 Concept	299
20.23.2 Advantages	300
20.23.3 Disadvantages	300
20.24RAID 5	300
20.24.1 Concept	300

20.24.2 Characteristics	300
20.24.3 Advantages	300
20.24.4 Disadvantages	300
20.25 RAID 6	300
20.25.1 Concept	300
20.25.2 Characteristics	300
20.25.3 Advantages	300
20.25.4 Disadvantages	301
20.26 RAID 10	301
20.26.1 Concept	301
20.26.2 Advantages	301
20.26.3 Disadvantages	301
20.27 RAID Comparison	301
20.28 SSD Internals	301
20.28.1 Organization	302
20.28.2 Characteristics	302
20.29 Wear Leveling	302
20.29.1 Problem	302
20.29.2 Solution	302
20.29.3 Goal	302
20.29.4 Types	302
20.30 TRIM	302
20.30.1 Definition	302
20.30.2 Benefits	302
20.30.3 Linux Example	303
20.31 Solved Numerical Example	303
20.32 Interview Tips	303
20.33 Key Takeaways	304
20.34 25 Interview Questions with Answers	304
20.35 25 MCQs with Answers	305

20.36Revision Notes	306
20.37Cheat Sheet	307

VIII Modern Operating Systems 309

21 Linux Internals 311

21.1 Introduction	311
21.2 Linux Architecture Overview	311
21.3 User Space vs Kernel Space	312
21.3.1 User Space	312
21.3.2 Kernel Space	312
21.4 Linux Kernel Architecture	312
21.5 Monolithic Kernel Design	313
21.6 Kernel Modules	313
21.7 System Call Interface	314
21.8 Process Management in Linux	314
21.9 Scheduler Overview	315
21.10Completely Fair Scheduler (CFS)	315
21.11Linux Memory Management	316
21.12Virtual Memory in Linux	316
21.13Linux File Systems Overview	316
21.14ext4 Architecture	317
21.15proc File System	317
21.16sysfs	318
21.17Signals	318
21.18IPC Mechanisms	318
21.19Daemons	319
21.20Linux Boot Process	319
21.21init and systemd	319
21.22Kernel Panic	320
21.23Common Linux Commands for OS Interviews	320

21.24	Real-World Backend Engineering Examples	321
21.24.1	Web Server	321
21.24.2	Database Server	321
21.24.3	Container Platform	321
21.25	Dedicated Interview Section: What Happens When a Linux Process Starts? . . .	321
21.26	Dedicated Interview Section: What Happens When a Process Terminates? . . .	322
21.27	Dedicated Interview Section: How a System Call Reaches the Kernel?	322
21.28	Key Takeaways	322
22	Modern Concurrency	323
22.1	Introduction	323
22.2	Learning Objectives	323
22.3	Concurrency vs Parallelism	324
22.3.1	Concurrency	324
22.3.2	Parallelism	324
22.3.3	Example	324
22.3.4	Comparison	324
22.4	Multicore Architectures	325
22.4.1	Definition	325
22.4.2	Benefits	325
22.5	Hyperthreading	325
22.5.1	Definition	325
22.5.2	Goal	325
22.5.3	Example	325
22.5.4	Advantages	326
22.5.5	Limitations	326
22.6	Symmetric Multiprocessing (SMP)	326
22.6.1	Definition	326
22.6.2	Characteristics	326
22.7	NUMA Systems	326
22.7.1	Definition	326

22.7.2	Idea	326
22.7.3	Interview Insight	327
22.8	Thread Pools	327
22.8.1	Definition	327
22.8.2	Why Needed	327
22.8.3	Architecture	327
22.8.4	Java Example	327
22.9	Work Queues	327
22.9.1	Definition	327
22.9.2	Benefits	327
22.10	Fork-Join Model	328
22.10.1	Definition	328
22.10.2	Example	328
22.10.3	Java Example	328
22.11	Task Scheduling	328
22.12	Futures	329
22.12.1	Definition	329
22.12.2	Java Example	329
22.12.3	Benefits	329
22.13	Promises	329
22.13.1	Definition	329
22.13.2	Workflow	329
22.13.3	Use Cases	329
22.14	Async Programming	329
22.14.1	Definition	329
22.14.2	Benefits	330
22.14.3	Python Example	330
22.15	Event Loops	330
22.15.1	Definition	330
22.15.2	Architecture	330

22.15.3 Examples	330
22.16 Non-Blocking Programming	330
22.16.1 Definition	330
22.16.2 Examples	331
22.16.3 Advantages	331
22.17 Lock-Free Data Structures	331
22.17.1 Definition	331
22.17.2 Tools	331
22.17.3 Example	331
22.17.4 Advantages	331
22.17.5 Disadvantages	331
22.18 Wait-Free Algorithms	331
22.18.1 Definition	331
22.18.2 Properties	332
22.18.3 Challenge	332
22.19 Software Transactional Memory	332
22.19.1 Definition	332
22.19.2 Workflow	332
22.19.3 Advantages	332
22.20 Actor Model	332
22.20.1 Definition	332
22.20.2 Principles	332
22.20.3 Examples	333
22.21 Reactive Programming Concepts	333
22.21.1 Definition	333
22.21.2 Features	333
22.21.3 Examples	333
22.22 Modern Concurrency Challenges	333
22.23 Thread-per-Request vs Event-Driven Architectures	334
22.23.1 Thread-per-Request	334

22.23.2 Event-Driven Architecture	334
22.24 How High-Concurrency Servers Work	335
22.25 Java Examples	335
22.25.1 CompletableFuture	335
22.25.2 ExecutorService	335
22.26 Python Examples	336
22.26.1 Asyncio	336
22.27 C++ Examples	336
22.27.1 Future	336
22.27.2 Thread	336
22.28 Backend Service Examples	336
22.29 Performance Discussion	336
22.30 Modern Concurrency Interview Questions	337
22.31 Key Takeaways	337
22.32 25 Interview Questions with Answers	338
22.33 25 MCQs with Answers	339
22.34 Revision Notes	340
22.35 Cheat Sheet	341
23 Virtualization and Containers	343
23.1 Introduction	343
23.2 Motivation for Virtualization	343
23.3 Virtual Machines	344
23.4 Hypervisors	344
23.5 Type 1 Hypervisors	345
23.6 Type 2 Hypervisors	345
23.7 Full Virtualization	346
23.8 Para-Virtualization	346
23.9 Hardware-Assisted Virtualization	346
23.10 CPU Virtualization	347
23.11 Memory Virtualization	347

23.12Storage Virtualization	347
23.13Network Virtualization	348
23.14Containers	348
23.15Linux Namespaces	348
23.16cgroups	349
23.17Container Isolation	349
23.18Docker Architecture	349
23.19Images	350
23.20Containers	350
23.21Registries	351
23.22Kubernetes Overview	351
23.23Pods	351
23.24Container Scheduling	352
23.25Security Considerations	352
23.26Dedicated Interview Section: VM vs Container	353
23.27Dedicated Interview Section: Docker vs Virtual Machine	353
23.28Dedicated Interview Section: How Containers Achieve Isolation	353
23.29Key Takeaways	354
24 Operating System Security	355
24.1 Introduction	355
24.2 Learning Objectives	355
24.3 Security Fundamentals	356
24.3.1 Definition	356
24.3.2 CIA Triad	356
24.3.3 Example	356
24.4 Threat Model Basics	356
24.4.1 Definition	356
24.4.2 Typical Threats	357
24.4.3 Security Architecture	357
24.5 Authentication	357

24.5.1	Definition	357
24.5.2	Methods	357
24.5.3	Linux Example	358
24.5.4	Windows Example	358
24.6	Authorization	358
24.6.1	Definition	358
24.6.2	Examples	358
24.7	Authentication vs Authorization	358
24.8	Access Control Matrix	359
24.8.1	Definition	359
24.8.2	Challenge	359
24.9	Access Control Lists (ACLs)	359
24.9.1	Definition	359
24.9.2	Example	359
24.9.3	Advantages	359
24.9.4	Disadvantages	359
24.10	Capabilities	360
24.10.1	Definition	360
24.10.2	Example	360
24.10.3	Advantages	360
24.10.4	Disadvantages	360
24.11	ACLs vs Capabilities	360
24.12	Privilege Separation	360
24.12.1	Definition	360
24.12.2	Example	360
24.12.3	Benefits	361
24.13	Principle of Least Privilege	361
24.13.1	Definition	361
24.13.2	Example	361
24.13.3	Benefits	361

24.14	Secure Kernel Design	361
24.14.1	Goals	362
24.15	Memory Protection	362
24.15.1	Definition	362
24.15.2	Mechanisms	362
24.15.3	Benefits	362
24.16	ASLR	362
24.16.1	Definition	362
24.16.2	Goal	362
24.16.3	Protected Areas	362
24.16.4	Benefit	363
24.16.5	Linux Example	363
24.17	DEP / NX Bit	363
24.17.1	Definition	363
24.17.2	Purpose	363
24.17.3	Mitigates	363
24.18	Sandboxing	363
24.18.1	Definition	363
24.18.2	Examples	364
24.18.3	Architecture	364
24.19	Secure Boot	364
24.19.1	Definition	364
24.19.2	Goal	364
24.19.3	Boot Chain	364
24.19.4	Benefits	364
24.20	SELinux Overview	365
24.20.1	Definition	365
24.20.2	Security Model	365
24.20.3	Features	365
24.20.4	Example	365

24.21AppArmor Overview	365
24.21.1 Definition	365
24.21.2 Approach	365
24.21.3 Advantages	365
24.21.4 Common Usage	366
24.22Container Security	366
24.22.1 Challenges	366
24.22.2 Security Mechanisms	366
24.22.3 Container Isolation	366
24.23Common Vulnerabilities	366
24.23.1 Buffer Overflow	366
24.23.2 Privilege Escalation	367
24.23.3 Race Conditions	367
24.23.4 Code Injection	367
24.23.5 Directory Traversal	367
24.23.6 Use-After-Free	367
24.24Real-World Attack Scenarios	367
24.24.1 Scenario 1	367
24.24.2 Scenario 2	367
24.24.3 Scenario 3	367
24.24.4 Scenario 4	368
24.25How Modern OSes Isolate Applications	368
24.25.1 Isolation Layers	368
24.26Security Best Practices	368
24.27Key Takeaways	369
24.2820 Interview Questions with Answers	369
24.2920 MCQs with Answers	370
24.30Revision Notes	371
24.31Cheat Sheet	372

IX	Operating Systems for System Design	373
25	High-Concurrency Systems	375
25.1	Introduction	375
25.2	What High Concurrency Means	375
25.3	Concurrency Challenges	376
25.4	Throughput vs Latency	376
25.4.1	Throughput	376
25.4.2	Latency	376
25.5	Thread-per-Request Model	377
25.6	Event-Driven Architecture	377
25.7	Reactor Pattern	378
25.8	Proactor Pattern	378
25.9	Non-Blocking I/O	379
25.10	Asynchronous I/O	379
25.11	Thread Pools	379
25.12	Connection Pools	380
25.13	Load Balancing	380
25.14	Context Switching Costs	381
25.15	Kernel Bottlenecks	381
25.16	Resource Contention	381
25.17	Backpressure	382
25.18	Queue-Based Architectures	382
25.19	High-Concurrency Web Servers	383
25.19.1	Nginx-Style Architecture	383
25.20	High-Concurrency Databases	383
25.21	Modern Backend Architectures	384
25.22	Dedicated Interview Section: How a Web Server Handles Millions of Requests	385
25.23	Dedicated Interview Section: Why Thread-per-Request Does Not Scale	385
25.24	Dedicated Interview Section: Event-Driven Systems Explained	385
25.25	Key Takeaways	386

26 Memory and Performance Optimization	387
26.1 Introduction	387
26.2 Performance Fundamentals	387
26.2.1 Definition	387
26.2.2 Performance Equation	388
26.2.3 Key Observation	388
26.3 CPU Cache Hierarchy	388
26.3.1 Motivation	388
26.3.2 Hierarchy	388
26.3.3 Characteristics	389
26.4 Cache Locality	389
26.4.1 Definition	389
26.4.2 Importance	389
26.5 Spatial Locality	389
26.5.1 Definition	389
26.5.2 Example	389
26.5.3 Benefit	390
26.6 Temporal Locality	390
26.6.1 Definition	390
26.6.2 Example	390
26.6.3 Benefit	390
26.7 Cache Lines	390
26.7.1 Definition	390
26.7.2 Illustration	390
26.7.3 Interview Insight	391
26.8 False Sharing	391
26.8.1 Definition	391
26.8.2 Example	391
26.8.3 Effects	391
26.8.4 Solution	391

26.9 TLB Overview	391
26.9.1 Definition	391
26.9.2 Address Translation	391
26.9.3 Purpose	392
26.10 TLB Performance Impact	392
26.10.1 TLB Hit	392
26.10.2 TLB Miss	392
26.10.3 Impact	392
26.10.4 Interview Insight	392
26.11 Memory Access Costs	392
26.11.1 Observation	392
26.12 Branch Prediction	393
26.12.1 Definition	393
26.12.2 Example	393
26.12.3 Correct Prediction	393
26.12.4 Misprediction	393
26.12.5 Impact	393
26.13 Pipeline Effects	393
26.13.1 Pipeline	393
26.13.2 Benefits	393
26.13.3 Problems	393
26.13.4 Result	394
26.14 Memory Allocation Costs	394
26.14.1 Dynamic Allocation	394
26.14.2 Costs	394
26.14.3 Optimization	394
26.15 NUMA Effects	394
26.15.1 NUMA	394
26.15.2 Concept	394
26.15.3 Performance Implications	395

26.15.4 Optimization	395
26.16 Memory Leaks	395
26.16.1 Definition	395
26.16.2 Example	395
26.16.3 Effects	395
26.17 Fragmentation	395
26.17.1 Internal Fragmentation	395
26.17.2 External Fragmentation	395
26.17.3 Effects	396
26.18 Garbage Collection Overview	396
26.18.1 Definition	396
26.18.2 Languages	396
26.18.3 Advantages	396
26.18.4 Disadvantages	396
26.19 Performance Profiling	396
26.19.1 Definition	396
26.19.2 Popular Tools	396
26.19.3 Goal	397
26.20 Benchmarking Fundamentals	397
26.20.1 Definition	397
26.20.2 Rules	397
26.20.3 Example	397
26.21 Performance Case Study	397
26.21.1 Scenario	397
26.22 Optimization Trade-Offs	398
26.23 Why Cache Locality Matters	398
26.24 Common Performance Bottlenecks	399
26.25 How Memory Affects Scalability	399
26.26 Key Takeaways	399
26.27 20 Interview Questions with Answers	400

26.2820 MCQs with Answers	401
26.29Revision Notes	401
26.30Cheat Sheet	402
 X Interview Mastery	 403

List of Figures

1.1	Operating System as an Interface	4
1.2	Evolution of Operating Systems	6
1.3	Multiprogramming Concept	7
1.4	Major OS Components	10
1.5	Mode Transition	11
1.6	Boot Sequence	12
2.1	Basic Computer System Architecture	18
2.2	CPU Internal Organization	19
2.3	Stack Structure	21
2.4	Memory Hierarchy	21
2.5	Fetch Decode Execute Cycle	22
2.6	Interrupt Processing	23
2.7	DMA Data Transfer	24
2.8	Multicore Processor	26
2.9	NUMA Organization	26
2.10	Mode Switching	28
3.1	Typical Process Memory Layout	33
3.2	Process State Transition Diagram	35
3.3	Process Control Block	36
3.4	Context Switch	36
3.5	Process Hierarchy	42
4.1	Multiple Threads Inside a Process	47
4.2	Thread State Diagram	48
4.3	Many-to-One Model	49

4.4	One-to-One Model	50
4.5	Many-to-Many Model	50
4.6	Thread Pool Architecture	52
4.7	Parallel Execution on Multiple Cores	53
5.1	Basic CPU Scheduling	56
6.1	Concurrent Execution Timeline	75
6.2	Critical Section Structure	77
6.3	Peterson's Algorithm States	79
6.4	Atomic Execution	81
6.5	Spinlock Behavior	83
6.6	Shared Memory Visibility Challenges	84
7.1	Concurrent Execution Timeline	87
7.2	Critical Section Structure	89
7.3	Peterson's Algorithm States	91
7.4	Atomic Execution	93
7.5	Spinlock Behavior	95
7.6	Shared Memory Visibility Challenges	96
8.1	Producer Consumer Architecture	100
8.2	Bounded Buffer	102
8.3	Readers Writers Problem	103
8.4	Dining Philosophers	104
8.5	Sleeping Barber	106
8.6	River Crossing Problem	108
8.7	Barrier Synchronization	109
9.1	Four-Way Traffic Deadlock	114
9.2	Circular Wait Condition	120
9.3	Deadlock in Resource Allocation Graph	121
9.4	Wait-For Graph Cycle	122

10.1 Deadlock Cycle	131
10.2 Safe State Transition	133
10.3 Unsafe State	133
10.4 Cycle Indicates Deadlock	138
10.5 Deadlock Avoidance Decision Flow	139
11.1 Address Binding Process	147
11.2 MMU Address Translation	152
11.3 Swapping	154
11.4 Contiguous Allocation	156
12.1 Fragmentation in Contiguous Allocation	165
12.2 Pages and Frames	166
12.3 Physical Address Format	168
12.4 Address Translation Process	168
12.5 TLB-Based Translation	170
12.6 Hierarchical Paging	171
12.7 Hashed Page Table	173
12.8 Shared Pages	173
12.9 TLB Miss Handling	175
12.10 TLB Shutdown Mechanism	175
13.1 User View vs Physical Memory View	179
13.2 Segmentation Address Translation	181
13.3 Segmentation with Paging	185
14.1 Virtual Memory Abstraction	192
14.2 Typical Virtual Address Space	192
14.3 Demand Paging	193
14.4 Page Fault Generation	194
14.5 Page Fault Handling Sequence	195
14.6 Copy-on-Write Shared Page	196
14.7 Shared Memory	197

14.8 Thrashing Cycle	198
15.1 Clock Page Replacement	212
16.1 Memory Allocation Layers	223
16.2 Fixed Partition Allocation	224
16.3 Dynamic Partition Allocation	225
16.4 Next Fit Search	228
16.5 Buddy Allocation Splitting	229
16.6 Slab Allocator	230
16.7 Heap Layout	231
16.8 External Fragmentation	232
17.1 Sequential Access	243
17.2 Direct Access	243
17.3 Indexed Access	244
17.4 Single-Level Directory	245
17.5 Two-Level Directory	245
17.6 Tree-Structured Directory	246
17.7 Hard Links	249
17.8 Symbolic Link	249
18.1 Layered File System Architecture	255
18.2 Typical On-Disk Layout	256
18.3 Simplified Inode Structure	258
18.4 Contiguous Allocation	259
18.5 Linked Allocation	259
18.6 Indexed Allocation	260
18.7 Bitmap Free Space Tracking	261
18.8 Journaling Workflow	262
18.9 Linux File Storage Path	264
19.1 Disk Structure	269

19.2 Cylinder Across Platters	270
19.3 Disk Request Positions	276
19.4 RAID 0	277
19.5 RAID 1	277
20.1 Disk Structure	291
20.2 Cylinder Across Platters	292
20.3 Disk Request Positions	298
20.4 RAID 0	299
20.5 RAID 1	299
21.1 Linux Architecture	311
21.2 Linux Kernel Subsystems	313
21.3 System Call Flow	314
21.4 Simplified CFS Tree	315
21.5 Simplified ext4 Layout	317
21.6 Linux Boot Sequence	319
21.7 Process Startup	322
21.8 System Call Path	322
22.1 Multicore Processor	325
22.2 NUMA Architecture	326
22.3 Thread Pool	327
22.4 Event Loop	330
22.5 Actor Communication	333
23.1 Virtual Machine Architecture	344
23.2 Type 1 Hypervisor	345
23.3 Type 2 Hypervisor	345
23.4 Docker Architecture	350
23.5 Kubernetes Pod	352
23.6 Container Isolation Mechanisms	354

24.1 Security Layers	357
24.2 Application Sandbox	364
24.3 Container Isolation	366
24.4 Application Isolation	368
25.1 Thread-per-Request Model	377
25.2 Event-Driven Architecture	377
25.3 Reactor Pattern	378
25.4 Asynchronous I/O	379
25.5 Thread Pool	379
25.6 Connection Pool	380
25.7 Load Balancing	381
25.8 Queue-Based Architecture	383
25.9 Nginx Worker Architecture	383
25.10 Modern Backend Architecture	384
26.1 Memory Hierarchy	388
26.2 Cache Line	390
26.3 NUMA Architecture	395

List of Tables

1.1	Goals of an Operating System	5
1.2	Hard vs Soft RTOS	9
1.3	Kernel Types	10
1.4	User Mode vs Kernel Mode	11
3.1	Program vs Process	32
3.2	fork() Return Values	37
3.3	Linux Process Commands	42
5.1	Scheduling Algorithm Comparison	68
11.1	Static vs Dynamic Linking	154
13.1	Example Segments	178
13.2	Segment Structure Example	180
13.3	Example Segment Table	180
13.4	Segment Protection	182
13.5	Segmentation vs Paging	187
15.1	Page Replacement Comparison	218
17.1	Common File Systems	240
17.2	Permission Symbols	248
17.3	Hard Link vs Symbolic Link	249
17.4	Directory Structure Comparison	251
18.1	Typical Superblock Fields	257
18.2	Allocation Method Comparison	265
19.1	RAID Comparison	279

20.1 RAID Comparison	301
21.1 User Space vs Kernel Space	312
21.2 Common Linux Signals	318
21.3 Important Linux Commands	320
22.1 Concurrency vs Parallelism	324
22.2 Architecture Comparison	334
22.3 Real-World Concurrency Models	336
23.1 VM vs Container	353
23.2 Docker vs Virtual Machine	353
24.1 CIA Triad	356
24.2 Authentication vs Authorization	358
24.3 Access Control Matrix	359
24.4 ACL vs Capability	360
25.1 Throughput vs Latency	376
26.1 Memory Hierarchy Characteristics	389
26.2 Memory Access Costs	392
26.3 Optimization Trade-Offs	398

Part I

Fundamentals

Introduction to Operating Systems

“An operating system is the most important software layer between hardware and applications.”Operating Systems Principle

1.1 Chapter Overview

This chapter introduces the foundational concepts of Operating Systems (OS), one of the most important subjects in Software Development Engineer (SDE) interviews. Understanding operating systems helps engineers write efficient software, debug performance issues, reason about concurrency, and design scalable systems.

Major technology companies such as Amazon, Google, Microsoft, Meta, Uber, Netflix, Atlassian, Apple, and Bloomberg frequently ask operating system questions during interviews.

1.2 Learning Objectives

After completing this chapter, you should be able to:

- Explain what an Operating System is.
- Describe major goals of operating systems.
- Understand the evolution of operating systems.
- Compare batch, multiprogramming, and time-sharing systems.
- Understand distributed, real-time, and embedded operating systems.
- Explain kernel responsibilities.
- Differentiate user mode and kernel mode.
- Understand system calls.
- Explain the OS boot process.
- Discuss why operating systems matter for software engineers.

1.3 What is an Operating System?

1.3.1 Definition

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides services to applications.

It acts as an intermediary between:

- Users
- Applications
- Hardware

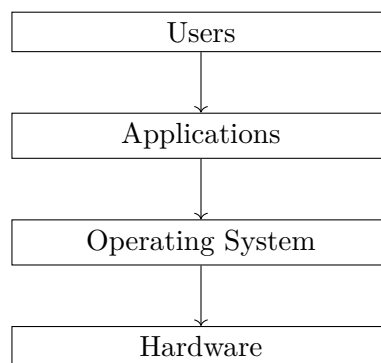


Figure 1.1: Operating System as an Interface

1.3.2 Simple Analogy

Consider a restaurant:

- Customers = Users
- Waiter = Operating System
- Kitchen = Hardware

Customers never directly communicate with the kitchen. The waiter coordinates requests and responses.

Similarly, applications do not directly access hardware. The operating system manages all interactions.

1.3.3 Examples of Operating Systems

- Linux
- Windows
- macOS

- Android
- iOS
- FreeBSD
- RTLinux
- VxWorks

1.4 Goals of an Operating System

The primary goals of an operating system are:

1.4.1 Convenience

Provide an easy environment for running applications.

1.4.2 Efficiency

Utilize hardware resources efficiently.

1.4.3 Resource Sharing

Allow multiple users and programs to share resources safely.

1.4.4 Reliability

Prevent crashes and ensure system stability.

1.4.5 Security

Protect users and resources from unauthorized access.

1.4.6 Scalability

Support growth in users, processes, and hardware.

Goal	Description
Convenience	Simplifies application development
Efficiency	Maximizes CPU and memory usage
Reliability	Ensures correct operation
Security	Protects data and resources
Scalability	Handles increasing workloads

Table 1.1: Goals of an Operating System

1.5 Evolution of Operating Systems

Operating systems evolved through several stages.

1. No Operating System
2. Batch Systems
3. Multiprogramming Systems
4. Time-Sharing Systems
5. Personal Computer Operating Systems
6. Distributed Systems
7. Mobile Operating Systems
8. Cloud and Container-Oriented Systems

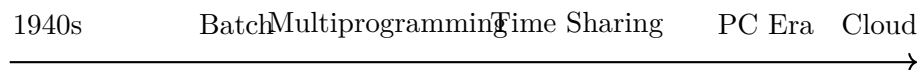


Figure 1.2: Evolution of Operating Systems

1.6 Batch Systems

1.6.1 Definition

Batch systems execute jobs in batches without user interaction.

1.6.2 Working

1. User submits jobs.
2. Jobs are grouped.
3. OS executes them sequentially.

1.6.3 Advantages

- Simple implementation
- Better CPU utilization than manual execution

1.6.4 Disadvantages

- No interaction
- Long waiting time

1.6.5 Example

Early IBM mainframe systems.

1.7 Multiprogramming Systems

1.7.1 Definition

Multiple programs reside in memory simultaneously.

When one process waits for I/O, another process uses the CPU.

1.7.2 Goal

Increase CPU utilization.

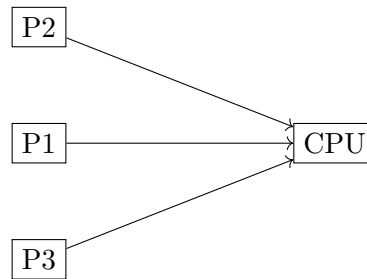


Figure 1.3: Multiprogramming Concept

1.7.3 Example

Early UNIX systems.

1.8 Time Sharing Systems

1.8.1 Definition

Time-sharing systems allow multiple users to interact with a computer simultaneously.

1.8.2 How It Works

CPU time is divided into small slices called **time quanta**.

1.8.3 Advantages

- Interactive computing

- Improved user experience
- Fair CPU allocation

1.8.4 Examples

- UNIX
- Linux
- Windows

1.9 Distributed Operating Systems

1.9.1 Definition

A distributed OS manages multiple computers and presents them as a single system.

1.9.2 Goals

- Resource sharing
- Fault tolerance
- Scalability

1.9.3 Examples

- Amoeba
- Plan 9
- Modern cloud infrastructures

1.10 Real-Time Operating Systems

1.10.1 Definition

A Real-Time Operating System (RTOS) guarantees response within strict timing constraints.

1.10.2 Types

1. Hard Real-Time
2. Soft Real-Time

Hard RTOS	Soft RTOS
Missing deadline unacceptable	Missing deadline tolerated
Aircraft systems	Video streaming

Table 1.2: Hard vs Soft RTOS

1.10.3 Examples

- VxWorks
- QNX
- RTLinux

1.11 Embedded Operating Systems

1.11.1 Definition

Embedded operating systems run on dedicated devices.

1.11.2 Characteristics

- Low memory footprint
- High reliability
- Energy efficient

1.11.3 Examples

- FreeRTOS
- Embedded Linux
- Zephyr

1.12 Components of an Operating System

Major OS components include:

- Process Management
- Memory Management
- File System
- I/O Management
- Security
- Networking

- Device Drivers

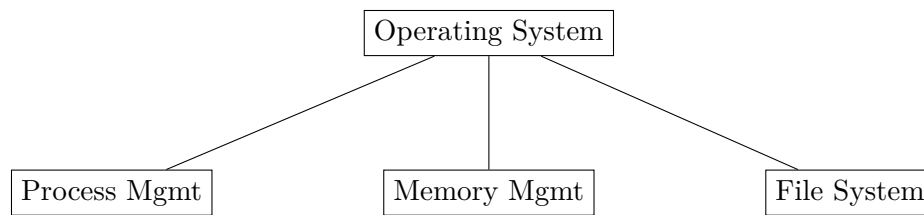


Figure 1.4: Major OS Components

1.13 Kernel Overview

1.13.1 Definition

The kernel is the core component of an operating system.

It manages:

- CPU
- Memory
- Devices
- Processes

1.13.2 Kernel Responsibilities

- Process scheduling
- Memory management
- Interrupt handling
- Device communication
- Security enforcement

1.13.3 Types of Kernels

Monolithic	Services inside kernel space
Microkernel	Minimal kernel functionality
Hybrid	Combination approach

Table 1.3: Kernel Types

1.13.4 Examples

Linux uses a monolithic kernel.

Windows uses a hybrid kernel.

1.14 User Mode vs Kernel Mode

Feature	User Mode	Kernel Mode
Privilege	Restricted	Full Access
Hardware Access	No	Yes
Crash Impact	Process Only	Entire System

Table 1.4: User Mode vs Kernel Mode

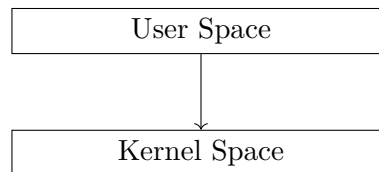


Figure 1.5: Mode Transition

1.15 System Calls

System calls provide an interface between applications and the operating system.

1.15.1 Examples

Linux:

```

read()
write()
fork()
exec()
open()
close()
  
```

Windows:

```

CreateProcess()
ReadFile()
WriteFile()
  
```

1.15.2 Flow

Application → System Call → Kernel

1.16 Boot Process Overview

1. Power On
2. BIOS/UEFI Initialization
3. Bootloader Execution
4. Kernel Loading
5. Hardware Initialization
6. System Services Start
7. User Login



Figure 1.6: Boot Sequence

1.17 Operating System Services

Operating systems provide:

- Program execution
- I/O operations
- File manipulation
- Communication
- Error detection
- Resource allocation
- Security

1.18 Why Operating Systems Matter for SDEs

1.18.1 Interview Perspective

Companies ask OS questions because engineers must understand:

- Concurrency
- Threading
- Memory usage
- Scheduling
- Performance bottlenecks
- Scalability

1.18.2 Real-World Examples

- High CPU usage debugging
- Memory leaks
- Deadlock investigation
- Container scheduling
- Distributed systems design

1.18.3 Interview Tip

Always connect operating system concepts with:

- Performance
- Scalability
- Reliability
- Security

1.19 Key Takeaways

- Operating systems manage hardware resources.
- Kernels are the core of operating systems.
- System calls enable user-kernel interaction.
- Multiprogramming improves CPU utilization.
- Time sharing enables interactivity.
- RTOS systems focus on deadlines.
- Distributed systems provide scalability.
- OS knowledge is critical for SDE interviews.

1.20 15 Interview Questions with Answers

1. What is an operating system?
Answer: System software managing hardware and providing services.
2. What are the main goals of an OS?
Answer: Convenience, efficiency, security, reliability.
3. What is a kernel?
Answer: Core component managing resources.
4. What is multiprogramming?
Answer: Multiple programs in memory simultaneously.

5. What is time sharing?
Answer: CPU sharing through time slices.
6. Difference between user and kernel mode?
Answer: Privilege level and hardware access.
7. What is a system call?
Answer: Interface to request kernel services.
8. What is a distributed OS?
Answer: OS managing multiple machines as one.
9. What is RTOS?
Answer: OS with deterministic response times.
10. What is a bootloader?
Answer: Program loading the kernel.
11. Why are interrupts important?
Answer: Enable event-driven processing.
12. Linux kernel type?
Answer: Monolithic.
13. Windows kernel type?
Answer: Hybrid.
14. Why does OS scheduling matter?
Answer: Efficient CPU allocation.
15. Why do SDE interviews ask OS questions?
Answer: To evaluate systems understanding.

1.21 15 MCQs with Answers

1. OS primarily acts as:
 - (a) Compiler
 - (b) Interface between hardware and applications
 - (c) Database
 - (d) BrowserAnswer: B
2. Kernel runs in: A) User mode B) Kernel mode Answer: B
3. Linux kernel is: A) Microkernel B) Hybrid C) Monolithic D) Exokernel Answer: C
4. Multiprogramming improves: Answer: CPU Utilization
5. RTOS stands for: Answer: Real-Time Operating System
6. System calls are used for: Answer: Accessing OS services
7. Time-sharing systems use: Answer: Time quanta
8. BIOS executes before: Answer: Kernel
9. Embedded OS targets: Answer: Dedicated devices

10. User mode has: Answer: Restricted privileges
11. Kernel mode has: Answer: Full privileges
12. Process management is handled by: Answer: OS
13. Example RTOS: Answer: VxWorks
14. Example desktop OS: Answer: Windows
15. Example distributed OS: Answer: Amoeba

1.22 Revision Notes

- OS = Resource Manager + Abstraction Layer.
- Kernel is the heart of the OS.
- User mode is restricted.
- Kernel mode has full access.
- System calls cross privilege boundaries.
- Multiprogramming increases CPU utilization.
- Time sharing improves responsiveness.
- RTOS guarantees timing constraints.
- Embedded OS focuses on resource efficiency.
- Distributed OS improves scalability.

1.23 Cheat Sheet

Operating System	Resource manager and abstraction layer
Kernel	Core OS component
System Call	Interface to kernel services
Multiprogramming	Multiple programs in memory
Time Sharing	CPU divided into quanta
RTOS	Deadline-oriented OS
Distributed OS	Multiple machines appear as one
User Mode	Restricted execution
Kernel Mode	Privileged execution
Boot Process	BIOS → Bootloader → Kernel

Computer System Architecture

2.1 Introduction

Computer System Architecture describes the organization and interaction of hardware components that execute programs and support operating systems. Understanding architecture is fundamental for operating systems because the OS directly manages processors, memory, devices, interrupts, and protection mechanisms.

From an interview perspective, computer architecture concepts frequently appear in discussions about processes, memory management, concurrency, virtualization, performance optimization, and system design.

2.2 Computer System Overview

A computer system consists of hardware and software working together to execute instructions and manage resources.

2.2.1 Major Components

- Central Processing Unit (CPU)
- Main Memory (RAM)
- Secondary Storage
- Input Devices
- Output Devices
- System Bus
- I/O Controllers
- Operating System

2.2.2 High-Level Architecture

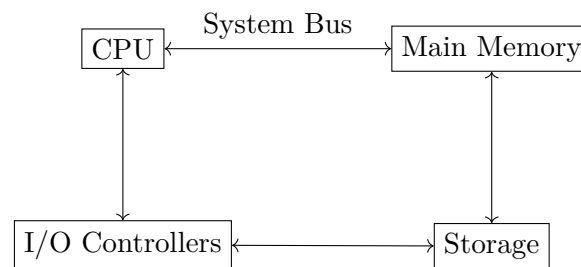


Figure 2.1: Basic Computer System Architecture

2.2.3 Von Neumann Architecture

Most modern systems follow the Von Neumann model.

Characteristics:

- Shared memory for instructions and data
- Sequential instruction execution
- Program stored in memory
- CPU fetches instructions from memory

2.3 CPU Architecture

The CPU executes instructions and coordinates system activities.

2.3.1 CPU Components

- Arithmetic Logic Unit (ALU)
- Control Unit
- Registers
- Cache
- Instruction Decoder
- Execution Units

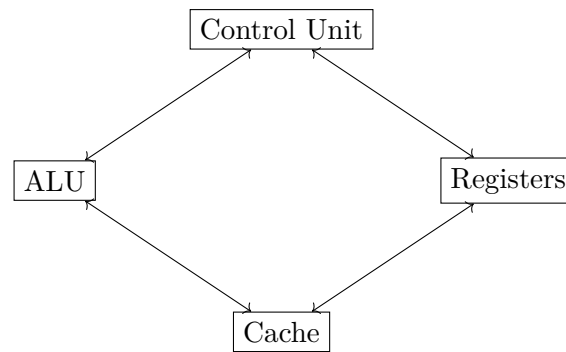


Figure 2.2: CPU Internal Organization

2.4 Registers

Registers are the fastest storage locations in a computer.

2.4.1 Characteristics

- Located inside CPU
- Very low access latency
- Hold frequently used data
- Limited quantity

2.4.2 Common Registers

Register	Purpose
General Purpose Registers	Store operands and intermediate results
Program Counter	Address of next instruction
Stack Pointer	Points to stack top
Instruction Register	Current instruction
Status Register	Condition flags
Base Register	Memory relocation
Index Register	Address calculation

2.5 Program Counter

Program Counter (PC) stores the address of the next instruction to execute.

2.5.1 Operation

1. Fetch instruction from PC address
2. Execute instruction
3. Increment PC
4. Update PC for branch instructions

2.5.2 Example

PC = 1000

Instruction at 1000 executes.

PC becomes:

1004

assuming instruction length is 4 bytes.

2.6 Stack Pointer

Stack Pointer (SP) points to the top of the stack.

2.6.1 Uses

- Function calls
- Local variables
- Return addresses
- Context switching
- Interrupt handling

2.6.2 Stack Layout

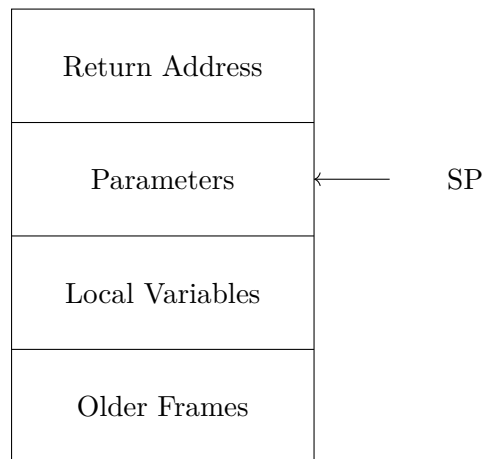


Figure 2.3: Stack Structure

2.7 Memory Hierarchy

Memory hierarchy balances cost, speed, and capacity.

2.7.1 Hierarchy Levels

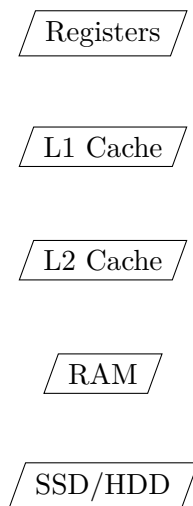


Figure 2.4: Memory Hierarchy

2.7.2 Hierarchy Comparison

Level	Speed	Cost	Capacity
Registers	Highest	Highest	Lowest
L1 Cache	Very High	High	Small

L2 Cache	High	Medium	Moderate
RAM	Moderate	Lower	Large
Disk	Slow	Lowest	Very Large

2.8 Instruction Cycle

Instruction cycle is the sequence of operations required to execute an instruction.

Stages:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

2.9 Fetch Decode Execute Cycle

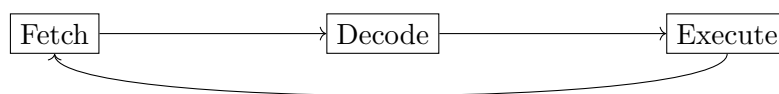


Figure 2.5: Fetch Decode Execute Cycle

2.9.1 Fetch

CPU retrieves instruction from memory.

2.9.2 Decode

Instruction decoder interprets operation.

2.9.3 Execute

ALU or execution units perform requested task.

2.10 Interrupts

Interrupts notify CPU about events requiring immediate attention.

2.10.1 Types

- Hardware Interrupts
- Software Interrupts
- Maskable Interrupts
- Non-Maskable Interrupts

2.10.2 Interrupt Handling Flow

1. Device generates interrupt
2. CPU saves context
3. Interrupt vector lookup
4. ISR executes
5. Context restored
6. Resume execution



Figure 2.6: Interrupt Processing

2.11 Trap Handling

A trap is a software-generated interrupt.

2.11.1 Examples

- System calls
- Breakpoints
- Debugging operations

2.11.2 Flow

1. User process issues trap
2. CPU switches to kernel mode
3. Kernel executes service
4. Return to user mode

2.12 Exceptions

Exceptions are unexpected conditions occurring during instruction execution.

2.12.1 Examples

- Divide by zero
- Invalid opcode
- Page fault
- Arithmetic overflow

2.12.2 Classification

Type	Description
Fault	Recoverable exception
Trap	Intentional exception
Abort	Severe unrecoverable error

2.13 DMA

Direct Memory Access allows devices to transfer data directly to memory without CPU involvement.

2.13.1 Advantages

- Reduces CPU overhead
- Improves throughput
- Enables parallelism

2.13.2 DMA Operation

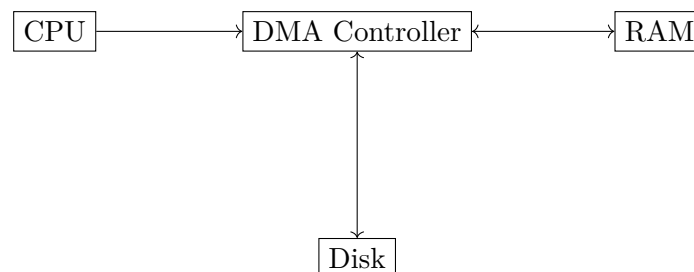


Figure 2.7: DMA Data Transfer

2.14 Cache Memory

Cache memory reduces average memory access time.

2.14.1 Why Cache Works

Based on locality principles:

- Temporal Locality
- Spatial Locality

2.14.2 Cache Levels

- L1 Cache
- L2 Cache
- L3 Cache

2.14.3 Cache Metrics

[Hit Ratio =====

Cache Hits $\frac{\text{Cache Hits}}{\text{Total Accesses}}$

2.14.4 Interview Insight

A cache hit is significantly faster than a cache miss because the CPU avoids accessing slower memory.

2.15 Multicore Systems

Modern CPUs contain multiple processing cores.

2.15.1 Benefits

- Parallel execution
- Better throughput
- Improved responsiveness
- Efficient multitasking

2.15.2 Architecture

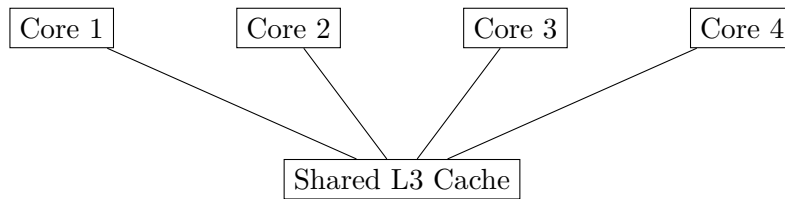


Figure 2.8: Multicore Processor

2.16 NUMA Architecture

NUMA stands for Non-Uniform Memory Access.

2.16.1 Concept

Memory access time depends on memory location relative to processor.

2.16.2 Architecture



Figure 2.9: NUMA Organization

2.16.3 Characteristics

- Local memory access is faster
- Remote memory access is slower
- Common in large servers

2.17 Hardware Protection Mechanisms

Protection mechanisms prevent unauthorized access.

2.17.1 Goals

- Isolation
- Security

- Reliability
- Resource control

2.17.2 Mechanisms

- Memory Protection
- Privilege Levels
- MMU
- Access Control Bits
- Segmentation
- Paging

2.18 User Mode and Privileged Instructions

Modern processors support multiple execution modes.

2.18.1 User Mode

- Restricted access
- Application execution
- Cannot execute privileged instructions

2.18.2 Kernel Mode

- Full hardware access
- Executes operating system code
- Handles interrupts and exceptions

2.18.3 Privileged Instructions

Examples:

- Modify page tables
- Disable interrupts
- Access device registers
- Change processor mode

2.18.4 Mode Transition

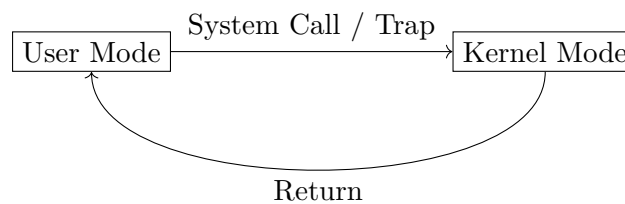


Figure 2.10: Mode Switching

2.19 Interview Explanations

2.19.1 Why Are Registers Faster Than RAM?

Registers reside inside the CPU and can be accessed without traversing memory buses.

2.19.2 Why Are Interrupts Important?

Without interrupts, CPUs would continuously poll devices, wasting processing cycles.

2.19.3 Why Is Cache Needed?

Processor speeds have increased much faster than memory speeds. Cache bridges this gap.

2.19.4 Difference Between Interrupt and Trap

Interrupts are usually external events while traps are software-generated.

2.19.5 Why Does NUMA Exist?

NUMA improves scalability in multiprocessor systems by distributing memory closer to processors.

2.20 Key Takeaways

Part II

Processes and Threads

Processes

3.1 Chapter

Overview

Processes are among the most frequently tested operating system topics in Software Development Engineer interviews. Companies such as Amazon, Google, Microsoft, Meta, Uber, Atlassian, Bloomberg, and Apple expect candidates to understand process creation, process management, context switching, scheduling, and Linux process internals.

This chapter builds a strong conceptual and interview-oriented understanding of processes.

3.2 Learning

Objectives

After completing this chapter, you should be able to:

- Define a process and explain its characteristics.
- Differentiate between a program and a process.
- Understand process memory layout.
- Explain process states and state transitions.
- Describe the Process Control Block (PCB).
- Understand context switching.
- Explain process creation and termination.
- Understand `fork()`, `exec()`, and `wait()`.
- Identify zombie and orphan processes.
- Understand daemon processes.
- Explain Linux process management.

3.3 Process

Concept

3.3.1 Definition

A **process** is a program in execution.

A process consists of:

- Program code
- Program counter
- CPU registers
- Stack
- Heap
- Open files
- Process state

3.3.2 Real-World

Analogy

Consider a recipe and a chef.

- Recipe = Program
- Chef executing recipe = Process

Multiple chefs can execute the same recipe simultaneously.

Similarly, multiple processes can execute the same program.

3.3.3 Examples

- Google Chrome
- VS Code
- MySQL Server
- Python Interpreter

Each running instance is a process.

3.4 Program

vs

Process

Feature	Program	Process
Definition	Passive entity	Active entity
Stored On	Disk	Memory
Execution	Not executing	Executing
State	Static	Dynamic
Resources	No resources allocated	Resources allocated
Example	chrome.exe	Running Chrome instance

Table 3.1: Program vs Process

3.4.1 Interview

Tip

A program becomes a process when loaded into memory and executed by the CPU.

3.5 Process

Memory

Layout

Every process has its own virtual address space.

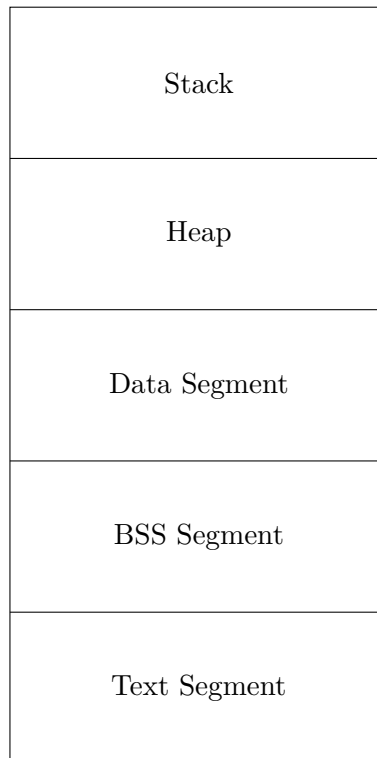


Figure 3.1: Typical Process Memory Layout

3.5.1 Text

Segment

Contains executable instructions.

3.5.2 Data

Segment

Stores initialized global and static variables.

3.5.3 BSS

Segment

Stores uninitialized global and static variables.

3.5.4 Heap

Used for dynamic memory allocation.

Examples:

`malloc()`

`calloc()`

`new`

3.5.5 Stack

Stores:

- Function calls
- Local variables
- Return addresses

3.5.6 Interview

Question

Stack grows in which direction?

Typically toward lower memory addresses.

Heap grows in which direction?

Typically toward higher memory addresses.

3.6 Process

States

3.6.1 Five-State

Model

1. New
2. Ready
3. Running
4. Waiting (Blocked)
5. Terminated

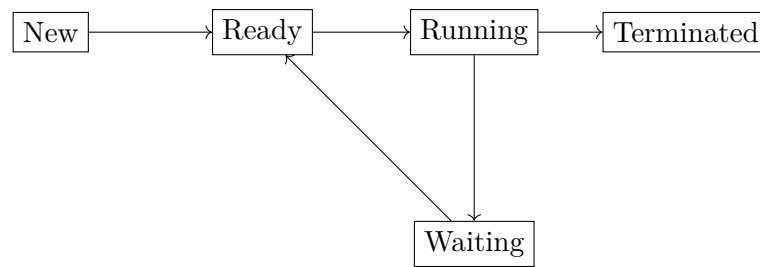


Figure 3.2: Process State Transition Diagram

3.6.2 Ready State

Waiting for CPU.

3.6.3 Running State

Currently executing.

3.6.4 Waiting State

Waiting for an event or I/O completion.

3.6.5 Terminated State

Execution completed.

3.7 Process Control Block (PCB)

3.7.1 Definition

PCB is a kernel data structure containing information about a process.

3.7.2 Contents of PCB

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Scheduling Information
- Memory Information
- Open Files

- Accounting Information

PID
State
Registers
Program Counter
Memory Info
Scheduling Info
Open Files
Accounting Info

Figure 3.3: Process Control Block

3.8 Context

Switching

3.8.1 Definition

Context switching is the process of saving the state of one process and loading the state of another.

3.8.2 Steps

1. Save current process context into PCB.
2. Select next process.
3. Load next process context.
4. Resume execution.



Figure 3.4: Context Switch

3.8.3 Interview

Insight

Context switching is pure overhead because no useful work is performed during the switch.

3.9 Process

Creation

Most operating systems allow processes to create child processes.

3.9.1 Linux

Model

Parent process creates child process using:

```
fork()
```

3.9.2 Windows

Model

Uses:

```
CreateProcess()
```

3.10 fork()

3.10.1 Definition

`fork()` creates a new process by duplicating the calling process.

3.10.2 Return

Values

Return Value	Meaning
0	Child Process
>0	Parent Process
-1	Error

Table 3.2: `fork()` Return Values

3.10.3 Example

```

1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main()
5  {
6      pid_t pid = fork();
7
8      if(pid == 0)
9          printf("Child Process\n");
10     else

```

```
11     printf("Parent Process\n");
12
13     return 0;
14 }
```

3.10.4 Interview

Insight

After fork():

- Parent and child have separate address spaces.
- Modern systems use Copy-On-Write (COW).

3.11 exec()

3.11.1 Definition

exec() replaces the current process image with a new program.

3.11.2 Common

Variants

- execl()
- execlp()
- execv()
- execvp()

3.11.3 Example

```
1  #include <unistd.h>
2
3  int main()
4  {
5      execl("/bin/ls", "ls", "-l", NULL);
6      return 0;
7  }
```

3.11.4 Important

Point

exec() does not create a new process.

It replaces the current process image.

3.12 wait()

3.12.1 Definition

wait() causes the parent process to wait for child termination.

3.12.2 Example

```
1 #include <sys/wait.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     if(fork()==0)
7         return 0;
8
9     wait(NULL);
10 }
```

3.12.3 Purpose

Prevents zombie processes.

3.13 Process

Scheduling

Overview

The scheduler selects a process from the ready queue.

Goals:

- High CPU utilization
- Fairness
- Low response time
- High throughput

Common scheduling algorithms:

- FCFS
- SJF
- Priority Scheduling
- Round Robin
- Multilevel Queue

3.14 Process

Termination

A process may terminate due to:

- Normal completion
- Error
- Fatal exception
- Parent termination
- Explicit kill signal

Linux commands:

```
kill PID  
kill -9 PID  
pkill process_name
```

3.15 Zombie

Processes

3.15.1 Definition

A zombie process is a terminated process whose PCB still exists.

3.15.2 Reason

Parent has not collected exit status.

3.15.3 Characteristics

- Process finished execution.
- Entry remains in process table.
- Consumes PID slot.

3.15.4 Fix

Parent must call:

```
wait()  
waitpid()
```


3.15.5 Linux**Example**

```
ps aux | grep Z
```

3.16 Orphan**Processes****3.16.1 Definition**

An orphan process is a child process whose parent terminates before it.

3.16.2 What**Happens?**

The orphan is adopted by:

```
init (PID 1)
```

or

```
systemd
```

3.16.3 Interview**Question****Zombie vs Orphan?**

Zombie: Parent alive, child terminated.

Orphan: Parent terminated, child alive.

3.17 Daemon**Processes****3.17.1 Definition**

A daemon is a background process providing system services.

3.17.2 Characteristics

- No terminal
- Long-running
- Starts during boot

Examples:

- sshd
- cron
- systemd
- httpd

Linux command:

```
ps -ef
```

3.18 Process

Hierarchies

Processes form a tree structure.

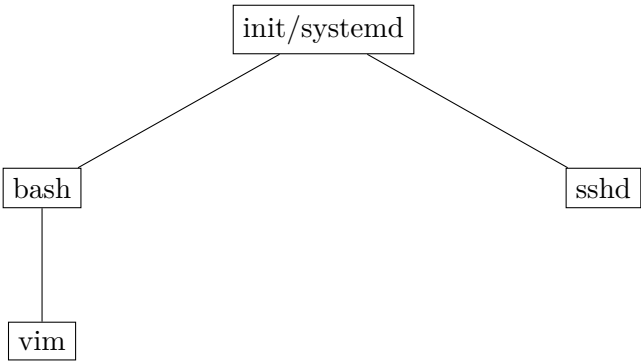


Figure 3.5: Process Hierarchy

3.19 Linux

Process

Management

3.19.1 Useful

Commands

Command	Purpose
ps	Display processes
top	Process monitoring
htop	Interactive monitoring
pstree	Process tree
kill	Terminate process
nice	Set priority
renice	Modify priority

Table 3.3: Linux Process Commands

3.19.2 Examples

View process tree:

`pstree`

Monitor processes:

`top`

Kill process:

`kill -9 1234`

3.20 Interview

Insights

Frequently asked topics:

- `fork()` vs `exec()`
- Zombie vs Orphan
- PCB contents
- Context switching overhead
- Copy-On-Write
- Process states
- Linux process commands
- Process hierarchy

Golden Rule:

If asked about process creation in Linux:

`fork()` → `exec()` → `wait()`

3.21 Key

Takeaways

- Process is a program in execution.
- PCB stores process information.
- Context switching saves and restores process state.
- `fork()` creates a child process.
- `exec()` replaces process image.
- `wait()` synchronizes parent and child.
- Zombies are terminated processes awaiting cleanup.
- Orphans are adopted by `init`/`systemd`.
- Daemons provide background services.
- Linux provides powerful process management tools.

3.22 20 Interview Questions with Answers

1. What is a process?
Answer: Program in execution.
2. Program vs Process?
Answer: Program is passive; process is active.
3. What is PCB?
Answer: Kernel data structure storing process information.
4. What is PID?
Answer: Process Identifier.
5. What is context switching?
Answer: Saving one process state and restoring another.
6. Why is context switching expensive?
Answer: It performs no useful work.
7. What does fork() do?
Answer: Creates a child process.
8. fork() return value in child?
Answer: 0.
9. fork() return value in parent?
Answer: Child PID.
10. What does exec() do?
Answer: Replaces process image.
11. Does exec() create a process?
Answer: No.
12. Purpose of wait()?
Answer: Waits for child termination.
13. What is a zombie process?
Answer: Terminated process awaiting cleanup.
14. What is an orphan process?
Answer: Child whose parent terminated.
15. Who adopts orphan processes?
Answer: init/systemd.
16. What is a daemon?
Answer: Background service process.
17. What is Copy-On-Write?
Answer: Memory copied only upon modification.
18. What is process scheduling?
Answer: Selecting next process for CPU.
19. What is a ready queue?
Answer: Queue of runnable processes.

20. What Linux command shows processes?

Answer: ps.

3.23 20 MCQs with Answers

1. Process is: A) Hardware B) Program in execution C) Compiler D) Kernel

Answer: B

2. PCB stands for: Answer: Process Control Block

3. fork() creates: Answer: Child Process

4. exec() creates a new process. Answer: False

5. wait() prevents: Answer: Zombie Processes

6. Process state waiting means: Answer: Waiting for event/I/O

7. Linux process ID is: Answer: PID

8. Stack stores: Answer: Function calls

9. Heap stores: Answer: Dynamic allocations

10. Zombie process is: Answer: Terminated but not reaped

11. Orphan process is: Answer: Parent terminated

12. Daemon process runs: Answer: Background

13. Scheduler selects from: Answer: Ready Queue

14. Context switch stores: Answer: CPU State

15. Linux tree command: Answer: pstree

16. kill -9 sends: Answer: SIGKILL

17. Child PID returned to: Answer: Parent

18. Program becomes process when: Answer: Executed

19. Copy-On-Write improves: Answer: Efficiency

20. Running process uses: Answer: CPU

3.24 Revision

Notes

- Process = Program + Execution State.
- PCB stores process metadata.
- fork() duplicates process.
- exec() replaces process image.
- wait() synchronizes parent and child.
- Context switching causes overhead.
- Zombie = dead child not reaped.
- Orphan = living child without parent.
- Daemon = background service.

- Linux process tree begins at init/systemd.

3.25 Cheat

Sheet

Process	Program in execution
PCB	Process metadata
fork()	Create child process
exec()	Replace process image
wait()	Wait for child
PID	Process identifier
Zombie	Dead child not cleaned
Orphan	Child without parent
Daemon	Background process
Ready State	Waiting for CPU
Running State	Using CPU
Waiting State	Waiting for event
Context Switch	Save + Restore CPU state
pstree	Show hierarchy
top	Monitor processes
kill -9	Force termination

Threads

4.1 Motivation for Threads

Threads enable concurrent execution within a process. They improve responsiveness, resource sharing, scalability, and CPU utilization.

Benefits:

- Improved responsiveness
- Better resource sharing
- Lower overhead than processes
- Efficient multicore utilization
- Increased throughput

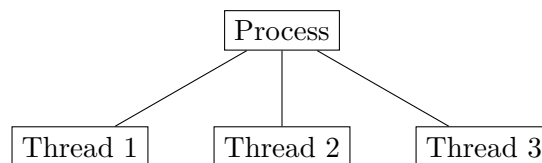


Figure 4.1: Multiple Threads Inside a Process

4.2 What is a Thread

A thread is the smallest unit of CPU execution.

Each thread contains:

- Program Counter
- Register Set
- Stack
- Thread State

Shared among threads:

- Code Segment

- Heap
- Open Files
- Global Variables

4.3 Process

vs

Thread

Feature	Process	Thread
Address Space	Separate	Shared
Creation Cost	High	Low
Context Switch	Expensive	Cheaper
Communication	IPC Needed	Shared Memory
Isolation	Strong	Weak
Failure Impact	Isolated	Can Affect Process

4.4 Thread

States

Typical thread states:

- New
- Ready
- Running
- Waiting
- Terminated

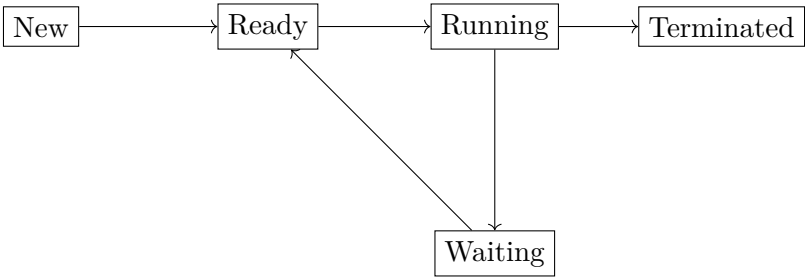


Figure 4.2: Thread State Diagram

4.5 User

Threads

User threads are managed entirely in user space.

Advantages:

- Fast creation

- Fast switching
- No kernel involvement

Disadvantages:

- Blocking system call blocks all threads
- Limited multicore support

4.6 Kernel

Threads

Kernel threads are managed by the operating system.

Advantages:

- True parallelism
- Better scheduling
- Blocking affects only one thread

Disadvantages:

- Higher overhead
- More expensive context switches

4.7 Multithreading

Models

Models define mapping between user threads and kernel threads.

4.8 Many-to-One

Multiple user threads mapped to a single kernel thread.

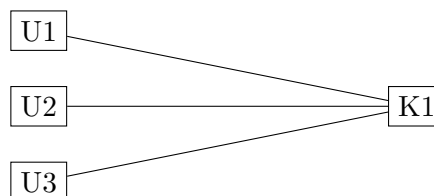


Figure 4.3: Many-to-One Model

Pros:

- Efficient

Cons:

- No parallel execution

4.9 One-to-One

Each user thread maps to one kernel thread.

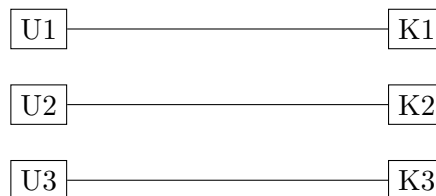


Figure 4.4: One-to-One Model

4.10 Many-to-Many

Many user threads mapped to many kernel threads.

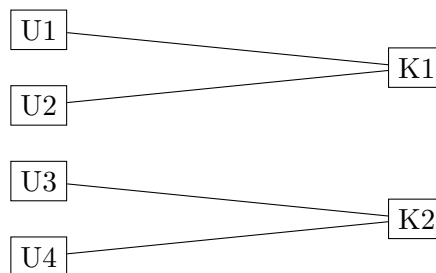


Figure 4.5: Many-to-Many Model

4.11 Thread

Libraries

Popular thread libraries:

- POSIX Threads (Pthreads)
- Java Threads
- Python threading module

4.12 POSIX

Threads

4.12.1 Pthreads

Example

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void* work(void* arg) {
5     printf("Hello Thread\n");
6     return NULL;
7 }
8
9 int main() {
10     pthread_t t;
11     pthread_create(&t, NULL, work, NULL);
12     pthread_join(t, NULL);
13     return 0;
14 }
```

4.13 Java

Threads

Java provides Thread class and Runnable interface.

4.13.1 Java

Example

```
1 class Worker extends Thread {
2     public void run() {
3         System.out.println("Running");
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         Worker t = new Worker();
10        t.start();
11    }
12 }
```

4.14 Python

Threads

Python provides the threading module.

4.14.1 Python

Example

```
1 import threading
2
```

```
3 def task():
4     print("Thread running")
5
6 t = threading.Thread(target=task)
7 t.start()
8 t.join()
```

4.15 Thread

Pools

Thread pools maintain a collection of reusable worker threads.

Advantages:

- Reduced creation overhead
- Better resource management
- Improved throughput
- Controlled concurrency

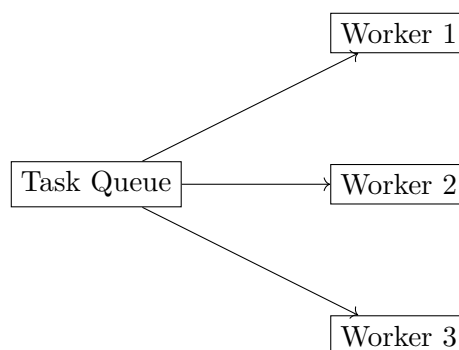


Figure 4.6: Thread Pool Architecture

4.16 Thread

Safety

Thread-safe code behaves correctly when accessed concurrently.

Techniques:

- Mutexes
- Semaphores
- Read-Write Locks
- Atomic Operations
- Immutability

4.17 Thread Local Storage

Thread Local Storage (TLS) provides thread-specific variables.

Characteristics:

- Private to each thread
- Prevents race conditions
- Useful for caches and contexts

4.18 Multicore Execution

Multiple threads can execute simultaneously on different CPU cores.



Figure 4.7: Parallel Execution on Multiple Cores

4.19 Interview Notes

- Threads share address space; processes do not.
- Context switching between threads is cheaper.
- Thread pools improve scalability.
- Kernel threads enable true parallelism.
- Thread safety is critical in concurrent systems.
- TLS provides per-thread data storage.

4.20 Key Takeaways

CPU Scheduling

5.1 Chapter

Overview

CPU Scheduling is one of the most important Operating Systems topics for Software Development Engineer interviews. It combines theoretical concepts with numerical problem-solving and is frequently asked by Amazon, Google, Microsoft, Meta, Uber, Atlassian, Apple, Bloomberg, and many other technology companies.

This chapter covers scheduling fundamentals, scheduling metrics, major scheduling algorithms, numerical techniques, interview tricks, and quick-solving strategies.

5.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand why CPU scheduling is necessary.
- Calculate scheduling performance metrics.
- Differentiate preemptive and non-preemptive scheduling.
- Solve scheduling numerals efficiently.
- Analyze FCFS, SJF, SRTF, Priority, Round Robin, MLQ, and MLFQ.
- Explain starvation and aging.
- Compare scheduling algorithms.
- Answer interview questions confidently.

5.3 CPU

Scheduling

Basics

5.3.1 Definition

CPU Scheduling is the process of selecting a process from the ready queue and allocating the CPU to it.

5.3.2 Why Scheduling is Needed

At any instant:

- Multiple processes may be ready.
- Only one process can use a CPU core.
- OS must decide who gets CPU access.

5.3.3 Scheduling Cycle

Ready Queue → Scheduler → CPU Execution

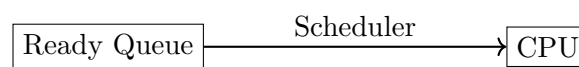


Figure 5.1: Basic CPU Scheduling

5.4 Scheduling Goals

A good scheduler should:

- Maximize CPU utilization.
- Maximize throughput.
- Minimize waiting time.
- Minimize turnaround time.
- Minimize response time.
- Ensure fairness.
- Avoid starvation.

5.5 Throughput

5.5.1 Definition

Number of processes completed per unit time.

$$Throughput = \frac{\text{Number of completed processes}}{\text{Time}}$$

5.5.2 Example

50 processes completed in 10 seconds.

$$Throughput = \frac{50}{10} = 5$$

processes/second.

5.6 Turnaround

Time

5.6.1 Definition

Total time spent by a process in the system.

$$TAT = Completion - Arrival$$

5.6.2 Example

$$Arrival = 2$$

$$Completion = 15$$

$$TAT = 15 - 2 = 13$$

5.7 Waiting

Time

5.7.1 Definition

Time spent waiting in the ready queue.

$$WT = TAT - Burst$$

5.7.2 Example

$$TAT = 20$$

$$Burst = 8$$

$$WT = 20 - 8 = 12$$

5.8 Response

Time

5.8.1 Definition

Time from arrival until first CPU allocation.

$$RT = First\ Start - Arrival$$

5.8.2 Importance

Critical for interactive systems.

5.9 CPU

Utilization

5.9.1 Definition

Percentage of time CPU remains busy.

$$CPU\ Utilization = \frac{CPU\ Busy\ Time}{Total\ Time} \times 100$$

5.9.2 Goal

Keep CPU utilization as high as possible.

5.10 Preemptive

Scheduling

5.10.1 Definition

OS can forcibly take CPU away from a running process.

5.10.2 Examples

- Round Robin
- SRTF
- Preemptive Priority

5.10.3 Advantages

- Better responsiveness
- Fair CPU allocation

5.10.4 Disadvantages

- More context switches
- Higher overhead

5.11 Non-Preemptive Scheduling

5.11.1 Definition

Once CPU is allocated, process keeps CPU until completion or blocking.

5.11.2 Examples

- FCFS
- SJF
- Non-preemptive Priority

5.11.3 Advantages

- Simpler
- Less overhead

5.11.4 Disadvantages

- Poor responsiveness
- Convoy effect

5.12 First Come First Serve (FCFS)

5.12.1 Definition

Processes are executed in order of arrival.

5.12.2 Working

Queue behaves like FIFO.

5.12.3 Advantages

- Simple
- Easy implementation

5.12.4 Disadvantages

- Convoy effect
- Poor average waiting time

5.12.5 Complexity

$$O(1)$$

5.12.6 Example

Process	Burst
P1	5
P2	3
P3	2

Gantt Chart:

0 P1 5 P2 8 P3 10

Waiting Times:

$$P1 = 0$$

$$P2 = 5$$

$$P3 = 8$$

Average WT:

$$\frac{0 + 5 + 8}{3} = 4.33$$

5.13 Shortest Job First (SJF)

5.13.1 Definition

Shortest burst process executes first.

5.13.2 Working

Select minimum burst time.

5.13.3 Advantages

- Minimum average waiting time

5.13.4 Disadvantages

- Burst prediction difficult
- Starvation possible

5.13.5 Complexity

$$O(n)$$

5.13.6 Example

Processes:

$$P1 = 6, P2 = 2, P3 = 8, P4 = 3$$

Order:

$$P2 \rightarrow P4 \rightarrow P1 \rightarrow P3$$

Gantt Chart:

0 2 5 11 19

Average WT:

$$\frac{0 + 2 + 5 + 11}{4} = 4.5$$

5.14 Shortest Remaining Time First (SRTF)

5.14.1 Definition

Preemptive version of SJF.

5.14.2 Working

Always execute process with smallest remaining burst.

5.14.3 Advantages

- Optimal average waiting time

5.14.4 Disadvantages

- High context switching
- Starvation possible

5.14.5 Complexity

$$O(n)$$

5.14.6 Example

Arrival:

$$P1 = (0, 8)$$

$$P2 = (1, 4)$$

At time 1:

P2 preempts P1.

Gantt:

P1 | P2 | P1

5.15 Priority

Scheduling

5.15.1 Definition

Highest priority process executes first.

5.15.2 Working

Priority determines scheduling order.

5.15.3 Advantages

- Important tasks run first

5.15.4 Disadvantages

- Starvation

5.15.5 Complexity

$$O(n)$$

5.15.6 Example

Process	Priority
P1	3
P2	1
P3	2

Order:

$$P2 \rightarrow P3 \rightarrow P1$$

5.16 Round Robin Scheduling

5.16.1 Definition

Processes receive CPU for a fixed time quantum.

5.16.2 Working

After quantum expires:

- Process preempted
- Moved to queue end

5.16.3 Advantages

- Fairness
- Good response time

5.16.4 Disadvantages

- High context switching
- Quantum selection critical

5.16.5 Complexity

$$O(1)$$

5.16.6 Example

$$\text{Quantum} = 2$$

$$P1 = 5$$

$$P2 = 4$$

Gantt:

P1 P2 P1 P2 P1

5.17 Multilevel Queue Scheduling

5.17.1 Definition

Ready queue divided into multiple fixed queues.

Example:

- System Processes
- Interactive Processes
- Batch Processes

5.17.2 Working

Each queue has its own scheduler.

5.17.3 Advantages

- Separation of workloads

5.17.4 Disadvantages

- Inflexible
- Starvation possible

5.17.5 Complexity

Depends on queue structure.

5.18 Multilevel Feedback Queue (MLFQ)

5.18.1 Definition

Processes can move between queues.

5.18.2 Working

- New jobs start at top queue.
- CPU-intensive jobs move downward.
- Interactive jobs remain near top.

5.18.3 Advantages

- Adaptive
- Excellent responsiveness

5.18.4 Disadvantages

- Complex implementation

5.18.5 Complexity

Implementation dependent.

5.18.6 Example

$Q1 : RR(4)$

$Q2 : RR(8)$

$Q3 : FCFS$

5.19 Dispatcher

5.19.1 Definition

Dispatcher transfers CPU control to selected process.

Responsibilities:

- Context switching
- User mode transition
- Jump to process location

5.19.2 Dispatch**Latency**

Time taken by dispatcher to stop one process and start another.

5.20 Context**Switch****Impact**

Context switching introduces overhead.

$$Useful\ Work = Total\ Time - Context\ Switch\ Time$$

Excessive context switching reduces:

- Throughput
- CPU efficiency

5.21 Starvation**5.21.1 Definition**

A process waits indefinitely because other processes continuously receive CPU.

5.21.2 Common**In**

- Priority Scheduling
- SJF
- SRTF

5.22 Aging**5.22.1 Definition**

Gradually increasing process priority over time.

5.22.2 Purpose

Prevents starvation.

5.22.3 Example

Priority increases every 5 seconds.

5.23 Comparative

Summary

Algorithm	Preemptive	Starvation	Response Time
FCFS	No	No	Poor
SJF	No	Yes	Good
SRTF	Yes	Yes	Excellent
Priority	Both	Yes	Good
RR	Yes	No	Excellent
MLQ	Usually	Possible	Good
MLFQ	Yes	Rare	Excellent

Table 5.1: Scheduling Algorithm Comparison

5.24 Common

Interview

Traps

1. Forgetting arrival times.
2. Confusing turnaround and waiting time.
3. Using wrong quantum in RR.
4. Ignoring preemption in SRTF.
5. Incorrect completion times.
6. Missing starvation discussion.
7. Not mentioning aging.
8. Forgetting context switch overhead.

5.25 How to Solve Scheduling Numericals Quickly

5.25.1 Step 1

Create process table.

5.25.2 Step 2

Draw Gantt chart.

5.25.3 Step 3

Find completion times.

5.25.4 Step**4**

Calculate:

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = FirstStart - AT$$

5.25.5 Step**5**

Compute averages.

5.25.6 Golden**Interview****Formula**

$$WT = TAT - BT$$

This formula solves most scheduling problems quickly.

5.26 Key**Takeaways**

- Scheduling selects the next CPU process.
- FCFS is simple but suffers convoy effect.
- SJF minimizes average waiting time.
- SRTF is preemptive SJF.
- Priority scheduling can starve low-priority jobs.
- Aging prevents starvation.
- Round Robin provides fairness.
- MLFQ is widely used in modern systems.
- Dispatcher performs context switches.
- Numerical problems require accurate Gantt charts.

5.27 25 Interview Questions with Answers

1. What is CPU scheduling?
Answer: Selecting a process for CPU execution.
2. What is throughput?
Answer: Completed processes per unit time.
3. Define turnaround time.
Answer: Completion time minus arrival time.
4. Define waiting time.
Answer: Time spent in ready queue.
5. Define response time.
Answer: First CPU allocation delay.
6. What is CPU utilization?
Answer: Percentage CPU remains busy.
7. FCFS stands for?
Answer: First Come First Serve.
8. Major issue in FCFS?
Answer: Convoy effect.
9. Which algorithm minimizes average waiting time?
Answer: SJF.
10. What is SRTF?
Answer: Preemptive SJF.
11. What causes starvation?
Answer: Continuous postponement.
12. How is starvation prevented?
Answer: Aging.
13. What is aging?
Answer: Gradual priority increase.
14. Round Robin is?
Answer: Preemptive.
15. What is time quantum?
Answer: Fixed CPU slice.
16. Very small quantum causes?
Answer: Excessive context switching.
17. Very large quantum becomes?
Answer: FCFS.
18. Priority scheduling problem?
Answer: Starvation.
19. What is dispatcher?
Answer: CPU transfer mechanism.

20. Dispatch latency?
Answer: Dispatcher overhead time.
21. Which algorithm is best for interactive systems?
Answer: Round Robin.
22. Which algorithm adapts dynamically?
Answer: MLFQ.
23. Which algorithm uses multiple queues?
Answer: MLQ.
24. What is context switching?
Answer: Process state change.
25. Why is scheduling important?
Answer: Efficient CPU utilization.

5.28 25 MCQs with Answers

1. FCFS is: A) Preemptive B) Non-preemptive Answer: B
2. SRTF is: Answer: Preemptive SJF
3. Turnaround Time = Answer: Completion - Arrival
4. Waiting Time = Answer: TAT - Burst
5. Response Time measures: Answer: First CPU allocation delay
6. RR uses: Answer: Time Quantum
7. Starvation occurs in: Answer: Priority Scheduling
8. Aging prevents: Answer: Starvation
9. Dispatcher performs: Answer: Context Switch
10. CPU utilization should be: Answer: High
11. Throughput should be: Answer: High
12. Waiting time should be: Answer: Low
13. Round Robin is: Answer: Preemptive
14. Convoy effect occurs in: Answer: FCFS
15. SJF gives: Answer: Minimum Average Waiting Time
16. Interactive systems prefer: Answer: RR
17. MLFQ stands for: Answer: Multilevel Feedback Queue
18. Dispatch latency is: Answer: Dispatcher Time
19. Priority scheduling can cause: Answer: Starvation
20. MLFQ allows: Answer: Queue Migration
21. Small quantum causes: Answer: High Overhead
22. Large quantum resembles: Answer: FCFS
23. Scheduler selects: Answer: Next Process

24. Throughput unit: Answer: Processes/Time
25. SRTF chooses: Answer: Shortest Remaining Burst

5.29 Revision

Notes

- CPU Scheduling selects next process.
- Throughput should increase.
- Waiting time should decrease.
- $TAT = CT - AT$.
- $WT = TAT - BT$.
- $RT = \text{First Start} - AT$.
- FCFS suffers convoy effect.
- SJF minimizes average waiting time.
- SRTF is preemptive SJF.
- RR uses time quantum.
- Aging prevents starvation.
- MLFQ is adaptive.

5.30 Cheat

Sheet

TAT	$CT - AT$
WT	$TAT - BT$
RT	$\text{First Start} - AT$
FCFS	FIFO Scheduling
SJF	Smallest Burst First
SRTF	Smallest Remaining Burst First
Priority	Highest Priority First
RR	Time Quantum Scheduling
MLQ	Fixed Multiple Queues
MLFQ	Dynamic Multiple Queues
Starvation	Indefinite Waiting
Aging	Priority Increase
Dispatcher	CPU Transfer Unit
Context Switch	Save + Restore State

Part III

Synchronization

The Critical Section Problem

6.1 Introduction to Concurrency

Concurrency refers to the execution of multiple tasks during overlapping time periods. Modern operating systems support concurrency through processes, threads, multicore processors, and distributed systems.

Concurrency improves:

- CPU utilization
- Responsiveness
- Throughput
- Scalability

However, concurrency introduces synchronization challenges when multiple execution units access shared resources.

6.1.1 Concurrent Execution

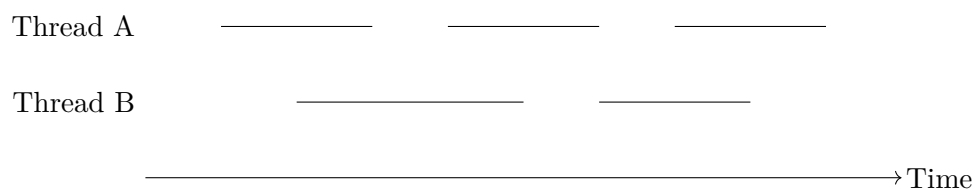


Figure 6.1: Concurrent Execution Timeline

6.2 Shared Resources

A shared resource is any resource accessible by multiple threads or processes.

Examples include:

- Shared variables
- Memory regions

- Files
- Databases
- Printers
- Network sockets

6.2.1 Shared

Counter

Example

```

1  int counter = 0;
2
3  Thread 1:
4  counter++;
5
6  Thread 2:
7  counter++;

```

At first glance the final value appears to be 2.

However concurrent execution can produce unexpected results.

6.3 Race

Conditions

A race condition occurs when the output depends on the timing or order of execution of concurrent operations.

6.3.1 Example

Assume:

counter = 5

Two threads execute:

```

1  counter++;

```

Internally:

1. Read counter
2. Increment value
3. Write back

6.3.2 Execution

Timeline

Step	Thread 1	Thread 2
1	Read 5	
2		Read 5
3	Compute 6	
4		Compute 6
5	Write 6	
6		Write 6

Final value:

6

Expected value:

7

This is a race condition.

6.4 Critical Sections

A critical section is a code segment that accesses shared resources.

Only one thread should execute a critical section at a time.

6.4.1 Structure



Figure 6.2: Critical Section Structure

6.5 The Critical Section Problem

The critical section problem asks:

How can multiple processes or threads safely share resources while preventing race conditions?

The solution must satisfy three fundamental requirements.

6.6 Requirements for a Correct Solution

6.6.1 Mutual Exclusion

At most one process can execute inside the critical section at any time.

Allowed:

$$P_1 \rightarrow CS$$

$$P_2 \rightarrow \textit{Waiting}$$

6.6.2 Progress

If no process is in the critical section and some wish to enter, selection should not be postponed indefinitely.

6.6.3 Bounded Waiting

A process should not wait forever after requesting access.

There must exist an upper bound on waiting time.

6.7 Software-Based Solutions

Before hardware synchronization primitives existed, software algorithms attempted to solve the critical section problem.

Examples:

- Peterson's Algorithm
- Bakery Algorithm

6.8 Peterson's Algorithm

Peterson's Algorithm solves the critical section problem for two processes.

6.8.1 Shared

Variables

```
1 bool flag[2];
2 int turn;
```

6.8.2 Process

i

```
1 flag[i] = true;
2 turn = j;
3
4 while(flag[j] && turn == j);
5
6 critical_section();
7
8 flag[i] = false;
```

6.8.3 Working

Principle

1. Process announces intent.
2. Gives turn to other process.
3. Waits if other process also wants entry.
4. Enters critical section safely.

6.8.4 State

Diagram

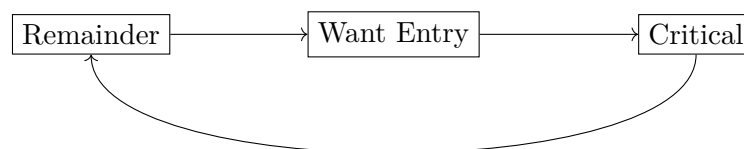


Figure 6.3: Peterson's Algorithm States

6.8.5 Interview

Note

Peterson's algorithm is important theoretically but rarely used in production systems because modern CPUs reorder memory operations.

6.9 Bakery

Algorithm

Proposed by Leslie Lamport.

Inspired by customers taking numbered tickets in a bakery.

6.9.1 Idea

1. Each process takes a number.
2. Smallest ticket enters first.
3. Ties broken using process ID.

6.9.2 Example

Process	Ticket
P1	5
P2	2
P3	7

Execution order:

$$P2 \rightarrow P1 \rightarrow P3$$

6.9.3 Advantages

- Supports multiple processes
- Guarantees fairness
- Satisfies bounded waiting

6.10 Hardware Support for Synchronization

Software-only solutions are difficult to implement correctly on modern hardware.

Processors provide atomic instructions.

Benefits:

- Faster synchronization
- Simpler implementation
- Better scalability

6.11 Atomic Operations

An atomic operation executes completely or not at all.

No interruption occurs during execution.

Examples:

- Test-and-Set
- Compare-and-Swap
- Exchange
- Atomic Increment

6.11.1 Atomicity

Illustration

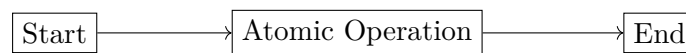


Figure 6.4: Atomic Execution

6.12 Test-and-Set

Instruction

Test-and-Set atomically reads and sets a memory location.

6.12.1 Pseudo

Code

```
1 bool TestAndSet(bool *lock)
2 {
3     bool old = *lock;
4     *lock = true;
5     return old;
6 }
```

6.12.2 Spinlock

Example

```
1 while(TestAndSet(&lock));
2
3 critical_section();
4
5 lock = false;
```

6.13 Compare-and-Swap

(CAS)

CAS compares memory content with an expected value.

If equal, replacement occurs atomically.

6.13.1 Pseudo**Code**

```
1 CAS(address, expected, newValue)
```

6.13.2 Example

```
1 int expected = 10;
2
3 CAS(&x, expected, 20);
```

If:

$x = 10$

then:

$x = 20$

Otherwise no change occurs.

6.14 Exchange**Instruction**

Exchange atomically swaps values.

6.14.1 Pseudo**Code**

```
1 Exchange(&a, &b);
```

6.14.2 Use**Cases**

- Locks
- Spinlocks
- Atomic ownership transfer

6.15 Spinlocks

A spinlock repeatedly checks a lock variable.

6.15.1 Operation

1. Attempt lock acquisition.
2. If unavailable, spin.
3. Acquire lock.
4. Execute critical section.
5. Release lock.

6.15.2 Visualization

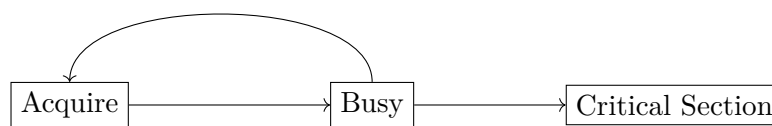


Figure 6.5: Spinlock Behavior

6.15.3 Advantages

- Low overhead
- Efficient for short waits

6.15.4 Disadvantages

- CPU wastage
- Poor for long waits

6.16 Modern CPU Challenges

Modern processors introduce complexities:

- Out-of-order execution
- Multiple cache levels
- Memory reordering
- Store buffers
- Multiple cores

Traditional algorithms may fail without memory barriers.

6.17 Memory Reordering Basics

Processors may reorder instructions to improve performance.

6.17.1 Example

Program order:

```
1 x = 1;
2 flag = true;
```

Observed order:

```
1 flag = true;
2 x = 1;
```

Another CPU may observe the reordered version.

6.17.2 Memory Illustration



Figure 6.6: Shared Memory Visibility Challenges

6.18 Real-World Race Condition Examples

6.18.1 Bank Account Update

```
1 balance = balance - 100;
```

Simultaneous withdrawals may corrupt the balance.

6.18.2 Inventory Management

Two orders simultaneously decrement stock.

Result:

Negative inventory.

6.18.3 Ticket Booking

Two users purchase the last seat concurrently.

Result:

Overbooking.

6.18.4 Web

Counter

```
1 visitors++;
```

Concurrent updates lose increments.

6.19 Interview

Explanations

6.19.1 Why is the Critical Section Problem Important?

Almost every concurrent application accesses shared resources.

Without synchronization:

- Data corruption occurs.
- Results become nondeterministic.
- Bugs become difficult to reproduce.

6.19.2 Why Isn't Peterson's Algorithm Used in Practice?

Modern hardware reorders memory operations.

The algorithm assumes strict memory ordering.

6.19.3 Why Is CAS Popular?

CAS is lock-free and supported by modern CPUs.

Many concurrent data structures use CAS internally.

6.19.4 Spinlock vs Mutex

Spinlocks waste CPU cycles while waiting.

Mutexes put threads to sleep.

Spinlocks are preferred only for short critical sections.

6.20 Common

Mistakes

- Assuming increment operations are atomic.

- Forgetting memory visibility issues.
- Holding locks too long.
- Using spinlocks for long waits.
- Ignoring fairness requirements.
- Accessing shared data without synchronization.
- Assuming multicore execution is deterministic.

6.21 Key

Takeaways

The Critical Section Problem

7.1 Introduction to Concurrency

Concurrency refers to the execution of multiple tasks during overlapping time periods. Modern operating systems support concurrency through processes, threads, multicore processors, and distributed systems.

Concurrency improves:

- CPU utilization
- Responsiveness
- Throughput
- Scalability

However, concurrency introduces synchronization challenges when multiple execution units access shared resources.

7.1.1 Concurrent Execution

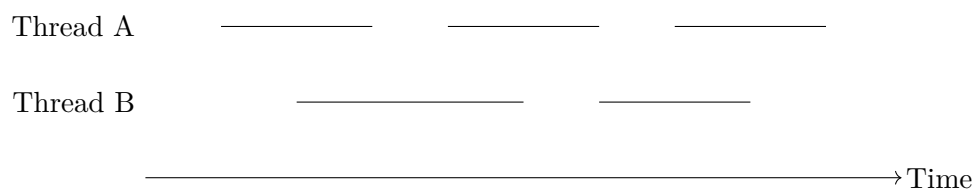


Figure 7.1: Concurrent Execution Timeline

7.2 Shared Resources

A shared resource is any resource accessible by multiple threads or processes.

Examples include:

- Shared variables
- Memory regions

- Files
- Databases
- Printers
- Network sockets

7.2.1 Shared

Counter

Example

```

1  int counter = 0;
2
3  Thread 1:
4  counter++;
5
6  Thread 2:
7  counter++;

```

At first glance the final value appears to be 2.

However concurrent execution can produce unexpected results.

7.3 Race

Conditions

A race condition occurs when the output depends on the timing or order of execution of concurrent operations.

7.3.1 Example

Assume:

counter = 5

Two threads execute:

```

1  counter++;

```

Internally:

1. Read counter
2. Increment value
3. Write back

7.3.2 Execution

Timeline

Step	Thread 1	Thread 2
1	Read 5	
2		Read 5
3	Compute 6	
4		Compute 6
5	Write 6	
6		Write 6

Final value:

6

Expected value:

7

This is a race condition.

7.4 Critical Sections

A critical section is a code segment that accesses shared resources.

Only one thread should execute a critical section at a time.

7.4.1 Structure



Figure 7.2: Critical Section Structure

7.5 The Critical Section Problem

The critical section problem asks:

How can multiple processes or threads safely share resources while preventing race conditions?

The solution must satisfy three fundamental requirements.

7.6 Requirements for a Correct Solution

7.6.1 Mutual Exclusion

At most one process can execute inside the critical section at any time.

Allowed:

$$P_1 \rightarrow CS$$

$$P_2 \rightarrow \textit{Waiting}$$

7.6.2 Progress

If no process is in the critical section and some wish to enter, selection should not be postponed indefinitely.

7.6.3 Bounded Waiting

A process should not wait forever after requesting access.

There must exist an upper bound on waiting time.

7.7 Software-Based Solutions

Before hardware synchronization primitives existed, software algorithms attempted to solve the critical section problem.

Examples:

- Peterson's Algorithm
- Bakery Algorithm

7.8 Peterson's Algorithm

Peterson's Algorithm solves the critical section problem for two processes.

7.8.1 Shared

Variables

```
1 bool flag[2];
2 int turn;
```

7.8.2 Process

i

```
1 flag[i] = true;
2 turn = j;
3
4 while(flag[j] && turn == j);
5
6 critical_section();
7
8 flag[i] = false;
```

7.8.3 Working

Principle

1. Process announces intent.
2. Gives turn to other process.
3. Waits if other process also wants entry.
4. Enters critical section safely.

7.8.4 State

Diagram

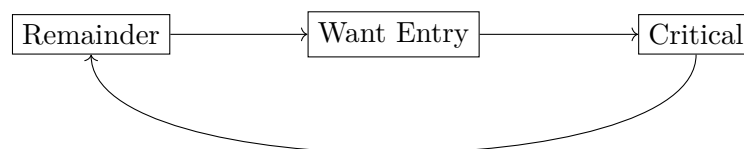


Figure 7.3: Peterson's Algorithm States

7.8.5 Interview

Note

Peterson's algorithm is important theoretically but rarely used in production systems because modern CPUs reorder memory operations.

7.9 Bakery

Algorithm

Proposed by Leslie Lamport.

Inspired by customers taking numbered tickets in a bakery.

7.9.1 Idea

1. Each process takes a number.
2. Smallest ticket enters first.
3. Ties broken using process ID.

7.9.2 Example

Process	Ticket
P1	5
P2	2
P3	7

Execution order:

$$P2 \rightarrow P1 \rightarrow P3$$

7.9.3 Advantages

- Supports multiple processes
- Guarantees fairness
- Satisfies bounded waiting

7.10 Hardware Support for Synchronization

Software-only solutions are difficult to implement correctly on modern hardware.

Processors provide atomic instructions.

Benefits:

- Faster synchronization
- Simpler implementation
- Better scalability

7.11 Atomic Operations

An atomic operation executes completely or not at all.

No interruption occurs during execution.

Examples:

- Test-and-Set
- Compare-and-Swap
- Exchange
- Atomic Increment

7.11.1 Atomicity

Illustration

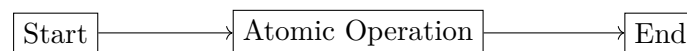


Figure 7.4: Atomic Execution

7.12 Test-and-Set

Instruction

Test-and-Set atomically reads and sets a memory location.

7.12.1 Pseudo

Code

```
1 bool TestAndSet(bool *lock)
2 {
3     bool old = *lock;
4     *lock = true;
5     return old;
6 }
```

7.12.2 Spinlock

Example

```
1 while(TestAndSet(&lock));
2
3 critical_section();
4
5 lock = false;
```

7.13 Compare-and-Swap

(CAS)

CAS compares memory content with an expected value.

If equal, replacement occurs atomically.

7.13.1 Pseudo**Code**

```
1 CAS(address, expected, newValue)
```

7.13.2 Example

```
1 int expected = 10;
2
3 CAS(&x, expected, 20);
```

If:

$x = 10$

then:

$x = 20$

Otherwise no change occurs.

7.14 Exchange**Instruction**

Exchange atomically swaps values.

7.14.1 Pseudo**Code**

```
1 Exchange(&a, &b);
```

7.14.2 Use**Cases**

- Locks
- Spinlocks
- Atomic ownership transfer

7.15 Spinlocks

A spinlock repeatedly checks a lock variable.

7.15.1 Operation

1. Attempt lock acquisition.
2. If unavailable, spin.
3. Acquire lock.
4. Execute critical section.
5. Release lock.

7.15.2 Visualization

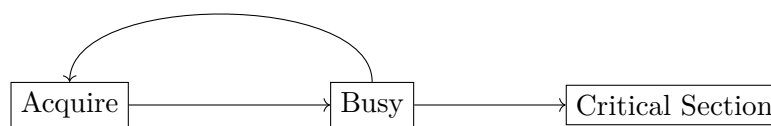


Figure 7.5: Spinlock Behavior

7.15.3 Advantages

- Low overhead
- Efficient for short waits

7.15.4 Disadvantages

- CPU wastage
- Poor for long waits

7.16 Modern CPU Challenges

Modern processors introduce complexities:

- Out-of-order execution
- Multiple cache levels
- Memory reordering
- Store buffers
- Multiple cores

Traditional algorithms may fail without memory barriers.

7.17 Memory Reordering Basics

Processors may reorder instructions to improve performance.

7.17.1 Example

Program order:

```
1 x = 1;
2 flag = true;
```

Observed order:

```
1 flag = true;
2 x = 1;
```

Another CPU may observe the reordered version.

7.17.2 Memory Illustration



Figure 7.6: Shared Memory Visibility Challenges

7.18 Real-World Race Condition Examples

7.18.1 Bank Account Update

```
1 balance = balance - 100;
```

Simultaneous withdrawals may corrupt the balance.

7.18.2 Inventory Management

Two orders simultaneously decrement stock.

Result:

Negative inventory.

7.18.3 Ticket Booking

Two users purchase the last seat concurrently.

Result:

Overbooking.

7.18.4 Web

Counter

```
1 visitors++;
```

Concurrent updates lose increments.

7.19 Interview

Explanations

7.19.1 Why is the Critical Section Problem Important?

Almost every concurrent application accesses shared resources.

Without synchronization:

- Data corruption occurs.
- Results become nondeterministic.
- Bugs become difficult to reproduce.

7.19.2 Why Isn't Peterson's Algorithm Used in Practice?

Modern hardware reorders memory operations.

The algorithm assumes strict memory ordering.

7.19.3 Why Is CAS Popular?

CAS is lock-free and supported by modern CPUs.

Many concurrent data structures use CAS internally.

7.19.4 Spinlock vs Mutex

Spinlocks waste CPU cycles while waiting.

Mutexes put threads to sleep.

Spinlocks are preferred only for short critical sections.

7.20 Common

Mistakes

- Assuming increment operations are atomic.

- Forgetting memory visibility issues.
- Holding locks too long.
- Using spinlocks for long waits.
- Ignoring fairness requirements.
- Accessing shared data without synchronization.
- Assuming multicore execution is deterministic.

7.21 Key

Takeaways

Classical Synchronization Problems

8.1 Introduction

Classical synchronization problems are fundamental operating systems problems that demonstrate coordination, mutual exclusion, deadlock avoidance, starvation prevention, and resource sharing among concurrent processes and threads.

These problems frequently appear in Software Development Engineer (SDE) interviews because they test understanding of:

- Concurrency
- Semaphores
- Monitors
- Mutexes
- Condition Variables
- Deadlocks
- Thread Coordination

8.2 Producer Consumer Problem

8.2.1 Problem Statement

A producer generates data and places it into a shared buffer. A consumer removes data from the buffer.

Requirements:

- Producer must not insert into a full buffer.
- Consumer must not remove from an empty buffer.
- Shared buffer access must be synchronized.

8.2.2 Visualization



Figure 8.1: Producer Consumer Architecture

8.2.3 Synchronization

Challenges

- Race conditions
- Buffer overflow
- Buffer underflow
- Mutual exclusion

8.2.4 Semaphore

Solution

Semaphores:

- `mutex = 1`
- `empty = N`
- `full = 0`

```
1 wait(empty);  
2 wait(mutex);  
3  
4 insert_item();  
5  
6 signal(mutex);  
7 signal(full);
```

8.2.5 Monitor

Solution

```
1 monitor Buffer {  
2   condition notFull;  
3   condition notEmpty;  
4 }
```

8.2.6 Modern

Implementation

- Java `BlockingQueue`
- Go Channels
- Python `Queue`
- Message Brokers

8.2.7 Complexity

Discussion

[Time = $O(1)$]

per insert/remove operation.

8.2.8 Execution

Example

Producer inserts:

[A,B,C]

Consumer removes:

[$A \rightarrow B \rightarrow C$]

8.2.9 C

Example

```
1 sem_wait(&empty);
2 sem_wait(&mutex);
3
4 buffer[in++] = item;
5
6 sem_post(&mutex);
7 sem_post(&full);
```

8.2.10 Java

Example

```
1 BlockingQueue<Integer> q =
2 new ArrayBlockingQueue<>(10);
```

8.2.11 Python

Example

```
1 from queue import Queue
2 q = Queue(10)
```

8.2.12 Interview

Follow-Up

How would you support multiple producers and consumers?

8.3 Bounded Buffer Problem

8.3.1 Problem Statement

Producer Consumer with finite buffer size.

8.3.2 Visualization

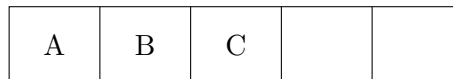


Figure 8.2: Bounded Buffer

8.3.3 Synchronization Challenges

- Finite capacity
- Concurrent access
- Blocking conditions

8.3.4 Semaphore Solution

Identical to Producer Consumer using:

- mutex
- full
- empty

8.3.5 Monitor Solution

Use condition variables:

- notFull
- notEmpty

8.3.6 Modern Implementation

Thread-safe queues and ring buffers.

8.3.7 Complexity Discussion

[$O(1)$]

for enqueue and dequeue.

8.4 Readers

Writers

Problem

8.4.1 Problem

Statement

Multiple readers can read simultaneously.

Writers require exclusive access.

8.4.2 Visualization

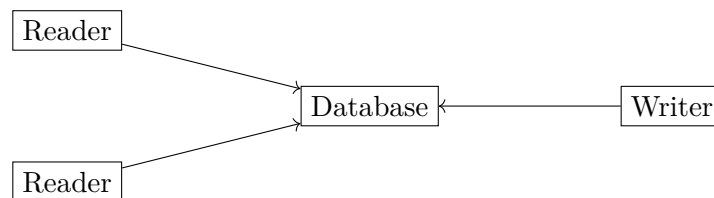


Figure 8.3: Readers Writers Problem

8.4.3 Synchronization

Challenges

- Reader starvation
- Writer starvation
- Fairness

8.4.4 Semaphore

Solution

```

1 wait(mutex);
2 readCount++;
3 if(readCount == 1)
4 wait(rw_mutex);
5 signal(mutex);
  
```

8.4.5 Monitor

Solution

Use:

- canRead
- canWrite

condition variables.

8.4.6 Modern**Implementation**

- ReadWriteLock (Java)
- `sharedmutex(C++)`

8.4.7 Complexity**Discussion**

[O(1)]

lock acquisition.

8.4.8 Interview**Follow-Up**

How would you eliminate writer starvation?

8.5 Dining**Philosophers****Problem****8.5.1 Problem****Statement**

Five philosophers share five chopsticks.

Each philosopher needs two chopsticks to eat.

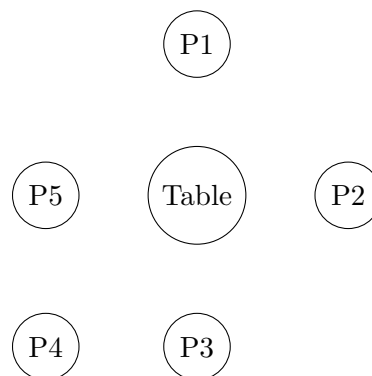
8.5.2 Visualization

Figure 8.4: Dining Philosophers

8.5.3 Synchronization**Challenges**

- Deadlock
- Starvation
- Resource contention

8.5.4 Semaphore**Solution**

Limit philosophers:
 $[N-1]$
 to eat simultaneously.

8.5.5 Monitor**Solution**

Monitor tracks fork states.

8.5.6 Modern**Implementation**

Lock ordering strategy.

8.5.7 Complexity**Discussion**

$[O(1)]$
 resource acquisition.

8.5.8 C**Example**

```

1 wait(leftFork);
2 wait(rightFork);
3 eat();
4 signal(leftFork);
5 signal(rightFork);

```

8.5.9 Interview**Follow-Up**

How do you prevent deadlock?

8.6 Sleeping**Barber****Problem****8.6.1 Problem****Statement**

A barber sleeps when no customers exist.

Customers either:

- Get haircut

- Wait
- Leave

8.6.2 Visualization



Figure 8.5: Sleeping Barber

8.6.3 Synchronization

Challenges

- Limited waiting chairs
- Barber sleep/wakeup
- Customer coordination

8.6.4 Semaphore

Solution

Use:

- customers
- barber
- mutex

8.6.5 Monitor

Solution

Condition variables manage waiting room.

8.6.6 Modern

Implementation

Task queues and worker pools.

8.6.7 Complexity

Discussion

[$O(1)$]

per customer.

8.7 Cigarette Smokers Problem

8.7.1 Problem Statement

Three smokers each possess one ingredient.

Agent supplies two ingredients.

Only the smoker with the missing ingredient can smoke.

8.7.2 Visualization

Tobacco + Paper $\rightarrow$ Smoker with Matches

8.7.3 Synchronization Challenges

- Resource matching
- Wakeup coordination

8.7.4 Semaphore Solution

Separate semaphores for each smoker.

8.7.5 Monitor Solution

Condition variables signal correct smoker.

8.7.6 Modern Implementation

Event-driven signaling.

8.7.7 Complexity Discussion

[$O(1)$]

selection.

8.8 River Crossing Problem

8.8.1 Problem Statement

Hackers and serfs must cross a river.

Boat capacity:

[4]

Unsafe combinations are disallowed.

8.8.2 Visualization



Figure 8.6: River Crossing Problem

8.8.3 Synchronization

Challenges

- Group formation
- Safety constraints

8.8.4 Semaphore

Solution

Counters plus semaphores coordinate boarding.

8.8.5 Monitor

Solution

Monitor validates safe combinations.

8.8.6 Modern

Implementation

Barrier plus locks.

8.8.7 Complexity

Discussion

[O(1)]

coordination decisions.

8.9 Barrier

Synchronization

Problem

8.9.1 Problem

Statement

All threads must reach a barrier before any proceed.

8.9.2 Visualization

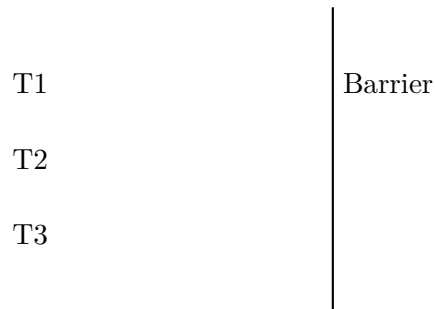


Figure 8.7: Barrier Synchronization

8.9.3 Synchronization

Challenges

- Waiting for all threads
- Reusable barriers

8.9.4 Semaphore

Solution

Use:

- count
- mutex
- barrier semaphore

8.9.5 Monitor

Solution

Condition variable waits until count reaches threshold.

8.9.6 Modern

Implementation

- `pthread_barrierCyclicBarrier`
- `CountDownLatch`

8.9.7 Complexity

Discussion

[$O(1)$]
per arrival.

8.9.8 Java

Example

```
1 CyclicBarrier barrier =  
2 new CyclicBarrier(4);
```

8.9.9 Python

Example

```
1 from threading import Barrier  
2 b = Barrier(4)
```

8.10 Key

Takeaways

Part IV

Deadlocks

Deadlocks

9.1 Chapter

Overview

Deadlocks are among the most important synchronization-related topics in Operating Systems and are frequently asked in Software Development Engineer interviews. Understanding deadlocks requires knowledge of processes, resources, synchronization, and resource allocation policies.

This chapter covers deadlock fundamentals, resource allocation models, Coffman conditions, resource allocation graphs, wait-for graphs, database deadlocks, distributed deadlocks, and deadlock detection concepts.

9.2 Learning

Objectives

After completing this chapter, you should be able to:

- Define a deadlock.
- Understand resource allocation concepts.
- Distinguish reusable and consumable resources.
- Explain all Coffman conditions.
- Analyze Resource Allocation Graphs (RAGs).
- Analyze Wait-For Graphs (WFGs).
- Identify deadlocks in practical systems.
- Understand database deadlocks.
- Understand distributed system deadlocks.
- Explain deadlock detection concepts.
- Solve interview questions involving deadlocks.

9.3 Introduction to Deadlocks

9.3.1 Motivation

Modern systems execute multiple processes and threads concurrently.

These processes compete for resources such as:

- CPU
- Memory
- Files
- Printers
- Database Locks
- Network Connections

Improper resource allocation may lead to a situation where no process can proceed.

This situation is called a **Deadlock**.

9.3.2 Real-World

Analogy

Imagine four cars arriving simultaneously at a four-way intersection.

Each car blocks the path of another car.

No car can move.

Everyone waits forever.

This is a deadlock.

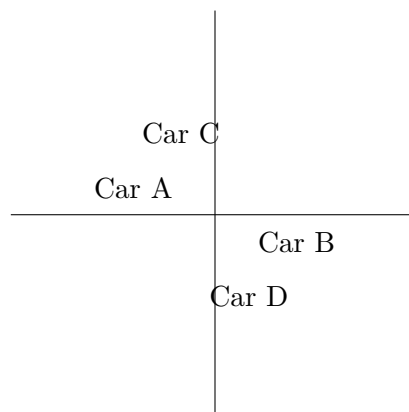


Figure 9.1: Four-Way Traffic Deadlock

9.4 Resource Allocation Concepts

9.4.1 Resource

A resource is anything required by a process to perform its task.

Examples:

- CPU Core
- Memory Page
- File Handle
- Database Lock
- Printer
- Network Socket

9.4.2 Resource Usage Cycle

1. Request Resource
2. Use Resource
3. Release Resource

9.4.3 Example

A process:

1. Requests Printer
2. Prints Document
3. Releases Printer

9.5 Reusable Resources

9.5.1 Definition

Reusable resources can be used repeatedly by different processes.

9.5.2 Characteristics

- Not consumed.
- Released after use.
- Reallocated later.

9.5.3 Examples

- CPU
- Printer
- Memory
- Disk Drive
- File Lock

9.5.4 Illustration

Process A → Uses Printer → Releases Printer

Printer remains available for future use.

9.6 Consumable

Resources

9.6.1 Definition

Consumable resources disappear once used.

9.6.2 Characteristics

- Produced and consumed.
- Not reused directly.

9.6.3 Examples

- Messages
- Signals
- Interrupts
- Data Packets

9.6.4 Example

A producer generates a message.

A consumer receives it.

The message is consumed.

9.7 Deadlock

Definition

9.7.1 Formal

Definition

A deadlock is a condition where a set of processes are permanently blocked because each process is waiting for a resource held by another process in the same set.

9.7.2 Example

Consider:

- Process P1 holds Resource R1.
- Process P1 requests Resource R2.
- Process P2 holds Resource R2.
- Process P2 requests Resource R1.

Neither process can proceed.

Deadlock occurs.

9.7.3 Visualization

$$P1 \rightarrow R2$$
$$P2 \rightarrow R1$$

while

$$R1 \rightarrow P1$$
$$R2 \rightarrow P2$$

9.8 Coffman

Conditions

9.8.1 Overview

A deadlock can occur only if all four Coffman conditions hold simultaneously.

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

If even one condition is removed, deadlock cannot occur.

9.8.2 Mutual

Exclusion

Definition

At least one resource must be non-shareable.

Only one process may use it at a time.

Example

Printer:

- Process A printing.
- Process B must wait.

Interview

Insight

Read-only files do not require mutual exclusion.

Printers do.

9.8.3 Hold

and

Wait

Definition

A process holds one resource while requesting additional resources.

Example

- Process P1 holds Printer.
- Requests Scanner.

Visualization

$P1 : Printer + Waiting(Scanner)$

9.8.4 No**Preemption****Definition**

Resources cannot be forcibly taken away.
They must be released voluntarily.

Example

Printer cannot usually be taken away mid-print.

Interview**Insight**

CPU scheduling uses preemption.
Printers generally do not.

9.8.5 Circular**Wait****Definition**

Processes form a circular chain of waiting.

Example

$$P1 \rightarrow P2$$

$$P2 \rightarrow P3$$

$$P3 \rightarrow P1$$

Visualization

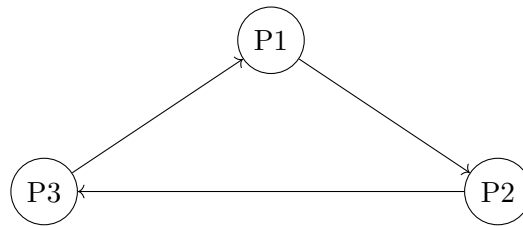


Figure 9.2: Circular Wait Condition

9.9 Resource Allocation Graphs

9.9.1 Definition

A Resource Allocation Graph (RAG) is a directed graph representing resource ownership and requests.

9.9.2 Components

- Process = Circle
- Resource = Rectangle

9.9.3 Edges

Request Edge:

$$P_i \rightarrow R_j$$

Allocation Edge:

$$R_j \rightarrow P_i$$

9.9.4 Example

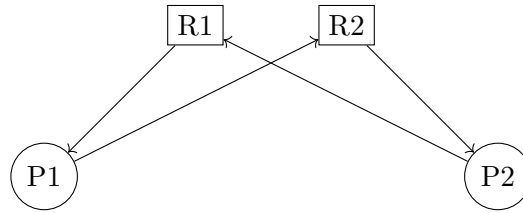


Figure 9.3: Deadlock in Resource Allocation Graph

9.9.5 Important

Observation

For systems with a single instance per resource:

Cycle implies deadlock.

9.10 Wait-For

Graphs

9.10.1 Definition

A Wait-For Graph removes resource nodes and represents only process dependencies.

9.10.2 Construction

If:

P_i

waits for a resource held by

P_j

draw:

$P_i \rightarrow P_j$

9.10.3 Example

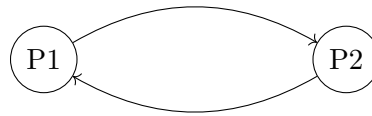


Figure 9.4: Wait-For Graph Cycle

9.10.4 Deadlock

Rule

Cycle in WFG implies deadlock.

9.11 Deadlock

Examples

9.11.1 Example

1: Printer and Scanner

P1:

- Holds Printer
- Requests Scanner

P2:

- Holds Scanner
- Requests Printer

Result:

Deadlock.

9.11.2 Example

2: Two Mutex Locks

Thread A:

```
lock(m1);
lock(m2);
```

Thread B:

```
lock(m2);
lock(m1);
```

Possible deadlock.

9.11.3 Example 3: Dining Philosophers

Each philosopher:

- Holds left fork.
- Waits for right fork.

Deadlock possible.

9.12 Deadlocks in Databases

9.12.1 Scenario

Transaction T1:

- Locks Row A
- Requests Row B

Transaction T2:

- Locks Row B
- Requests Row A

Deadlock occurs.

9.12.2 Example

T1:

LOCK A

LOCK B

T2:

LOCK B

LOCK A

9.12.3 Database Handling

DBMS periodically:

- Detects deadlocks
- Rolls back transactions

9.12.4 Case

Study

MySQL InnoDB:

- Builds wait-for graph.
- Detects cycle.
- Chooses victim transaction.
- Performs rollback.

9.13 Deadlocks in Distributed Systems

9.13.1 Challenge

Resources are distributed across machines.

No global state exists.

9.13.2 Examples

- Distributed Databases
- Distributed File Systems
- Cloud Platforms

9.13.3 Problem

Detecting cycles becomes difficult because:

- Information is distributed.
- Network delays occur.
- Partial failures occur.

9.13.4 Case

Study

Distributed database transaction locks spanning multiple servers.

Global deadlock may involve machines in different data centers.

9.14 Deadlocks in Modern Applications

9.14.1 Microservices

Service A:

- Calls Service B.
- Waits for response.

Service B:

- Calls Service A.
- Waits for response.

Distributed deadlock.

9.14.2 Cloud Systems

Examples:

- Kubernetes controllers
- Distributed locks
- Resource schedulers

9.14.3 Multithreaded Applications

Common source:

- Inconsistent lock ordering.

9.15 Deadlock Detection Concepts

9.15.1 Purpose

Determine whether deadlock currently exists.

9.15.2 Basic Idea

- Build dependency graph.
- Search for cycles.

9.15.3 For Single Instance Resources

Use Wait-For Graph.

Cycle indicates deadlock.

9.15.4 For Multiple Instances

Use matrix-based detection algorithms.

9.15.5 Detection Frequency

Tradeoff:

- Frequent detection = More overhead
- Infrequent detection = Longer deadlocks

9.15.6 Deadlock Recovery

After detection:

- Kill process
- Rollback transaction
- Resource preemption

9.16 Interview Case Studies

9.16.1 Case Study 1

Two threads locking resources in opposite order.

Question:

Will deadlock occur?

Answer:

Possible if timing permits.

9.16.2 Case Study 2

A cycle exists in a wait-for graph.

Question:

What does it imply?

Answer:

Deadlock.

9.16.3 Case

Study

3

Remove one Coffman condition.

Question:

Can deadlock occur?

Answer:

No.

9.17 Key

Takeaways

- Deadlock is permanent waiting among processes.
- Resources may be reusable or consumable.
- Four Coffman conditions are necessary.
- Mutual exclusion is often unavoidable.
- Circular wait is easiest to visualize.
- Resource Allocation Graphs model ownership and requests.
- Wait-For Graphs simplify deadlock detection.
- Databases frequently encounter deadlocks.
- Distributed deadlocks are harder to detect.
- Deadlock detection relies heavily on cycle detection.

9.18 20 Interview Questions with Answers

1. What is a deadlock?

Answer: Permanent waiting among processes due to circular resource dependencies.

2. What are the Coffman conditions?

Answer: Mutual exclusion, hold and wait, no preemption, circular wait.

3. Are all four conditions necessary?

Answer: Yes.

4. What is mutual exclusion?

Answer: Resource cannot be shared simultaneously.

5. What is hold and wait?

Answer: Holding resources while requesting others.

6. What is no preemption?
Answer: Resources cannot be forcibly removed.
7. What is circular wait?
Answer: Circular chain of waiting processes.
8. What is a reusable resource?
Answer: Resource used repeatedly after release.
9. Give a reusable resource example.
Answer: Printer.
10. What is a consumable resource?
Answer: Resource consumed when used.
11. Example of consumable resource?
Answer: Message.
12. What is a Resource Allocation Graph?
Answer: Graph representing requests and allocations.
13. Process node shape in RAG?
Answer: Circle.
14. Resource node shape in RAG?
Answer: Rectangle.
15. What is a Wait-For Graph?
Answer: Process dependency graph.
16. What does a cycle indicate in WFG?
Answer: Deadlock.
17. Why do databases experience deadlocks?
Answer: Conflicting locks.
18. What is a deadlock victim?
Answer: Process selected for termination/rollback.
19. Why are distributed deadlocks difficult?
Answer: No global state.
20. How are deadlocks detected?
Answer: Cycle detection.

9.19 20 MCQs with Answers

1. Deadlock occurs when: A) CPU idle B) Infinite waiting C) Paging D) Swapping
Answer: B
2. Number of Coffman conditions: Answer: 4
3. Printer is: Answer: Reusable Resource
4. Message is: Answer: Consumable Resource
5. Process nodes in RAG are: Answer: Circles

6. Resource nodes are: Answer: Rectangles
7. Circular wait means: Answer: Circular dependency
8. Hold and wait means: Answer: Holding while requesting
9. Deadlock requires: Answer: All four conditions
10. WFG stands for: Answer: Wait-For Graph
11. Cycle in WFG indicates: Answer: Deadlock
12. Database deadlock often involves: Answer: Locks
13. Resource allocation graph contains: Answer: Processes and Resources
14. Mutual exclusion means: Answer: Exclusive access
15. No preemption means: Answer: Cannot forcibly reclaim
16. Distributed deadlock is: Answer: Harder to detect
17. Detection commonly uses: Answer: Graph Cycles
18. Recovery may involve: Answer: Rollback
19. InnoDB can: Answer: Detect deadlocks
20. Deadlock is a: Answer: Resource allocation problem

9.20 Revision

Notes

- Deadlock = Permanent waiting.
- Four Coffman conditions are mandatory.
- Reusable resources are released after use.
- Consumable resources disappear after use.
- RAG contains processes and resources.
- WFG contains only processes.
- Cycle in WFG implies deadlock.
- Databases frequently encounter deadlocks.
- Distributed deadlocks are challenging.
- Detection relies on dependency analysis.

9.21 Cheat**Sheet**

Deadlock	Permanent Waiting
Reusable Resource	Printer, CPU, Memory
Consumable Resource	Message, Signal
Mutual Exclusion	Exclusive Access
Hold and Wait	Hold + Request
No Preemption	Cannot Force Release
Circular Wait	Circular Dependency
RAG	Resource Allocation Graph
WFG	Wait-For Graph
Cycle in WFG	Deadlock
Database Deadlock	Conflicting Locks
Distributed Deadlock	Cross-Machine Dependency
Detection	Cycle Detection
Recovery	Kill / Rollback

Deadlock Handling

10.1 Introduction

Deadlock handling is one of the most important topics in Operating Systems and Software Engineering interviews. A deadlock occurs when a set of processes are permanently blocked because each process is waiting for a resource held by another process.

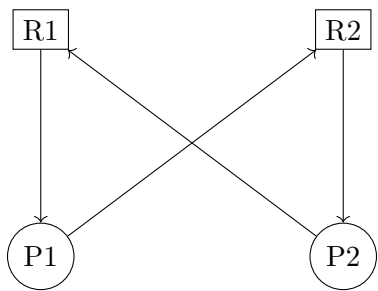


Figure 10.1: Deadlock Cycle

10.2 Deadlock

Prevention

Deadlock prevention ensures that at least one of the four necessary conditions for deadlock never holds.

10.2.1 Necessary

Conditions

- 1. Mutual Exclusion
- 2. Hold and Wait
- 3. No Preemption
- 4. Circular Wait

10.2.2 Prevention

Techniques

Condition	Prevention Strategy
Mutual Exclusion	Make resources sharable whenever possible

Hold and Wait	Request all resources at once
No Preemption	Allow resource preemption
Circular Wait	Impose resource ordering

10.3 Resource

Ordering

Assign a unique number to every resource type.

$$R_1 < R_2 < R_3 < R_4$$

Processes must request resources in increasing order.

Example:

$$R_1 \rightarrow R_3 \rightarrow R_4$$

Allowed.

$$R_3 \rightarrow R_1$$

Not allowed.

10.3.1 Advantages

- Eliminates circular wait
- Easy to implement
- Common in databases

10.4 Resource

Preemption

Resources may be forcibly taken from processes.

10.4.1 Example

1. P1 holds Printer
2. P2 requests Printer

3. OS preempts Printer from P1
4. Printer assigned to P2

Useful for:

- CPU scheduling
- Memory management
- Virtual machines

10.5 Deadlock

Avoidance

Deadlock avoidance dynamically analyzes resource allocation requests.

The system grants requests only if the resulting state remains safe.

10.6 Safe

State

A safe state guarantees the existence of a safe sequence.

10.6.1 Safe

Sequence

A sequence where every process can complete.

Example:

$$\langle P_1, P_3, P_4, P_0, P_2 \rangle$$

10.6.2 Visualization

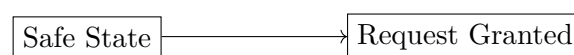


Figure 10.2: Safe State Transition

10.7 Unsafe

State

Unsafe state does not necessarily imply deadlock.

However, future requests may lead to deadlock.



Figure 10.3: Unsafe State

10.8 Banker's

Algorithm

Developed by Edsger Dijkstra.

The algorithm determines whether granting a request keeps the system safe.

10.8.1 Data

Structures

- Available
- Max
- Allocation
- Need

Formula:

$$Need = Max - Allocation$$

10.9 Numerical

Example

10.9.1 Available

(3, 3, 2)

10.9.2 Allocation

Matrix

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

10.9.3 Max**Matrix**

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

10.9.4 Need**Matrix**

$$Need = Max - Allocation$$

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

10.10 Safety**Algorithm****10.10.1 Algorithm****Steps**

1. Initialize Work = Available
2. Finish[i] = false
3. Find process whose Need $\leq$ Work
4. Execute process
5. Release resources
6. Update Work
7. Repeat

10.10.2 Walkthrough

Initially:

$$Work = (3, 3, 2)$$

P1 satisfies:

$$(1, 2, 2) \leq (3, 3, 2)$$

After completion:

$$Work = (5, 3, 2)$$

P3 satisfies:

$$(0, 1, 1) \leq (5, 3, 2)$$

After completion:

$$Work = (7, 4, 3)$$

P4 satisfies.

After P4:

$$Work = (7, 4, 5)$$

P0 satisfies.

After P0:

$$Work = (7, 5, 5)$$

P2 satisfies.

Safe sequence:

$$\langle P_1, P_3, P_4, P_0, P_2 \rangle$$

Therefore state is safe.

10.11 Resource Request Algorithm

Suppose process P_i requests resources.

10.11.1 Checks

$$Request_i \leq Need_i$$

$$Request_i \leq Available$$

Temporarily allocate.

Run Safety Algorithm.

Grant only if safe.

10.11.2 Example

Available:

$$(3, 3, 2)$$

Request by P1:

$$(1, 0, 2)$$

Conditions satisfied.

After simulation, system remains safe.

Request granted.

10.12 Deadlock Detection

Instead of preventing deadlocks, the system allows them and periodically detects them.

10.12.1 Resource Allocation Graph

Used when each resource has a single instance.

Cycle implies deadlock.

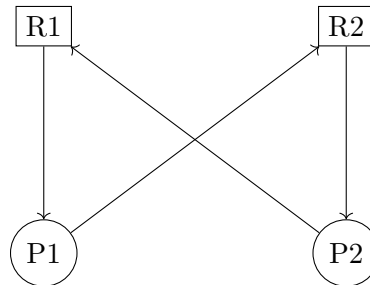


Figure 10.4: Cycle Indicates Deadlock

10.12.2 Detection Algorithm

For multiple resource instances:

- Similar to Banker's Safety Algorithm
- Detect unfinished processes
- Report deadlocked set

10.13 Deadlock Recovery

After detection, recovery is required.

Two major approaches:

1. Process Termination
2. Resource Preemption

10.14 Recovery by Process Termination

10.14.1 Option 1

Terminate all deadlocked processes.

Advantages:

- Simple
- Immediate

Disadvantages:

- Large loss of work

10.14.2 Option

2

Terminate one process at a time.

Factors:

- Priority
- Execution time
- Resources used
- Remaining execution time

10.15 Recovery by Resource Preemption

10.15.1 Steps

1. Select victim
2. Preempt resources
3. Rollback process
4. Restart later

10.15.2 Victim Selection Criteria

- Minimum cost
- Lowest priority
- Least work completed

10.16 Flowchart for Deadlock Handling

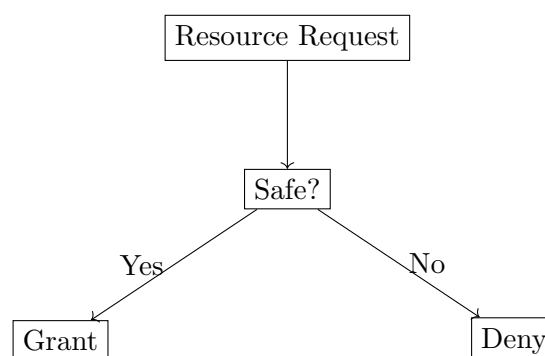


Figure 10.5: Deadlock Avoidance Decision Flow

10.17 Practical Deadlock Handling

Most operating systems use a combination of:

- Prevention
- Avoidance
- Detection
- Recovery

Examples:

- Linux file locks
- Windows synchronization objects
- JVM monitors
- Database transaction managers

10.18 Database Deadlock Handling

Databases frequently encounter deadlocks.

Example:

- Transaction T1 locks Row A
- Transaction T2 locks Row B
- T1 requests B
- T2 requests A

Deadlock occurs.

10.18.1 Solutions

- Wait-Die
- Wound-Wait
- Timeout
- Deadlock Detection Graph

10.18.2 Wait-Die

Older transaction waits.

Younger transaction aborts.

10.18.3 Wound-Wait

Older transaction preempts younger transaction.

10.19 Distributed Deadlock Handling

Deadlocks may span multiple machines.

Challenges:

- No global clock
- Distributed resources
- Communication delays

10.19.1 Techniques

- Centralized Detection
- Hierarchical Detection
- Distributed Detection

10.19.2 Probe-Based Detection

Special probe messages traverse wait-for graph.

Cycle implies deadlock.

10.20 Interview Strategies

- Always distinguish prevention, avoidance, detection, and recovery.
- Know Banker's Algorithm thoroughly.
- Memorize safe versus unsafe state.
- Understand database deadlock handling.
- Explain trade-offs clearly.
- Practice numerical Banker's Algorithm problems.

10.21 Common Pitfalls

- Unsafe state is not deadlock.
- Safe state guarantees completion.
- Deadlock avoidance requires maximum resource information.

- Prevention may reduce utilization.
- Recovery can be expensive.
- Detection incurs runtime overhead.

10.22 Key

Takeaways

Part V

Memory Management

Memory Management Fundamentals

11.1 Chapter

Overview

Memory management is one of the core responsibilities of an Operating System. It determines how memory is allocated, protected, shared, relocated, and reclaimed. Understanding memory management is essential for Software Development Engineer interviews because it directly impacts performance, scalability, security, and system reliability.

This chapter introduces the foundational concepts required for advanced topics such as paging, segmentation, virtual memory, page replacement, and memory allocation.

11.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand memory management responsibilities.
- Explain address binding mechanisms.
- Differentiate logical and physical addresses.
- Understand MMU functionality.
- Explain dynamic loading and linking.
- Compare static and dynamic linking.
- Understand swapping.
- Explain memory protection mechanisms.
- Understand relocation.
- Analyze contiguous memory allocation.
- Differentiate internal and external fragmentation.
- Explain memory compaction.

11.3 Memory Management Overview

11.3.1 Definition

Memory Management is the process of controlling and coordinating main memory usage among processes.

11.3.2 Responsibilities

- Allocation of memory
- Deallocation of memory
- Address translation
- Memory protection
- Process isolation
- Efficient utilization
- Sharing mechanisms

11.3.3 Why It Matters

Without proper memory management:

- Processes may overwrite each other.
- Security vulnerabilities may arise.
- Memory utilization becomes inefficient.
- System crashes may occur.

11.3.4 Real-World Analogy

Consider a hotel:

- Hotel = Memory
- Rooms = Memory Blocks
- Guests = Processes
- Reception = Operating System

The OS decides which process gets which memory region.

11.4 Address Binding

11.4.1 Definition

Address binding is the process of mapping program addresses to memory addresses.

11.4.2 Why Binding is Needed

Source programs use symbolic addresses:

```
count  
sum  
buffer
```

These must eventually be converted into actual memory addresses.

11.4.3 Stages

1. Compile-Time Binding
2. Load-Time Binding
3. Execution-Time Binding

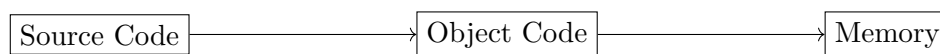


Figure 11.1: Address Binding Process

11.5 Compile-Time Binding

11.5.1 Definition

Addresses are fixed during compilation.

11.5.2 Characteristics

- Compiler generates absolute addresses.
- Program must load at predetermined location.
- Relocation is impossible.

11.5.3 Example

Suppose compiler assumes:

$$Base = 1000$$

Variable:

$$x = 1020$$

Address is fixed permanently.

11.5.4 Advantages

- Simple
- Fast execution

11.5.5 Disadvantages

- Inflexible
- Rarely used in modern systems

11.6 Load-Time

Binding

11.6.1 Definition

Address binding occurs when the program is loaded into memory.

11.6.2 Characteristics

- Compiler generates relocatable code.
- Loader determines actual addresses.

11.6.3 Example

Program can load at:

1000

or

5000

depending on memory availability.

11.6.4 Advantages

- Flexible
- Supports relocation during loading

11.6.5 Disadvantages

- Cannot move after execution starts

11.7 Execution-Time

Binding

11.7.1 Definition

Address translation occurs while the program executes.

11.7.2 Characteristics

- Most modern operating systems use it.
- Requires hardware support.
- Uses MMU.

11.7.3 Advantages

- Maximum flexibility
- Supports virtual memory
- Allows process relocation

11.7.4 Disadvantages

- Additional hardware complexity

11.8 Logical

Addresses

11.8.1 Definition

Addresses generated by the CPU.

Also called:

- Virtual Addresses
- Program Addresses

11.8.2 Characteristics

- Visible to processes.
- Not actual RAM locations.
- Translated by MMU.

11.8.3 Example

CPU generates:

2500

Logical address.

11.9 Physical

Addresses

11.9.1 Definition

Actual addresses in main memory.

11.9.2 Characteristics

- Used by RAM hardware.
- Generated after address translation.

11.9.3 Example

Logical:

2500

Translated to:

10500

Physical address.

11.10 MMU (Memory Management Unit)

11.10.1 Definition

Hardware component responsible for address translation.

11.10.2 Working

$$Physical = Logical + Base$$

11.10.3 Example

Base Register:

10000

Logical Address:

500

Physical Address:

10500



Figure 11.2: MMU Address Translation

11.10.4 Interview

Note

MMU is a favorite interview topic because it bridges hardware and operating systems.

11.11 Dynamic

Loading

11.11.1 Definition

Program modules are loaded into memory only when required.

11.11.2 Example

A rarely used error-handling module loads only when an error occurs.

11.11.3 Advantages

- Reduced memory usage
- Faster startup

11.11.4 Disadvantages

- Additional runtime overhead

11.12 Dynamic

Linking

11.12.1 Definition

Libraries are linked during execution.

11.12.2 Working

Executable contains references instead of full library code.

At runtime:

- Loader locates library.
- Addresses resolved.

11.12.3 Advantages

- Smaller executable
- Shared libraries
- Easy updates

11.12.4 Disadvantages

- Runtime dependency

11.13 Shared Libraries

11.13.1 Definition

Libraries shared among multiple processes.

11.13.2 Linux Examples

```
libc.so  
libpthread.so  
libm.so
```

11.13.3 Windows Examples

```
kernel32.dll  
user32.dll  
gdi32.dll
```

11.13.4 Advantages

- Memory savings
- Reduced disk usage

11.14 Static Linking vs Dynamic Linking

Feature	Static Linking	Dynamic Linking
Library Inclusion	Embedded	External
Executable Size	Large	Small
Memory Usage	Higher	Lower
Runtime Dependency	No	Yes
Updates	Rebuild Required	Easy
Performance	Slightly Faster	Slight Overhead

Table 11.1: Static vs Dynamic Linking

11.15 Swapping

11.15.1 Definition

Swapping moves processes between memory and secondary storage.

11.15.2 Purpose

Allows more processes than available RAM.

11.15.3 Diagram

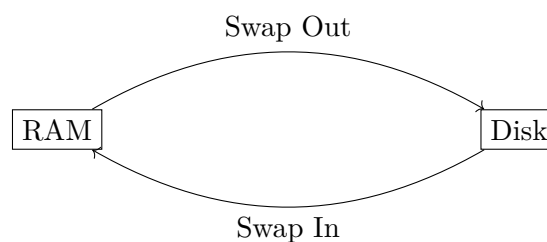


Figure 11.3: Swapping

11.15.4 Advantages

- Better memory utilization

11.15.5 Disadvantages

- Disk I/O overhead
- Performance degradation

11.16 Memory Protection

11.16.1 Definition

Mechanisms preventing unauthorized memory access.

11.16.2 Need

One process must not corrupt another process.

11.16.3 Protection Mechanisms

- Base Register
- Limit Register
- Access Permissions
- Virtual Memory

11.16.4 Example

Valid range:

1000 – 5000

Accessing:

6000

causes protection fault.

11.17 Relocation

11.17.1 Definition

Ability to move a process in memory during execution.

11.17.2 Why Needed

- Memory compaction

- Swapping
- Virtual memory

11.17.3 Interview

Insight

Execution-time binding enables relocation.

11.18 Contiguous Memory Allocation

11.18.1 Definition

Each process occupies one continuous block of memory.

11.18.2 Example

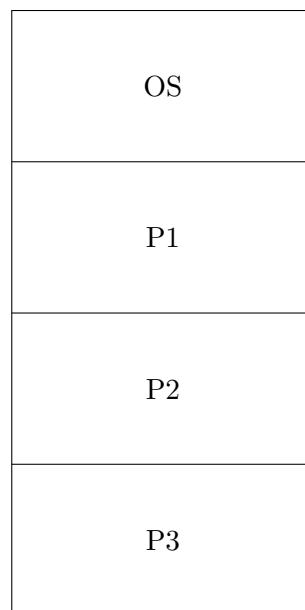


Figure 11.4: Contiguous Allocation

11.18.3 Advantages

- Simple implementation
- Fast access

11.18.4 Disadvantages

- Fragmentation problems

11.19 Fragmentation

11.19.1 Definition

Unused memory resulting from allocation inefficiencies.

Two types:

1. Internal Fragmentation
2. External Fragmentation

11.20 Internal

Fragmentation

11.20.1 Definition

Unused space inside allocated memory blocks.

11.20.2 Example

Process requires:

14KB

Allocated:

16KB

Waste:

2KB

11.20.3 Visualization

|Used|Used|Unused|

11.21 External Fragmentation

11.21.1 Definition

Free memory exists but is scattered into small pieces.

11.21.2 Example

Free blocks:

$$5KB + 4KB + 3KB$$

Total:

$$12KB$$

Request:

$$10KB$$

Cannot allocate because no single contiguous block exists.

11.21.3 Visualization

$$|P1|Free|P2|Free|P3|Free|$$

11.22 Memory Compaction

11.22.1 Definition

Technique of moving processes to combine scattered free memory.

11.22.2 Before Compaction

$$|P1|Free|P2|Free|P3|Free|$$

11.22.3 After**Compaction**

$|P1|P2|P3|FreeFreeFree|$

11.22.4 Advantages

- Eliminates external fragmentation

11.22.5 Disadvantages

- Expensive operation
- Requires relocation support

11.23 Linux**Examples**

View memory usage:

```
free -h
```

Process memory:

```
top
```

Shared libraries:

```
ldd executable
```

Memory maps:

```
cat /proc/pid/maps
```

11.24 Windows**Examples**

Memory monitoring:

Task Manager

Memory diagnostics:

Resource Monitor

DLL inspection:

Dependency Walker

11.25 Interview

Notes

Frequently asked topics:

- Logical vs Physical Address
- MMU Working
- Static vs Dynamic Linking
- Internal vs External Fragmentation
- Swapping
- Relocation
- Dynamic Loading vs Dynamic Linking

11.25.1 Golden

Rule

Logical Address $\rightarrow$ MMU $\rightarrow$ Physical Address

11.26 Key

Takeaways

- Memory management allocates and protects memory.
- Address binding may occur at compile, load, or execution time.
- Logical addresses differ from physical addresses.
- MMU performs address translation.
- Dynamic loading loads modules on demand.
- Dynamic linking links libraries at runtime.
- Shared libraries save memory.
- Swapping enables better memory utilization.
- Fragmentation reduces efficiency.
- Compaction reduces external fragmentation.

11.27 20 Interview Questions with Answers

1. What is memory management?
Answer: Allocation and control of memory resources.
2. What is address binding?
Answer: Mapping program addresses to memory addresses.
3. Types of address binding?
Answer: Compile-time, load-time, execution-time.
4. What is compile-time binding?

Answer: Addresses fixed during compilation.

5. What is load-time binding?

Answer: Addresses resolved during loading.

6. What is execution-time binding?

Answer: Addresses resolved during execution.

7. What is a logical address?

Answer: Address generated by CPU.

8. What is a physical address?

Answer: Actual RAM location.

9. What is MMU?

Answer: Memory Management Unit.

10. MMU performs?

Answer: Address translation.

11. What is dynamic loading?

Answer: Load modules when needed.

12. What is dynamic linking?

Answer: Link libraries at runtime.

13. Shared library example in Linux?

Answer: libc.so.

14. Shared library example in Windows?

Answer: kernel32.dll.

15. What is swapping?

Answer: Moving processes between RAM and disk.

16. What is relocation?

Answer: Moving process memory location.

17. What is contiguous allocation?

Answer: Continuous memory block allocation.

18. What is internal fragmentation?

Answer: Waste inside allocated block.

19. What is external fragmentation?

Answer: Scattered free memory.

20. What is compaction?

Answer: Combining free memory blocks.

11.28 20 MCQs with Answers

1. MMU stands for: A) Memory Mapping Unit B) Memory Management Unit C) Main Memory Unit D) Memory Monitor Unit

Answer: B

2. Logical address is generated by: Answer: CPU

3. Physical address belongs to: Answer: RAM
4. Dynamic loading occurs: Answer: On demand
5. Dynamic linking occurs: Answer: Runtime
6. Linux shared library extension: Answer: .so
7. Windows shared library extension: Answer: .dll
8. Swapping uses: Answer: Disk
9. Internal fragmentation occurs: Answer: Inside allocated blocks
10. External fragmentation occurs: Answer: Between allocated blocks
11. Compile-time binding is: Answer: Fixed
12. Execution-time binding requires: Answer: MMU
13. Shared libraries save: Answer: Memory
14. Compaction removes: Answer: External Fragmentation
15. Relocation moves: Answer: Process Memory
16. Contiguous allocation uses: Answer: Continuous Blocks
17. Dynamic loading improves: Answer: Memory Utilization
18. Memory protection prevents: Answer: Illegal Access
19. Address translation performed by: Answer: MMU
20. Fragmentation reduces: Answer: Memory Efficiency

11.29 Revision

Notes

- Memory management controls memory allocation.
- Address binding maps addresses.
- Logical address generated by CPU.
- Physical address exists in RAM.
- MMU performs translation.
- Dynamic loading loads code on demand.
- Dynamic linking resolves libraries at runtime.
- Shared libraries reduce memory usage.
- Swapping moves processes between RAM and disk.
- Internal fragmentation = inside allocated blocks.
- External fragmentation = scattered free space.
- Compaction removes external fragmentation.

11.30 Cheat**Sheet**

Logical Address	Generated by CPU
Physical Address	Actual RAM Address
MMU	Address Translator
Compile-Time Binding	Fixed at Compilation
Load-Time Binding	Fixed During Loading
Execution-Time Binding	Runtime Translation
Dynamic Loading	Load on Demand
Dynamic Linking	Link at Runtime
Shared Library	Shared Executable Code
Swapping	RAM $\leftrightarrow$ Disk
Relocation	Move Process Memory
Contiguous Allocation	Continuous Block
Internal Fragmentation	Waste Inside Block
External Fragmentation	Scattered Free Space
Compaction	Merge Free Space

Paging

12.1 Motivation for Paging

Paging is a memory management technique that eliminates the need for contiguous physical memory allocation.

Problems with contiguous allocation:

- External fragmentation
- Difficult memory allocation
- Process growth limitations
- Inefficient memory utilization

Paging solves these problems by dividing memory into fixed-size blocks.

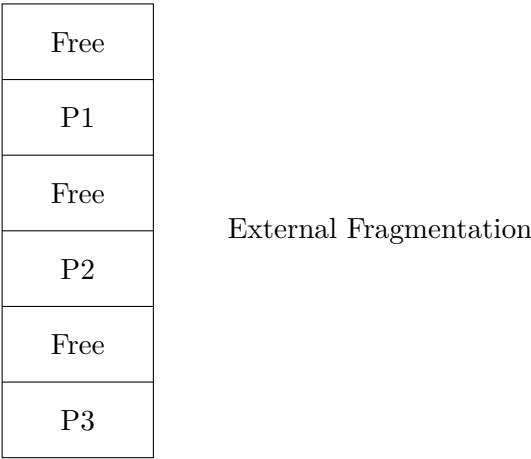


Figure 12.1: Fragmentation in Contiguous Allocation

12.2 Paging Fundamentals

Paging divides:

- Logical memory into pages
- Physical memory into frames

Page size equals frame size.

Page Size = Frame Size

Common page sizes:

- 4 KB
- 8 KB
- 16 KB
- 2 MB (Huge Page)
- 1 GB (Huge Page)

12.3

Pages

and

Frames

12.3.1 Pages

Fixed-size logical memory blocks.

12.3.2 Frames

Fixed-size physical memory blocks.

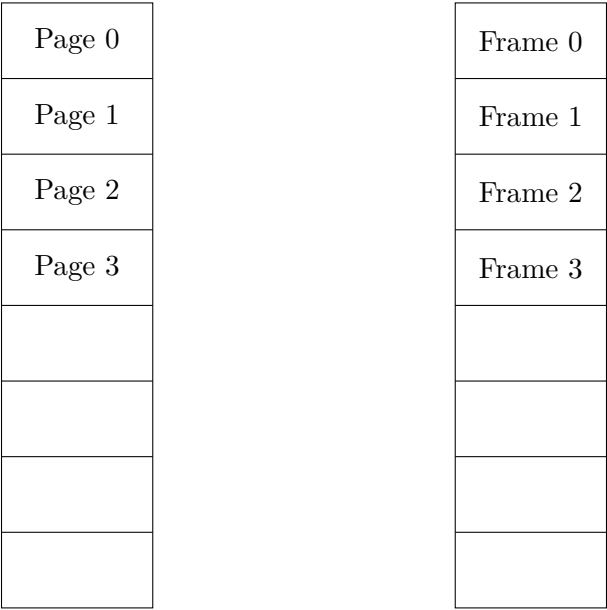


Figure 12.2: Pages and Frames

12.4 Logical Address Structure

Logical address consists of:

$$\text{Logical Address} = \langle \text{Page Number}, \text{Offset} \rangle$$

Example:

16-bit Address

Page Size:

$$1KB = 2^{10}$$

Offset bits:

$$10$$

Page Number bits:

$$16 - 10 = 6$$

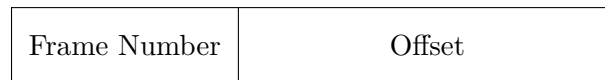
Logical Address:

$$\langle p, d \rangle$$

12.5 Physical Address Structure

Physical address consists of:

$$\langle \text{Frame Number}, \text{Offset} \rangle$$

**Figure 12.3:** Physical Address Format

12.6 Address

Translation

Address translation converts logical addresses into physical addresses.

12.6.1 Example

Logical Address:

(5, 100)

Page Table:

Page 5 → *Frame 12*

Physical Address:

(12, 100)

**Figure 12.4:** Address Translation Process

12.7 Page

Tables

A page table maps pages to frames.

12.7.1 Example

Page	Frame
0	5
1	2
2	8
3	9
4	1
5	12

12.7.2 Purpose

- Address translation
- Memory protection
- Page tracking

12.8 Page Table Entries

Typical Page Table Entry (PTE):

Field	Purpose
Frame Number	Physical Frame
Valid Bit	Page Present?
Protection Bits	Access Rights
Dirty Bit	Modified Page
Reference Bit	Recently Used

12.9 Translation Lookaside Buffer (TLB)

TLB is a special hardware cache storing recent page table entries.

12.9.1 Motivation

Without TLB:

1. Access page table
2. Access memory

Two memory accesses required.

With TLB:

1. Check TLB
2. Access memory

One memory access on hit.

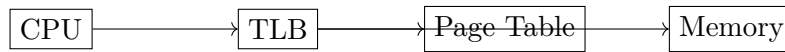


Figure 12.5: TLB-Based Translation

12.10 Effective Memory Access Time

12.10.1 Formula

Let:

$h = TLB \text{ hit ratio}$

$t = TLB \text{ lookup time}$

$m = \text{Memory access time}$

Then:

$$EMAT = h(t + m) + (1 - h)(t + 2m)$$

12.10.2 Numerical

Example

Given:

$$t = 10ns$$

$$m = 100ns$$

$$h = 0.95$$

Calculation:

$$EMAT = 0.95(10 + 100) + 0.05(10 + 200)$$

$$EMAT = 104.5 + 10.5$$

$$EMAT = 115ns$$

12.11 Hierarchical

Paging

Large address spaces require huge page tables.

Solution:

Multi-level hierarchy.

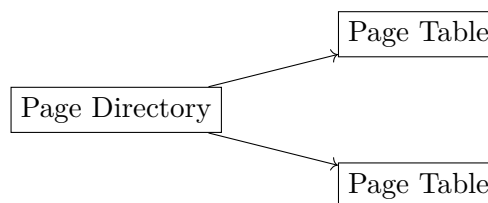


Figure 12.6: Hierarchical Paging

12.12 Multilevel

Paging

Modern systems use multiple levels.

Example:

32-bit system:

- Page Directory
- Page Table
- Offset

64-bit systems may use:

- 4-level paging
- 5-level paging

Example:

48 – bit Virtual Address

$$9 + 9 + 9 + 9 + 12$$

12.13 Inverted Page Tables

Instead of one entry per page:

One entry per frame.

12.13.1 Advantages

- Smaller memory overhead
- Suitable for large address spaces

12.13.2 Disadvantages

- Slower lookup
- Requires hashing

12.14 Hashed Page Tables

Used for very large virtual address spaces.

Hash Function:

$$Hash(VPN)$$

VPN:

Virtual Page Number.



Figure 12.7: Hashed Page Table

12.15 Shared

Pages

Multiple processes may share pages.

Common examples:

- Shared libraries
- Read-only code
- Memory-mapped files

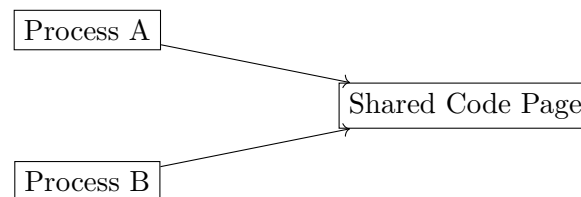


Figure 12.8: Shared Pages

12.16 Huge

Pages

Huge pages reduce:

- TLB misses
- Page table size
- Translation overhead

Common huge page sizes:

- 2 MB
- 1 GB

12.16.1 Advantages

- Better performance
- Fewer TLB entries

12.17 Paging in Linux

Linux supports:

- Demand paging
- Copy-on-write
- Huge pages
- Transparent Huge Pages (THP)
- Multilevel page tables

Typical x86-64 paging:

- PGD
- PUD
- PMD
- PTE

12.18 Paging in Windows

Windows supports:

- Demand paging
- Working sets
- Large pages
- Memory-mapped files

Features:

- Pagefile
- Virtual memory manager
- TLB optimization

12.19 TLB Miss Handling

12.19.1 TLB Hit

1. Lookup succeeds
2. Frame found
3. Memory accessed

12.19.2 TLB**Miss**

1. Search page table
2. Load entry into TLB
3. Retry access



TLB Miss

Figure 12.9: TLB Miss Handling**12.20 TLB****Shootdown**

Occurs in multiprocessor systems.

Problem:

- CPU1 modifies page mapping
- CPU2 still has stale TLB entry

Solution:

- Send Inter-Processor Interrupt (IPI)
- Invalidate remote TLB entries

TLB Shootdown

**Figure 12.10:** TLB Shootdown Mechanism**12.21 Interview-Focused****Explanations****12.21.1 Why****Paging?**

Eliminates external fragmentation.

12.21.2 Why**TLB?**

Reduces address translation overhead.

12.21.3 Why Multilevel Paging?

Reduces page table memory usage.

12.21.4 Why Huge Pages?

Improves TLB coverage and performance.

12.21.5 Most Common Interview Formula

$$EMAT = h(t + m) + (1 - h)(t + 2m)$$

12.22 Key Takeaways

Segmentation

13.1 Introduction

Segmentation is a memory management technique that divides a program into logical units called **segments**. Unlike paging, which divides memory into fixed-size blocks, segmentation reflects the programmer's view of memory and groups related data together.

Typical program components naturally map to segments:

- Code Segment
- Data Segment
- Stack Segment
- Heap Segment
- Shared Library Segment

Segmentation improves program organization, protection, and sharing.

13.2 Motivation for Segmentation

13.2.1 Problem with Linear Address Spaces

Consider a program containing:

- Instructions
- Global Variables
- Functions
- Stack
- Dynamic Memory

Treating all of these as a single contiguous memory block is inefficient.

Different parts of a program require different:

- Protection Levels

- Access Rights
- Growth Patterns
- Sharing Policies

13.2.2 Logical

Organization

Programmers think of memory as:

Code + Data + Stack + Heap

not as one large array of bytes.

Segmentation aligns memory management with the programmer's perspective.

13.3 Segmentation

Fundamentals

13.3.1 Definition

Segmentation divides a process into variable-sized logical units called segments.

Each segment contains logically related information.

13.3.2 Examples

Segment Number	Contents	height
Code 1	Global Data	2
Heap 3	Stack	4
Shared Libraries	height	

Table 13.1: Example Segments

13.3.3 Logical

Address

Format

In segmentation, a logical address consists of:

[$\langle \textit{Segment Number}, \textit{Offset} \rangle$]

Example:

[$\langle 2, 150 \rangle$]

means:

- Segment Number = 2
- Offset = 150

13.4 User View vs Physical Memory View

13.4.1 User View

Programmers see:

- Functions
- Variables
- Stack
- Heap
- Libraries

13.4.2 Physical Memory View

Hardware sees:

- Physical Memory Addresses
- Memory Frames
- Actual RAM Locations

13.4.3 Illustration

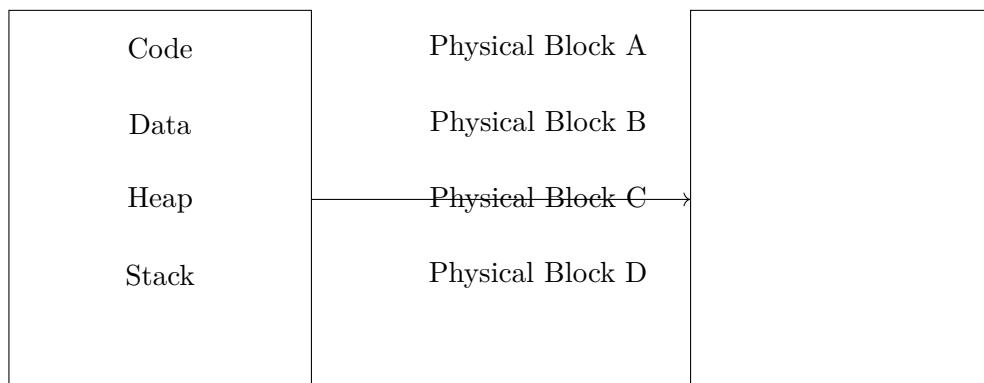


Figure 13.1: User View vs Physical Memory View

13.5 Segment Structure

Each segment contains:

- Base Address
- Length (Limit)
- Protection Information

13.5.1 Example

Segment	Base	Limit	height0
1000	500	1	4000
800	2	9000	300
			height

Table 13.2: Segment Structure Example

13.6 Segment

Tables

13.6.1 Definition

A Segment Table stores information about all segments belonging to a process.

13.6.2 Fields

Typically contains:

- Segment Number
- Base Address
- Limit
- Protection Bits

13.6.3 Example

Segment

Table

Segment	Base	Limit	height0
1000	600	1	4000
800	2	7000	500
			3
9000	1000	height	

Table 13.3: Example Segment Table

13.7 Address

Translation

in

Segmentation

13.7.1 Translation

Process

Logical Address:

$[\langle s, d \rangle]$

where:

- (s) = segment number

- (d) = offset

Steps:

1. Locate segment table entry.
2. Check offset against limit.
3. Add offset to base address.
4. Generate physical address.

13.7.2 Formula

If
 $[d < \text{Limit}]$
 then
 $[\text{Physical Address} = \text{Base} + d]$

13.7.3 Numerical

Example

Segment Table:

Segment 1: Base = 4000, Limit = 800

Logical Address:

$[\langle 1, 200 \rangle]$

Physical Address:

$[4000 + 200 = 4200]$

13.8 Address

Translation

Diagram

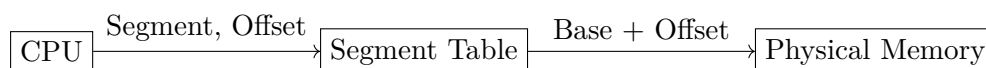


Figure 13.2: Segmentation Address Translation

13.9 Protection

Mechanisms

Segmentation naturally supports protection.

13.9.1 Protection

Bits

Each segment may have:

- Read Permission
- Write Permission
- Execute Permission

13.9.2 Example

Segment		Permission	heightCode
Execute Only	Data	Read/Write	Stack
Read/Write	height		

Table 13.4: Segment Protection

13.10 Sharing

Mechanisms

Segmentation allows efficient sharing.

13.10.1 Example

Two processes may share:

- Code Segment
- Shared Libraries

while maintaining separate:

- Data Segments
- Stack Segments

13.10.2 Benefits

- Reduced Memory Usage
- Improved Efficiency

13.11 Segmentation

Hardware

Hardware support typically includes:

- Segment Table Base Register (STBR)

- Segment Table Length Register (STLR)
- MMU Support

13.11.1 STBR

Points to segment table location.

13.11.2 STLR

Stores number of segments.

13.12 External Fragmentation

13.12.1 Problem

Segments have variable sizes.

Memory develops holes over time.

13.12.2 Example

| P1 | Free | P2 | Free | P3 |

Although sufficient free memory exists, large requests may fail.

13.12.3 Result

External fragmentation occurs.

13.13 Compaction

13.13.1 Definition

Compaction moves segments to create contiguous free space.

13.13.2 Before

| P1 | Free | P2 | Free | P3 |

13.13.3 After

| P1 | P2 | P3 | Large Free Block |

13.13.4 Drawback

Compaction is expensive.

13.14 Pure Segmentation

13.14.1 Definition

Memory management using only segmentation.

13.14.2 Characteristics

- Variable-sized segments
- Logical organization
- External fragmentation

13.15 Segmentation with Paging

13.15.1 Motivation

Segmentation suffers from external fragmentation.

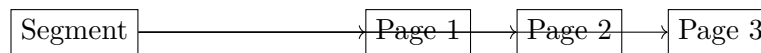
Paging solves fragmentation problems.

13.15.2 Approach

Each segment is divided into pages.

13.15.3 Benefits

- Logical Organization
- Reduced Fragmentation
- Better Memory Utilization

**Figure 13.3:** Segmentation with Paging

13.15.4 Architecture

13.16 Intel x86 Segmentation Overview

13.16.1 Historical Design

Intel x86 originally relied heavily on segmentation.

Logical address:

[Segment:Offset]

Example:

[CS:IP]

[DS:Offset]

13.16.2 Segment Registers

- CS (Code Segment)
- DS (Data Segment)
- SS (Stack Segment)
- ES
- FS
- GS

13.16.3 Modern x86

Modern operating systems use:

- Flat Address Spaces
- Paging Dominantly
- Minimal Segmentation

13.17 Modern Operating Systems and Segmentation

13.17.1 Linux

Modern Linux primarily uses:

- Paging
- Virtual Memory

Segmentation usage is minimal.

13.17.2 Windows

Modern Windows also uses:

- Paging-Based Memory Management
- Flat Memory Model

13.17.3 Reason

Paging provides:

- Better Flexibility
- Reduced Fragmentation
- Easier Memory Management

13.18 Advantages of Segmentation

1. Matches Programmer View
2. Natural Protection Support
3. Easy Sharing
4. Logical Program Organization
5. Independent Segment Growth

13.19 Disadvantages of Segmentation

1. External Fragmentation
2. Compaction Overhead
3. Complex Allocation
4. Variable Size Management
5. Additional Hardware Support

13.20 Real-World Use Cases

13.20.1 Program Organization

Separate:

- Code
- Data
- Stack

13.20.2 Shared Libraries

Shared code segments among processes.

13.20.3 Security

Different permissions per segment.

13.20.4 Legacy x86 Systems

Used segmentation extensively.

13.21 Segmentation vs Paging

Feature	Segmentation	Paging
Logical Units	Fixed Size Pages	Division Basis
Fixed Programmer Visible	Size	Variable
External	Yes	No
Supported via Page Tables	Internal Protection	Fragmentation
Sharing	Easy	Natural
Complex	Simpler	Possible Allocation
Dominant	Modern Usage	Limited

Table 13.5: Segmentation vs Paging

13.22 Key Takeaways

- Segmentation divides memory into logical units.

- Logical addresses contain segment number and offset.
- Segment tables store base and limit values.
- Segmentation supports protection naturally.
- Segmentation supports sharing efficiently.
- External fragmentation is a major drawback.
- Compaction reduces fragmentation.
- Modern systems primarily use paging.
- Segmentation with paging combines strengths of both.
- Intel x86 historically used segmentation extensively.

13.23 20 Interview Questions with Answers

1. What is segmentation? Answer: A memory management technique dividing memory into logical segments.
2. Why was segmentation introduced? Answer: To match programmer view of memory.
3. What is a segment? Answer: A logical unit of a program.
4. What is a logical address in segmentation? Answer: Segment number and offset.
5. What is a segment table? Answer: A table storing segment information.
6. What does a segment table contain? Answer: Base address and limit.
7. What is STBR? Answer: Segment Table Base Register.
8. What is STLR? Answer: Segment Table Length Register.
9. How is physical address calculated? Answer: Base + Offset.
10. What happens if offset exceeds limit? Answer: Trap or protection fault.
11. Does segmentation support protection? Answer: Yes.
12. Does segmentation support sharing? Answer: Yes.
13. What fragmentation occurs in segmentation? Answer: External fragmentation.
14. What is compaction? Answer: Rearranging memory to create large free blocks.
15. What is pure segmentation? Answer: Segmentation without paging.
16. Why combine segmentation with paging? Answer: To reduce fragmentation.
17. What segment register stores code segment? Answer: CS.
18. What segment register stores stack segment? Answer: SS.
19. Do modern Linux systems rely heavily on segmentation? Answer: No.
20. Which memory technique dominates modern OSes? Answer: Paging.

13.24 20 MCQs with Answers

1. Segmentation divides memory into: A) Frames B) Pages C) Segments D) Blocks
Answer: C

2. Segment sizes are: A) Fixed B) Variable C) Equal D) Random
Answer: B
3. Logical address contains: A) Page + Frame B) Segment + Offset C) Base + Limit D) Frame + Offset
Answer: B
4. Segment table stores: A) Base and Limit B) Frames C) Pages D) Cache Lines
Answer: A
5. STBR stands for: Answer: Segment Table Base Register
6. STLR stands for: Answer: Segment Table Length Register
7. Segmentation supports: A) Protection B) Sharing C) Both D) Neither
Answer: C
8. Physical address equals: A) Base + Offset B) Page + Offset C) Limit + Offset D) Segment + Offset
Answer: A
9. Main fragmentation issue: A) Internal B) External C) Cache D) TLB
Answer: B
10. Compaction addresses: A) Internal Fragmentation B) External Fragmentation C) TLB Misses D) Cache Misses
Answer: B
11. Segment register for code: A) DS B) SS C) CS D) ES
Answer: C
12. Segment register for stack: A) CS B) SS C) DS D) FS
Answer: B
13. Segmentation is: A) Programmer Visible B) Invisible C) Kernel Only D) Hardware Only
Answer: A
14. Paging uses: A) Variable Sizes B) Fixed Sizes C) Logical Units D) Functions
Answer: B
15. Segmentation uses: A) Logical Units B) Fixed Blocks C) Cache Lines D) Frames Only
Answer: A
16. Modern Linux primarily uses: A) Segmentation B) Paging C) Swapping Only D) Overlaying
Answer: B
17. Modern Windows primarily uses: A) Segmentation B) Paging C) Compaction D) Overlays
Answer: B
18. Shared libraries benefit from: A) Segmentation B) Cache C) DMA D) TLB
Answer: A
19. Segment table entry includes: A) Limit B) Base C) Protection D) All
Answer: D
20. Segmentation with paging combines: A) Benefits of both techniques B) Only paging C) Only segmentation D) Neither
Answer: A

13.25 Revision

Notes

- Segmentation divides memory into logical units.
- Logical address = Segment Number + Offset.
- Segment Table stores Base and Limit.
- MMU performs address translation.
- Protection and sharing are natural in segmentation.
- External fragmentation is the primary problem.
- Compaction reduces fragmentation.
- Pure segmentation uses variable-sized segments.
- Segmentation with paging is more efficient.
- Modern operating systems primarily rely on paging.

13.26 Cheat

Sheet

Logical Address	Segment Number + Offset
	Physical Address
Base + Offset	Base + Limit + Protection
Segment Table	STBR
Segment Table Base Register	Segment Table Length Register
STLR	Major Advantage
Logical Organization	External Fragmentation
Major Disadvantage	Compaction
Removes External Fragmentation	Natural Support
Protection	Sharing
Easy Support	Variable Sized Segments
Pure Segmentation	Segmentation + Paging
Hybrid Approach	Historically Segmentation Based
Intel x86	Modern Linux
Paging Dominant	Paging Dominant
Modern Windows	

Virtual Memory

14.1 Introduction

Virtual Memory is one of the most important abstractions provided by modern operating systems. It allows processes to execute as if they have access to a large, contiguous memory space, regardless of the actual amount of installed physical memory.

Virtual memory provides:

- Process isolation
- Efficient memory utilization
- Larger logical address spaces
- Simplified programming model
- Multiprogramming support

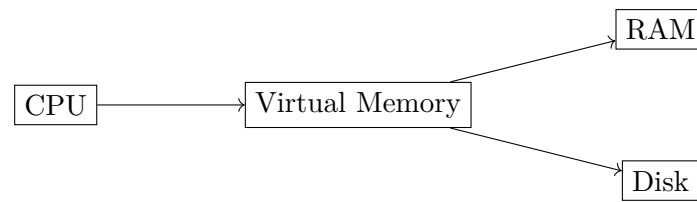
14.2 Motivation for Virtual Memory

Without virtual memory:

- Entire program must fit in RAM
- Large applications cannot execute
- Multiprogramming becomes difficult
- Memory utilization decreases

With virtual memory:

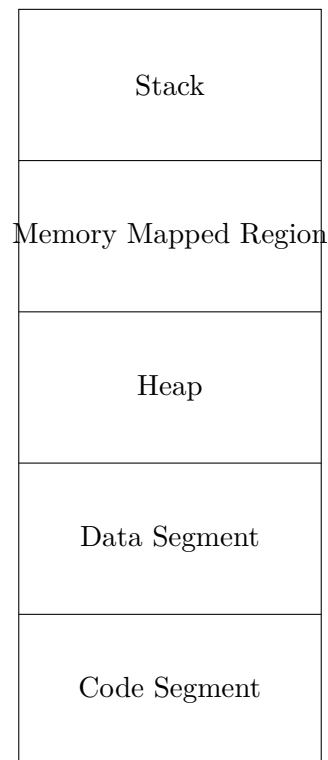
- Only actively used portions remain in RAM
- Large applications can run on smaller systems
- More processes can coexist
- Better system throughput

**Figure 14.1:** Virtual Memory Abstraction

14.3 Virtual Address Space

Each process sees its own virtual address space.

Typical layout:

**Figure 14.2:** Typical Virtual Address Space

14.3.1 Advantages

- Process isolation
- Protection
- Easier relocation
- Shared memory support

14.4 Demand

Paging

Demand paging loads pages only when required.

Instead of loading an entire program:

- Load minimal pages
- Fetch remaining pages on demand

Benefits:

- Faster startup
- Lower memory consumption
- Better scalability



Figure 14.3: Demand Paging

14.5 Demand

Segmentation

Demand segmentation applies the same idea to segments.

Only required segments are loaded.

Examples:

- Code segment
- Data segment
- Shared libraries

Modern systems primarily use demand paging.

14.6 Lazy

Loading

Lazy loading postpones loading until actual use.

Examples:

- Shared libraries
- Dynamic modules
- Memory mapped files

Advantages:

- Reduced startup time
- Reduced memory usage

14.7 Page

Faults

A page fault occurs when a process references a page that is not currently in physical memory.

14.7.1 Causes

- Demand paging
- Swapped-out page
- Invalid reference
- Copy-on-write trigger



Figure 14.4: Page Fault Generation

14.8 Page

Fault

Handling

Procedure

When a page fault occurs:

1. CPU generates trap
2. Kernel gains control
3. Validate memory reference
4. Locate page on disk
5. Find free frame
6. Read page into RAM
7. Update page table
8. Restart instruction

14.8.1 Detailed

Walkthrough

Suppose:

- Process accesses Page 25
- Page not present
- Valid bit = 0

Sequence:

1. MMU detects missing page
2. Trap generated
3. OS checks legality
4. Page located in swap area
5. Free frame allocated
6. Disk I/O performed
7. Page table updated
8. Process resumes



Figure 14.5: Page Fault Handling Sequence

14.9 Valid and Invalid References

14.9.1 Valid Reference

Reference belongs to process address space.

Example:

`0x00400000`

inside allocated memory.

14.9.2 Invalid Reference

Reference outside process address space.

Example:

`0xFFFFFFFFFFFFFFFF`

Results:

- Segmentation fault
- Access violation
- Process termination

14.10 Copy-on-Write (COW)

Copy-on-Write delays copying until modification occurs.

14.10.1 Motivation

Traditional fork:

- Copy entire address space
- Expensive operation

COW:

- Share pages initially
- Copy only on write

14.10.2 Operation

Parent and child share page:

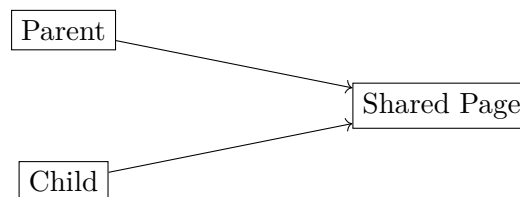


Figure 14.6: Copy-on-Write Shared Page

When child writes:

1. Page fault occurs
2. New page allocated
3. Data copied
4. Child receives private copy

14.11 Memory Mapped Files

Files can be mapped directly into virtual memory.

Advantages:

- Faster I/O
- Shared access

- Simpler programming

14.11.1 Example

```

1  int fd = open("data.txt", O_RDONLY);
2
3  void* ptr = mmap(
4      NULL,
5      size,
6      PROT_READ,
7      MAP_SHARED,
8      fd,
9      0
10 );

```

14.12 Shared Memory

Shared memory enables multiple processes to access common memory regions.

Benefits:

- Fast IPC
- Low overhead
- High throughput

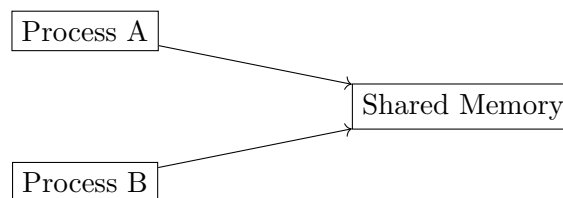


Figure 14.7: Shared Memory

14.13 Working Set Model

Working Set = Pages actively used by a process during a time interval.

Notation:

$$WSS_i$$

where:

$WSS = \text{Working Set Size}$

Benefits:

- Prevents thrashing
- Improves performance
- Better memory allocation

14.14 Thrashing

Thrashing occurs when excessive page faults consume most CPU time.

Symptoms:

- High disk activity
- Low CPU utilization
- Poor throughput

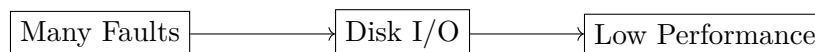


Figure 14.8: Thrashing Cycle

14.14.1 Solutions

- Increase memory
- Working set model
- Better allocation policies
- Reduce multiprogramming level

14.15 Local Allocation Policies

Processes receive a fixed set of frames.

Advantages:

- Isolation
- Predictability

Disadvantages:

- Possible underutilization

14.16 Global Allocation Policies

Frames may be taken from any process.

Advantages:

- Better utilization
- Adaptive allocation

Disadvantages:

- Unpredictable behavior

Policy	Advantage	Disadvantage
Local	Isolation	Less Flexible
Global	Better Utilization	Interference

14.17 Overcommitment

Operating systems may allocate more virtual memory than available physical memory.

Example:

64GB Virtual

with:

16GB RAM

Advantages:

- Efficient resource usage
- Supports large workloads

Risks:

- Thrashing
- OOM situations

14.18 Swap Space

Swap space stores inactive pages on disk.

14.18.1 Purpose

- Extend apparent memory
- Support demand paging
- Handle memory pressure

14.18.2 Drawbacks

- Disk is slower than RAM
- Excessive swapping hurts performance

14.19 Virtual Memory in Linux

Linux uses:

- Demand paging
- Copy-on-write
- `mmap()`
- Shared memory
- Swap partitions/files
- Transparent Huge Pages

14.19.1 Linux Commands

Check memory:

```
1 free -h
```

View mappings:

```
1 cat /proc/self/maps
```

Check virtual memory:

```
1 vmstat
```


14.20 Virtual Memory in Windows

Windows uses:

- Demand paging
- Pagefile.sys
- Working sets
- Memory-mapped files
- Copy-on-write

14.20.1 Important Components

- Virtual Memory Manager
- Pagefile
- Working Set Manager
- Memory Compression

14.21 Case Study: Launching a Large Application

Suppose a browser executable is 500 MB.

Without virtual memory:

- Entire executable loaded
- Slow startup

With virtual memory:

- Only necessary pages loaded
- Remaining pages fetched on demand
- Faster startup

14.22 Case Study: `fork()` in Linux

Without COW:

- Entire process copied
- High overhead

With COW:

- Shared pages
- Copy only when modified
- Significant performance gain

14.23 Interview Focus: What Happens During a Page Fault?

Expected answer:

1. Memory reference occurs
2. MMU checks page table
3. Page absent
4. Trap generated
5. OS validates reference
6. Page fetched from disk
7. Page table updated
8. Instruction restarted

14.24 Interview Focus: Why Virtual Memory Exists

Key points:

- Larger logical memory
- Process isolation
- Efficient memory usage
- Multiprogramming support
- Demand loading

14.25 Interview Focus: How Copy-on-Write Works

Key points:

1. Parent forks child
2. Pages shared as read-only
3. Write attempt causes fault
4. OS allocates new page
5. Data copied
6. Writer gets private page

14.26 Key**Takeaways**

Page Replacement Algorithms

15.1 Introduction

Virtual memory allows processes to execute even when their entire address space is not loaded into physical memory. Since physical memory is limited, the operating system must decide which page should be removed when a new page needs to be loaded.

This decision is made by a **Page Replacement Algorithm**.

Page replacement is one of the most important interview topics in Operating Systems because it combines theory, numerical problem solving, performance analysis, and system design considerations.

15.2 Learning

Objectives

After studying this chapter, you should be able to:

- Explain why page replacement is necessary.
- Calculate page faults for common algorithms.
- Analyze reference strings.
- Compare replacement algorithms.
- Understand Belady's Anomaly.
- Solve page replacement numericals quickly.
- Discuss practical implementations in modern operating systems.

15.3 Why Page Replacement is Needed

When a process accesses a page that is not currently in memory:

1. A page fault occurs.
2. OS loads the required page.
3. If all frames are occupied, a page must be removed.

The choice of which page to remove directly affects performance.

15.3.1 Example

Available Frames:

[3]

Current Pages:

[[1,2,3]]

Incoming Page:

[4]

One page must be evicted.

15.4 Frame

Allocation

Basics

15.4.1 Frame

A frame is a fixed-size block in physical memory.

15.4.2 Page

A page is a fixed-size block in virtual memory.

15.4.3 Relationship

[Page $\rightarrow$ Frame]

Each loaded page occupies exactly one frame.

15.4.4 Allocation

Strategies

- Equal Allocation
- Proportional Allocation
- Priority-Based Allocation

15.5 Page

Faults

15.5.1 Definition

A page fault occurs when a referenced page is not present in memory.

15.5.2 Steps

1. Trap to OS.
2. Locate page on disk.
3. Select victim page.
4. Load required page.
5. Resume execution.

15.5.3 Impact

Page faults are expensive because disk access is much slower than RAM access.

15.6 Reference

Strings

15.6.1 Definition

A reference string is a sequence of page references generated by a process.

15.6.2 Example

[7,0,1,2,0,3,0,4,2,3,0,3,2]

This sequence is used to analyze replacement algorithms.

15.7 Performance

Considerations

Important metrics include:

- Number of page faults
- Hit ratio
- Miss ratio
- CPU utilization
- Disk I/O overhead

15.7.1 Goal

Minimize page faults.

15.8 FIFO (First-In First-Out)

15.8.1 Theory

The oldest loaded page is replaced first.

15.8.2 Working Principle

Pages are maintained in queue order.

15.8.3 Example

Frames = 3

Reference String:

[1,2,3,4]

Memory Evolution:

[[1]]

[[1,2]]

[[1,2,3]]

[[4,2,3]]

Page 1 is removed first.

15.8.4 Advantages

- Very simple
- Easy implementation

15.8.5 Disadvantages

- Ignores usage history
- Suffers from Belady's Anomaly

15.8.6 Complexity

[$O(1)$]

with queue implementation.

15.9 Optimal Page Replacement

15.9.1 Theory

Replace the page that will not be used for the longest future duration.

15.9.2 Example

Frames = 3

Reference String:

[7,0,1,2]

Current Memory:

[7,0,1]

Incoming Page:

[2]

Future references indicate page 7 is never used again.

Replace page 7.

15.9.3 Advantages

- Minimum possible page faults
- Benchmark algorithm

15.9.4 Disadvantages

- Requires future knowledge
- Not implementable in practice

15.9.5 Complexity

[$O(n)$]

future scan per replacement.

15.10 LRU (Least Recently Used)

15.10.1 Theory

Replace the page that has not been used for the longest time in the past.

15.10.2 Example

Frames:

[2,3,4]

Recent Usage:

[4,3,2]

Page 2 is least recently used.

Replace page 2.

15.10.3 Advantages

- Good practical approximation
- Stack algorithm

15.10.4 Disadvantages

- Tracking history is expensive

15.10.5 Complexity

[$O(1)$]

with specialized hardware support.

15.11 Second Chance Algorithm

15.11.1 Theory

FIFO enhanced with reference bits.

15.11.2 Working

If Reference Bit = 1:

- Clear bit
- Give page another chance

If Reference Bit = 0:

Replace page.

15.11.3 Advantages

- Better than FIFO
- Simple implementation

15.11.4 Disadvantages

- Approximation only

15.11.5 Complexity

Approximately:

[$O(1)$]

Average case.

15.12 Clock

Algorithm

15.12.1 Theory

Efficient implementation of Second Chance.

15.12.2 Structure

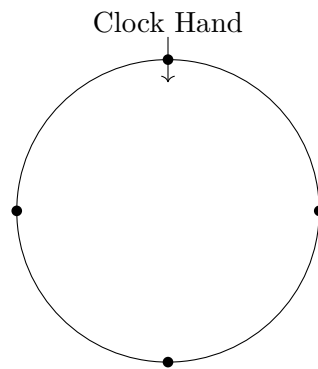
Pages arranged in circular form.

15.12.3 Advantages

- Efficient
- Widely used

15.12.4 Disadvantages

- Approximate LRU

**Figure 15.1:** Clock Page Replacement

15.13 Enhanced Clock Algorithm

15.13.1 Theory

Uses:

- Reference Bit (R)
- Modified Bit (M)

15.13.2 Priority Order

[(0,0)]

[(0,1)]

[(1,0)]

[(1,1)]

Pages with lowest priority are replaced first.

15.13.3 Advantages

- Considers dirty pages
- Better decisions

15.13.4 Disadvantages

- More complexity

15.14 LFU (Least Frequently Used)

15.14.1 Theory

Replace page with lowest access frequency.

15.14.2 Example

Frequency Counts:

[P1=10]

[P2=3]

[P3=7]

Replace:

[P2]

15.14.3 Advantages

- Uses long-term history

15.14.4 Disadvantages

- Old references dominate

15.14.5 Complexity

[$O(\log n)$]

using priority queues.

15.15 MFU (Most Frequently Used)

15.15.1 Theory

Replace the page with highest access count.

15.15.2 Idea

Frequently used pages may have completed their purpose.

15.15.3 Advantages

Useful in specialized workloads.

15.15.4 Disadvantages

Generally performs worse than LRU.

15.15.5 Complexity

[$O(\log n)$]

15.16 NRU (Not Recently Used)

15.16.1 Theory

Uses:

- Reference Bit
- Modified Bit

Pages classified into four classes.

15.16.2 Classes

[(0,0)]

[(0,1)]

[(1,0)]

[(1,1)]

Replace lowest class page.

15.16.3 Advantages

- Simple approximation

15.16.4 Disadvantages

- Coarse-grained decisions

15.17 Random Replacement

15.17.1 Theory

Select victim page randomly.

15.17.2 Advantages

- Extremely simple
- No bookkeeping

15.17.3 Disadvantages

- Unpredictable performance

15.17.4 Complexity

[$O(1)$]

15.18 Solved Numerical Example 1

Frames = 3

Reference String:

[1,2,3,4,1,2,5]

FIFO Simulation:

1 F

2 F

3 F

4 F

1 F

2 F

5 F

Total Page Faults:

[7]

15.19 Solved Numerical Example 2

Frames = 3

Reference String:

[1,2,3,1,4,5]

LRU Simulation:

Faults:

[1,2,3,4,5]

Total:

[5]

15.20 Solved Numerical Example 3

Frames = 4

Reference String:

[7,0,1,2,0,3,0,4]

Optimal replacement yields minimum page faults.

Result:

[6]

page faults.

15.21 Belady's Anomaly

15.21.1 Definition

Increasing the number of frames causes more page faults.

15.21.2 Example

FIFO:

Frames = 3

Page Faults = 9

Frames = 4

Page Faults = 10

More memory produced worse performance.

15.21.3 Observation

Occurs in:

- FIFO

Does not occur in:

- LRU
- Optimal

15.22 Stack

Algorithms

15.22.1 Definition

Algorithms where pages in (n) frames are always a subset of pages in $(n+1)$ frames.

15.22.2 Examples

- LRU
- Optimal

15.22.3 Property

Never exhibit Belady's Anomaly.

15.23 Working Set Based Replacement

15.23.1 Theory

Maintain set of pages actively used by a process.

15.23.2 Working Set

Pages referenced during recent interval.

$$[W(t, \Delta)]$$

15.23.3 Advantages

- Prevents thrashing
- Good locality support

15.23.4 Disadvantages

- Tracking overhead

15.24 Page

Buffering

15.24.1 Definition

Maintain pool of free frames for quick allocation.

15.24.2 Benefits

- Faster page fault handling
- Reduced latency

15.24.3 Working

Victim pages moved to buffer before permanent removal.

15.25 Algorithm

Comparison

Algorithm	Optimality	Practical	Belady
Poor	Yes	Yes	FIFO
No	No	Optimal	Best
No	LRU	Excellent	Yes
Second Chance	Good	Yes	Rare
Good	Yes	Rare	Clock
		Enhanced	Better
Yes	Rare	Clock	Limited
No	LFU	Moderate	
MFU	Moderate	Limited	No
Moderate	Yes	No	NRU
		Random	Weak
Yes	No		

Table 15.1: Page Replacement Comparison

15.26 Interview Shortcuts

15.26.1 Shortcut 1

Optimal always gives minimum page faults.

15.26.2 Shortcut 2

LRU is a stack algorithm.

15.26.3 Shortcut 3

FIFO may exhibit Belady's Anomaly.

15.26.4 Shortcut 4

Clock = Efficient Second Chance.

15.26.5 Shortcut 5

Working Set helps prevent thrashing.

15.27 Key Takeaways

- Page replacement handles full memory situations.
- FIFO is simple but imperfect.
- Optimal provides theoretical lower bound.
- LRU is widely used.
- Clock approximates LRU efficiently.
- LFU uses frequency information.
- NRU uses reference and modify bits.
- Belady's Anomaly affects FIFO.
- Stack algorithms avoid Belady's Anomaly.
- Working Set reduces thrashing.

15.28 25 Interview Questions with Answers

1. What is page replacement? Answer: Selecting a victim page during page faults.
2. Why is page replacement needed? Answer: Physical memory is limited.

3. What is a page fault? Answer: Referenced page not present in memory.
4. What is a frame? Answer: Physical memory block.
5. What is FIFO? Answer: First-In First-Out replacement.
6. FIFO disadvantage? Answer: Belady's Anomaly.
7. What is Optimal replacement? Answer: Replace farthest future page.
8. Why is Optimal impractical? Answer: Future knowledge unavailable.
9. What is LRU? Answer: Least Recently Used.
10. LRU approximates what principle? Answer: Temporal locality.
11. What is Second Chance? Answer: FIFO with reference bits.
12. What is Clock algorithm? Answer: Circular Second Chance.
13. Enhanced Clock uses which bits? Answer: Reference and Modified.
14. LFU means? Answer: Least Frequently Used.
15. MFU means? Answer: Most Frequently Used.
16. NRU means? Answer: Not Recently Used.
17. What is Random replacement? Answer: Random victim selection.
18. What is Belady's Anomaly? Answer: More frames causing more faults.
19. Which algorithm suffers from Belady? Answer: FIFO.
20. What is a stack algorithm? Answer: Subset property algorithm.
21. Give stack algorithm examples. Answer: LRU, Optimal.
22. Working Set purpose? Answer: Prevent thrashing.
23. What is page buffering? Answer: Maintain free-frame pool.
24. Best theoretical algorithm? Answer: Optimal.
25. Most commonly discussed practical algorithm? Answer: LRU.

15.29 25 MCQs with Answers

1. FIFO stands for: Answer: First In First Out
2. LRU stands for: Answer: Least Recently Used
3. LFU stands for: Answer: Least Frequently Used
4. MFU stands for: Answer: Most Frequently Used
5. NRU stands for: Answer: Not Recently Used
6. Optimal replacement requires: Answer: Future knowledge
7. Page fault occurs when: Answer: Page absent from memory
8. Frame belongs to: Answer: Physical Memory
9. Page belongs to: Answer: Virtual Memory
10. FIFO suffers from: Answer: Belady's Anomaly
11. LRU is: Answer: Stack Algorithm
12. Optimal is: Answer: Stack Algorithm

13. Clock is based on: Answer: Second Chance
14. Enhanced Clock uses: Answer: R and M bits
15. Working Set helps prevent: Answer: Thrashing
16. Random replacement complexity: Answer: $O(1)$
17. LRU exploits: Answer: Temporal Locality
18. FIFO uses: Answer: Queue
19. LFU uses: Answer: Frequency Count
20. Optimal gives: Answer: Minimum Faults
21. Belady's anomaly affects: Answer: FIFO
22. Page buffering improves: Answer: Fault Handling
23. NRU classification uses: Answer: Two Bits
24. Clock structure is: Answer: Circular
25. Best benchmark algorithm: Answer: Optimal

15.30 Revision

Notes

- Page replacement occurs after page faults.
- FIFO removes oldest page.
- Optimal removes farthest future page.
- LRU removes least recently used page.
- Clock approximates LRU.
- LFU uses access frequency.
- NRU uses reference and modified bits.
- FIFO may show Belady's Anomaly.
- LRU and Optimal are stack algorithms.
- Working Set helps prevent thrashing.

15.31 Cheat

Sheet

FIFO	Oldest Page First
Farthest Future Use	Optimal
LRU	Least Recent Use
FIFO + Reference Bit	Second Chance
Clock	Circular Second Chance
R Bit + M Bit	Enhanced Clock
LFU	Lowest Frequency
Highest Frequency	MFU
NRU	Not Recently Used
Random Victim	Random
Belady's Anomaly	More Frames, More Faults
LRU, Optimal	Stack Algorithms
Working Set	Active Pages
Free Frame Pool	Page Buffering
Goal	Minimize Page Faults

Memory Allocation Strategies

16.1 Introduction

Memory allocation is the process of assigning portions of memory to programs, processes, threads, kernel subsystems, and data structures. Efficient allocation is critical for performance, scalability, and memory utilization.

Memory allocators aim to balance:

- Fast allocation
- Fast deallocation
- Low fragmentation
- Good cache locality
- Scalability on multicore systems

16.2 Memory

Allocation

Overview

Memory allocation occurs at multiple levels:

1. Physical memory allocation
2. Virtual memory allocation
3. Kernel memory allocation
4. User-space heap allocation



Figure 16.1: Memory Allocation Layers

16.3 Fixed

Partition

Allocation

Physical memory is divided into fixed-size partitions.

Each process is assigned one partition.

16.3.1 Example

Memory Size:

64 MB

Partitions:

16 MB, 16 MB, 16 MB, 16 MB

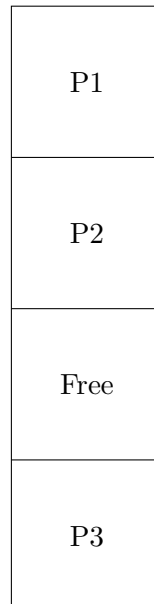


Figure 16.2: Fixed Partition Allocation

16.3.2 Advantages

- Simple implementation
- Fast allocation

16.3.3 Disadvantages

- Internal fragmentation
- Limited flexibility

16.4 Dynamic Partition Allocation

Partitions are created dynamically according to process requirements.

16.4.1 Example

Processes:

10MB, 20MB, 15MB

Memory allocated exactly as needed.

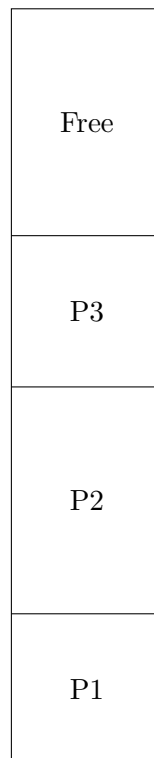


Figure 16.3: Dynamic Partition Allocation

16.4.2 Advantages

- Better utilization
- Flexible sizing

16.4.3 Disadvantages

- External fragmentation

16.5 First

Fit

Allocate the first free block large enough to satisfy the request.

16.5.1 Example

Free blocks:

100, 500, 200, 300, 600

Request:

212

Allocated from:

500

Remaining:

288

16.5.2 Advantages

- Fast
- Simple

16.5.3 Disadvantages

- Fragmentation near beginning

16.6 Best

Fit

Allocate the smallest free block that satisfies the request.

16.6.1 Example

Blocks:

100, 500, 200, 300, 600

Request:

212

Chosen block:

300

Remaining:

88

16.6.2 Advantages

- Less wasted space per allocation

16.6.3 Disadvantages

- Slower search
- Many tiny fragments

16.7 Worst

Fit

Allocate the largest available block.

16.7.1 Example

Request:

212

Chosen block:

600

Remaining:

388

16.7.2 Advantages

- Leaves large usable blocks

16.7.3 Disadvantages

- Generally poor utilization

16.8 Next

Fit

Variation of First Fit.

Search begins from the last allocated location.

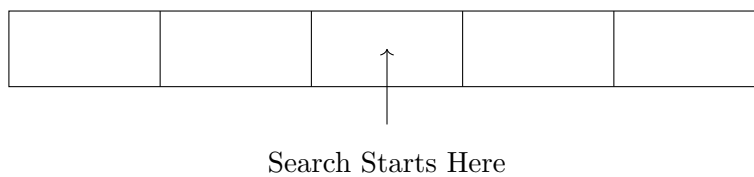


Figure 16.4: Next Fit Search

16.8.1 Advantages

- Avoids repeated scanning

16.8.2 Disadvantages

- Can increase fragmentation

16.9 Buddy

System

Memory is managed in power-of-two blocks.

Available sizes:

$$2^k$$

Example:

1024KB

Split into:

$$512 + 512$$

Then:

$$256 + 256$$

Until suitable size found.

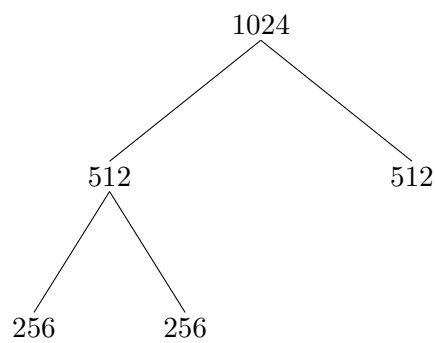


Figure 16.5: Buddy Allocation Splitting

16.9.1 Advantages

- Fast splitting
- Fast merging

16.9.2 Disadvantages

- Internal fragmentation

16.10 Slab

Allocation

Used heavily in kernels.

Stores pre-allocated objects of identical size.

Examples:

- Process descriptors
- Inodes
- File structures

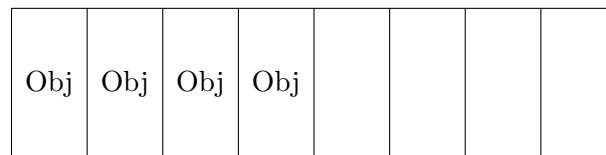


Figure 16.6: Slab Allocator

16.10.1 Advantages

- Low overhead
- Excellent cache locality
- Reduced fragmentation

16.11 Kernel

Memory

Allocation

Kernel allocations differ from user-space allocations.

Requirements:

- Low latency
- Predictability
- Concurrency support

Kernel allocators often use:

- Buddy allocator
- Slab allocator
- SLUB

- SLOB

16.12 Heap Management

Heap stores dynamically allocated memory.

Common operations:

- malloc()
- calloc()
- realloc()
- free()

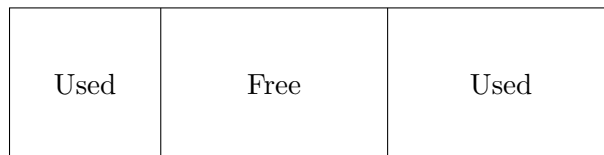


Figure 16.7: Heap Layout

16.13 Free Lists

Free memory blocks are maintained in linked lists.

16.13.1 Example

100 → 200 → 500 → 300

Allocation traverses the list to find a suitable block.

16.13.2 Advantages

- Simple
- Flexible

16.13.3 Disadvantages

- Search overhead

16.14 Fragmentation

Analysis

16.14.1 Internal

Fragmentation

Allocated block larger than requested.

Example:

$Request = 10KB$

$Allocated = 16KB$

Waste:

$6KB$

16.14.2 External

Fragmentation

Enough total memory exists but is scattered.

Free	Used	Free	Used	Free	
------	------	------	------	------	--

Figure 16.8: External Fragmentation

16.15 Memory

Pools

Preallocated memory regions for fixed-size allocations.

Advantages:

- Constant-time allocation
- Predictable performance
- Reduced fragmentation

Used in:

- Game engines
- Networking systems

- Embedded systems

16.16 Allocators in Modern Systems

Modern allocators emphasize:

- Thread scalability
- NUMA awareness
- Cache locality
- Low contention

Features:

- Per-thread caches
- Lock-free structures
- Object reuse

16.17 Linux Memory Allocation

Linux uses multiple allocators.

Physical memory:

- Buddy allocator

Kernel objects:

- SLAB
- SLUB
- SLOB

User-space:

- glibc malloc
- jemalloc
- tcmalloc

16.18 malloc Internals Overview

Typical malloc implementation:

1. Request arrives

2. Search free list/bin
3. Split large block if necessary
4. Return pointer
5. Merge adjacent blocks on free

16.18.1 Common Data Structures

- Free lists
- Bins
- Arenas
- Metadata headers

16.19 jemalloc and tcmalloc Overview

16.19.1 jemalloc

Developed for scalability.

Features:

- Multiple arenas
- Reduced lock contention
- Good fragmentation behavior

Used by:

- FreeBSD
- Firefox
- Various databases

16.19.2 tcmalloc

Developed by Google.

Features:

- Thread-local caches
- Fast allocation
- Efficient multicore scaling

Used by:

- Large-scale backend systems

- Performance-sensitive services

16.20 Detailed Comparison of Allocation Methods

Method	Speed	Fragmentation	Complexity	Typical Usage
Fixed Partition	High	Internal	Low	Legacy Systems
Dynamic Parti- tion	Medium	External	Medium	General Alloca- tion
First Fit	High	Medium	Low	OS Allocation
Best Fit	Low	High Tiny Frag- ments	Medium	Research
Worst Fit	Medium	High	Medium	Educational
Next Fit	Medium	Medium	Low	Variants
Buddy	High	Internal	Medium	Kernel Memory
Slab	Very High	Very Low	Medium	Kernel Objects
Memory Pool	Very High	Low	Medium	Real-Time Sys- tems
jemalloc	Very High	Low	High	Servers
tcmalloc	Very High	Low	High	Large Services

16.21 Performance

Analysis

Allocator	Allocation Speed	Scalability
First Fit	Good	Moderate
Best Fit	Poor	Moderate
Buddy	Good	Good
Slab	Excellent	Excellent
jemalloc	Excellent	Excellent
tcmalloc	Excellent	Excellent

16.22 Interview

Discussions

16.22.1 Why is Buddy Allocation Popular in Kernels?

- Fast splitting
- Fast merging
- Efficient physical page management

16.22.2 Why is Slab Allocation Used?

- Frequent object reuse
- Better cache locality
- Lower fragmentation

16.22.3 Why Do Modern Servers Use jemalloc or tcmalloc?

- Better multicore scalability
- Reduced contention
- Improved performance

16.23 Key Takeaways

Part VI

File Systems

File System Fundamentals

17.1 Introduction

A file system is one of the most important components of an operating system. It provides a structured way to store, organize, retrieve, protect, and manage data on storage devices such as hard disks, SSDs, USB drives, and network storage systems.

For Software Development Engineer interviews, file systems are frequently discussed because they directly impact storage efficiency, application performance, reliability, security, and scalability.

17.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand the purpose of file systems.
- Explain file concepts and attributes.
- Describe common file operations.
- Differentiate file access methods.
- Analyze directory structures.
- Understand metadata and permissions.
- Explain hard links and symbolic links.
- Solve common interview questions on file systems.

17.3 What is a File System

17.3.1 Definition

A File System is a collection of methods and data structures that an operating system uses to store, organize, retrieve, and manage files on storage devices.

17.3.2 Responsibilities

- File Storage
- File Retrieval
- Directory Management
- Access Control
- Space Management
- Metadata Maintenance
- Data Integrity

17.3.3 Examples

Operating System	File System
Linux	ext4, XFS, Btrfs
Windows	NTFS, FAT32, exFAT
macOS	APFS, HFS+

Table 17.1: Common File Systems

17.4 File

Concept

17.4.1 Definition

A file is a named collection of related information stored on secondary storage.

17.4.2 Examples

- source.cpp
- report.pdf
- image.jpg
- database.db
- video.mp4

17.4.3 Real-World

Analogy

Think of a file cabinet:

- Cabinet = Storage Device
- Folder = Directory
- Document = File

17.5 File

Attributes

Every file contains metadata called file attributes.

17.5.1 Common

Attributes

- Name
- Identifier
- Type
- Location
- Size
- Creation Time
- Modification Time
- Access Time
- Owner
- Permissions

17.5.2 Example

Linux:

```
-rw-r--r-- 1 user user 2048 Jan 1 notes.txt
```

Contains:

- Permissions
- Owner
- Size
- Timestamp
- Filename

17.6 File

Operations

Common file operations include:

1. Create
2. Open
3. Read
4. Write

5. Append
6. Seek
7. Rename
8. Close
9. Delete
10. Truncate

17.6.1 Linux

Example

Create file:

```
touch notes.txt
```

Delete file:

```
rm notes.txt
```

17.6.2 Windows

Example

Create file:

```
type nul > notes.txt
```

Delete file:

```
del notes.txt
```

17.7 Access

Methods

File systems provide different ways to access data.

1. Sequential Access
2. Direct Access
3. Indexed Access

17.8 Sequential

Access

17.8.1 Definition

Records are accessed in sequence.

17.8.2 Example

Reading a text file line-by-line.



Figure 17.1: Sequential Access

17.8.3 Advantages

- Simple
- Efficient for streaming

17.8.4 Disadvantages

- Slow random access

17.9 Direct

Access

17.9.1 Definition

Any record can be accessed directly.

17.9.2 Example

Database record lookup.



Figure 17.2: Direct Access

17.9.3 Advantages

- Fast retrieval
- Suitable for databases

17.9.4 Disadvantages

- Additional indexing overhead

17.10 Indexed

Access

17.10.1 Definition

An index maps records to storage locations.

17.10.2 Example

Database B+ Tree index.

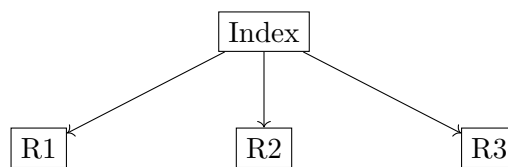


Figure 17.3: Indexed Access

17.10.3 Advantages

- Efficient searching
- Supports large datasets

17.10.4 Disadvantages

- Extra storage for index

17.11 Directory

Structures

Directories organize files into a hierarchy.

Common structures:

1. Single-Level Directory
2. Two-Level Directory
3. Tree-Structured Directory
4. Acyclic Graph Directory
5. General Graph Directory

17.12 Single-Level

Directory

17.12.1 Definition

All files reside in one directory.

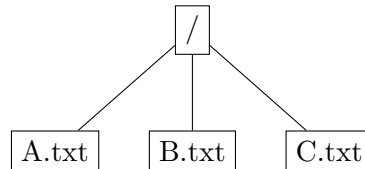


Figure 17.4: Single-Level Directory

17.12.2 Advantages

- Simple

17.12.3 Disadvantages

- Filename conflicts
- Poor scalability

17.13 Two-Level

Directory

17.13.1 Definition

Each user has a separate directory.

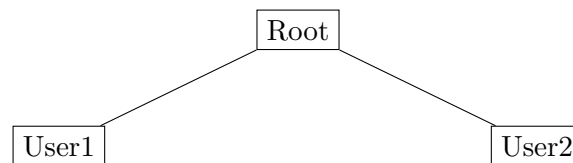


Figure 17.5: Two-Level Directory

17.13.2 Advantages

- Avoids name conflicts

17.13.3 Disadvantages

- Limited organization

17.14 Tree-Structured

Directory

17.14.1 Definition

Directories may contain files and subdirectories.

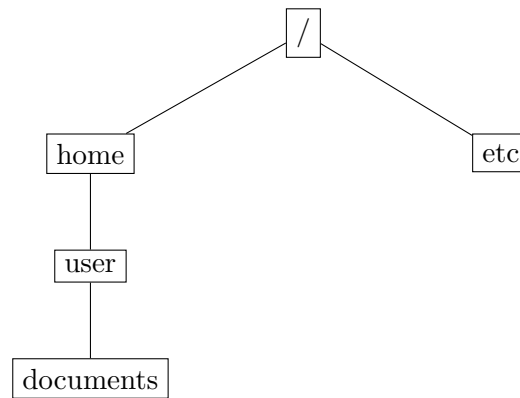


Figure 17.6: Tree-Structured Directory

17.14.2 Advantages

- Highly scalable
- Natural hierarchy

17.14.3 Disadvantages

- More complex traversal

17.15 Acyclic

Graph

Directories

17.15.1 Definition

Allows shared files and directories without cycles.

17.15.2 Advantages

- Resource sharing
- No infinite traversal loops

17.15.3 Disadvantages

- More complex management

17.16 General Graph Directories

17.16.1 Definition

Allows arbitrary links, including cycles.

17.16.2 Advantages

- Maximum flexibility

17.16.3 Disadvantages

- Infinite traversal risk
- Garbage collection challenges

17.17 File Metadata

17.17.1 Definition

Metadata is information about a file rather than file content.

17.17.2 Examples

- Owner
- Size
- Permissions
- Creation Time
- Modification Time

Linux command:

```
stat file.txt
```

17.18 File Permissions

17.18.1 Linux Permissions

```
-rwxr-xr--
```

17.18.2 Meaning

Symbol	Meaning
r	Read
w	Write
x	Execute

Table 17.2: Permission Symbols

17.18.3 Permission

Groups

- Owner
- Group
- Others

17.19 Links

A link is a directory entry referencing a file.

Types:

1. Hard Link
2. Symbolic Link

17.20 Hard

Links

17.20.1 Definition

A hard link is another name for the same inode.

17.20.2 Linux

Example

```
ln file1 file2
```

17.20.3 Properties

- Same inode
- Cannot span file systems
- Usually not allowed for directories

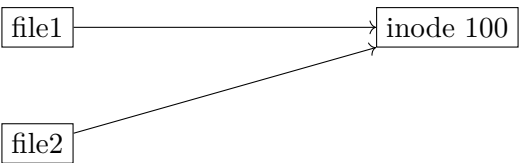


Figure 17.7: Hard Links

17.20.4 Illustration

17.21 Symbolic

Links

17.21.1 Definition

A symbolic link stores a pathname pointing to another file.

17.21.2 Linux

Example

```
ln -s target link
```

17.21.3 Properties

- Different inode
- Can span file systems
- Can point to directories
- Can become dangling

17.21.4 Illustration



Figure 17.8: Symbolic Link

17.22 Hard

Link

vs

Soft

Link

Feature	Hard Link	Soft Link
References	Inode	Pathname
Same Inode	Yes	No
Cross Filesystems	No	Yes
Directory Support	Usually No	Yes
Dangling Possible	No	Yes
Delete Original	Still Works	Breaks Link

Table 17.3: Hard Link vs Symbolic Link

17.23 Linux

Examples

List files:

```
ls
```

Detailed listing:

```
ls -l
```

Create directory:

```
mkdir project
```

Show metadata:

```
stat file.txt
```

Create hard link:

```
ln a.txt b.txt
```

Create symbolic link:

```
ln -s a.txt link.txt
```

17.24 Windows

Examples

Create directory:

```
mkdir project
```

List files:

```
dir
```

Create symbolic link:

```
mklink link.txt target.txt
```

Create directory junction:

```
mklink /J linkdir targetdir
```

17.25 Directory Structure Comparison

Structure	Advantage	Disadvantage
Single-Level	Simple	Name Conflicts
Two-Level	User Isolation	Limited Hierarchy
Tree	Scalable	More Complex
Acyclic Graph	Sharing	Management Overhead
General Graph	Flexible	Cycles Possible

Table 17.4: Directory Structure Comparison

17.26 Interview Focus

17.26.1 Frequently Asked Topics

- Hard Link vs Soft Link
- Directory Structures
- Inodes
- Metadata
- File Permissions
- Access Methods

17.26.2 Golden Rule

Hard Link = Same Inode

Soft Link = Path Reference

17.27 Key Takeaways

- File systems organize and manage storage.
- Files are named collections of data.
- Metadata describes file properties.
- Sequential, direct, and indexed access are common methods.
- Tree directories dominate modern systems.
- Hard links share the same inode.
- Symbolic links store paths.
- Linux and Windows support hierarchical directories.
- Permissions provide security.
- Directory design affects scalability.

17.28 20 Interview Questions with Answers

1. What is a file system?
Answer: A mechanism for organizing and managing files on storage devices.
2. What is a file?
Answer: A named collection of related data.
3. What are file attributes?
Answer: Metadata such as size, owner, permissions, and timestamps.
4. What is metadata?
Answer: Information about a file.
5. What is sequential access?
Answer: Accessing records in order.
6. What is direct access?
Answer: Accessing any record directly.
7. What is indexed access?
Answer: Using an index for retrieval.
8. What is a directory?
Answer: A structure that organizes files.
9. What is a single-level directory?
Answer: All files stored in one directory.
10. What is a two-level directory?
Answer: Separate directories per user.
11. What is a tree directory?
Answer: Hierarchical directory structure.
12. What is an acyclic graph directory?
Answer: Shared files without cycles.
13. What is a general graph directory?
Answer: Directory structure allowing cycles.
14. What are file permissions?
Answer: Access control settings.
15. What does chmod modify?
Answer: File permissions.
16. What is a hard link?
Answer: Another name for the same inode.
17. What is a symbolic link?
Answer: A path reference to another file.
18. Can hard links cross file systems?
Answer: No.
19. Can symbolic links cross file systems?
Answer: Yes.

20. Which directory structure is most common?

Answer: Tree-structured directory.

17.29 20 MCQs with Answers

1. A file system manages: A) CPU B) Memory C) Storage D) Cache

Answer: C

2. File metadata contains: Answer: File Attributes

3. Sequential access reads: Answer: In Order

4. Direct access allows: Answer: Random Retrieval

5. Indexed access uses: Answer: Index Structure

6. Tree directories are: Answer: Hierarchical

7. Hard links share: Answer: Inode

8. Symbolic links store: Answer: Pathname

9. Linux file permissions include: Answer: r, w, x

10. chmod changes: Answer: Permissions

11. Hard links can cross filesystems: Answer: No

12. Soft links can cross filesystems: Answer: Yes

13. Dangling links occur with: Answer: Symbolic Links

14. Common Linux filesystem: Answer: ext4

15. Common Windows filesystem: Answer: NTFS

16. File size is: Answer: Metadata

17. Tree structure provides: Answer: Scalability

18. Single-level directories suffer: Answer: Name Conflicts

19. Acyclic graph allows: Answer: Sharing

20. Most common directory structure: Answer: Tree

17.30 Revision

Notes

- File systems organize storage.
- Files contain data.
- Metadata describes files.
- Access methods include sequential, direct, and indexed.
- Directories organize files.
- Tree structure is most common.
- Hard links share inodes.
- Symbolic links store paths.
- Permissions provide security.

- Linux and Windows use hierarchical directories.

17.31 Cheat

Sheet

File	Named Collection of Data
Metadata	Data About Data
Sequential Access	Ordered Access
Direct Access	Random Access
Indexed Access	Index-Based Access
Single-Level Directory	One Directory
Two-Level Directory	Per User Directory
Tree Directory	Hierarchical Structure
Acyclic Graph	Sharing Without Cycles
General Graph	Cycles Allowed
Hard Link	Same Inode
Symbolic Link	Path Reference
Hard Link Cross FS	No
Soft Link Cross FS	Yes
Permissions	r, w, x
Linux FS	ext4
Windows FS	NTFS

File System Implementation

18.1 Introduction

A file system is responsible for organizing, storing, retrieving, protecting, and managing data on storage devices. File system implementation determines how files are represented on disk, how free space is tracked, and how reliability is maintained.

Modern file systems must provide:

- Efficient storage utilization
- Fast file access
- Reliability and recovery
- Scalability
- Security and permissions

18.2 File

System

Architecture

A typical file system consists of several layers.

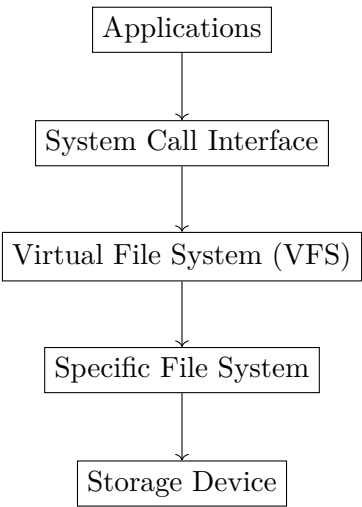


Figure 18.1: Layered File System Architecture

Responsibilities include:

- Namespace management
- Metadata management
- Data allocation
- Caching
- Recovery

18.3 On-Disk

Structures

On-disk structures persist across system reboots.

Common structures include:

- Boot Block
- Superblock
- Inode Table
- Data Blocks
- Journals
- Free Space Information

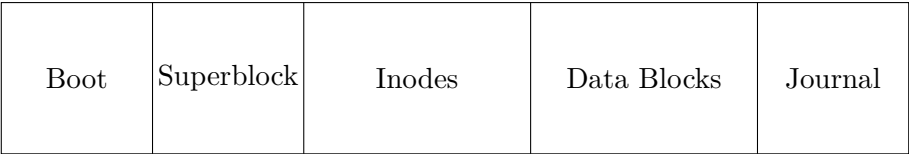


Figure 18.2: Typical On-Disk Layout

18.4 In-Memory

Structures

To improve performance, operating systems maintain memory-resident structures.

Examples:

- Mount table
- Buffer cache
- Open file table
- Directory cache
- Inode cache

18.5 Boot

Block

The boot block resides at the beginning of the storage device.

Purpose:

- Contains bootstrap code
- Used during system startup
- Loads operating system kernel

18.6 Superblock

The superblock stores global file system metadata.

Typical contents:

- File system size
- Block size
- Free block count
- Free inode count
- Mount information
- File system state

Field	Description
Block Size	Size of disk block
Total Blocks	Total blocks in file system
Free Blocks	Available blocks
Inode Count	Number of inodes
State	Clean/Dirty state

Table 18.1: Typical Superblock Fields

18.7 Inodes

An inode is a data structure describing a file.

An inode stores:

- Ownership
- Permissions
- File size
- Timestamps
- Block pointers

An inode does not store the filename.

Permissions
Owner
File Size
Timestamps
Direct Blocks
Indirect Blocks

Figure 18.3: Simplified Inode Structure

18.8

File Control Blocks

A File Control Block (FCB) stores file metadata.

Information includes:

- File attributes
- Location
- Ownership
- Access rights

Unix systems typically use inodes instead of traditional FCBs.

18.9

Allocation Methods

Allocation methods determine how file data is placed on disk.

Three classic methods:

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

18.10

Contiguous Allocation

File occupies consecutive disk blocks.

Example:

File A → *Blocks 20 – 29*



Figure 18.4: Contiguous Allocation

Advantages:

- Fast sequential access
- Fast random access

Disadvantages:

- External fragmentation
- Difficult file growth

18.11 Linked

Allocation

Each block contains pointer to next block.

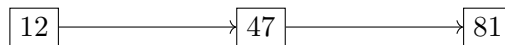


Figure 18.5: Linked Allocation

Advantages:

- No external fragmentation
- Easy growth

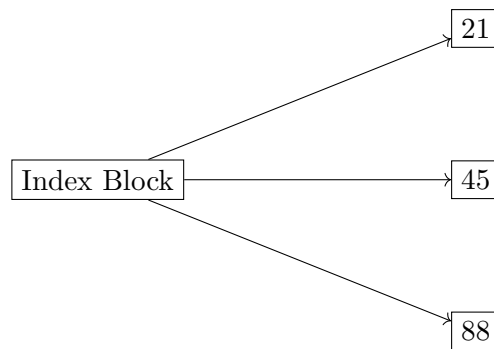
Disadvantages:

- Poor random access
- Pointer overhead

18.12 Indexed

Allocation

A special index block contains addresses of all file blocks.

**Figure 18.6:** Indexed Allocation

Advantages:

- Efficient random access
- No external fragmentation

Disadvantages:

- Index overhead
- Additional block accesses

18.13 Free Space Management

The operating system tracks unused disk blocks.

Requirements:

- Fast allocation
- Fast deallocation
- Low overhead

Common techniques:

- Bitmaps
- Free Lists

18.14 Bitmaps

Each block is represented by one bit.

$0 = \textit{Free}$

1 = *Allocated*

Example:

111001010000111

Advantages:

- Compact
- Efficient searching

1	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---

Figure 18.7: Bitmap Free Space Tracking

18.15 Free

Lists

Free blocks are linked together.

Example:

14 → 27 → 65 → 102

Advantages:

- Simple implementation

Disadvantages:

- Slower search
- Poor locality

18.16 Journaling

Journaling improves reliability.

Before modifying metadata:

1. Write operation to journal
2. Commit journal entry
3. Apply actual changes

Advantages:

- Faster recovery
- Reduced corruption



Figure 18.8: Journaling Workflow

18.17 Log Structured File Systems

Log Structured File Systems (LFS) treat storage as an append-only log.

Characteristics:

- Sequential writes
- Better write performance
- Crash recovery support

Challenges:

- Cleaning old segments
- Space reclamation

18.18 ext4

Overview

ext4 is one of the most widely used Linux file systems.

Features:

- Journaling
- Extents
- Delayed allocation
- Large file support
- Fast fsck

Advantages:

- Mature

- Reliable
- High performance

18.19 NTFS

Overview

NTFS is the default Windows file system.

Features:

- Journaling
- Access Control Lists
- Compression
- Encryption
- Large volume support

Central structure:

- Master File Table (MFT)

18.20 File

System

Consistency

Consistency means metadata correctly reflects file system state.

Potential issues:

- Power failure
- Kernel crash
- Incomplete writes

Consistency mechanisms:

- Journaling
- Checksums
- Recovery tools

18.21 fsck

Overview

fsck (File System Check) repairs inconsistent file systems.

Responsibilities:

- Verify metadata

- Repair inode errors
- Recover orphaned files
- Correct allocation maps

Typical checks:

1. Superblock validation
2. Inode validation
3. Block consistency
4. Directory consistency
5. Reference count verification

18.22 Dedicated Interview Section: How Linux Stores Files Internally

Linux stores files using inodes.

When a file is created:

1. Allocate inode
2. Allocate data blocks
3. Create directory entry
4. Associate filename with inode number

Example:

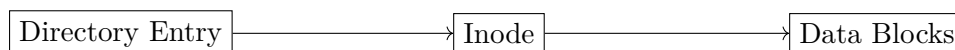


Figure 18.9: Linux File Storage Path

Important interview fact:

- Filename is stored in directory entries.
- Metadata is stored in inode.
- Actual contents are stored in data blocks.

18.23 Allocation Method Comparison

Method	Random Access	Growth	Fragmentation
Contiguous	Excellent	Poor	External
Linked	Poor	Excellent	None
Indexed	Good	Good	None

Table 18.2: Allocation Method Comparison

18.24 Key Takeaways

Disk Management and Disk Scheduling

19.1 Introduction

Storage systems are critical components of modern operating systems. Efficient disk management directly affects application performance, database throughput, file system responsiveness, and overall system efficiency.

This chapter covers traditional magnetic disks, modern SSDs, disk scheduling algorithms, RAID technologies, and interview-focused numerical problem solving.

19.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand magnetic disk organization.
- Explain SSD architecture.
- Calculate seek times and head movements.
- Solve disk scheduling numericals.
- Compare scheduling algorithms.
- Understand RAID levels and fault tolerance.
- Explain wear leveling and TRIM.
- Answer common interview questions.

19.3 Magnetic

Disks

Overview

19.3.1 Definition

A magnetic disk stores data using magnetized surfaces on rotating platters.

19.3.2 Components

- Platters
- Spindle
- Read/Write Heads
- Disk Controller

19.3.3 Characteristics

- Mechanical movement
- Large storage capacity
- Low cost per GB
- Higher latency than SSDs

19.4 SSD

Overview

19.4.1 Definition

Solid State Drives (SSDs) store data in flash memory rather than magnetic platters.

19.4.2 Characteristics

- No moving parts
- Lower latency
- Faster reads
- Lower power consumption
- More expensive per GB

19.4.3 Advantages

over

HDDs

- Faster boot times
- Better random access
- Silent operation
- Shock resistance

19.5 Disk

Structure

Magnetic disks are organized into:

- Platters
- Tracks
- Sectors
- Cylinders

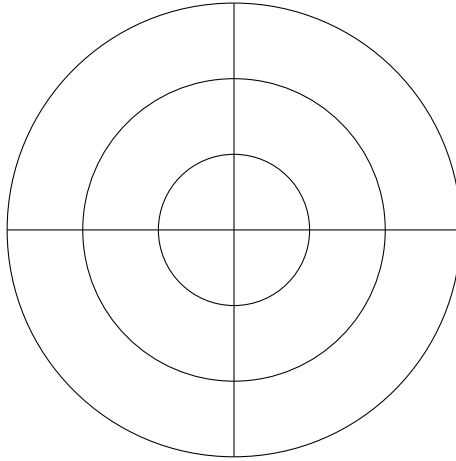


Figure 19.1: Disk Structure

19.6 Tracks

19.6.1 Definition

A track is a circular path on the disk surface where data is recorded.

19.6.2 Characteristics

- Concentric circles
- Divided into sectors
- Multiple tracks per platter

19.7 Sectors

19.7.1 Definition

A sector is the smallest physical storage unit on a disk.

19.7.2 Typical

Sizes

- 512 Bytes
- 4 KB (Advanced Format)

19.8 Cylinders

19.8.1 Definition

A cylinder consists of all tracks located at the same radius across multiple platters.

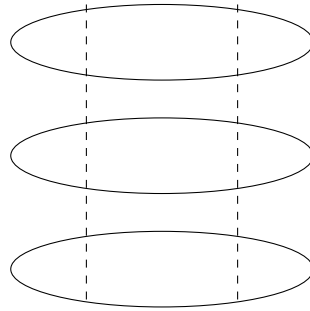


Figure 19.2: Cylinder Across Platters

19.9 Seek

Time

19.9.1 Definition

Time required to move the read/write head to the target track.

19.9.2 Formula

$$\text{Seek Time} \propto \text{Distance Moved}$$

19.9.3 Example

Current Track:

50

Target Track:

120

Head Movement:

$$|120 - 50| = 70$$

tracks.

19.10 Rotational Latency

19.10.1 Definition

Time waiting for the desired sector to rotate under the read/write head.

19.10.2 Average Rotational Latency

$$\frac{1}{2} \times \textit{Rotation Time}$$

19.10.3 Example

7200 RPM disk:

$$\textit{Rotation Time} = \frac{60}{7200}$$

$$= 8.33ms$$

Average Latency:

$$4.17ms$$

19.11 Transfer Time

19.11.1 Definition

Time required to transfer data once the correct sector reaches the head.

19.11.2 Factors

- Data Size
- Disk Speed
- Controller Speed

19.12 Disk Access Time

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

19.13 Disk Scheduling Concepts

Disk scheduling determines the order in which pending disk requests are serviced.

19.13.1 Goals

- Reduce seek time
- Improve throughput
- Reduce waiting time
- Improve response time

19.13.2 Example Queue

98, 183, 37, 122, 14, 124, 65, 67

Initial Head:

53

This queue will be used throughout the chapter.

19.14 FCFS (First Come First Serve)

19.14.1 Working

Requests are serviced in arrival order.

19.14.2 Sequence

$$53 \rightarrow 98 \rightarrow 183 \rightarrow 37$$

$$\rightarrow 122 \rightarrow 14 \rightarrow 124$$

$$\rightarrow 65 \rightarrow 67$$

19.14.3 Head**Movement**

$$45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

$$= 640$$

tracks.

19.14.4 Advantages

- Fair
- Simple

19.14.5 Disadvantages

- Poor performance
- High seek time

19.15 SSTF (Shortest Seek Time First)**19.15.1 Working**

Choose request closest to current head.

19.15.2 Sequence

$$53 \rightarrow 65 \rightarrow 67 \rightarrow 37$$

$\rightarrow 14 \rightarrow 98 \rightarrow 122$

$\rightarrow 124 \rightarrow 183$

19.15.3 Head

Movement

$$12 + 2 + 30 + 23 + 84 + 24 + 2 + 59$$

$$= 236$$

tracks.

19.15.4 Advantages

- Better throughput
- Lower seek time

19.15.5 Disadvantages

- Starvation possible

19.16 SCAN

Algorithm

19.16.1 Working

Disk arm moves in one direction servicing requests.

At end, reverses direction.

19.16.2 Nickname

Elevator Algorithm.

19.16.3 Example

Direction:

→

Sequence:

53 → 65 → 67 → 98

→ 122 → 124 → 183

Then reverse.

19.16.4 Advantages

- Good throughput
- Fair

19.16.5 Disadvantages

- End tracks may wait longer

19.17 CSCAN

Algorithm

19.17.1 Working

Head moves only in one direction.

After reaching end:

- Returns immediately to beginning.
- Continues scanning.

19.17.2 Advantages

- Uniform waiting times

19.17.3 Disadvantages

- Additional movement

19.18 LOOK

Algorithm

19.18.1 Working

Like SCAN but reverses at last request.

Does not move to physical disk end.

19.18.2 Advantages

- Less movement
- Better efficiency

19.18.3 Disadvantages

- Slightly more complex

19.19 CLOOK

Algorithm

19.19.1 Working

Like CSCAN but jumps from last request to first request.

19.19.2 Advantages

- Reduced unnecessary movement

19.19.3 Disadvantages

- More bookkeeping

19.20 Scheduling

Diagram

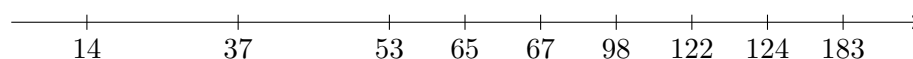


Figure 19.3: Disk Request Positions

19.21 RAID

Levels

19.21.1 Definition

RAID (Redundant Array of Independent Disks) combines multiple disks to improve:

- Performance
- Reliability
- Fault Tolerance

19.22 RAID

0

19.22.1 Concept

Striping only.

Disk 1	Disk 2
A1	A2
B1	B2
C1	C2

Figure 19.4: RAID 0

19.22.2 Advantages

- Highest performance

19.22.3 Disadvantages

- No fault tolerance

19.23 RAID

1

19.23.1 Concept

Mirroring.

Disk 1	Disk 2
A	A
B	B
C	C

Figure 19.5: RAID 1

19.23.2 Advantages

- High reliability

19.23.3 Disadvantages

- 50% storage efficiency

19.24 RAID

5

19.24.1 Concept

Striping with distributed parity.

19.24.2 Characteristics

- Minimum 3 disks
- Survives one disk failure

19.24.3 Advantages

- Good balance

19.24.4 Disadvantages

- Write penalty

19.25 RAID

6

19.25.1 Concept

Dual distributed parity.

19.25.2 Characteristics

- Survives two disk failures

19.25.3 Advantages

- Higher reliability

19.25.4 Disadvantages

- More parity overhead

19.26 RAID

10

19.26.1 Concept

Mirroring + Striping.

RAID1 + RAID0

19.26.2 Advantages

- Excellent performance
- Excellent reliability

19.26.3 Disadvantages

- Expensive

19.27 RAID

Comparison

RAID	Performance	Fault Tolerance	Efficiency
0	Excellent	None	100%
1	Good	High	50%
5	Good	1 Disk	High
6	Moderate	2 Disks	Moderate
10	Excellent	High	50%

Table 19.1: RAID Comparison

19.28 SSD

Internals

SSD consists of:

- NAND Flash
- Controller
- DRAM Cache
- Firmware

19.28.1 Organization

Page → Block → Chip

19.28.2 Characteristics

- Reads fast
- Writes more expensive
- Erase-before-write

19.29 Wear

Leveling

19.29.1 Problem

Flash cells have limited write cycles.

19.29.2 Solution

Wear leveling distributes writes evenly.

19.29.3 Goal

Prevent premature failure.

19.29.4 Types

- Dynamic Wear Leveling
- Static Wear Leveling

19.30 TRIM

19.30.1 Definition

TRIM informs SSD which blocks are no longer needed.

19.30.2 Benefits

- Faster writes
- Better garbage collection
- Longer SSD lifespan

19.30.3 Linux**Example**

```
fstrim -v /
```

19.31 Solved**Numerical****Example**

Requests:

98, 183, 37, 122, 14, 124, 65, 67

Head:

53

Using SSTF:

53 → 65 → 67 → 37

→ 14 → 98 → 122

→ 124 → 183

Total Movement:

236

tracks.

19.32 Interview**Tips**

- SSTF minimizes immediate seek time.

- SCAN is called Elevator Algorithm.
- LOOK avoids unnecessary end movement.
- RAID 0 provides performance only.
- RAID 1 provides mirroring.
- RAID 5 survives one failure.
- RAID 6 survives two failures.
- RAID 10 combines speed and redundancy.
- SSDs do not require traditional disk scheduling.
- Wear leveling is a common interview topic.

19.33 Key

Takeaways

- Disk performance depends on seek time, latency, and transfer time.
- Disk scheduling reduces head movement.
- SSTF improves over FCFS.
- SCAN and LOOK are commonly discussed algorithms.
- RAID improves reliability and performance.
- SSDs differ fundamentally from magnetic disks.
- Wear leveling extends SSD life.
- TRIM improves SSD performance.

19.34 25 Interview Questions with Answers

1. What is seek time?
Answer: Time to move disk head.
2. What is rotational latency?
Answer: Waiting for sector rotation.
3. What is transfer time?
Answer: Time to transfer data.
4. What is FCFS?
Answer: Arrival-order scheduling.
5. What is SSTF?
Answer: Closest request first.
6. SSTF drawback?
Answer: Starvation.
7. SCAN nickname?
Answer: Elevator Algorithm.
8. What is CSCAN?

Answer: Circular SCAN.

9. What is LOOK?

Answer: SCAN without going to disk end.

10. What is CLOOK?

Answer: Circular LOOK.

11. Define RAID.

Answer: Redundant Array of Independent Disks.

12. RAID 0 provides?

Answer: Striping.

13. RAID 1 provides?

Answer: Mirroring.

14. RAID 5 survives?

Answer: One disk failure.

15. RAID 6 survives?

Answer: Two disk failures.

16. RAID 10 combines?

Answer: Mirroring and Striping.

17. SSD stores data using?

Answer: Flash Memory.

18. SSD has moving parts?

Answer: No.

19. What is wear leveling?

Answer: Even write distribution.

20. Why TRIM?

Answer: Identify unused blocks.

21. What is a track?

Answer: Circular recording path.

22. What is a sector?

Answer: Smallest physical unit.

23. What is a cylinder?

Answer: Same-radius tracks.

24. Best FCFS advantage?

Answer: Simplicity.

25. SSD scheduling needed?

Answer: Generally less important.

19.35 25 MCQs with Answers

1. Smallest disk storage unit: Answer: Sector

2. Circular storage path: Answer: Track
3. Same-radius tracks: Answer: Cylinder
4. FCFS stands for: Answer: First Come First Serve
5. SSTF stands for: Answer: Shortest Seek Time First
6. Elevator algorithm: Answer: SCAN
7. Circular SCAN: Answer: CSCAN
8. LOOK is improvement over: Answer: SCAN
9. RAID 0 uses: Answer: Striping
10. RAID 1 uses: Answer: Mirroring
11. RAID 5 tolerates: Answer: One Failure
12. RAID 6 tolerates: Answer: Two Failures
13. RAID 10 combines: Answer: RAID 0 and RAID 1
14. SSD uses: Answer: NAND Flash
15. SSD moving parts: Answer: None
16. Wear leveling improves: Answer: Lifespan
17. TRIM improves: Answer: SSD Performance
18. SSTF may cause: Answer: Starvation
19. SCAN is: Answer: Fairer than SSTF
20. FCFS is: Answer: Simple
21. Average latency depends on: Answer: Rotation Speed
22. Seek time depends on: Answer: Head Movement
23. SSD latency compared to HDD: Answer: Lower
24. RAID improves: Answer: Reliability
25. Disk scheduling mainly reduces: Answer: Seek Time

19.36 Revision

Notes

- Disk access time = Seek + Latency + Transfer.
- SSTF selects nearest request.
- SCAN = Elevator Algorithm.
- LOOK avoids unnecessary movement.
- RAID 0 = Striping.
- RAID 1 = Mirroring.
- RAID 5 = Single parity.
- RAID 6 = Dual parity.
- RAID 10 = Mirroring + Striping.
- SSDs use flash memory.
- Wear leveling extends SSD lifespan.

- TRIM improves SSD efficiency.

19.37 Cheat

Sheet

Track	Circular Path
Sector	Smallest Physical Unit
Cylinder	Same Radius Tracks
Seek Time	Head Movement Time
Rotational Latency	Rotation Wait Time
Transfer Time	Data Transfer Time
FCFS	Arrival Order
SSTF	Closest Request
SCAN	Elevator Algorithm
CSCAN	Circular SCAN
LOOK	Optimized SCAN
CLOOK	Optimized CSCAN
RAID 0	Striping
RAID 1	Mirroring
RAID 5	Single Parity
RAID 6	Dual Parity
RAID 10	Mirroring + Striping
SSD	Flash Storage
Wear Leveling	Even Writes
TRIM	Free Block Notification

Part VII

Input/Output Systems

Disk Management and Disk Scheduling

20.1 Introduction

Storage systems are critical components of modern operating systems. Efficient disk management directly affects application performance, database throughput, file system responsiveness, and overall system efficiency.

This chapter covers traditional magnetic disks, modern SSDs, disk scheduling algorithms, RAID technologies, and interview-focused numerical problem solving.

20.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand magnetic disk organization.
- Explain SSD architecture.
- Calculate seek times and head movements.
- Solve disk scheduling numericals.
- Compare scheduling algorithms.
- Understand RAID levels and fault tolerance.
- Explain wear leveling and TRIM.
- Answer common interview questions.

20.3 Magnetic

Disks

Overview

20.3.1 Definition

A magnetic disk stores data using magnetized surfaces on rotating platters.

20.3.2 Components

- Platters
- Spindle
- Read/Write Heads
- Disk Controller

20.3.3 Characteristics

- Mechanical movement
- Large storage capacity
- Low cost per GB
- Higher latency than SSDs

20.4 SSD

Overview

20.4.1 Definition

Solid State Drives (SSDs) store data in flash memory rather than magnetic platters.

20.4.2 Characteristics

- No moving parts
- Lower latency
- Faster reads
- Lower power consumption
- More expensive per GB

20.4.3 Advantages

over

HDDs

- Faster boot times
- Better random access
- Silent operation
- Shock resistance

20.5 Disk

Structure

Magnetic disks are organized into:

- Platters
- Tracks
- Sectors
- Cylinders

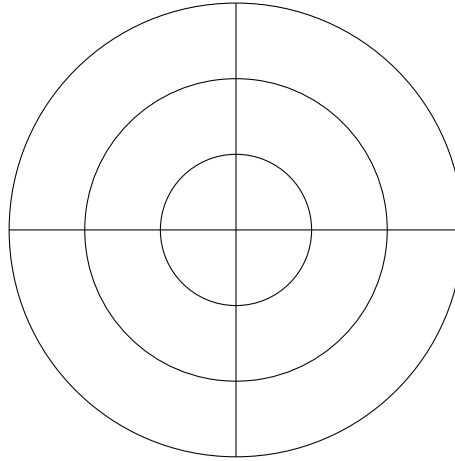


Figure 20.1: Disk Structure

20.6 Tracks

20.6.1 Definition

A track is a circular path on the disk surface where data is recorded.

20.6.2 Characteristics

- Concentric circles
- Divided into sectors
- Multiple tracks per platter

20.7 Sectors

20.7.1 Definition

A sector is the smallest physical storage unit on a disk.

20.7.2 Typical

Sizes

- 512 Bytes
- 4 KB (Advanced Format)

20.8 Cylinders

20.8.1 Definition

A cylinder consists of all tracks located at the same radius across multiple platters.

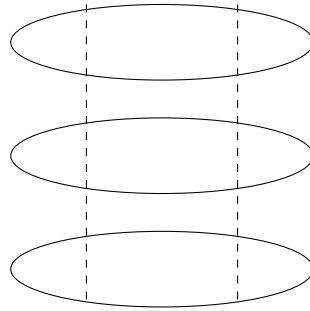


Figure 20.2: Cylinder Across Platters

20.9 Seek

Time

20.9.1 Definition

Time required to move the read/write head to the target track.

20.9.2 Formula

$$\text{Seek Time} \propto \text{Distance Moved}$$

20.9.3 Example

Current Track:

50

Target Track:

120

Head Movement:

$$|120 - 50| = 70$$

tracks.

20.10 Rotational Latency

20.10.1 Definition

Time waiting for the desired sector to rotate under the read/write head.

20.10.2 Average Rotational Latency

$$\frac{1}{2} \times \textit{Rotation Time}$$

20.10.3 Example

7200 RPM disk:

$$\textit{Rotation Time} = \frac{60}{7200}$$

$$= 8.33ms$$

Average Latency:

$$4.17ms$$

20.11 Transfer Time

20.11.1 Definition

Time required to transfer data once the correct sector reaches the head.

20.11.2 Factors

- Data Size
- Disk Speed
- Controller Speed

20.12 Disk Access Time

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

20.13 Disk Scheduling Concepts

Disk scheduling determines the order in which pending disk requests are serviced.

20.13.1 Goals

- Reduce seek time
- Improve throughput
- Reduce waiting time
- Improve response time

20.13.2 Example Queue

98, 183, 37, 122, 14, 124, 65, 67

Initial Head:

53

This queue will be used throughout the chapter.

20.14 FCFS (First Come First Serve)

20.14.1 Working

Requests are serviced in arrival order.

20.14.2 Sequence

$$53 \rightarrow 98 \rightarrow 183 \rightarrow 37$$

$$\rightarrow 122 \rightarrow 14 \rightarrow 124$$

$$\rightarrow 65 \rightarrow 67$$

20.14.3 Head**Movement**

$$45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

$$= 640$$

tracks.

20.14.4 Advantages

- Fair
- Simple

20.14.5 Disadvantages

- Poor performance
- High seek time

20.15 SSTF (Shortest Seek Time First)**20.15.1 Working**

Choose request closest to current head.

20.15.2 Sequence

$$53 \rightarrow 65 \rightarrow 67 \rightarrow 37$$

$\rightarrow 14 \rightarrow 98 \rightarrow 122$

$\rightarrow 124 \rightarrow 183$

20.15.3 Head

Movement

$$12 + 2 + 30 + 23 + 84 + 24 + 2 + 59$$

$$= 236$$

tracks.

20.15.4 Advantages

- Better throughput
- Lower seek time

20.15.5 Disadvantages

- Starvation possible

20.16 SCAN

Algorithm

20.16.1 Working

Disk arm moves in one direction servicing requests.

At end, reverses direction.

20.16.2 Nickname

Elevator Algorithm.

20.16.3 Example

Direction:

→

Sequence:

53 → 65 → 67 → 98

→ 122 → 124 → 183

Then reverse.

20.16.4 Advantages

- Good throughput
- Fair

20.16.5 Disadvantages

- End tracks may wait longer

20.17 CSCAN

Algorithm

20.17.1 Working

Head moves only in one direction.

After reaching end:

- Returns immediately to beginning.
- Continues scanning.

20.17.2 Advantages

- Uniform waiting times

20.17.3 Disadvantages

- Additional movement

20.18 LOOK

Algorithm

20.18.1 Working

Like SCAN but reverses at last request.

Does not move to physical disk end.

20.18.2 Advantages

- Less movement
- Better efficiency

20.18.3 Disadvantages

- Slightly more complex

20.19 CLOOK

Algorithm

20.19.1 Working

Like CSCAN but jumps from last request to first request.

20.19.2 Advantages

- Reduced unnecessary movement

20.19.3 Disadvantages

- More bookkeeping

20.20 Scheduling

Diagram

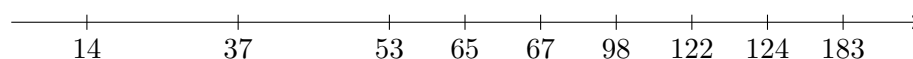


Figure 20.3: Disk Request Positions

20.21 RAID

Levels

20.21.1 Definition

RAID (Redundant Array of Independent Disks) combines multiple disks to improve:

- Performance
- Reliability
- Fault Tolerance

20.22 RAID

0

20.22.1 Concept

Striping only.

Disk 1	Disk 2
A1	A2
B1	B2
C1	C2

Figure 20.4: RAID 0

20.22.2 Advantages

- Highest performance

20.22.3 Disadvantages

- No fault tolerance

20.23 RAID

1

20.23.1 Concept

Mirroring.

Disk 1	Disk 2
A	A
B	B
C	C

Figure 20.5: RAID 1

20.23.2 Advantages

- High reliability

20.23.3 Disadvantages

- 50% storage efficiency

20.24 RAID

5

20.24.1 Concept

Striping with distributed parity.

20.24.2 Characteristics

- Minimum 3 disks
- Survives one disk failure

20.24.3 Advantages

- Good balance

20.24.4 Disadvantages

- Write penalty

20.25 RAID

6

20.25.1 Concept

Dual distributed parity.

20.25.2 Characteristics

- Survives two disk failures

20.25.3 Advantages

- Higher reliability

20.25.4 Disadvantages

- More parity overhead

20.26 RAID

10

20.26.1 Concept

Mirroring + Striping.

RAID1 + RAID0

20.26.2 Advantages

- Excellent performance
- Excellent reliability

20.26.3 Disadvantages

- Expensive

20.27 RAID

Comparison

RAID	Performance	Fault Tolerance	Efficiency
0	Excellent	None	100%
1	Good	High	50%
5	Good	1 Disk	High
6	Moderate	2 Disks	Moderate
10	Excellent	High	50%

Table 20.1: RAID Comparison

20.28 SSD

Internals

SSD consists of:

- NAND Flash
- Controller
- DRAM Cache
- Firmware

20.28.1 Organization

Page → Block → Chip

20.28.2 Characteristics

- Reads fast
- Writes more expensive
- Erase-before-write

20.29 Wear

Leveling

20.29.1 Problem

Flash cells have limited write cycles.

20.29.2 Solution

Wear leveling distributes writes evenly.

20.29.3 Goal

Prevent premature failure.

20.29.4 Types

- Dynamic Wear Leveling
- Static Wear Leveling

20.30 TRIM

20.30.1 Definition

TRIM informs SSD which blocks are no longer needed.

20.30.2 Benefits

- Faster writes
- Better garbage collection
- Longer SSD lifespan

20.30.3 Linux**Example**

```
fstrim -v /
```

20.31 Solved**Numerical****Example**

Requests:

98, 183, 37, 122, 14, 124, 65, 67

Head:

53

Using SSTF:

53 → 65 → 67 → 37

→ 14 → 98 → 122

→ 124 → 183

Total Movement:

236

tracks.

20.32 Interview**Tips**

- SSTF minimizes immediate seek time.

- SCAN is called Elevator Algorithm.
- LOOK avoids unnecessary end movement.
- RAID 0 provides performance only.
- RAID 1 provides mirroring.
- RAID 5 survives one failure.
- RAID 6 survives two failures.
- RAID 10 combines speed and redundancy.
- SSDs do not require traditional disk scheduling.
- Wear leveling is a common interview topic.

20.33 Key

Takeaways

- Disk performance depends on seek time, latency, and transfer time.
- Disk scheduling reduces head movement.
- SSTF improves over FCFS.
- SCAN and LOOK are commonly discussed algorithms.
- RAID improves reliability and performance.
- SSDs differ fundamentally from magnetic disks.
- Wear leveling extends SSD life.
- TRIM improves SSD performance.

20.34 25 Interview Questions with Answers

1. What is seek time?
Answer: Time to move disk head.
2. What is rotational latency?
Answer: Waiting for sector rotation.
3. What is transfer time?
Answer: Time to transfer data.
4. What is FCFS?
Answer: Arrival-order scheduling.
5. What is SSTF?
Answer: Closest request first.
6. SSTF drawback?
Answer: Starvation.
7. SCAN nickname?
Answer: Elevator Algorithm.
8. What is CSCAN?

Answer: Circular SCAN.

9. What is LOOK?

Answer: SCAN without going to disk end.

10. What is CLOOK?

Answer: Circular LOOK.

11. Define RAID.

Answer: Redundant Array of Independent Disks.

12. RAID 0 provides?

Answer: Striping.

13. RAID 1 provides?

Answer: Mirroring.

14. RAID 5 survives?

Answer: One disk failure.

15. RAID 6 survives?

Answer: Two disk failures.

16. RAID 10 combines?

Answer: Mirroring and Striping.

17. SSD stores data using?

Answer: Flash Memory.

18. SSD has moving parts?

Answer: No.

19. What is wear leveling?

Answer: Even write distribution.

20. Why TRIM?

Answer: Identify unused blocks.

21. What is a track?

Answer: Circular recording path.

22. What is a sector?

Answer: Smallest physical unit.

23. What is a cylinder?

Answer: Same-radius tracks.

24. Best FCFS advantage?

Answer: Simplicity.

25. SSD scheduling needed?

Answer: Generally less important.

20.35 25 MCQs with Answers

1. Smallest disk storage unit: Answer: Sector

2. Circular storage path: Answer: Track
3. Same-radius tracks: Answer: Cylinder
4. FCFS stands for: Answer: First Come First Serve
5. SSTF stands for: Answer: Shortest Seek Time First
6. Elevator algorithm: Answer: SCAN
7. Circular SCAN: Answer: CSCAN
8. LOOK is improvement over: Answer: SCAN
9. RAID 0 uses: Answer: Striping
10. RAID 1 uses: Answer: Mirroring
11. RAID 5 tolerates: Answer: One Failure
12. RAID 6 tolerates: Answer: Two Failures
13. RAID 10 combines: Answer: RAID 0 and RAID 1
14. SSD uses: Answer: NAND Flash
15. SSD moving parts: Answer: None
16. Wear leveling improves: Answer: Lifespan
17. TRIM improves: Answer: SSD Performance
18. SSTF may cause: Answer: Starvation
19. SCAN is: Answer: Fairer than SSTF
20. FCFS is: Answer: Simple
21. Average latency depends on: Answer: Rotation Speed
22. Seek time depends on: Answer: Head Movement
23. SSD latency compared to HDD: Answer: Lower
24. RAID improves: Answer: Reliability
25. Disk scheduling mainly reduces: Answer: Seek Time

20.36 Revision

Notes

- Disk access time = Seek + Latency + Transfer.
- SSTF selects nearest request.
- SCAN = Elevator Algorithm.
- LOOK avoids unnecessary movement.
- RAID 0 = Striping.
- RAID 1 = Mirroring.
- RAID 5 = Single parity.
- RAID 6 = Dual parity.
- RAID 10 = Mirroring + Striping.
- SSDs use flash memory.
- Wear leveling extends SSD lifespan.

- TRIM improves SSD efficiency.

20.37 Cheat

Sheet

Track	Circular Path
Sector	Smallest Physical Unit
Cylinder	Same Radius Tracks
Seek Time	Head Movement Time
Rotational Latency	Rotation Wait Time
Transfer Time	Data Transfer Time
FCFS	Arrival Order
SSTF	Closest Request
SCAN	Elevator Algorithm
CSCAN	Circular SCAN
LOOK	Optimized SCAN
CLOOK	Optimized CSCAN
RAID 0	Striping
RAID 1	Mirroring
RAID 5	Single Parity
RAID 6	Dual Parity
RAID 10	Mirroring + Striping
SSD	Flash Storage
Wear Leveling	Even Writes
TRIM	Free Block Notification

Part VIII

Modern Operating Systems

Linux Internals

21.1 Introduction

Linux is one of the most widely used operating systems in the world. It powers servers, cloud infrastructure, embedded devices, supercomputers, smartphones (via Android), and modern backend systems.

For Software Development Engineer (SDE) interviews, Linux internals form a critical topic because they connect operating system theory with real-world system implementation.

21.2 Linux

Architecture

Overview

Linux follows a layered architecture.

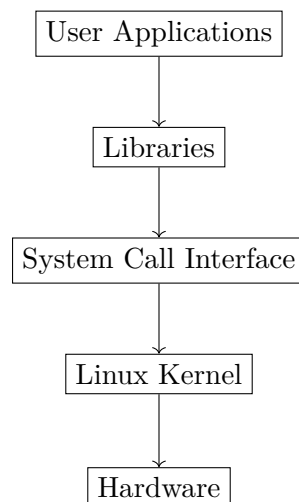


Figure 21.1: Linux Architecture

Major components include:

- User Space
- System Call Interface
- Kernel Space
- Hardware Layer

21.3 User Space vs Kernel Space

Linux divides execution into two protection domains.

21.3.1 User Space

Applications execute in user mode.

Characteristics:

- Restricted privileges
- Cannot access hardware directly
- Cannot execute privileged instructions

21.3.2 Kernel Space

Kernel executes in privileged mode.

Characteristics:

- Full hardware access
- Memory management
- Scheduling
- Device management

Feature	User Space	Kernel Space
Privilege	Low	High
Hardware Access	No	Yes
Memory Access	Restricted	Full
Crash Impact	Process Only	Entire System

Table 21.1: User Space vs Kernel Space

21.4 Linux Kernel Architecture

The Linux kernel contains several subsystems.

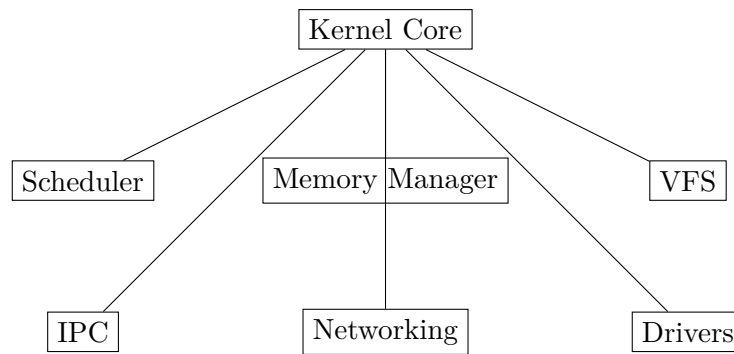


Figure 21.2: Linux Kernel Subsystems

21.5 Monolithic Kernel Design

Linux uses a monolithic kernel.

Characteristics:

- All major services run in kernel space
- High performance
- Shared address space

Advantages:

- Fast communication
- Lower overhead
- Better performance

Disadvantages:

- Large codebase
- Bug may crash entire system

21.6 Kernel Modules

Kernel modules are dynamically loadable components.

Examples:

- Device drivers
- File systems
- Network protocols

Common commands:

```
1 lsmod
2 insmod module.ko
3 rmmod module
4 modprobe module
```

Benefits:

- Extensibility
- Reduced kernel size
- Runtime loading

21.7 System Call Interface

System calls provide controlled access to kernel services.

Examples:

- `open()`
- `read()`
- `write()`
- `fork()`
- `execve()`
- `mmap()`

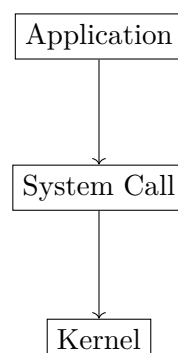


Figure 21.3: System Call Flow

21.8 Process Management in Linux

Linux represents processes using the `task_struct` structure.

Information stored:

- PID

- State
- Memory mappings
- Open files
- Scheduling information

Common process states:

- Running
- Ready
- Sleeping
- Stopped
- Zombie

21.9 Scheduler

Overview

The scheduler decides which task gets CPU time.

Goals:

- Fairness
- Throughput
- Low latency
- Scalability

21.10 Completely Fair Scheduler (CFS)

CFS is the default Linux scheduler.

Core idea:

Every runnable process should receive a fair share of CPU time.

CFS maintains a Red-Black Tree ordered by virtual runtime.

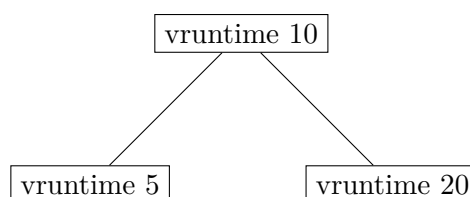


Figure 21.4: Simplified CFS Tree

Process with smallest virtual runtime executes next.

21.11 Linux Memory Management

Linux memory management includes:

- Physical memory management
- Virtual memory
- Paging
- Page cache
- Slab allocator

Memory zones:

- DMA
- Normal
- High Memory

21.12 Virtual Memory in Linux

Each process receives a private virtual address space.

Advantages:

- Isolation
- Security
- Larger logical memory
- Shared libraries

Key concepts:

- Page Tables
- TLB
- Demand Paging
- Swap

21.13 Linux File Systems Overview

Linux supports multiple file systems.

Examples:

- ext4
- XFS

- Btrfs
- NFS
- tmpfs

The Virtual File System (VFS) provides a unified interface.

21.14 ext4

Architecture

ext4 is the most common Linux file system.

Features:

- Journaling
- Extents
- Delayed Allocation
- Large File Support

Disk layout:

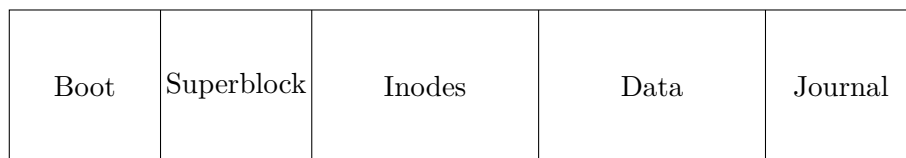


Figure 21.5: Simplified ext4 Layout

21.15 proc

File

System

/proc is a virtual file system exposing kernel information.

Examples:

```
1 cat /proc/cpuinfo
2 cat /proc/meminfo
3 cat /proc/self/status
```

Uses:

- Monitoring
- Diagnostics
- Performance analysis

21.16 sysfs

`/sys` exposes kernel objects and device information.

Examples:

```
1 ls /sys/class
2 ls /sys/devices
```

Uses:

- Device inspection
- Driver interaction
- Hardware configuration

21.17 Signals

Signals provide asynchronous process notifications.

Common signals:

Signal	Purpose
SIGINT	Interrupt
SIGTERM	Graceful Termination
SIGKILL	Forced Termination
SIGSEGV	Segmentation Fault
SIGSTOP	Pause Process

Table 21.2: Common Linux Signals

21.18 IPC

Mechanisms

Inter-Process Communication methods include:

- Pipes
- Named Pipes
- Shared Memory
- Message Queues
- Semaphores
- Sockets

21.19 Daemons

Daemons are background processes.

Examples:

- sshd
- cron
- systemd
- nginx

Characteristics:

- No controlling terminal
- Long-running
- Service oriented

21.20 Linux

Boot

Process

Linux startup sequence:

1. Power On
2. BIOS/UEFI
3. Bootloader
4. Kernel Loading
5. Init/Systemd
6. User Services



Figure 21.6: Linux Boot Sequence

21.21 init

and

systemd

Historically Linux used `init`.

Modern distributions use `systemd`.

Features:

- Parallel startup
- Service management

- Logging
- Dependency tracking

Commands:

```
1 systemctl status nginx
2 systemctl start nginx
3 systemctl stop nginx
```

21.22 Kernel

Panic

Kernel panic occurs when the kernel encounters an unrecoverable error.

Causes:

- Invalid memory access
- Driver bugs
- Hardware failures
- Corrupted kernel structures

Result:

- System halt
- Diagnostic information displayed

21.23 Common Linux Commands for OS Interviews

Command	Purpose
ps	Process Information
top	System Monitoring
htop	Interactive Monitoring
free	Memory Usage
vmstat	Virtual Memory Statistics
iostat	I/O Statistics
lsof	Open Files
netstat	Network Statistics
ss	Socket Information
kill	Send Signal
strace	Trace System Calls
df	Disk Usage
du	Directory Usage

Table 21.3: Important Linux Commands

21.24 Real-World Backend Engineering Examples

21.24.1 Web Server

A high-performance web server:

- Uses epoll
- Uses non-blocking sockets
- Uses multiple worker processes

21.24.2 Database Server

A database system:

- Uses mmap
- Uses page cache
- Relies on virtual memory

21.24.3 Container Platform

Containers rely on:

- Namespaces
- cgroups
- Process isolation

21.25 Dedicated Interview Section: What Happens When a Linux Process Starts?

Process startup sequence:

1. Parent invokes `fork()`
2. Child created
3. `execve()` loads executable
4. Page tables created
5. Program image loaded
6. Dynamic linker resolves libraries
7. `main()` executes

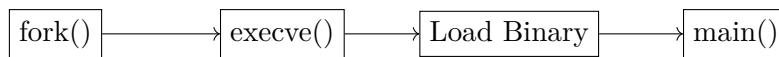


Figure 21.7: Process Startup

21.26 Dedicated Interview Section: What Happens When a Process Terminates?

Termination sequence:

1. `exit()` called
2. Resources released
3. Open files closed
4. Exit status stored
5. Process becomes zombie
6. Parent collects status using `wait()`

21.27 Dedicated Interview Section: How a System Call Reaches the Kernel?

Sequence:

1. Application invokes library wrapper
2. CPU executes `syscall` instruction
3. Mode switch occurs
4. Kernel handler executes
5. Service completed
6. Return to user mode

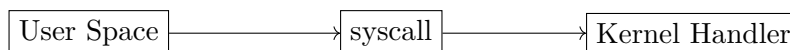


Figure 21.8: System Call Path

21.28 Key

Takeaways

Modern Concurrency

22.1 Introduction

Modern software systems must serve millions of users, process massive amounts of data, and utilize multicore processors efficiently. Concurrency has therefore become one of the most important topics for Software Development Engineers.

Understanding modern concurrency is essential for:

- Backend Development
- Distributed Systems
- Cloud Computing
- High-Performance Computing
- System Design Interviews
- Operating Systems Interviews

This chapter covers the concepts, architectures, programming models, and interview perspectives required for modern software engineering roles.

22.2 Learning

Objectives

After completing this chapter, you should be able to:

- Differentiate concurrency and parallelism.
- Understand multicore architectures.
- Explain thread pools and task scheduling.
- Use futures and promises.
- Understand asynchronous programming.
- Explain lock-free and wait-free algorithms.
- Understand actor systems and reactive programming.
- Analyze high-concurrency server architectures.
- Solve modern concurrency interview questions.

22.3 Concurrency vs Parallelism

22.3.1 Concurrency

Concurrency means multiple tasks make progress during overlapping time periods.

Tasks may not execute simultaneously.

22.3.2 Parallelism

Parallelism means multiple tasks execute at exactly the same time.

Requires multiple processing units.

22.3.3 Example

Single-core CPU:

$$Task_A \rightarrow Task_B \rightarrow Task_A$$

Concurrent but not parallel.

Multi-core CPU:

$$Core_1 : Task_A$$

$$Core_2 : Task_B$$

Parallel execution.

22.3.4 Comparison

Feature	Concurrency	Parallelism
Simultaneous Execution	Not Required	Required
Multiple Cores	Optional	Required
Goal	Responsiveness	Performance
Example	Async Server	Matrix Multiplication

Table 22.1: Concurrency vs Parallelism

22.4 Multicore

Architectures

22.4.1 Definition

A multicore processor contains multiple CPU cores on a single chip.

22.4.2 Benefits

- Higher throughput
- Better energy efficiency
- Parallel execution
- Improved responsiveness

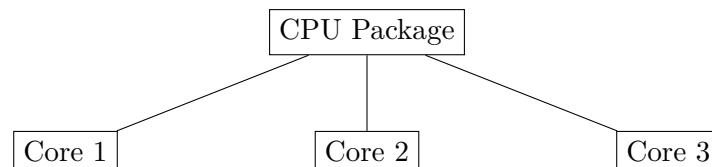


Figure 22.1: Multicore Processor

22.5 Hyperthreading

22.5.1 Definition

Hyperthreading allows a physical core to appear as multiple logical processors.

22.5.2 Goal

Improve utilization of execution units.

22.5.3 Example

8 Physical Cores

with Hyperthreading:

16 Logical CPUs

22.5.4 Advantages

- Better CPU utilization
- Increased throughput

22.5.5 Limitations

- Not equivalent to additional physical cores

22.6 Symmetric Multiprocessing (SMP)

22.6.1 Definition

Multiple processors share:

- Main memory
- I/O subsystem
- Operating system

22.6.2 Characteristics

- Equal access to resources
- Shared memory model
- Common in modern servers

22.7 NUMA Systems

22.7.1 Definition

NUMA stands for Non-Uniform Memory Access.

22.7.2 Idea

Memory closer to a CPU can be accessed faster.

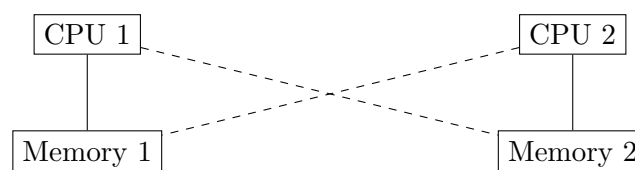


Figure 22.2: NUMA Architecture

22.7.3 Interview

Insight

NUMA awareness is critical for high-performance databases and backend systems.

22.8 Thread

Pools

22.8.1 Definition

A thread pool is a collection of reusable worker threads.

22.8.2 Why

Needed

Creating threads repeatedly is expensive.

22.8.3 Architecture

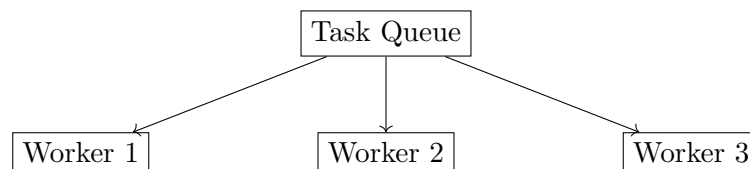


Figure 22.3: Thread Pool

22.8.4 Java

Example

```
ExecutorService pool =
Executors.newFixedThreadPool(4);

pool.submit(() -> {
    System.out.println("Task");
});
```

22.9 Work

Queues

22.9.1 Definition

A work queue stores tasks waiting for execution.

22.9.2 Benefits

- Load balancing

- Scalability
- Better resource utilization

22.10 Fork-Join

Model

22.10.1 Definition

Problem is split into smaller subtasks.

1. Fork
2. Execute
3. Join

22.10.2 Example

Parallel Merge Sort.

22.10.3 Java

Example

```
ForkJoinPool pool =  
new ForkJoinPool();
```

22.11 Task

Scheduling

Modern runtimes schedule tasks rather than threads.

Examples:

- Java Virtual Threads
- Go Goroutines
- Kotlin Coroutines
- Tokio Tasks

Benefits:

- Lightweight
- Scalable
- Millions of concurrent tasks

22.12 Futures

22.12.1 Definition

A Future represents a value available later.

22.12.2 Java

Example

```
Future<Integer> result =  
    executor.submit(() -> 42);
```

22.12.3 Benefits

- Non-blocking workflows
- Async execution

22.13 Promises

22.13.1 Definition

A Promise is a writable counterpart of a Future.

22.13.2 Workflow

Producer $\rightarrow$ *Promise*

Consumer $\rightarrow$ *Future*

22.13.3 Use

Cases

- Async APIs
- Distributed systems

22.14 Async

Programming

22.14.1 Definition

Execution proceeds without waiting for long-running operations.

22.14.2 Benefits

- Better scalability
- Higher throughput
- Reduced blocking

22.14.3 Python

Example

```
import asyncio

async def hello():
    print("Hello")

asyncio.run(hello())
```

22.15 Event

Loops

22.15.1 Definition

An event loop continuously processes events and callbacks.

22.15.2 Architecture



Figure 22.4: Event Loop

22.15.3 Examples

- Node.js
- Python asyncio
- Nginx

22.16 Non-Blocking

Programming

22.16.1 Definition

Operations return immediately without blocking threads.

22.16.2 Examples

- Async I/O
- Epoll
- `io_uring`

22.16.3 Advantages

- Better scalability
- Fewer threads

22.17 Lock-Free Data Structures

22.17.1 Definition

System-wide progress is guaranteed without locks.

22.17.2 Tools

- CAS
- Atomic Operations

22.17.3 Example

Lock-free queue.

22.17.4 Advantages

- High performance
- No deadlocks

22.17.5 Disadvantages

- Complex implementation

22.18 Wait-Free Algorithms

22.18.1 Definition

Every thread completes its operation in finite steps.

22.18.2 Properties

- Stronger than lock-free
- No starvation

22.18.3 Challenge

Very difficult to implement.

22.19 Software Transactional Memory

22.19.1 Definition

Treat memory operations as transactions.

22.19.2 Workflow

1. Begin Transaction
2. Execute
3. Commit
4. Retry on Conflict

22.19.3 Advantages

- Easier reasoning
- Reduced locking

22.20 Actor Model

22.20.1 Definition

Actors communicate through messages.

No shared memory.

22.20.2 Principles

- Encapsulation
- Message Passing
- Isolation

22.20.3 Examples

- Erlang
- Akka
- Orleans



Figure 22.5: Actor Communication

22.21 Reactive Programming Concepts

22.21.1 Definition

Programming based on asynchronous streams.

22.21.2 Features

- Event-driven
- Non-blocking
- Backpressure support

22.21.3 Examples

- RxJava
- Reactor
- Akka Streams

22.22 Modern Concurrency Challenges

- Race Conditions
- Deadlocks
- Livelocks
- Starvation
- False Sharing
- Cache Coherency
- NUMA Effects
- Memory Ordering

22.23 Thread-per-Request vs Event-Driven Architectures

22.23.1 Thread-per-Request

Each request gets a dedicated thread.

Examples:

- Traditional Java Servers
- Apache HTTP Server

Advantages:

- Simpler programming model

Disadvantages:

- High memory usage
- Context switching overhead

22.23.2 Event-Driven

Architecture

Single or few threads process many requests.

Examples:

- Nginx
- Node.js
- Redis

Advantages:

- Massive scalability

Disadvantages:

- More complex logic

Feature	Thread-per-Request	Event Driven
Threads	Many	Few
Memory	High	Low
Scalability	Moderate	High
Complexity	Low	High

Table 22.2: Architecture Comparison

22.24 How High-Concurrency Servers Work

Modern servers use:

- Event Loops
- Async I/O
- Thread Pools
- Work Queues
- Lock-Free Structures
- Load Balancing

Examples:

- Nginx
- Netty
- Redis
- Kafka
- Envoy

Typical architecture:

Network $\rightarrow$ *Event Loop* $\rightarrow$ *Task Queue*

$\rightarrow$ *Worker Pool* $\rightarrow$ *Response*

22.25 Java Examples

22.25.1 CompletableFuture

```
CompletableFuture.supplyAsync(  
    () -> "Hello"  
);
```

22.25.2 ExecutorService

```
ExecutorService executor =  
Executors.newFixedThreadPool(8);
```

22.26 Python

Examples

22.26.1 Asyncio

```
import asyncio

async def work():
    await asyncio.sleep(1)

asyncio.run(work())
```

22.27 C++

Examples

22.27.1 Future

```
auto result =
std::async([] {
    return 42;
});
```

22.27.2 Thread

```
std::thread t(task);
t.join();
```

22.28 Backend

Service

Examples

System	Concurrency Model
Nginx	Event Driven
Redis	Single Thread Event Loop
Kafka	Thread Pools
Netty	Event Loop Groups
Go Services	Goroutines
Akka	Actor Model

Table 22.3: Real-World Concurrency Models

22.29 Performance

Discussion

Key performance factors:

- Context Switches
- Lock Contention
- Cache Locality
- False Sharing
- Memory Bandwidth
- NUMA Awareness
- Thread Scheduling

Interview insight:

Scalable systems minimize blocking and maximize useful CPU work.

22.30 Modern Concurrency Interview Questions

Common interview themes:

- Futures vs Promises
- Thread Pools
- Event Loops
- Actor Model
- Async vs Multithreading
- Lock-Free Programming
- NUMA
- High-Concurrency Servers

22.31 Key

Takeaways

- Concurrency and parallelism are different concepts.
- Modern systems rely heavily on multicore processors.
- Thread pools improve efficiency.
- Futures and promises simplify async workflows.
- Event loops power scalable servers.
- Lock-free algorithms improve throughput.
- Actor systems eliminate shared memory issues.
- Reactive programming supports asynchronous streams.
- High-concurrency servers avoid blocking.
- Modern SDE interviews frequently test concurrency knowledge.

22.32 25 Interview Questions with Answers

1. What is concurrency?
Answer: Multiple tasks making progress during overlapping time.
2. What is parallelism?
Answer: Simultaneous execution of tasks.
3. Difference between concurrency and parallelism?
Answer: Parallelism requires simultaneous execution.
4. What is hyperthreading?
Answer: Multiple logical CPUs per physical core.
5. What is SMP?
Answer: Shared-memory multiprocessor architecture.
6. What is NUMA?
Answer: Non-Uniform Memory Access.
7. Why use thread pools?
Answer: Reduce thread creation overhead.
8. What is a work queue?
Answer: Queue of pending tasks.
9. What is fork-join?
Answer: Divide and merge parallel computation.
10. What is a Future?
Answer: Result available later.
11. What is a Promise?
Answer: Producer side of a Future.
12. What is async programming?
Answer: Non-blocking execution model.
13. What is an event loop?
Answer: Dispatcher for asynchronous events.
14. What is non-blocking I/O?
Answer: I/O that does not stall threads.
15. What is lock-free programming?
Answer: Progress without locks.
16. What is wait-free programming?
Answer: Every thread completes in finite steps.
17. What is STM?
Answer: Software Transactional Memory.
18. What is the Actor Model?
Answer: Message-based concurrency.
19. What is reactive programming?
Answer: Programming with asynchronous streams.

20. What causes false sharing?
Answer: Multiple threads updating same cache line.
21. Why is NUMA important?
Answer: Memory locality affects performance.
22. What powers Node.js?
Answer: Event loop.
23. Why avoid excessive threads?
Answer: Context-switch overhead.
24. How does Nginx scale?
Answer: Event-driven architecture.
25. What is lock contention?
Answer: Multiple threads competing for locks.

22.33 25 MCQs with Answers

1. Parallelism requires: Answer: Multiple Execution Units
2. Hyperthreading creates: Answer: Logical CPUs
3. NUMA stands for: Answer: Non-Uniform Memory Access
4. Thread pools improve: Answer: Efficiency
5. Future represents: Answer: Future Result
6. Promise is: Answer: Writable Future
7. Node.js uses: Answer: Event Loop
8. Redis primarily uses: Answer: Event Loop
9. Async programming reduces: Answer: Blocking
10. Lock-free structures use: Answer: Atomic Operations
11. Wait-free guarantees: Answer: Per-thread Progress
12. Actor model uses: Answer: Message Passing
13. STM stands for: Answer: Software Transactional Memory
14. Nginx is: Answer: Event Driven
15. Go uses: Answer: Goroutines
16. Reactive systems process: Answer: Streams
17. Thread-per-request creates: Answer: Many Threads
18. Event-driven servers create: Answer: Few Threads
19. False sharing affects: Answer: Cache Performance
20. NUMA impacts: Answer: Memory Latency
21. CompletableFuture belongs to: Answer: Java
22. asyncio belongs to: Answer: Python
23. std::future belongs to: Answer: C++

24. Fork-Join is useful for: Answer: Parallel Tasks
25. Lock contention decreases: Answer: Throughput

22.34 Revision

Notes

- Concurrency Parallelism.
- Multicore processors enable parallel execution.
- Thread pools reduce overhead.
- Futures represent future results.
- Promises complete futures.
- Event loops power scalable servers.
- Async programming avoids blocking.
- Lock-free algorithms improve throughput.
- Wait-free guarantees completion.
- Actor systems use message passing.
- NUMA affects memory access speed.
- Modern backend systems are heavily asynchronous.

22.35 Cheat**Sheet**

Concurrency	Overlapping Progress
Parallelism	Simultaneous Execution
Hyperthreading	Multiple Logical CPUs
SMP	Shared Memory Multiprocessing
NUMA	Non-Uniform Memory Access
Thread Pool	Reusable Threads
Work Queue	Pending Tasks
Future	Future Result
Promise	Future Producer
Event Loop	Async Dispatcher
Async Programming	Non-Blocking Execution
Lock-Free	System Progress Guaranteed
Wait-Free	Per-Thread Progress Guaranteed
STM	Transactional Memory
Actor Model	Message Passing
Reactive Programming	Async Streams
Nginx	Event Driven
Node.js	Event Loop
Redis	Event Loop
Go	Goroutines

Virtualization and Containers

23.1 Introduction

Virtualization and containers are foundational technologies in modern cloud computing, distributed systems, backend infrastructure, DevOps, and platform engineering.

Almost every large-scale software system today runs inside virtual machines, containers, or both.

Understanding virtualization and containers is essential for Software Development Engineer interviews, Site Reliability Engineering interviews, Cloud Engineering interviews, and Backend Engineering interviews.

23.2 Motivation for Virtualization

Before virtualization became mainstream, applications ran directly on physical servers.

Problems included:

- Low hardware utilization
- High infrastructure costs
- Poor scalability
- Difficult deployment
- Resource fragmentation

Virtualization addresses these challenges by allowing multiple isolated environments to share the same physical hardware.

Benefits:

- Better utilization
- Resource isolation
- Easier management
- High availability
- Scalability
- Cost reduction

23.3 Virtual

Machines

A Virtual Machine (VM) is a software emulation of a physical computer.

Each VM includes:

- Virtual CPU
- Virtual Memory
- Virtual Storage
- Virtual Network Interfaces
- Guest Operating System

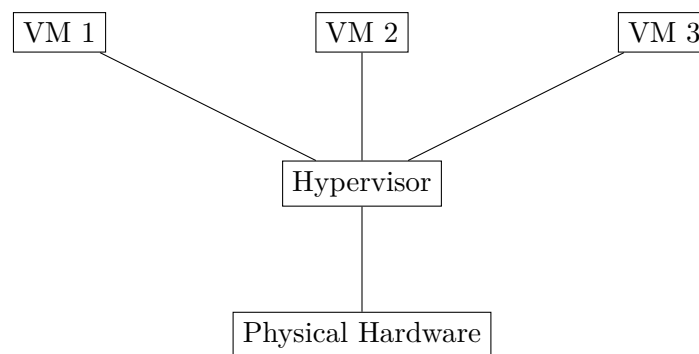


Figure 23.1: Virtual Machine Architecture

23.4 Hypervisors

A hypervisor manages virtual machines.

Responsibilities:

- CPU allocation
- Memory allocation
- Device virtualization
- Scheduling
- Isolation

Examples:

- VMware ESXi
- KVM
- Hyper-V
- Xen
- VirtualBox

23.5 Type 1 Hypervisors

Type 1 hypervisors run directly on hardware.

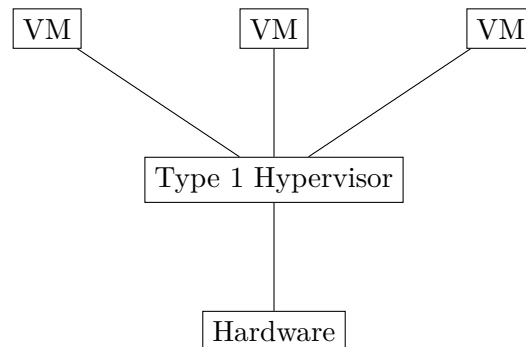


Figure 23.2: Type 1 Hypervisor

Examples:

- VMware ESXi
- Xen
- Hyper-V

Advantages:

- Better performance
- Strong isolation
- Enterprise deployment

23.6 Type 2 Hypervisors

Type 2 hypervisors run on a host operating system.

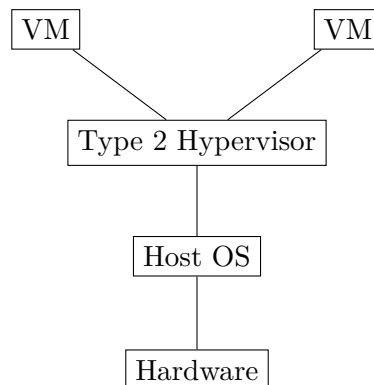


Figure 23.3: Type 2 Hypervisor

Examples:

- VirtualBox
- VMware Workstation

23.7 Full

Virtualization

Full virtualization completely emulates hardware.

Guest operating systems run unmodified.

Advantages:

- Compatibility
- Flexibility

Disadvantages:

- Higher overhead

23.8 Para-Virtualization

Guest operating systems are modified to cooperate with the hypervisor.

Advantages:

- Better performance
- Lower overhead

Disadvantages:

- Requires guest OS modifications

Example:

- Xen PV

23.9 Hardware-Assisted

Virtualization

Modern CPUs provide virtualization support.

Examples:

- Intel VT-x
- Intel VT-d
- AMD-V

- AMD-Vi

Benefits:

- Reduced hypervisor overhead
- Faster context switching
- Improved isolation

23.10 CPU

Virtualization

CPU virtualization allows multiple VMs to share physical processors.

Techniques:

- Time slicing
- Virtual CPUs (vCPUs)
- Hardware support

23.11 Memory

Virtualization

Memory virtualization provides isolated address spaces.

Techniques:

- Shadow page tables
- Nested paging
- Extended Page Tables (EPT)

Benefits:

- Isolation
- Security
- Flexibility

23.12 Storage

Virtualization

Storage resources are abstracted from physical devices.

Examples:

- SAN
- Virtual disks

- Cloud block storage

Advantages:

- Scalability
- Simplified management

23.13 Network

Virtualization

Network virtualization creates logical networks independent of physical topology.

Examples:

- VLANs
- SDN
- Virtual Switches
- Overlay Networks

23.14 Containers

Containers provide lightweight operating-system-level virtualization.

Containers share the host kernel while maintaining isolation.

Advantages:

- Fast startup
- Low overhead
- High density
- Portability

23.15 Linux

Namespaces

Namespaces isolate system resources.

Types:

- PID Namespace
- Mount Namespace
- Network Namespace
- IPC Namespace
- UTS Namespace

- User Namespace

Example:

A container may see process IDs starting at 1 while the host sees different PIDs.

23.16 cgroups

Control Groups (cgroups) limit and monitor resource usage.

Resources managed:

- CPU
- Memory
- Disk I/O
- Network bandwidth

Example:

- Limit container memory to 1 GB
- Restrict CPU usage to two cores

23.17 Container

Isolation

Container isolation relies on:

- Namespaces
- cgroups
- Capabilities
- Seccomp
- AppArmor
- SELinux

23.18 Docker

Architecture

Docker is the most popular container platform.

Components:

- Docker Client
- Docker Daemon
- Docker Images

- Docker Registry
- Containers

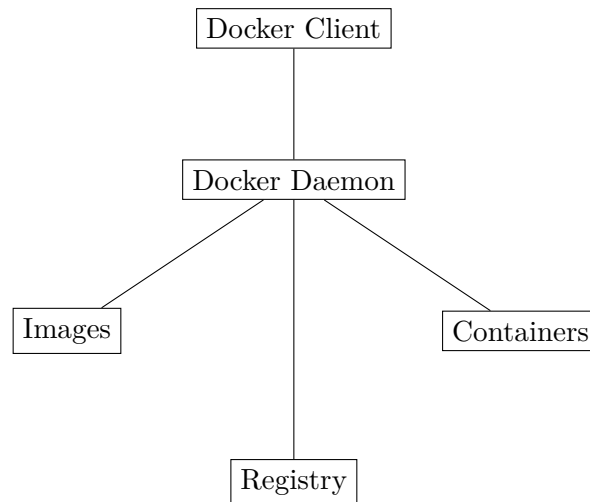


Figure 23.4: Docker Architecture

23.19 Images

Images are immutable templates used to create containers.

Characteristics:

- Layered
- Read-only
- Reusable

Example:

```
1 docker pull nginx
```

23.20 Containers

Containers are runtime instances of images.

Example:

```
1 docker run nginx
```

Lifecycle:

- Created
- Running

- Paused
- Stopped
- Removed

23.21 Registries

Container registries store and distribute images.

Examples:

- Docker Hub
- Amazon ECR
- Google Artifact Registry
- GitHub Container Registry

Workflow:

```
1 docker build
2 docker push
3 docker pull
```

23.22 Kubernetes

Overview

Kubernetes is a container orchestration platform.

Responsibilities:

- Scheduling
- Scaling
- Service discovery
- Self-healing
- Rolling updates

23.23 Pods

A Pod is the smallest deployable Kubernetes unit.

Characteristics:

- One or more containers
- Shared network namespace
- Shared storage

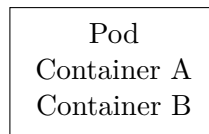


Figure 23.5: Kubernetes Pod

23.24 Container

Scheduling

Schedulers determine where workloads execute.

Scheduling factors:

- CPU availability
- Memory availability
- Affinity rules
- Resource requests
- Taints and tolerations

23.25 Security

Considerations

Common security concerns:

- Privileged containers
- Container escapes
- Vulnerable images
- Secret management
- Supply chain attacks

Best practices:

- Least privilege
- Image scanning
- Read-only filesystems
- Resource limits
- Network policies

23.26 Dedicated Interview Section: VM vs Container

Feature	Virtual Machine	Container
Kernel	Own Kernel	Shared Kernel
Startup Time	Seconds/Minutes	Milliseconds
Resource Usage	High	Low
Isolation	Strong	Moderate
Density	Lower	Higher
Portability	Good	Excellent

Table 23.1: VM vs Container

23.27 Dedicated Interview Section: Docker vs Virtual Machine

Aspect	Docker	VM
Boot Time	Very Fast	Slower
Memory Usage	Low	Higher
Kernel Sharing	Yes	No
Overhead	Low	High
Density	High	Moderate

Table 23.2: Docker vs Virtual Machine

23.28 Dedicated Interview Section: How Containers Achieve Isolation

Containers achieve isolation through:

1. Namespaces isolate visibility.
2. cgroups isolate resources.
3. Capabilities reduce privileges.
4. Seccomp restricts system calls.
5. AppArmor/SELinux enforce policies.

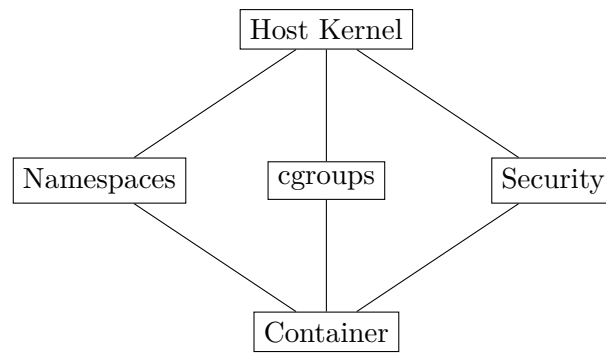


Figure 23.6: Container Isolation Mechanisms

23.29 Key

Takeaways

Operating System Security

24.1 Introduction

Operating System (OS) security focuses on protecting system resources, processes, memory, files, devices, and users from unauthorized access, misuse, and attacks.

Modern operating systems such as Linux, Windows, macOS, Android, and iOS implement multiple layers of defense to provide confidentiality, integrity, and availability.

For Software Development Engineers (SDEs), OS security knowledge is important because security vulnerabilities often arise from improper privilege handling, memory misuse, weak authentication mechanisms, and insecure system design.

24.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand core OS security concepts.
- Explain authentication and authorization mechanisms.
- Compare ACLs and capabilities.
- Understand memory protection techniques.
- Explain ASLR and DEP.
- Understand sandboxing and secure boot.
- Describe SELinux and AppArmor.
- Understand container security fundamentals.
- Identify common operating system vulnerabilities.
- Answer OS security interview questions confidently.

24.3 Security

Fundamentals

24.3.1 Definition

Security is the protection of computing resources against unauthorized access, modification, destruction, or disclosure.

24.3.2 CIA

Triad

The foundation of information security consists of:

1. Confidentiality
2. Integrity
3. Availability

Property	Description
Confidentiality	Prevent unauthorized access to information
Integrity	Prevent unauthorized modification
Availability	Ensure resources remain accessible

Table 24.1: CIA Triad

24.3.3 Example

Online banking systems require:

- Confidentiality of account data.
- Integrity of transactions.
- Availability of services.

24.4 Threat

Model

Basics

24.4.1 Definition

A threat model identifies:

- Assets
- Threats
- Attackers
- Attack Surfaces
- Mitigations

24.4.2 Typical

Threats

- Malware
- Privilege Escalation
- Data Theft
- Insider Threats
- Remote Exploits
- Denial of Service

24.4.3 Security

Architecture

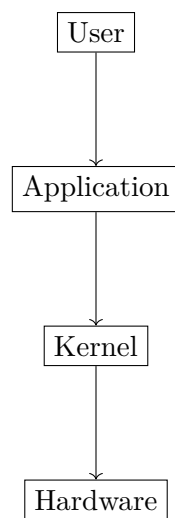


Figure 24.1: Security Layers

24.5 Authentication

24.5.1 Definition

Authentication verifies identity.

Question answered:

Who are you?

24.5.2 Methods

- Passwords
- Biometrics
- Smart Cards
- OTPs

- Multi-Factor Authentication

24.5.3 Linux

Example

/etc/passwd

/etc/shadow

24.5.4 Windows

Example

Authentication is commonly integrated with:

- Active Directory
- Kerberos
- NTLM

24.6 Authorization

24.6.1 Definition

Authorization determines allowed actions.

Question answered:

What are you allowed to do?

24.6.2 Examples

- Read file
- Write file
- Execute program
- Access device

24.7 Authentication

vs

Authorization

Authentication	Authorization
Identity Verification	Permission Verification
Who Are You?	What Can You Do?
Occurs First	Occurs After Authentication
Password Login	File Access Check

Table 24.2: Authentication vs Authorization

24.8 Access Control Matrix

24.8.1 Definition

An Access Control Matrix defines permissions between subjects and objects.

	File A	File B
Alice	RW	R
Bob	R	RWX

Table 24.3: Access Control Matrix

24.8.2 Challenge

Matrices become very large in real systems.

24.9 Access Control Lists (ACLs)

24.9.1 Definition

ACLs store permissions with each object.

24.9.2 Example

File:

`report.txt`

ACL:

- Alice : Read
- Bob : Read, Write
- Charlie : Read

24.9.3 Advantages

- Easy per-object management
- Widely used

24.9.4 Disadvantages

- Large ACL maintenance

24.10 Capabilities

24.10.1 Definition

Capabilities associate permissions with subjects instead of objects.

24.10.2 Example

User Alice possesses:

- Read(File A)
- Write(File B)

24.10.3 Advantages

- Fine-grained security
- Easier delegation

24.10.4 Disadvantages

- Capability management complexity

24.11 ACLs vs Capabilities

Feature	ACL	Capability
Stored With	Object	Subject
Lookup	Object Based	User Based
Delegation	Difficult	Easier
Usage	Common	Specialized Systems

Table 24.4: ACL vs Capability

24.12 Privilege Separation

24.12.1 Definition

Separate privileged and unprivileged components.

24.12.2 Example

Web Browser:

- Renderer Process
- Network Process
- Sandbox Process

24.12.3 Benefits

- Reduced attack impact
- Better isolation

24.13 Principle of Least Privilege

24.13.1 Definition

Grant only the minimum permissions required.

24.13.2 Example

A web server should not run as:

`root`

Instead:

`www-data`

24.13.3 Benefits

- Reduced damage
- Improved security

24.14 Secure Kernel Design

Secure kernels focus on:

- Isolation
- Access Control
- Memory Protection
- Auditing
- Secure IPC

24.14.1 Goals

- Small trusted computing base
- Minimal attack surface
- Strong privilege boundaries

24.15 Memory Protection

24.15.1 Definition

Memory protection prevents unauthorized access between processes.

24.15.2 Mechanisms

- Virtual Memory
- MMU
- Page Tables
- User Mode
- Kernel Mode

24.15.3 Benefits

- Process Isolation
- Fault Containment
- Security Enforcement

24.16 ASLR

24.16.1 Definition

ASLR stands for Address Space Layout Randomization.

24.16.2 Goal

Randomize memory locations.

24.16.3 Protected Areas

- Stack

- Heap
- Libraries
- Executable Regions

24.16.4 Benefit

Makes exploitation significantly harder.

24.16.5 Linux

Example

```
cat /proc/sys/kernel/randomize_va_space
```

24.17 DEP / NX Bit

24.17.1 Definition

DEP (Data Execution Prevention) prevents execution from data pages.

NX = No Execute.

24.17.2 Purpose

Stop code execution in:

- Stack
- Heap
- Data Regions

24.17.3 Mitigates

- Buffer Overflow Attacks
- Code Injection

24.18 Sandboxing

24.18.1 Definition

Sandboxing isolates applications from the rest of the system.

24.18.2 Examples

- Chrome Sandbox
- Android App Sandbox
- iOS App Sandbox
- Firejail

24.18.3 Architecture

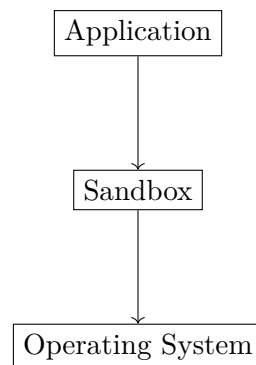


Figure 24.2: Application Sandbox

24.19 Secure

Boot

24.19.1 Definition

Secure Boot verifies cryptographic signatures during boot.

24.19.2 Goal

Prevent unauthorized software from loading.

24.19.3 Boot

Chain

1. Firmware
2. Bootloader
3. Kernel
4. Operating System

24.19.4 Benefits

- Rootkit Protection

- Boot-Time Integrity

24.20 SELinux

Overview

24.20.1 Definition

SELinux stands for Security Enhanced Linux.

24.20.2 Security

Model

Mandatory Access Control (MAC)

24.20.3 Features

- Fine-grained policies
- Process isolation
- File labeling
- Security contexts

24.20.4 Example

```
getenforce
```

Possible output:

```
Enforcing
```

24.21 AppArmor

Overview

24.21.1 Definition

AppArmor is a Linux security framework.

24.21.2 Approach

Path-based access control.

24.21.3 Advantages

- Easier policy management

- Simpler configuration

24.21.4 Common

Usage

- Ubuntu
- SUSE Linux

24.22 Container

Security

24.22.1 Challenges

Containers share the host kernel.

24.22.2 Security

Mechanisms

- Namespaces
- Cgroups
- Capabilities
- Seccomp
- SELinux
- AppArmor

24.22.3 Container

Isolation

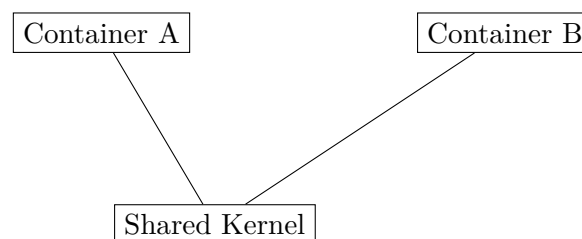


Figure 24.3: Container Isolation

24.23 Common

Vulnerabilities

24.23.1 Buffer

Overflow

Writing beyond memory boundaries.

24.23.2 Privilege Escalation

Obtaining higher permissions.

24.23.3 Race Conditions

Timing-based attacks.

24.23.4 Code Injection

Injecting malicious code.

24.23.5 Directory Traversal

Accessing unintended files.

24.23.6 Use-After-Free

Accessing freed memory.

24.24 Real-World Attack Scenarios

24.24.1 Scenario 1

Web server running as root.

Result:

- Full system compromise.

24.24.2 Scenario 2

No ASLR.

Result:

- Easier exploitation.

24.24.3 Scenario 3

Weak container isolation.

Result:

- Container escape attack.

24.24.4 Scenario

4

Misconfigured ACLs.

Result:

- Data leakage.

24.25 How Modern OSes Isolate Applications

Modern operating systems isolate applications using:

- Virtual Memory
- Process Boundaries
- User/Kernel Modes
- Sandboxing
- Containers
- SELinux
- AppArmor
- Access Control Systems

24.25.1 Isolation

Layers

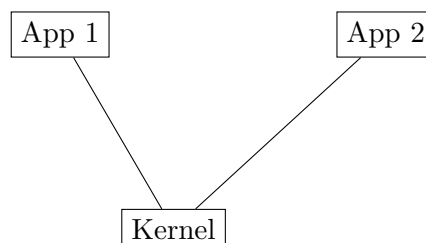


Figure 24.4: Application Isolation

24.26 Security Best Practices

- Follow least privilege.
- Keep systems updated.
- Use strong authentication.
- Enable ASLR.
- Enable DEP/NX.

- Use secure boot.
- Audit permissions regularly.
- Isolate workloads.
- Minimize attack surface.
- Monitor security logs.

24.27 Key

Takeaways

- Authentication verifies identity.
- Authorization verifies permissions.
- ACLs and capabilities implement access control.
- Least privilege is a fundamental security principle.
- Memory protection isolates processes.
- ASLR randomizes memory layouts.
- DEP prevents execution from data pages.
- Sandboxing limits application damage.
- SELinux and AppArmor strengthen Linux security.
- Containers require additional security controls.

24.28 20 Interview Questions with Answers

1. What is OS security?
Answer: Protection of system resources from unauthorized access.
2. What is authentication?
Answer: Identity verification.
3. What is authorization?
Answer: Permission verification.
4. Difference between authentication and authorization?
Answer: Authentication identifies users; authorization determines permissions.
5. What is an ACL?
Answer: Object-based permission list.
6. What is a capability?
Answer: Subject-held permission token.
7. What is least privilege?
Answer: Grant minimum necessary permissions.
8. What is ASLR?
Answer: Address Space Layout Randomization.
9. Why use ASLR?

Answer: Prevent predictable memory layouts.

10. What is DEP?

Answer: Data Execution Prevention.

11. What is NX Bit?

Answer: Marks memory non-executable.

12. What is sandboxing?

Answer: Application isolation mechanism.

13. What is secure boot?

Answer: Verified boot process.

14. What is SELinux?

Answer: Mandatory access control framework.

15. What is AppArmor?

Answer: Path-based Linux security framework.

16. What is privilege escalation?

Answer: Gaining unauthorized privileges.

17. What is container isolation?

Answer: Separation using namespaces and cgroups.

18. What is a threat model?

Answer: Structured analysis of threats and defenses.

19. Why is memory protection important?

Answer: Prevents process interference.

20. What is the CIA triad?

Answer: Confidentiality, Integrity, Availability.

24.29 20 MCQs with Answers

1. Authentication verifies: Answer: Identity
2. Authorization verifies: Answer: Permissions
3. CIA stands for: Answer: Confidentiality, Integrity, Availability
4. ACLs are associated with: Answer: Objects
5. Capabilities are associated with: Answer: Subjects
6. ASLR randomizes: Answer: Memory Layout
7. DEP prevents: Answer: Code Execution in Data Pages
8. NX stands for: Answer: No Execute
9. SELinux implements: Answer: Mandatory Access Control
10. AppArmor is: Answer: Path-Based Security
11. Secure Boot protects: Answer: Boot Process
12. Least privilege means: Answer: Minimum Permissions
13. Container isolation uses: Answer: Namespaces

14. Rootkits commonly target: Answer: Boot and Kernel Layers
15. Sandboxing provides: Answer: Isolation
16. Buffer overflow affects: Answer: Memory Safety
17. Privilege escalation increases: Answer: Access Rights
18. Threat modeling identifies: Answer: Risks
19. Virtual memory improves: Answer: Isolation
20. Secure kernels reduce: Answer: Attack Surface

24.30 Revision

Notes

- Authentication = Identity.
- Authorization = Permissions.
- ACL = Object-based permissions.
- Capability = Subject-based permissions.
- Least privilege minimizes risk.
- ASLR randomizes memory addresses.
- DEP blocks execution from data memory.
- Sandboxing isolates applications.
- Secure Boot protects startup integrity.
- SELinux uses MAC policies.
- AppArmor uses path-based controls.
- Containers require layered security.

24.31 Cheat

Sheet

Authentication	Identity Verification
Authorization	Permission Verification
ACL	Object-Based Permissions
Capability	Subject-Based Permissions
Least Privilege	Minimum Required Access
ASLR	Randomized Memory Layout
DEP	Data Execution Prevention
NX Bit	Non-Executable Memory
Sandbox	Application Isolation
Secure Boot	Verified Startup Chain
SELinux	Mandatory Access Control
AppArmor	Path-Based Access Control
Container Security	Namespaces + Cgroups
Memory Protection	Process Isolation
CIA	Confidentiality Integrity Availability
Threat Model	Assets Threats Defenses
Privilege Separation	Isolated Components
Buffer Overflow	Memory Corruption Attack
Privilege Escalation	Unauthorized Permission Gain
Secure Kernel	Protected Core OS Component

Part IX

Operating Systems for System Design

High-Concurrency Systems

25.1 Introduction

Modern internet-scale systems must handle thousands to millions of concurrent users while maintaining low latency and high throughput. High-concurrency systems combine operating system principles, networking, distributed systems, and efficient software architecture.

Examples include:

- Web servers
- API gateways
- Database servers
- Message brokers
- Streaming platforms
- Cloud services

Understanding high-concurrency systems is essential for SDE, Backend Engineering, Distributed Systems, and System Design interviews.

25.2 What High Concurrency Means

Concurrency refers to the ability of a system to manage multiple tasks simultaneously.

Examples:

- 10,000 simultaneous TCP connections
- Millions of HTTP requests per minute
- Thousands of database queries per second

High concurrency does not necessarily mean high parallelism.

- Concurrency = Managing many tasks
- Parallelism = Executing many tasks simultaneously

25.3 Concurrency

Challenges

Major challenges include:

- Resource contention
- Synchronization overhead
- Memory consumption
- Context switching
- Lock contention
- Kernel bottlenecks
- Queue buildup
- Latency spikes

25.4 Throughput

vs

Latency

25.4.1 Throughput

Amount of work completed per unit time.

Examples:

- Requests per second (RPS)
- Transactions per second (TPS)
- Queries per second (QPS)

25.4.2 Latency

Time required to complete a request.

Examples:

- 10 ms API response
- 2 ms database query

Metric	Meaning	Goal
Throughput	Work Completed	High
Latency	Response Time	Low

Table 25.1: Throughput vs Latency

25.5 Thread-per-Request

Model

Traditional servers allocate one thread per request.

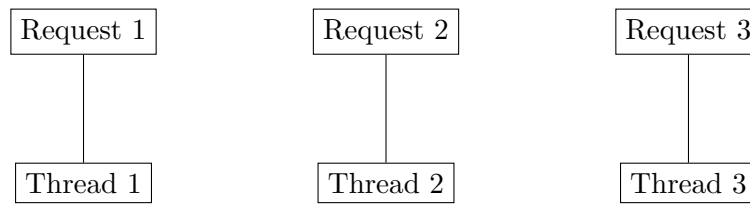


Figure 25.1: Thread-per-Request Model

Advantages:

- Simple design
- Easy programming model

Problems:

- High memory usage
- Context switching overhead
- Poor scalability

25.6 Event-Driven

Architecture

Instead of dedicating one thread per request, an event loop handles many connections.

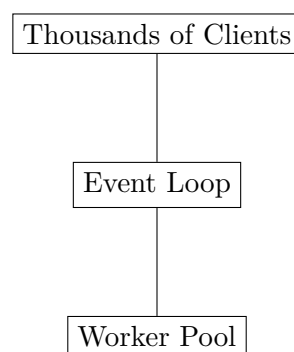


Figure 25.2: Event-Driven Architecture

Benefits:

- Low memory usage
- Better scalability
- Efficient I/O handling

25.7 Reactor

Pattern

The Reactor pattern waits for events and dispatches handlers.

Workflow:

1. Event arrives
2. Event loop detects readiness
3. Appropriate handler executes
4. Response generated

Examples:

- Nginx
- Redis
- Node.js

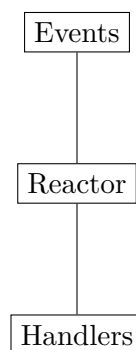


Figure 25.3: Reactor Pattern

25.8 Proactor

Pattern

The Proactor pattern uses asynchronous operations.

Workflow:

1. Submit asynchronous request
2. Kernel performs work
3. Completion event generated
4. Completion handler executes

Examples:

- Windows IOCP
- Linux `io_uring`

25.9 Non-Blocking

I/O

Non-blocking I/O returns immediately.

Example:

```
1 fcntl(fd, F_SETFL, O_NONBLOCK);
```

Benefits:

- Efficient event loops
- Better scalability

25.10 Asynchronous

I/O

Asynchronous I/O allows processing to continue while operations execute.

Submit —→ Continue Work —→ Completion Event

Figure 25.4: Asynchronous I/O

Advantages:

- High concurrency
- Reduced idle waiting

25.11 Thread

Pools

Creating threads for every request is expensive.

Thread pools maintain reusable workers.

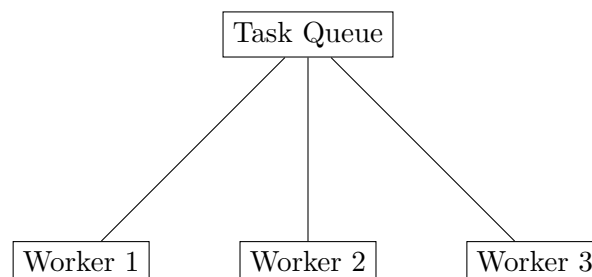


Figure 25.5: Thread Pool

Benefits:

- Reduced creation overhead

- Better throughput
- Predictable resource usage

25.12 Connection

Pools

Opening database connections is expensive.

Connection pools maintain reusable connections.

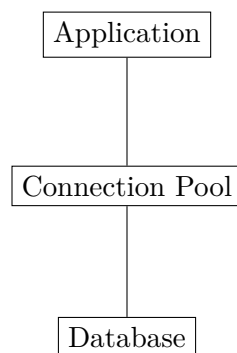


Figure 25.6: Connection Pool

Benefits:

- Reduced latency
- Better scalability
- Resource reuse

25.13 Load

Balancing

Load balancers distribute traffic across servers.

Algorithms:

- Round Robin
- Least Connections
- Weighted Round Robin
- Consistent Hashing

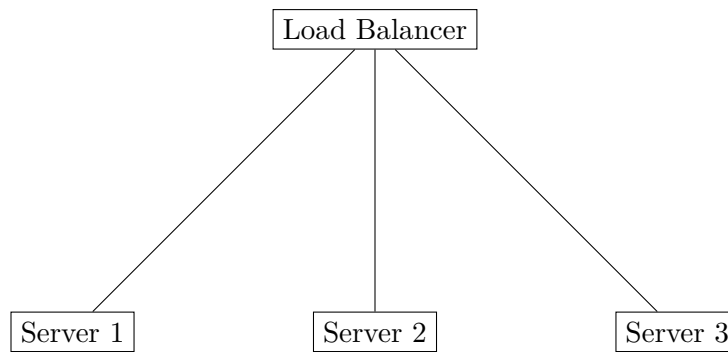


Figure 25.7: Load Balancing

25.14 Context Switching Costs

Context switching requires:

- Register saving
- Register restoration
- Cache disruption
- Scheduler overhead

Excessive threads lead to:

- Increased CPU overhead
- Reduced throughput
- Higher latency

25.15 Kernel Bottlenecks

Common bottlenecks include:

- Lock contention
- Scheduler overhead
- Interrupt handling
- Socket management
- File descriptor limits

25.16 Resource Contention

Multiple threads competing for resources can reduce performance.

Examples:

- CPU contention
- Memory contention
- Lock contention
- Disk contention

Mitigation strategies:

- Lock-free structures
- Sharding
- Partitioning
- Horizontal scaling

25.17 Backpressure

Backpressure prevents overload.

Example:

1. Producer generates faster than consumer
2. Queue grows
3. System slows producer

Benefits:

- Stability
- Controlled latency
- Avoiding crashes

25.18 Queue-Based

Architectures

Queues decouple producers and consumers.

Examples:

- Kafka
- RabbitMQ
- Amazon SQS

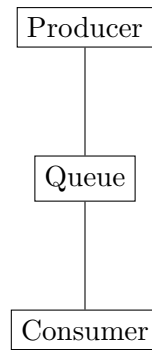


Figure 25.8: Queue-Based Architecture

25.19 High-Concurrency Web Servers

Modern web servers rely on:

- epoll
- Event loops
- Worker pools
- Non-blocking sockets

Examples:

- Nginx
- Envoy
- HAProxy

25.19.1 Nginx-Style Architecture

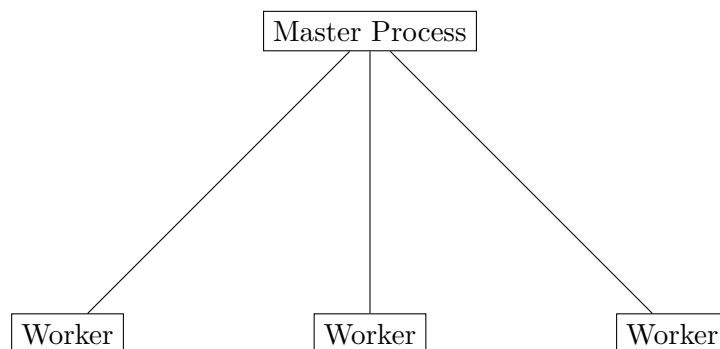


Figure 25.9: Nginx Worker Architecture

25.20 High-Concurrency Databases

Databases use:

- Connection pools
- Buffer pools
- Async replication
- Lock management
- Query optimization

Examples:

- PostgreSQL
- MySQL
- Cassandra
- MongoDB

25.21 Modern Backend Architectures

Modern systems commonly combine:

- Load balancers
- API gateways
- Microservices
- Message queues
- Databases
- Distributed caches

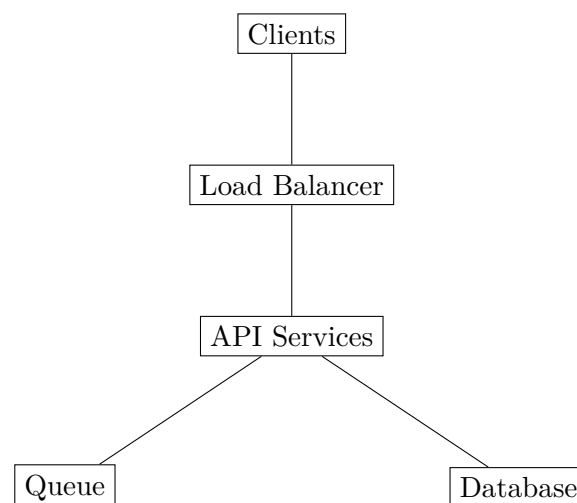


Figure 25.10: Modern Backend Architecture

25.22 Dedicated Interview Section: How a Web Server Handles Millions of Requests

Modern web servers use:

1. Non-blocking sockets
2. epoll or kqueue
3. Event loops
4. Worker processes
5. Load balancing
6. Efficient memory management

A single worker can manage thousands of connections because idle connections consume minimal CPU.

25.23 Dedicated Interview Section: Why Thread-per-Request Does Not Scale

Problems:

- Large memory consumption
- High context switching overhead
- Scheduler pressure
- Lock contention

Example:

- 100,000 requests
- 100,000 threads
- Several hundred GB memory required

Event-driven architectures solve this issue.

25.24 Dedicated Interview Section: Event-Driven Systems Explained

Event-driven systems process events rather than blocking per request.

Workflow:

1. Event arrives

2. Event loop receives notification
3. Handler executes
4. Response returned

Examples:

- Node.js
- Nginx
- Redis
- Netty

Benefits:

- High scalability
- Low resource consumption
- Excellent concurrency

25.25 Key

Takeaways

Memory and Performance Optimization

26.1 Introduction

Modern software performance is heavily influenced by memory behavior. While CPU frequencies have increased over the years, memory latency improvements have been much slower. As a result, understanding operating system memory concepts is essential for building scalable, high-performance systems.

In system design interviews and real-world engineering, memory efficiency often determines:

- Application throughput
- Response latency
- Scalability
- Resource utilization
- Infrastructure cost

This chapter connects operating system memory management concepts with practical software engineering performance optimization.

26.2 Performance

Fundamentals

26.2.1 Definition

Performance measures how efficiently a system completes work.

Common metrics include:

- Throughput
- Latency
- CPU Utilization
- Memory Usage
- Requests Per Second (RPS)

- Tail Latency (P95, P99)

26.2.2 Performance

Equation

$$Performance = \frac{Work}{Time}$$

26.2.3 Key

Observation

Many applications are not CPU-bound but memory-bound.

26.3 CPU

Cache

Hierarchy

26.3.1 Motivation

Main memory is much slower than CPU execution speed.

Caches reduce memory access latency.

26.3.2 Hierarchy

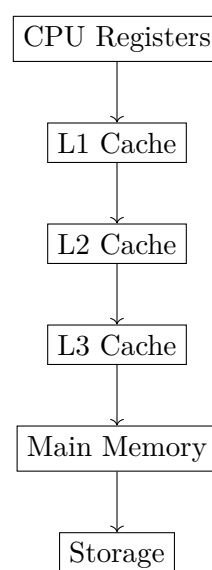


Figure 26.1: Memory Hierarchy

Level	Speed	Size
Registers	Fastest	Smallest
L1 Cache	Very Fast	Small
L2 Cache	Fast	Medium
L3 Cache	Slower	Larger
RAM	Slow	Large
Disk	Slowest	Huge

Table 26.1: Memory Hierarchy Characteristics

26.3.3 Characteristics

26.4 Cache

Locality

26.4.1 Definition

Cache locality refers to the tendency of programs to access the same or nearby memory locations repeatedly.

26.4.2 Importance

Good locality improves:

- Cache hit rate
- CPU efficiency
- Throughput

Poor locality causes:

- Cache misses
- Memory stalls
- Increased latency

26.5 Spatial

Locality

26.5.1 Definition

If a memory location is accessed, nearby locations are likely to be accessed soon.

26.5.2 Example

```
for(int i=0;i<n;i++)
    sum += arr[i];
```

Sequential array access exhibits strong spatial locality.

26.5.3 Benefit

Entire cache lines become useful.

26.6 Temporal

Locality

26.6.1 Definition

Recently accessed memory is likely to be accessed again soon.

26.6.2 Example

```
count++;  
count++;  
count++;
```

The variable remains in cache.

26.6.3 Benefit

Reduces repeated memory fetches.

26.7 Cache

Lines

26.7.1 Definition

Caches transfer data in fixed-size units called cache lines.

Typical size:

64 Bytes

26.7.2 Illustration

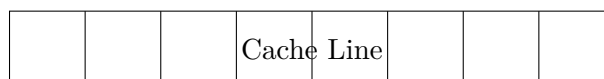


Figure 26.2: Cache Line

26.7.3 Interview

Insight

Most cache optimization discussions revolve around cache lines.

26.8 False

Sharing

26.8.1 Definition

False sharing occurs when multiple threads update different variables located on the same cache line.

26.8.2 Example

Thread A -> counter1

Thread B -> counter2

Even though variables differ, cache invalidations occur.

26.8.3 Effects

- Cache contention
- Reduced scalability
- Throughput degradation

26.8.4 Solution

Padding structures to separate cache lines.

26.9 TLB

Overview

26.9.1 Definition

TLB stands for Translation Lookaside Buffer.

It caches virtual-to-physical address translations.

26.9.2 Address

Translation

$VirtualAddress \rightarrow TLB \rightarrow PhysicalAddress$

26.9.3 Purpose

Avoid expensive page table lookups.

26.10 TLB Performance Impact

26.10.1 TLB Hit

Address translation is immediate.

26.10.2 TLB Miss

Page table lookup is required.

26.10.3 Impact

TLB misses can significantly increase memory access time.

26.10.4 Interview Insight

Many high-performance systems use huge pages to reduce TLB misses.

26.11 Memory Access Costs

Approximate latencies:

Operation	Latency
Register Access	1 Cycle
L1 Cache	4 Cycles
L2 Cache	12 Cycles
L3 Cache	40 Cycles
RAM Access	100-300 Cycles
SSD Access	Thousands of Cycles

Table 26.2: Memory Access Costs

26.11.1 Observation

A RAM access can be hundreds of times slower than an L1 cache access.

26.12 Branch

Prediction

26.12.1 Definition

Modern CPUs predict branch outcomes.

26.12.2 Example

```
if(x > 0)
    doWork();
```

CPU predicts likely path.

26.12.3 Correct

Prediction

Pipeline continues smoothly.

26.12.4 Misprediction

Pipeline flush occurs.

26.12.5 Impact

Can significantly affect performance.

26.13 Pipeline

Effects

26.13.1 Pipeline

CPU instructions execute in stages.

26.13.2 Benefits

- Higher throughput
- Better CPU utilization

26.13.3 Problems

- Branch mispredictions
- Cache misses

- Data hazards

26.13.4 Result

Pipeline stalls reduce performance.

26.14 Memory

Allocation

Costs

26.14.1 Dynamic

Allocation

Examples:

`malloc()`

`new`

26.14.2 Costs

- Metadata management
- Synchronization
- Fragmentation
- Cache disruption

26.14.3 Optimization

Use:

- Memory pools
- Object pools
- Arena allocators

26.15 NUMA

Effects

26.15.1 NUMA

Non-Uniform Memory Access.

26.15.2 Concept

Memory near a CPU is faster than remote memory.

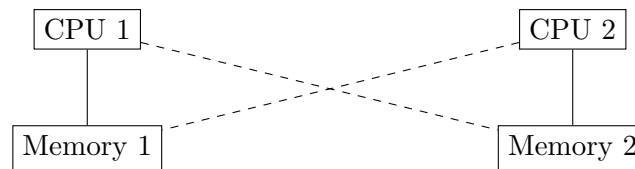


Figure 26.3: NUMA Architecture

26.15.3 Performance

Implications

Remote memory accesses are slower.

26.15.4 Optimization

Keep threads close to their memory.

26.16 Memory

Leaks

26.16.1 Definition

Allocated memory that is never released.

26.16.2 Example

```
int* p = new int[100];
// Missing delete[]
```

26.16.3 Effects

- Increased memory usage
- Performance degradation
- Application crashes

26.17 Fragmentation

26.17.1 Internal

Fragmentation

Unused space inside allocated blocks.

26.17.2 External

Fragmentation

Free memory exists but is scattered.

26.17.3 Effects

- Allocation failures
- Increased memory overhead

26.18 Garbage Collection Overview

26.18.1 Definition

Automatic memory reclamation.

26.18.2 Languages

- Java
- C#
- Go
- Python

26.18.3 Advantages

- Simpler memory management
- Reduced memory leaks

26.18.4 Disadvantages

- Pause times
- CPU overhead
- Memory overhead

26.19 Performance Profiling

26.19.1 Definition

Measuring where execution time is spent.

26.19.2 Popular Tools

Linux:

- perf

- valgrind
- gprof
- flame graphs

Java:

- VisualVM
- JProfiler
- YourKit

26.19.3 Goal

Find bottlenecks before optimizing.

26.20 Benchmarking

Fundamentals

26.20.1 Definition

Controlled measurement of performance.

26.20.2 Rules

- Warm up application
- Measure multiple runs
- Remove noise
- Use realistic workloads

26.20.3 Example

Time taken:

Array Access = 10 ms

Linked List Access = 150 ms

26.21 Performance

Case

Study

26.21.1 Scenario

Backend service processes millions of requests.

Initial implementation:

- Linked Lists
- Frequent allocations
- Poor cache locality

Problems:

- High latency
- Low throughput

Optimization:

- Arrays
- Object pooling
- Reduced allocations

Result:

- Better cache locality
- Lower latency
- Higher throughput

26.22 Optimization

Trade-Offs

Optimization	Trade-Off
Caching	More Memory Usage
Object Pooling	Complexity
Lock-Free Structures	Difficult Debugging
Huge Pages	Memory Waste
Manual Memory Management	Bug Risk

Table 26.3: Optimization Trade-Offs

26.23 Why

Cache

Locality

Matters

Cache locality often provides larger performance gains than algorithmic micro-optimizations.

Benefits:

- Higher cache hit rates
- Reduced memory latency
- Better CPU utilization
- Improved scalability

Interview Tip:

Arrays are often faster than linked lists because of cache locality.

26.24 Common Performance Bottlenecks

- Cache misses
- TLB misses
- Memory allocations
- Garbage collection pauses
- Lock contention
- False sharing
- NUMA effects
- Excessive I/O

26.25 How Memory Affects Scalability

Memory influences:

- Request throughput
- CPU efficiency
- Horizontal scaling cost
- Response latency
- Cloud infrastructure expenses

Poor memory behavior often limits scalability before CPU becomes the bottleneck.

26.26 Key Takeaways

- Memory performance is critical for modern applications.
- Cache locality significantly affects execution speed.
- Cache misses are expensive.
- False sharing harms scalability.
- TLB performance affects address translation costs.
- NUMA introduces memory access asymmetry.
- Memory leaks reduce long-term stability.
- Profiling should precede optimization.
- Benchmarking requires realistic workloads.

- Every optimization has trade-offs.

26.27 20 Interview Questions with Answers

1. What is cache locality?
Answer: Tendency to access nearby or recently used memory.
2. What is spatial locality?
Answer: Accessing nearby memory locations.
3. What is temporal locality?
Answer: Reusing recently accessed memory.
4. What is a cache line?
Answer: Smallest cache transfer unit.
5. Typical cache line size?
Answer: 64 bytes.
6. What is false sharing?
Answer: Threads modifying data on same cache line.
7. What is a TLB?
Answer: Translation Lookaside Buffer.
8. Why is TLB important?
Answer: Speeds address translation.
9. What causes TLB misses?
Answer: Missing cached translations.
10. What is NUMA?
Answer: Non-Uniform Memory Access.
11. Why is NUMA important?
Answer: Memory access latency varies.
12. What is branch prediction?
Answer: CPU predicts branch outcomes.
13. What is pipeline flushing?
Answer: Discarding incorrect speculative execution.
14. What is memory fragmentation?
Answer: Inefficient memory utilization.
15. What is a memory leak?
Answer: Unreleased allocated memory.
16. Why profile before optimizing?
Answer: To identify actual bottlenecks.
17. What is benchmarking?
Answer: Measuring performance systematically.
18. Why are arrays cache-friendly?
Answer: Contiguous memory layout.

19. What is garbage collection?

Answer: Automatic memory reclamation.

20. How does memory affect scalability?

Answer: Influences latency, throughput, and resource usage.

26.28 20 MCQs with Answers

1. Fastest memory component? Answer: Registers
2. Slowest storage layer? Answer: Disk
3. Typical cache line size? Answer: 64 Bytes
4. Spatial locality accesses: Answer: Nearby Memory
5. Temporal locality accesses: Answer: Recently Used Memory
6. False sharing impacts: Answer: Performance
7. TLB stores: Answer: Address Translations
8. NUMA stands for: Answer: Non-Uniform Memory Access
9. Cache misses increase: Answer: Latency
10. Branch misprediction causes: Answer: Pipeline Flush
11. Garbage collection performs: Answer: Memory Reclamation
12. Memory leaks cause: Answer: Memory Growth
13. External fragmentation means: Answer: Scattered Free Space
14. Profiling identifies: Answer: Bottlenecks
15. Benchmarking measures: Answer: Performance
16. Arrays provide: Answer: Good Locality
17. Linked lists often suffer: Answer: Poor Locality
18. L1 cache is: Answer: Fastest Cache
19. Object pools reduce: Answer: Allocation Cost
20. NUMA-aware placement improves: Answer: Performance

26.29 Revision

Notes

- Memory hierarchy dominates performance.
- Cache locality is critical.
- Spatial locality = nearby access.
- Temporal locality = repeated access.
- Cache lines are typically 64 bytes.
- False sharing hurts scalability.
- TLB caches address translations.
- NUMA affects memory latency.

- Profile before optimizing.
- Benchmark using realistic workloads.
- Memory leaks waste resources.
- Every optimization has trade-offs.

26.30 Cheat

Sheet

L1 Cache	Fastest Cache
Spatial Locality	Nearby Accesses
Temporal Locality	Repeated Accesses
Cache Line	64 Bytes (Typical)
False Sharing	Shared Cache Line Contention
TLB	Address Translation Cache
TLB Miss	Page Table Lookup Required
NUMA	Non-Uniform Memory Access
Memory Leak	Unreleased Allocation
Fragmentation	Wasted Memory Space
Garbage Collection	Automatic Reclamation
Branch Prediction	Predict Execution Path
Pipeline Flush	Misprediction Penalty
Profiling	Find Bottlenecks
Benchmarking	Measure Performance
Object Pool	Reuse Objects
Cache Miss	Slow Memory Access
Cache Hit	Fast Access
Locality	Performance Multiplier
Optimization	Trade-Off Driven

Distributed Systems Through the Lens of Operating Systems

27.1 Introduction

Operating systems manage resources, communication, concurrency, synchronization, memory, scheduling, and fault handling within a single machine. Distributed systems extend many of these concepts across multiple machines connected through a network.

Many distributed-system interview questions become easier when viewed through familiar operating-system concepts.

Examples:

- Processes become services
- Threads become workers
- Shared memory becomes distributed state
- IPC becomes RPC
- Mutexes become distributed locks
- Scheduling becomes orchestration
- Fault handling becomes distributed fault tolerance

27.2 Why Distributed Systems Exist

Single machines eventually hit limits.

Common limitations:

- CPU capacity
- Memory capacity
- Disk throughput
- Network throughput
- Availability requirements

Distributed systems address these limitations through:

- Horizontal scaling
- Replication
- Fault tolerance
- Geographic distribution
- Load sharing

Examples:

- Google Search
- Amazon
- Netflix
- Uber
- Meta

27.3 OS Concepts vs Distributed Systems Concepts

Operating Systems	Distributed Systems
Process	Service
Thread	Worker
Memory	Distributed State
IPC	RPC
Mutex	Distributed Lock
Scheduler	Orchestrator
Shared Memory	Distributed Cache
File System	Distributed Storage
Kernel	Control Plane
Crash Recovery	Fault Tolerance

Table 27.1: OS Concepts vs Distributed Systems

27.4 Processes vs Services

In operating systems:

- Process = executing program

In distributed systems:

- Service = independently deployable process



Figure 27.1: Services Communicating Across a Network

27.5 Threads vs Workers

Threads execute tasks within a process.

Distributed systems commonly use workers.

Examples:

- Kafka consumers
- Celery workers
- Background job processors
- Spark executors

Worker pools resemble thread pools.

27.6 Memory vs Distributed State

Operating systems use local RAM.

Distributed systems store state across machines.

Examples:

- Redis clusters
- Cassandra
- DynamoDB
- Distributed caches

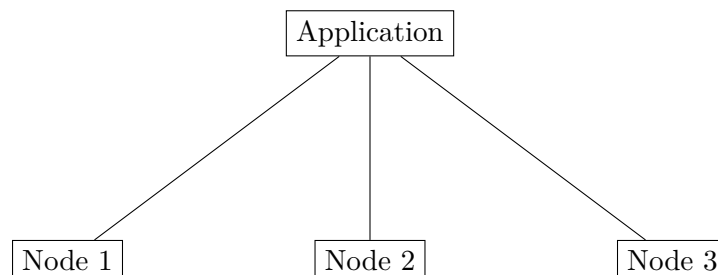


Figure 27.2: Distributed State

27.7 Local IPC vs Network Communication

Operating systems provide IPC mechanisms:

- Pipes
- Shared memory
- Message queues
- Sockets

Distributed systems use:

- HTTP
- RPC
- gRPC
- Message brokers

Key difference:

- IPC assumes local machine reliability
- Network communication assumes failures

27.8 RPC Fundamentals

Remote Procedure Call (RPC) allows one service to invoke functionality on another service as if calling a local function.

27.8.1 RPC Workflow

1. Client invokes function
2. Request serialized
3. Sent across network
4. Server executes request
5. Response serialized
6. Returned to client



Figure 27.3: RPC Communication

27.9 gRPC

Overview

gRPC is a high-performance RPC framework developed by **Google**.

Features:

- HTTP/2
- Protocol Buffers
- Streaming
- Strong typing
- Cross-language support

Advantages:

- Low latency
- Compact messages
- Efficient communication

27.10 Distributed

Locks

A mutex protects resources on one machine.

Distributed locks protect resources across multiple machines.

Common implementations:

- Redis Redlock
- ZooKeeper
- etcd
- Consul

Example:

1. Service A acquires lock
2. Service B waits
3. Service A releases lock
4. Service B proceeds

27.11 Consensus

Basics

Consensus is the process of multiple nodes agreeing on a value.

Requirements:

- Agreement
- Validity
- Termination

Applications:

- Metadata storage
- Cluster management
- Configuration systems
- Leader election

Popular algorithms:

- Paxos
- Raft
- Zab

27.12 Leader

Election

Distributed systems often elect one node as leader.

Responsibilities:

- Coordination
- Metadata management
- Decision making
- Replication control

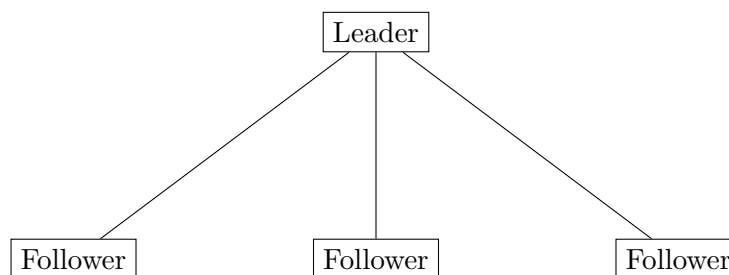


Figure 27.4: Leader Election

27.13 Clock

Synchronization

Distributed systems lack a perfectly shared clock.

Challenges:

- Network delay
- Clock drift
- Hardware differences

Common approaches:

- NTP
- PTP
- GPS clocks

27.14 Logical

Clocks

Introduced by Lamport.

Purpose:

- Order events
- Capture causality

Rule:

$$C(a) < C(b)$$

implies event a happened before event b .

27.15 Vector

Clocks

Vector clocks improve causal tracking.

Each node maintains a vector.

Example:

$$[3, 5, 2]$$

Advantages:

- Detect concurrent events
- Capture causal relationships

27.16 CAP

Theorem

A distributed system cannot simultaneously guarantee:

- Consistency
- Availability
- Partition Tolerance

During a partition:

- Choose Consistency
- Or choose Availability

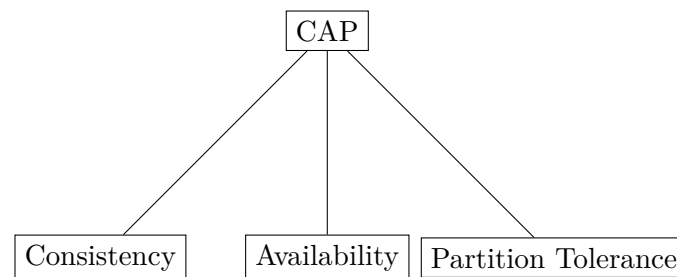


Figure 27.5: CAP Theorem

27.17 Consistency

Models

27.17.1 Strong

Consistency

Reads always return the latest write.

27.17.2 Eventual

Consistency

Replicas eventually converge.

27.17.3 Causal

Consistency

Causally related operations preserve order.

27.17.4 Read-Your-Writes

A client sees its own updates.

27.17.5 Monotonic

Reads

Reads never move backward in time.

27.18 Replication

Basics

Replication improves:

- Availability
- Scalability
- Fault tolerance

27.18.1 Primary-Replica

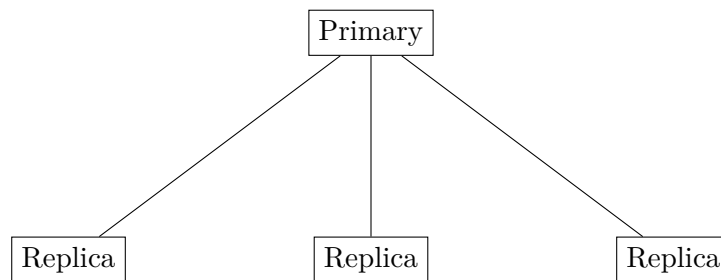


Figure 27.6: Primary-Replica Replication

27.18.2 Multi-Leader

Replication

Multiple leaders accept writes.

Advantages:

- Better geographic distribution

Challenges:

- Conflict resolution

27.19 Fault

Tolerance

Distributed systems assume failures.

Common failures:

- Node failures
- Disk failures

- Network failures
- Data-center failures

Strategies:

- Replication
- Retries
- Timeouts
- Circuit breakers
- Failover

27.20 Distributed

Transactions

Transactions spanning multiple nodes.

Challenges:

- Partial failures
- Network partitions
- Consistency requirements

Common protocols:

- Two-Phase Commit (2PC)
- Three-Phase Commit
- Saga Pattern

27.20.1 Two-Phase

Commit

1. Prepare Phase
2. Commit Phase

Guarantees atomicity but may block.

27.21 Modern

Cloud

Architectures

Modern systems typically include:

- Microservices
- Service mesh
- Distributed databases

- Message queues
- Object storage
- Containers
- Kubernetes

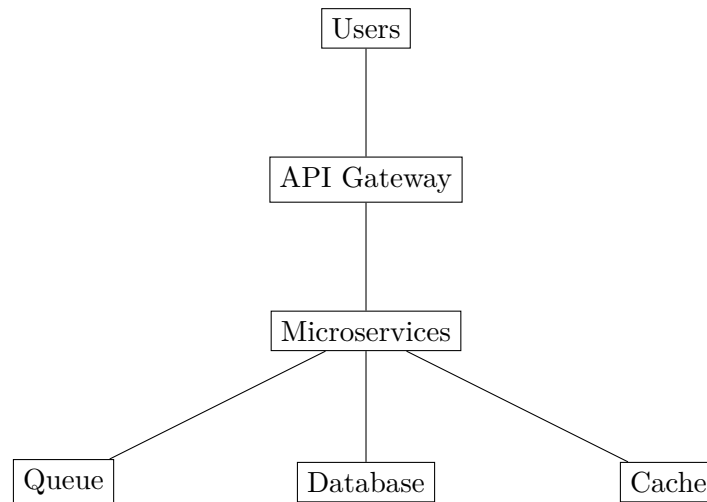


Figure 27.7: Modern Cloud Architecture

27.22 Dedicated Interview Section: How OS Concepts Appear in Distributed Systems

Operating System Concept	Distributed Equivalent
Process	Service
Thread	Worker
Mutex	Distributed Lock
Shared Memory	Distributed Cache
IPC	RPC
Scheduler	Kubernetes Scheduler
Crash Recovery	Failover
File System	Distributed Storage

Table 27.2: Concept Mapping

27.23 Dedicated Interview Section: Distributed Lock Interview Questions

Why are distributed locks needed?

Multiple machines may attempt to modify the same resource.

How are distributed locks implemented?

Commonly using:

- Redis
- ZooKeeper
- etcd

What can go wrong?

- Lock expiration
- Network partitions
- Split-brain scenarios

27.24 Dedicated Interview Section: Consistency Model Interview Questions

What is strong consistency?

Reads always return latest committed value.

What is eventual consistency?

Replicas converge eventually.

Why choose eventual consistency?

Improved availability and scalability.

Examples

- Strong Consistency: etcd
- Eventual Consistency: Dynamo-style systems

27.25 Key

Takeaways

Part X

Interview Mastery

Operating Systems Interview Questions

28.1 Introduction

This chapter is a consolidated interview-preparation resource covering the most important Operating Systems topics frequently asked in software engineering interviews.

28.2 Operating System Fundamentals

1. **Interview Question:** What is an operating system? **Short Answer:** Resource manager and abstraction layer. **Detailed Explanation:** Manages CPU, memory, storage, and devices. **Common Follow-up Questions:** Kernel? System calls? **Interview Tips:** Mention abstraction and resource allocation.
2. **Interview Question:** What are system calls? **Short Answer:** Interface to kernel services. **Detailed Explanation:** User programs access OS functionality through them. **Common Follow-up Questions:** fork()? exec()? **Interview Tips:** Explain mode switch.
3. **Interview Question:** What is a kernel?
4. **Interview Question:** Monolithic vs microkernel?
5. **Interview Question:** User mode vs kernel mode?
6. **Interview Question:** What is a trap?
7. **Interview Question:** What is an interrupt?
8. **Interview Question:** What is DMA?
9. **Interview Question:** What is bootstrapping?
10. **Interview Question:** What is a context switch?

28.3 Processes

11. Process lifecycle?
12. Process Control Block?
13. fork() vs exec()?

14. Zombie process?
15. Orphan process?
16. Process states?
17. Process creation?
18. Process termination?
19. Daemon process?
20. Copy-on-write during fork?

28.4 Threads

21. What is a thread?
22. Process vs thread?
23. User vs kernel threads?
24. Thread states?
25. Thread pool?
26. Multithreading benefits?
27. Thread safety?
28. Reentrant functions?
29. Thread-local storage?
30. Why are threads lightweight?

28.5 CPU

Scheduling

31. Scheduling objective?
32. FCFS scheduling?
33. SJF scheduling?
34. SRTF scheduling?
35. Round Robin?
36. Priority scheduling?
37. Starvation?
38. Aging?
39. Throughput vs turnaround time?
40. Context-switch overhead?

28.6 Synchronization

41. Race condition?

- 42. Critical section?
- 43. Mutual exclusion?
- 44. Progress condition?
- 45. Bounded waiting?
- 46. Peterson's algorithm?
- 47. Spinlock?
- 48. Semaphore?
- 49. Mutex?
- 50. Monitor?

28.7 Deadlocks

- 51. What is deadlock?
- 52. Four deadlock conditions?
- 53. Mutual exclusion condition?
- 54. Hold and wait?
- 55. No preemption?
- 56. Circular wait?
- 57. Deadlock prevention?
- 58. Deadlock avoidance?
- 59. Deadlock detection?
- 60. Deadlock recovery?

28.8 Memory

Management

- 61. Logical vs physical memory?
- 62. Memory allocation?
- 63. Fragmentation?
- 64. Internal fragmentation?
- 65. External fragmentation?
- 66. Compaction?
- 67. Buddy allocator?
- 68. Slab allocator?
- 69. Heap management?
- 70. Free lists?

28.9 Virtual

Memory

71. Why virtual memory?
72. Virtual address space?
73. Demand paging?
74. Page fault?
75. Valid vs invalid reference?
76. Working set model?
77. Thrashing?
78. Swap space?
79. Copy-on-write?
80. Memory-mapped files?

28.10 Paging

and

Segmentation

81. What is paging?
82. Page vs frame?
83. Page table?
84. TLB?
85. Effective access time?
86. Multilevel paging?
87. Inverted page table?
88. Hashed page table?
89. Segmentation?
90. Paging vs segmentation?

28.11 Page

Replacement

91. FIFO replacement?
92. Optimal replacement?
93. LRU replacement?
94. LFU replacement?
95. Second chance?
96. Clock algorithm?
97. Belady's anomaly?
98. Reference string?
99. Working set replacement?

- 100. Global vs local replacement?

28.12 File

Systems

- 101. File system purpose?
- 102. Inode?
- 103. Superblock?
- 104. File Control Block?
- 105. Contiguous allocation?
- 106. Linked allocation?
- 107. Indexed allocation?
- 108. Journaling?
- 109. ext4 architecture?
- 110. NTFS overview?

28.13 Disk

Scheduling

- 111. Disk scheduling objective?
- 112. FCFS disk scheduling?
- 113. SSTF?
- 114. SCAN?
- 115. C-SCAN?
- 116. LOOK?
- 117. C-LOOK?
- 118. Seek time?
- 119. Rotational latency?
- 120. Throughput optimization?

28.14 Linux

Internals

- 121. Linux architecture?
- 122. System call path?
- 123. proc filesystem?
- 124. sysfs?
- 125. CFS scheduler?
- 126. Signals?
- 127. Kernel modules?

- 128. systemd?
- 129. Linux boot process?
- 130. Kernel panic?

28.15 Containers and Virtualization

- 131. Why virtualization?
- 132. Hypervisor?
- 133. Type 1 hypervisor?
- 134. Type 2 hypervisor?
- 135. Full virtualization?
- 136. Para-virtualization?
- 137. Containers?
- 138. Namespaces?
- 139. cgroups?
- 140. Docker vs VM?

28.16 Operating System Security

- 141. Principle of least privilege?
- 142. Access control?
- 143. Authentication vs authorization?
- 144. Capability-based security?
- 145. Sandboxing?
- 146. ASLR?
- 147. Stack canaries?
- 148. Secure boot?
- 149. SELinux?
- 150. Privileged instructions?

28.17 High-Concurrency Systems

- 151. What is high concurrency?
- 152. Thread-per-request model?
- 153. Event-driven architecture?
- 154. Reactor pattern?
- 155. Proactor pattern?

- 156. Non-blocking I/O?
- 157. Asynchronous I/O?
- 158. epoll?
- 159. Backpressure?
- 160. How does Nginx scale?

28.18 Top 25 Most Frequently Asked Questions

- 1. Process vs Thread
- 2. Mutex vs Semaphore
- 3. Deadlock Conditions
- 4. What Happens During a Page Fault
- 5. Paging vs Segmentation
- 6. Virtual Memory
- 7. TLB
- 8. Context Switch
- 9. fork vs exec
- 10. Zombie Process
- 11. Orphan Process
- 12. CPU Scheduling Algorithms
- 13. Critical Section Problem
- 14. Peterson's Algorithm
- 15. Producer Consumer Problem
- 16. Readers Writers Problem
- 17. Dining Philosophers
- 18. Banker's Algorithm
- 19. LRU Page Replacement
- 20. Belady's Anomaly
- 21. Inode Structure
- 22. Linux Boot Process
- 23. System Calls
- 24. Docker vs VM
- 25. epoll vs select

28.19 Top 25 One-Line Rapid Revision Questions

- 1. What is a process?

2. What is a thread?
3. What is a PCB?
4. What is a page?
5. What is a frame?
6. What is TLB?
7. What is DMA?
8. What is a trap?
9. What is an interrupt?
10. What is deadlock?
11. What is starvation?
12. What is aging?
13. What is a semaphore?
14. What is a mutex?
15. What is thrashing?
16. What is swapping?
17. What is journaling?
18. What is an inode?
19. What is CFS?
20. What is a namespace?
21. What is a cgroup?
22. What is epoll?
23. What is a reactor?
24. What is a hypervisor?
25. What is copy-on-write?

28.20 Top**25****Deep-Dive****Questions**

1. Explain page-fault handling in detail.
2. Explain Linux process creation.
3. Explain Linux memory management.
4. Explain TLB shootdown.
5. Explain NUMA architecture.
6. Explain lock-free programming.
7. Explain compare-and-swap.
8. Explain monitor implementation.
9. Explain distributed locks.
10. Explain CFS scheduler internals.
11. Explain ext4 journaling.

12. Explain virtual memory in Linux.
13. Explain container isolation.
14. Explain namespaces and cgroups.
15. Explain epoll internals.
16. Explain event-driven architectures.
17. Explain context-switch costs.
18. Explain Banker's Algorithm.
19. Explain demand paging.
20. Explain working-set model.
21. Explain slab allocation.
22. Explain page replacement trade-offs.
23. Explain kernel-user transitions.
24. Explain memory ordering.
25. Explain high-concurrency server design.

28.21 Interview Preparation Strategy

1. Master operating-system fundamentals.
2. Understand processes and threads thoroughly.
3. Practice synchronization problems.
4. Solve deadlock scenarios.
5. Learn memory management numericals.
6. Practice page replacement questions.
7. Understand Linux internals.
8. Learn virtualization and containers.
9. Connect OS concepts to system design.
10. Conduct mock interviews regularly.

28.22 Last-Minute Revision Advice

- Revise Process vs Thread.
- Revise Mutex vs Semaphore.
- Memorize deadlock conditions.
- Practice page-fault handling.
- Revise TLB and paging.
- Understand copy-on-write.
- Review CPU scheduling algorithms.

- Revise producer-consumer and readers-writers.
- Review Linux process lifecycle.
- Revise Docker and container isolation.
- Review epoll and event-driven systems.
- Practice concise interview explanations.

28.23 Key

Takeaways

Operating Systems Numerical Problems

29.1 Introduction

This chapter serves as a comprehensive workbook for Operating Systems numerical problems commonly asked in:

- Software Development Engineer Interviews
- University Examinations
- GATE
- Competitive Programming Assessments
- Systems Engineering Interviews

Each section contains:

- Formula Reviews
- Solved Numerical Examples
- Interview Tricks
- Practice Problems
- Answers

29.2 Important	Formula	Summary
29.2.1 CPU		Scheduling

$$[\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}]$$

$$[\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}]$$

$$[\text{Response Time} = \text{First Execution} - \text{Arrival Time}]$$

$$[\text{Average Waiting Time} = \frac{\sum \text{Waiting Times}}{\text{Number of Processes}}]$$

$$[\text{Average Turnaround Time} = \frac{\sum \text{Turnaround Times}}{\text{Number of Processes}}]$$

29.2.2 Paging

$$[\text{Page Number} = \left\lfloor \frac{\text{Logical Address}}{\text{Page Size}} \right\rfloor]$$

$$[\text{Offset} = \text{Logical Address} \bmod \text{Page Size}]$$

$$[\text{Physical Address} = \text{Frame} \times \text{Page Size} + \text{Offset}]$$

29.2.3 Effective**Access****Time**

$$[\text{EAT} = (1-p) \times m + p \times (\text{page fault cost})]$$

29.2.4 Disk**Scheduling**

$$[\text{Total Head Movement} = \text{=====}]$$

$$\sum | \text{Current} - \text{Next} |]$$

29.3 CPU Scheduling Numericals

29.3.1 FCFS Formula Review

Processes execute in arrival order.

29.3.2 Solved Problem 1

Given

Process	Burst Time	height
P1	10	5
P2	8	3
P3	15	2

Gantt Chart

[0 P1 10 P2 15 P3 23]

Waiting Times:

[WT(P1)=0]

[WT(P2)=10]

[WT(P3)=15]

Average Waiting Time:

[$0+10+15$ $\frac{\quad}{3}$]

8.33]

Answer: 8.33 ms

29.3.3 Solved Problem 2

Burst Times:

[4,;6,;3,;7]

Waiting Times:

[0,;4,;10,;13]

Average:

[27 $\frac{\quad}{4}$]

6.75]

Answer: 6.75

29.3.4 Solved Problem 3

Arrival Times:

[0,,2,,4]

Burst Times:

[5,,3,,1]

Gantt Chart:

[0-5-8-9]

Turnaround Times:

[5,,6,,5]

Average TAT:

[16 $\frac{\text{=====}}{3}$
5.33]

Answer: 5.33

29.3.5 Solved Problems 4–10

1. Bursts: 2,4,6 $\rightarrow$ Avg WT = 2
2. Bursts: 1,3,5 $\rightarrow$ Avg WT = 1.67
3. Bursts: 8,4,2 $\rightarrow$ Avg WT = 6.67
4. Bursts: 9,5,1 $\rightarrow$ Avg WT = 7.67
5. Bursts: 3,3,3 $\rightarrow$ Avg WT = 3
6. Bursts: 4,8,12 $\rightarrow$ Avg WT = 5.33
7. Bursts: 10,20,30 $\rightarrow$ Avg WT = 13.33

29.3.6 Interview Shortcut

For FCFS:

[WT_i =====

$\sum \text{Burst Times Before Process}_i$]

29.3.7 SJF Formula Review

Shortest burst executes first.

29.3.8 Solved**Problem****11**

Burst Times:

[6,;8,;7,;3]

Order:

[P4,P1,P3,P2]

Waiting Times:

[0,;3,;9,;16]

Average:

$$\left[\frac{28}{4} \right]$$
Answer: 7**29.3.9 Solved****Problems****12–20**

1. Bursts: 4,2,8 $\rightarrow$ Avg WT = 2
2. Bursts: 9,1,5 $\rightarrow$ Avg WT = 2.33
3. Bursts: 10,3,6 $\rightarrow$ Avg WT = 4
4. Bursts: 8,4,2,1 $\rightarrow$ Avg WT = 2.75
5. Bursts: 7,7,7 $\rightarrow$ Avg WT = 7
6. Bursts: 3,2,1 $\rightarrow$ Avg WT = 1.33
7. Bursts: 5,10,15 $\rightarrow$ Avg WT = 5
8. Bursts: 12,4,8 $\rightarrow$ Avg WT = 5.33
9. Bursts: 11,1,2,3 $\rightarrow$ Avg WT = 2.5

29.3.10 SRTF**Problems****29.3.11 Solved****Problem****21**

Processes:

P	AT	BT	height
0	8	P2	1
4	P3	2	2

Execution:

[P1 $\rightarrow$ P2 $\rightarrow$ P3]

Completion Times:

[14,;7,;4]

Average Waiting Time:

[2.67]

Answer: 2.67

29.3.12 Solved

Problems

22–30

Nine additional SRTF numericals:

1. Avg WT = 1.5
2. Avg WT = 2
3. Avg WT = 2.33
4. Avg WT = 4
5. Avg WT = 3
6. Avg WT = 1
7. Avg WT = 5
8. Avg WT = 2.75
9. Avg WT = 3.5

29.3.13 Priority

Scheduling

Problems

29.3.14 Solved

Problems

31–40

1. Priority Order: 1,2,3 $\rightarrow$ Avg WT = 2
2. Avg WT = 4
3. Avg WT = 5
4. Avg WT = 3.5
5. Avg WT = 2.5
6. Avg WT = 6
7. Avg WT = 1
8. Avg WT = 4.33
9. Avg WT = 7
10. Avg WT = 2.67

29.3.15 Round

Robin

Problems

29.3.16 Solved

Problem

41

Time Quantum:

[Q=2]

Processes:

[P1=5,; P2=4,; P3=2]

Gantt Chart:

[P1;P2;P3;P1;P2;P1]

Average WT:

[4]

Answer: 4

29.3.17 Solved

Problems

42–50

1. Q=3 $\rightarrow$ Avg WT = 3.5
2. Q=4 $\rightarrow$ Avg WT = 2
3. Q=2 $\rightarrow$ Avg WT = 5
4. Q=1 $\rightarrow$ Avg WT = 6
5. Q=5 $\rightarrow$ Avg WT = 2.5
6. Q=2 $\rightarrow$ Avg WT = 4.5
7. Q=3 $\rightarrow$ Avg WT = 3
8. Q=4 $\rightarrow$ Avg WT = 1.5
9. Q=2 $\rightarrow$ Avg WT = 4

29.3.18 Multilevel

Queue

Problems

29.3.19 Solved

Problems

51–55

1. System Queue before User Queue
2. Avg WT = 3
3. Avg WT = 4
4. Avg WT = 5.5
5. Avg WT = 2

29.4 Deadlock**Numericals**

29.4.1	Resource	Allocation	Graph	Problems
---------------	-----------------	-------------------	--------------	-----------------

29.4.2	Solved	Problem	56
---------------	---------------	----------------	-----------

Graph:

[P1 $\rightarrow$ R1]

[R1 $\rightarrow$ P2]

[P2 $\rightarrow$ R2]

[R2 $\rightarrow$ P1]

Cycle exists.

Answer: Deadlock Present

29.4.3	Solved	Problems	57–60
---------------	---------------	-----------------	--------------

1. No Cycle $\rightarrow$ No Deadlock
2. Cycle Exists $\rightarrow$ Deadlock
3. Single Instance Cycle $\rightarrow$ Deadlock
4. Multiple Instance Cycle $\rightarrow$ Not Necessarily Deadlock

29.4.4	Wait-For	Graph	Problems
---------------	-----------------	--------------	-----------------

29.4.5	Solved	Problems	61–65
---------------	---------------	-----------------	--------------

1. Cycle Found $\rightarrow$ Deadlock
2. No Cycle $\rightarrow$ Safe
3. Triangle Cycle $\rightarrow$ Deadlock
4. Linear Chain $\rightarrow$ Safe
5. Self-loop $\rightarrow$ Deadlock

29.4.6	Banker's	Algorithm	Problems
---------------	-----------------	------------------	-----------------

29.4.7	Solved	Problems	66–75
---------------	---------------	-----------------	--------------

1. Safe Sequence = P1,P3,P4,P2
2. Safe Sequence = P2,P1,P3
3. Unsafe State

4. Safe State
5. Safe Sequence = P0,P2,P3,P1
6. Safe
7. Unsafe
8. Safe
9. Safe
10. Unsafe

29.5 Paging

Numericals

29.5.1 Address

Translation

Problems

29.5.2 Solved

Problem

76

Logical Address:

[2050]

Page Size:

[1024]

Page Number:

[$2050/1024 = 2$]

Offset:

[$2050-2048=2$]

Frame:

[Frame=5]

Physical Address:

[$5 \times 1024 + 2 =$]

5122]

Answer: 5122

29.5.3 Solved

Problems

77–85

1. PA = 2049
2. PA = 4098
3. PA = 6145
4. PA = 7168

5. PA = 9001
6. PA = 10050
7. PA = 12000
8. PA = 13024
9. PA = 15000

29.5.4 Multilevel Paging Problems

29.5.5 Solved Problems 86–90

1. Outer Index = 3, Inner = 10
2. Outer Index = 2, Inner = 7
3. Physical Address = 4096
4. Physical Address = 8192
5. Physical Address = 12288

29.5.6 TLB Problems

29.5.7 Solved Problems 91–95

1. TLB Hit Ratio=80
2. Hit Ratio=90
3. Hit Ratio=95
4. Hit Ratio=70
5. Hit Ratio=60

29.6 Segmentation Numericals

29.6.1 Solved Problem 96

Segment Table:

Segment	Base	Limit
1000	500	3000
800	height	

Logical Address:

[(1,200)]

Physical Address:

[3000+200 =====

3200]

Answer: 3200

29.6.2 Solved Problems 97–100

1. PA = 1500
2. Protection Fault
3. Shared Segment Valid
4. PA = 6400

29.7 Virtual Memory Numericals

29.7.1 Page Fault Problems

29.7.2 Solved Problems 101–105

1. Fault Rate = 10
2. Fault Rate = 20
3. Fault Rate = 5
4. Fault Rate = 15
5. Fault Rate = 30

29.7.3 Working Set Problems

29.7.4 Solved Problems 106–110

1. Working Set Size = 4
2. Working Set Size = 5
3. Working Set Size = 6
4. Working Set Size = 7
5. Working Set Size = 3

29.7.5 Thrashing Problems

29.7.6 Solved Problems 111–115

1. Thrashing Present
2. Thrashing Absent

3. Increase Frames
4. Reduce Multiprogramming
5. Working Set Policy Needed

29.8 Page Replacement Numericals

Reference String:

[7,0,1,2,0,3,0,4,2,3,0,3,2]

Frames:

[3]

29.8.1 FIFO Problems

29.8.2 Solved Problem 116

Page Faults:

[10]

Answer: 10

29.8.3 Optimal Problems

29.8.4 Solved Problem 117

Page Faults:

[7]

Answer: 7

29.8.5 LRU Problems

29.8.6 Solved Problem 118

Page Faults:

[9]

Answer: 9

29.8.7 Clock	Algorithm	Problems
---------------------	------------------	-----------------

29.8.8 Solved	Problems	119–122
----------------------	-----------------	----------------

1. Faults = 8
2. Faults = 9
3. Faults = 7
4. Faults = 10

29.8.9 Belady's		Anomaly
------------------------	--	----------------

29.8.10 Solved	Problems	123–125
-----------------------	-----------------	----------------

1. FIFO Frames 3 = 9 Faults
2. FIFO Frames 4 = 10 Faults
3. Belady's Anomaly Observed

29.9 Disk	Scheduling	Numericals
------------------	-------------------	-------------------

Requests:

[98,,183,,37,,122,,14,,124,,65,,67]

Initial Head:

[53]

29.9.1 FCFS		Problems
--------------------	--	-----------------

29.9.2 Solved	Problem	126
----------------------	----------------	------------

Total Movement:

[640]

Answer: 640

29.9.3 SSTF		Problems
--------------------	--	-----------------

29.9.4 Solved	Problem	127
----------------------	----------------	------------

Total Movement:

[236]

Answer: 236

29.9.5 SCAN Problems**29.9.6 Solved Problem 128**

Total Movement:

[208]

Answer: 208**29.9.7 CSCAN Problems****29.9.8 Solved Problem 129**

Total Movement:

[382]

Answer: 382**29.9.9 LOOK Problems****29.9.10 Solved Problem 130**

Total Movement:

[299]

Answer: 299**29.9.11 CLOOK Problems****29.9.12 Solved Problem 131**

Total Movement:

[322]

Answer: 322**29.10 Rapid Numerical Problem Solving Techniques**

1. Draw Gantt charts immediately.
2. Always sort burst times for SJF.
3. Track remaining time for SRTF.
4. For RR, use queue simulation.

5. Learn standard FIFO/LRU examples.
6. Memorize Banker's Algorithm procedure.
7. Split logical addresses into:
 - Page Number
 - Offset
8. For segmentation: [$PA = Base + Offset$]
9. For disk scheduling always draw head movement.
10. Calculate averages at the end only.

29.11 Formula

Sheet

$$[WT = TAT - BT]$$

$$[TAT = CT - AT]$$

$$[PA = Frame \times PageSize + Offset]$$

$$[PA = Base + Offset]$$

$$[EAT = (1-p)m + p(PageFaultCost)]$$

$$[Average = \frac{\sum Values}{N}]$$

29.12 Numerical

Cheat

Sheet

- FCFS: Execute by arrival order.
- SJF: Smallest burst first.
- SRTF: Smallest remaining time first.
- RR: Rotate after quantum.
- Paging: Split into page number and offset.
- Segmentation: Base + Offset.
- FIFO may show Belady's anomaly.
- Optimal gives minimum page faults.
- LRU approximates Optimal.
- SSTF minimizes seek movement locally.
- LOOK stops at last request.
- CSCAN provides uniform waiting.

29.13 Key

Takeaways

- Numerical problems dominate OS examinations.

- CPU scheduling requires mastery of waiting and turnaround calculations.
- Paging and segmentation require address translation practice.
- Banker's Algorithm is a frequent interview topic.
- Page replacement numericals are among the most commonly asked OS questions.
- Disk scheduling problems become easy once head movement diagrams are drawn.
- Fast problem solving comes from recognizing standard patterns and formulas.

Operating Systems Cheat Sheets

30.1 Introduction

This chapter is designed for rapid interview revision and last-minute preparation. Every section focuses on concise recall, common interview questions, key comparisons, and essential formulas.

30.2 Operating System Fundamentals

Key Definitions

- Operating System: Resource manager and abstraction layer.
- Kernel: Core component of the OS.
- System Call: Interface between user programs and kernel.
- Interrupt: Signal requiring CPU attention.
- Trap: Software-generated interrupt.
- DMA: Direct Memory Access.

Important Facts

- OS manages CPU, memory, storage, and I/O devices.
- User programs cannot directly access hardware.
- Kernel mode has full privileges.
- User mode is restricted.

Most Common Interview Questions

1. What is an operating system?
2. What is a kernel?
3. What is a system call?
4. User mode vs kernel mode?
5. What is context switching?

Memory**Trick**

OS = Resource Manager + Abstraction Layer

30.3 Computer**Architecture****Key****Definitions**

- CPU
- Register
- Program Counter
- Stack Pointer
- Cache
- MMU

Memory**Hierarchy**

[Registers > L1 > L2 > L3 > RAM > SSD > HDD]

Interview**Favorites**

- Fetch-Decode-Execute Cycle
- Cache Memory
- Interrupt Handling
- DMA
- NUMA

30.4 Processes**Key****Definitions**

- Process = Program in execution.
- PCB = Process Control Block.
- Context Switch = Switching CPU execution.

Process**States**

[New → Ready → Running → Waiting → Terminated]

Common**Questions**

- fork vs exec
- Zombie process
- Orphan process
- Context switching

30.5 Threads**Key****Definitions**

- Thread = Lightweight execution unit.
- Shares process memory.
- Own stack and registers.

Benefits

- Responsiveness
- Resource sharing
- Scalability
- Economy

30.6 CPU**Scheduling****Algorithms**

- FCFS
- SJF
- SRTF
- Round Robin
- Priority
- Multilevel Queue

Metrics

- Turnaround Time
- Waiting Time
- Response Time
- Throughput

- CPU Utilization

Formula**Summary**

$$[\text{Turnaround} = \text{Completion} - \text{Arrival}]$$

$$[\text{Waiting} = \text{Turnaround} - \text{Burst}]$$

30.7 Synchronization**Goals**

- Mutual Exclusion
- Progress
- Bounded Waiting

Common**Questions**

- Race Condition
- Critical Section
- Semaphore
- Mutex
- Monitor

30.8 Critical**Section****Problem****Requirements**

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Classic**Solutions**

- Peterson's Algorithm
- Bakery Algorithm
- Test-and-Set
- Compare-and-Swap

30.9 Semaphores

Definition

Integer synchronization variable.

Operations

- wait()
- signal()

Types

- Binary Semaphore
- Counting Semaphore

30.10 Mutexes

Definition

Lock mechanism providing ownership semantics.

Key

Difference

Mutex owner must unlock.

30.11 Deadlocks

Necessary

Conditions

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Strategies

- Prevention
- Avoidance

- Detection
- Recovery

30.12 Memory

Management

Key

Terms

- Logical Address
- Physical Address
- Fragmentation
- Compaction

Fragmentation

- Internal
- External

30.13 Paging

Formula

$$[\text{PhysicalAddress} = (\text{FrameNumber} \times \text{FrameSize}) + \text{Offset}]$$

Key

Terms

- Page
- Frame
- Page Table
- TLB

30.14 Segmentation

Address

Format

[(segment, offset)]

Characteristics

- Logical View

- Variable Sized
- External Fragmentation

30.15 Virtual

Memory

Key

Concepts

- Demand Paging
- Page Fault
- Thrashing
- Swap Space
- Copy-on-Write

Interview

Favorite

What happens during a page fault?

30.16 Page

Replacement

Algorithms

- FIFO
- Optimal
- LRU
- LFU
- Clock

Key

Fact

Optimal is theoretical benchmark.

30.17 Memory

Allocation

Algorithms

- First Fit
- Best Fit
- Worst Fit

- Next Fit
- Buddy System
- Slab Allocator

30.18 File Systems

Key Terms

- File
- Directory
- Inode
- Superblock

Allocation Methods

- Contiguous
- Linked
- Indexed

30.19 File System Implementation

On-Disk Structures

- Boot Block
- Superblock
- Inodes
- Data Blocks

Consistency

- Journaling
- fsck

30.20 Disk Scheduling

Algorithms

- FCFS

- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK

30.21 I/O

Systems

I/O

Types

- Programmed I/O
- Interrupt-Driven I/O
- DMA

Models

- Blocking
- Non-Blocking
- Asynchronous

30.22 Linux

Internals

Must

Know

Topics

- Process Creation
- fork()
- exec()
- CFS Scheduler
- Signals
- systemd
- proc
- sysfs

30.23 Modern

Concurrency

Key

Topics

- Lock-Free Programming

- Atomic Operations
- CAS
- Memory Ordering
- Thread Pools

30.24 Containers

Core

Concepts

- Namespaces
- cgroups
- Images
- Containers

30.25 Virtualization

Key

Concepts

- Hypervisor
- Type 1
- Type 2
- Hardware Virtualization

30.26 Operating

System

Security

Key

Topics

- Authentication
- Authorization
- Access Control
- ASLR
- Secure Boot
- Sandboxing

30.27 High-Concurrency

Systems

Must

Know

- Thread Pools
- Event Loops
- epoll
- Reactor Pattern
- Proactor Pattern
- Backpressure

30.28 Comparison

Tables

Process

vs

Thread

Aspect	Process	Thread
Memory	Separate	Shared
Creation Cost	High	Low
Context Switch	Expensive	Cheaper
Isolation	Strong	Weak

Mutex

vs

Semaphore

Aspect	Mutex	Semaphore
Ownership	Yes	No
Counter	No	Yes
Unlocker	Owner Only	Any Thread

Paging

vs

Segmentation

Aspect	Paging	Segmentation
Unit Size	Fixed	Variable
Fragmentation	Internal	External
View	Physical	Logical

VM

vs

Container

Aspect	VM	Container
--------	----	-----------

Kernel	Separate	Shared
Startup	Slow	Fast
Isolation	Strong	Moderate

FCFS vs SJF vs RR

Aspect	FCFS	SJF	RR
Preemptive	No	Optional	Yes
Fairness	Medium	Poor	High
Response Time	Poor	Good	Good

Blocking vs Non-Blocking I/O

Aspect	Blocking	Non-Blocking
Waits	Yes	No
CPU Usage	Low	Higher
Concurrency	Lower	Higher

Synchronization Mechanisms

Mechanism	Ownership	Blocking	Use Case
Mutex	Yes	Yes	Critical Sections
Semaphore	No	Yes	Resource Counting
Spinlock	Yes	No	Short Waits
Monitor	Yes	Yes	High-Level Sync

30.29 Top 100 One-Line Facts

1. OS is a resource manager.
2. Kernel runs in privileged mode.
3. Threads share address space.
4. Processes provide isolation.
5. TLB caches translations.
6. Page faults trigger OS intervention.
7. Context switches are expensive.
8. Mutex provides ownership.
9. Semaphore is a counter.
10. Deadlock requires four conditions.

11. Paging eliminates external fragmentation.
12. Segmentation reflects logical program structure.
13. Virtual memory enables large address spaces.
14. LRU approximates optimal replacement.
15. Linux uses copy-on-write fork.
16. ext4 uses journaling.
17. DMA bypasses CPU copying.
18. epoll scales well.
19. Containers share kernels.
20. Hypervisors run VMs.
21. CFS schedules fairly.
22. Signals provide notifications.
23. ASLR randomizes memory layout.
24. NUMA affects memory latency.
25. Thrashing destroys performance.
26. Buddy allocators split powers of two.
27. Slab allocators cache kernel objects.
28. Working set prevents thrashing.
29. Page tables map addresses.
30. Inodes store metadata.
31. SCAN resembles an elevator.
32. SSTF minimizes seek distance.
33. Round Robin improves responsiveness.
34. Starvation affects priority scheduling.
35. Aging reduces starvation.
36. Critical sections require protection.
37. CAS supports lock-free programming.
38. Monitors encapsulate synchronization.
39. Namespaces isolate containers.
40. cgroups enforce limits.
41. Docker uses namespaces and cgroups.
42. Linux boot starts with firmware.
43. System calls cross protection boundaries.
44. Cache is faster than RAM.
45. Registers are fastest storage.
46. User mode is restricted.
47. Kernel mode is privileged.
48. File descriptors identify open files.

49. IPC enables process communication.
50. Shared memory is fastest IPC.
51. Pipes are unidirectional.
52. Sockets support networking.
53. Fork duplicates a process.
54. Exec replaces process image.
55. Zombie retains exit status.
56. Orphan is adopted by init.
57. Daemons run in background.
58. Virtualization improves isolation.
59. Containers improve density.
60. Event loops scale efficiently.
61. Thread pools reduce overhead.
62. Connection pools improve throughput.
63. Backpressure prevents overload.
64. Reactor handles readiness events.
65. Proactor handles completion events.
66. Memory-mapped files use virtual memory.
67. Swap extends memory.
68. Belady's anomaly affects FIFO.
69. Optimal replacement is theoretical.
70. Page size impacts performance.
71. Huge pages reduce TLB misses.
72. Kernel panic is fatal.
73. `proc` exposes kernel data.
74. `sysfs` exposes device information.
75. Scheduler chooses runnable tasks.
76. Locks serialize access.
77. Deadlocks stop progress.
78. Recovery breaks cycles.
79. Distributed locks extend mutexes.
80. Strong consistency returns latest writes.
81. Eventual consistency converges later.
82. Replication improves availability.
83. Consensus enables agreement.
84. Leader election selects coordinator.
85. Vector clocks track causality.
86. NTP synchronizes clocks.

87. Authentication verifies identity.
88. Authorization verifies permissions.
89. Secure boot validates startup.
90. SELinux enforces policies.
91. Cache coherence matters in multicore CPUs.
92. NUMA favors local memory access.
93. CPU scheduling affects responsiveness.
94. Throughput measures completed work.
95. Latency measures delay.
96. Availability measures uptime.
97. Reliability measures correctness.
98. Scalability measures growth capability.
99. Concurrency differs from parallelism.
100. Synchronization prevents races.
101. Protection prevents misuse.
102. Isolation improves reliability.
103. OS concepts underpin distributed systems.

30.30 Top 100 Interview Rapid Fire Questions

1. What is an OS?
2. What is a kernel?
3. What is a system call?
4. What is a process?
5. What is a thread?
6. Process vs thread?
7. What is PCB?
8. What is context switching?
9. What is a race condition?
10. What is a semaphore?
11. What is a mutex?
12. What is a monitor?
13. What is deadlock?
14. Four deadlock conditions?
15. What is starvation?
16. What is aging?
17. What is paging?
18. What is segmentation?

19. What is TLB?
20. What is a page fault?
21. What is virtual memory?
22. What is thrashing?
23. What is swap?
24. What is copy-on-write?
25. What is FIFO?
26. What is LRU?
27. What is Optimal replacement?
28. What is fragmentation?
29. Internal fragmentation?
30. External fragmentation?
31. What is buddy allocation?
32. What is slab allocation?
33. What is an inode?
34. What is a superblock?
35. What is journaling?
36. What is ext4?
37. What is DMA?
38. What is interrupt-driven I/O?
39. What is blocking I/O?
40. What is non-blocking I/O?
41. What is asynchronous I/O?
42. What is select?
43. What is poll?
44. What is epoll?
45. What is Linux CFS?
46. What is fork()?
47. What is exec()?
48. What is a zombie process?
49. What is an orphan process?
50. What is a signal?
51. What is proc?
52. What is sysfs?
53. What is a namespace?
54. What is a cgroup?
55. What is Docker?
56. What is Kubernetes?

57. What is a hypervisor?
58. Type 1 hypervisor?
59. Type 2 hypervisor?
60. What is virtualization?
61. What is a VM?
62. What is a container?
63. What is ASLR?
64. What is secure boot?
65. What is SELinux?
66. What is NUMA?
67. What is cache coherence?
68. What is a spinlock?
69. What is CAS?
70. What is memory ordering?
71. What is a thread pool?
72. What is a connection pool?
73. What is backpressure?
74. What is a reactor?
75. What is a proactor?
76. What is throughput?
77. What is latency?
78. What is scalability?
79. What is availability?
80. What is consistency?
81. What is consensus?
82. What is Raft?
83. What is Paxos?
84. What is a distributed lock?
85. What is leader election?
86. What is NTP?
87. What is a logical clock?
88. What is a vector clock?
89. What is CAP theorem?
90. Strong consistency?
91. Eventual consistency?
92. What is replication?
93. What is fault tolerance?
94. What is 2PC?

95. What is IPC?
96. Shared memory?
97. Pipe?
98. Socket?
99. Why does thread-per-request not scale?
100. How does Nginx scale?

30.31 **Final** **Interview** **Day** **Checklist**

- Revise Process vs Thread.
- Revise Mutex vs Semaphore.
- Memorize Deadlock Conditions.
- Review Page Fault Handling.
- Review TLB and Paging.
- Review Virtual Memory.
- Review Scheduling Algorithms.
- Review Synchronization Problems.
- Review Linux Process Lifecycle.
- Review Containers and Virtualization.
- Review epoll and Event Loops.
- Practice concise answers.
- Focus on trade-offs.
- Explain concepts visually.
- Connect OS to system design.

30.32 **Complete** **OS** **Revision** **Map**

1. Fundamentals
2. Computer Architecture
3. Processes
4. Threads
5. Scheduling
6. Synchronization
7. Deadlocks
8. Memory Management
9. Paging
10. Segmentation

11. Virtual Memory
12. Page Replacement
13. Allocation
14. File Systems
15. Storage
16. I/O
17. Linux
18. Security
19. Virtualization
20. Containers
21. Concurrency
22. Distributed Systems Connections
23. High-Concurrency Systems
24. System Design Applications

30.33 Key

Takeaways

Final Revision Notes and Interview Strategy

31.1 Purpose of This Chapter

This chapter is designed as the final revision guide for Operating Systems interview preparation. It condenses the most important concepts from the entire handbook into a structured review plan, interview framework, and rapid revision reference.

31.2 30-Day Revision Plan

Week 1: Foundations

- Operating System Basics
- Computer Architecture
- Processes
- Threads
- Process Management

Week 2: Concurrency

- CPU Scheduling
- Critical Section Problem
- Mutexes
- Semaphores
- Monitors
- Classical Synchronization Problems

Week 3: Memory

- Memory Management

- Paging
- Segmentation
- Virtual Memory
- Page Replacement

Week

4:

Advanced

Topics

- File Systems
- Disk Scheduling
- Linux Internals
- Containers
- Virtualization
- Security
- System Design Connections

31.3 14-Day

Revision

Plan

Day	Topics height1
OS Fundamentals 2	Architecture + Boot Process 3
Processes 4	Threads 5
Scheduling 6	Synchronization 7
Deadlocks 8	Memory Management 9
Paging 10	Segmentation 11
Virtual Memory 12	File Systems 13
Linux + Containers 14	Full Revision height

31.4 7-Day

Revision

Plan

1. Fundamentals + Architecture
2. Processes + Threads
3. Scheduling + Synchronization
4. Deadlocks + Memory
5. Paging + Virtual Memory
6. File Systems + Disk Scheduling
7. Linux + Security + System Design

31.5 3-Day Revision Plan

Day 1

- Processes
- Threads
- Scheduling
- Synchronization

Day 2

- Deadlocks
- Paging
- Segmentation
- Virtual Memory

Day 3

- File Systems
- Linux
- Security
- Containers
- Revision Questions

31.6 1-Day Revision Plan

1. Process vs Thread
2. Scheduling Algorithms
3. Semaphores vs Mutexes
4. Deadlock Handling
5. Paging vs Segmentation
6. Page Replacement Algorithms
7. File Systems
8. Disk Scheduling
9. Linux Concepts
10. Containers and Virtualization

31.7 Interview

Morning

Checklist

- Review cheat sheets.
- Review scheduling formulas.
- Review page replacement algorithms.
- Review synchronization mechanisms.
- Review Linux commands.
- Revise top interview questions.
- Sleep well.
- Stay calm and structured.

31.8 OS Fundamentals Revision

- OS manages hardware resources.
- Kernel is the core component.
- User Mode and Kernel Mode provide isolation.
- System calls provide OS services.
- Examples:
 - Linux
 - Windows
 - macOS

31.9 Computer Architecture Revision

- CPU executes instructions.
- Registers are fastest memory.
- Cache improves performance.
- MMU performs address translation.
- Interrupts transfer control to kernel.

31.10 Processes Revision

- Process = Program in execution.
- Own address space.
- PCB stores process information.
- Context switch changes running process.
- `fork()` creates child process.
- `exec()` loads new program.
- `wait()` waits for child completion.

31.11 Threads Revision

- Lightweight execution units.
- Share process memory.
- Faster creation than processes.
- Context switch cheaper than process switch.

31.12 Scheduling

Revision

- FCFS
- SJF
- SRTF
- Priority
- Round Robin
- Multilevel Queue
- MLFQ

Remember:

$$[WT = TAT - BT]$$

$$[TAT = CT - AT]$$

31.13 Synchronization

Revision

- Race condition
- Critical section
- Mutex
- Semaphore
- Monitor
- Read-write lock
- Atomic operations

31.14 Deadlocks

Revision

Coffman Conditions:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Solutions:

- Prevention
- Avoidance
- Detection
- Recovery

31.15 Memory Management Revision

- Logical Address
- Physical Address
- MMU
- Swapping
- Relocation
- Fragmentation

31.16 Paging Revision

[PhysicalAddress =====

$\text{Frame} \times \text{PageSize} + \text{Offset}$]

Key Concepts:

- Page
- Frame
- Page Table
- TLB
- Address Translation

31.17 Segmentation Revision

[PhysicalAddress =====

Base+Offset]

Key Concepts:

- Segment Table
- Protection
- Sharing
- External Fragmentation

31.18 Virtual Memory Revision

- Demand Paging
- Page Faults
- Thrashing

- Working Set
- Locality

31.19 Page Replacement Revision

Algorithms:

- FIFO
- Optimal
- LRU
- Clock
- LFU

Important:

- Belady's Anomaly affects FIFO.
- Stack Algorithms avoid Belady's anomaly.

31.20 File Systems Revision

- File
- Directory
- Metadata
- Inodes
- Permissions
- Hard Links
- Symbolic Links

31.21 Disk Scheduling Revision

Algorithms:

- FCFS
- SSTF
- SCAN
- CSCAN
- LOOK
- CLOOK

31.22 Linux Internals Revision

- Process Management
- Scheduling
- CFS Scheduler
- Virtual Memory
- VFS
- Signals
- cgroups
- Namespaces

31.23 Containers Revision

- Namespaces
- cgroups
- Shared Kernel
- Docker
- Kubernetes

31.24 Virtualization Revision

- Hypervisor
- Type 1
- Type 2
- Virtual Machines
- Resource Isolation

31.25 Security Revision

- Authentication
- Authorization
- ACLs
- Capabilities
- ASLR
- DEP
- SELinux
- AppArmor

31.26 High-Concurrency**Systems****Revision**

- Thread Pools
- Event Loops
- Async Programming
- Futures
- Promises
- Lock-Free Structures

31.27 Top 50 Questions Every Candidate Must Know

1. What is an Operating System?
2. What is a kernel?
3. User mode vs kernel mode?
4. What is a system call?
5. Process vs program?
6. Process vs thread?
7. What is a PCB?
8. What is context switching?
9. Explain `fork()`.
10. Explain `exec()`.
11. Explain `wait()`.
12. What is CPU scheduling?
13. FCFS vs SJF?
14. SJF vs SRTF?
15. What is starvation?
16. What is aging?
17. What is a race condition?
18. What is a mutex?
19. What is a semaphore?
20. Binary vs counting semaphore?
21. What is a monitor?
22. What is deadlock?
23. Coffman conditions?
24. Deadlock prevention?
25. Deadlock avoidance?
26. Banker's Algorithm?
27. What is paging?
28. What is a page fault?
29. What is a TLB?
30. What is segmentation?
31. Paging vs segmentation?
32. Internal fragmentation?
33. External fragmentation?
34. What is virtual memory?
35. What is thrashing?
36. FIFO page replacement?

37. LRU page replacement?
38. Belady's anomaly?
39. What is an inode?
40. Hard link vs symbolic link?
41. What is disk scheduling?
42. SSTF vs SCAN?
43. What is RAID?
44. What is a namespace?
45. What are cgroups?
46. Container vs VM?
47. What is ASLR?
48. What is DEP?
49. Authentication vs authorization?
50. How does Linux scheduling work?

31.28 Top 50 Mistakes Candidates Make

1. Confusing process and program.
2. Confusing process and thread.
3. Mixing waiting and turnaround time.
4. Forgetting response time.
5. Incorrect Gantt charts.
6. Misunderstanding SRTF.
7. Forgetting starvation.
8. Confusing mutex and semaphore.
9. Ignoring race conditions.
10. Wrong deadlock definitions.
11. Forgetting all Coffman conditions.
12. Incorrect Banker's Algorithm.
13. Confusing page and frame.
14. Wrong address translation.
15. Forgetting TLB role.
16. Mixing paging and segmentation.
17. Confusing internal/external fragmentation.
18. Misunderstanding virtual memory.
19. Wrong page replacement counts.
20. Forgetting Belady's anomaly.
21. Misunderstanding thrashing.

22. Confusing inode and file.
23. Hard link confusion.
24. Soft link confusion.
25. Wrong disk scheduling calculations.
26. Ignoring seek time.
27. VM/container confusion.
28. Linux command uncertainty.
29. Security terminology confusion.
30. ACL/capability confusion.
31. ASLR misunderstanding.
32. Weak trade-off explanations.
33. Memorizing without understanding.
34. No diagrams.
35. No examples.
36. Not explaining assumptions.
37. Not asking clarifying questions.
38. Jumping to implementation.
39. Missing edge cases.
40. Poor communication.
41. Long-winded answers.
42. No structure.
43. Lack of system design connection.
44. Lack of concurrency understanding.
45. Ignoring scalability.
46. Ignoring performance.
47. Ignoring memory behavior.
48. Ignoring security implications.
49. Ignoring follow-up questions.
50. Lack of revision.

31.29 Top 50 Concepts Frequently Confused

Concept A	Concept B
Process Thread	Program
Kernel Mode Concurrency	Process User Mode
Semaphore Semaphore	Parallelism Mutex
Starvation Prevention	Monitor Deadlock
Segmentation Logical Address	Avoidance Paging
Frame Internal Fragmentation	Physical Address Page
Physical Memory FIFO	External Fragmentation Virtual Memory
Symbolic Link RAID 0	LRU Hard Link
Virtual Machine Authentication	RAID 1 Container
Capability Hypervisor	Authorization ACL
	Container Runtime

31.30 FAANG Interview Preparation Guidance

Amazon

Focus on:

- Trade-offs
- Scalability
- Linux
- Concurrency

Google

Focus on:

- Deep fundamentals
- Scheduling
- Memory
- Distributed systems

Meta

Focus on:

- Performance

- Concurrency
- Scalability

Microsoft

Focus on:

- OS internals
- Synchronization
- Memory management

31.31 How to Answer OS Questions

Recommended structure:

1. Definition
2. Internal Working
3. Example
4. Advantages
5. Disadvantages
6. Trade-Offs
7. Real-World Usage

31.32 How to Handle Follow-Up Questions

1. Listen carefully.
2. Clarify assumptions.
3. Explain reasoning.
4. Use examples.
5. Draw diagrams.
6. Discuss trade-offs.

31.33 How to Explain Trade-Offs

Framework:

- Performance
- Memory
- Complexity

- Scalability
- Reliability
- Security

Example:

Mutex:

- Simpler
- Lower overhead
- Less flexible

Semaphore:

- More flexible
- More complex

31.34 How to Draw OS Diagrams During Interviews

Useful diagrams:

- Process States
- Address Translation
- Page Tables
- Segment Tables
- Resource Allocation Graph
- Scheduling Gantt Chart
- File System Tree
- Container Architecture

31.35 The Complete Operating Systems Knowledge Map

OS → Processes → Threads → Scheduling → Synchronization → Deadlocks → Memory →
Paging → Virtual Memory → File Systems → Storage → Linux → Containers → Security →
Scalable Systems

31.36 One-Hour

Revision

Guide

1. Processes (10 min)
2. Threads (5 min)

3. Scheduling (10 min)
4. Synchronization (10 min)
5. Deadlocks (5 min)
6. Memory (10 min)
7. File Systems (5 min)
8. Linux + Security (5 min)

31.37 Thirty-Minute Revision Guide

- Process vs Thread
- Scheduling Algorithms
- Mutex vs Semaphore
- Deadlocks
- Paging vs Segmentation
- LRU vs FIFO
- Containers vs VMs
- Authentication vs Authorization

31.38 Fifteen-Minute Revision Guide

- Top 50 Questions
- Formula Sheet
- Key Comparisons
- Linux Concepts

31.39 Five-Minute Emergency Revision Guide

Remember:

- Process vs Thread
- Mutex vs Semaphore
- Deadlock Conditions
- Paging vs Segmentation
- FIFO vs LRU
- Hard vs Soft Link
- Container vs VM
- Authentication vs Authorization

31.40 Final Key Takeaways

- Understand concepts before memorizing.
- Explain internal working.
- Use diagrams frequently.
- Always discuss trade-offs.
- Connect OS concepts to real systems.
- Practice scheduling numericals.
- Practice page replacement problems.
- Review Linux internals.
- Understand concurrency deeply.
- Communicate clearly and confidently.

31.41 Final Interview Checklist

- Processes
- Threads
- Scheduling
- Synchronization
- Deadlocks
- Paging
- Segmentation
- Virtual Memory
- File Systems
- Disk Scheduling
- Linux
- Containers
- Security
- System Design Connections
- Numerical Problems

31.42 Closing Revision Notes

Operating Systems is one of the highest-leverage subjects in software engineering interviews because it connects directly to performance, scalability, concurrency, reliability, and security.

Mastering Operating Systems means understanding how software interacts with hardware, how resources are managed, and how modern systems achieve efficiency at scale.

Focus on fundamentals, practice explaining concepts aloud, solve numerical problems regularly, and always connect theoretical concepts to real-world systems. Candidates who can explain *why* an operating system behaves a certain way—not just *what* it does—consistently perform better in interviews.

The goal is not memorization. The goal is building a mental model of how computer systems work. Once that model is clear, interview questions become variations of concepts you already understand.

Bibliography
